

The Amazon Book

Ejercicios LeetCode para entrevistas

Aldo Zetina

13 de julio de 2026

Índice general

Introducción	3
1 Grafos	4
200. Number of Islands (M)	5
994. Rotting Oranges (M)	8
695. Max Area of Island (M)	11
542. 01 Matrix (M)	14
1091. Shortest Path in Binary Matrix (M)	17
417. Pacific Atlantic Water Flow (M)	21
79. Word Search (M)	26
207. Course Schedule (M)	30
210. Course Schedule II (M)	34
269. Alien Dictionary (H)	39
127. Word Ladder (H)	45
126. Word Ladder II (H)	49
261. Graph Valid Tree (M)	54
323. Number of Connected Components (M)	59
128. Longest Consecutive Sequence (M)	63
547. Number of Provinces (M)	67
743. Network Delay Time (M)	71
787. Cheapest Flights Within K Stops (M)	75
133. Clone Graph (M)	79
1192. Critical Connections in a Network (H)	84
2 Árboles	90
236. Lowest Common Ancestor of a Binary Tree (M)	91
1644. Lowest Common Ancestor of a Binary Tree II (M)	95
103. Binary Tree Zigzag Level Order Traversal (M)	99
199. Binary Tree Right Side View (M)	103
987. Vertical Order Traversal of a Binary Tree (H)	108
124. Binary Tree Maximum Path Sum (H)	112
543. Diameter of Binary Tree (Fácil)	117
863. All Nodes Distance K in Binary Tree (Media)	121
98. Validate Binary Search Tree (M)	126
105. Construct Binary Tree from Preorder and Inorder Traversal (M)	130
337. House Robber III (M)	134
297. Serialize and Deserialize Binary Tree (D)	138

Introducción

Este libro reúne ejercicios de LeetCode frecuentes en entrevistas de Amazon, resueltos en C++ y explicados para todo público. Cada encabezado de problema es un enlace: al hacer clic se abre el archivo `.cpp` correspondiente en Visual Studio Code, donde el código es compilable y trae casos de prueba.

Dónde está el código. Todo el código fuente vive en la carpeta `src/` del repositorio, organizada por bloque (`src/01-grafos/`, `src/02-arboles/`, ...). El libro lo importa directamente desde ahí (no se duplica). Bajo el título de cada problema se muestra la ruta exacta del archivo, por si quieres abrirlo o consultarlo directamente en `src/`.

Animaciones interactivas. Además del código, cada problema incluye el enlace ▷ *Ver animación* con dos accesos: **local** (abre el visualizador del repo descargado, funciona sin internet) y **web** (abre la versión publicada en GitHub Pages desde cualquier equipo). Muestra una visualización del algoritmo paso a paso: la cuadrícula se pinta, la cola del BFS avanza, el árbol se recorre. El visualizador vive en la carpeta `rep-visual/`, funciona sin instalar nada (basta un navegador) y permite alternar el idioma entre español e inglés. Los enlaces piden abrirse en una ventana nueva para no reemplazar la vista del PDF.

Cómo compilar. Ejecuta `build.ps1` en PowerShell. El script detecta la ruta del repositorio e inyecta las rutas de los enlaces, luego compila con `latexmk -xelatex`.

Capítulo 1

Grafos

200. Number of Islands

(M)

Código fuente: [src/01-grafos/200-number-of-islands.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de caracteres donde '1' es tierra y '0' es agua. Una *isla* es un grupo de tierras conectadas en horizontal o vertical. Queremos contar cuántas islas hay.

La idea, con un ejemplo de la vida real. Imagina que esa cuadrícula es un mapa y que tú lo miras desde un avión. Vas leyendo casilla por casilla, como cuando lees un libro: de izquierda a derecha y de arriba abajo. Cuando ves un pedazo de tierra por primera vez, gritas «¡isla nueva!» y sumas 1 a tu conteo.

Pero ojo: esa isla puede ser grande y tener muchas casillas pegadas. Si las cuentas otra vez, te dará un número equivocado. ¿El truco? En cuanto pisas una casilla de tierra, la «inundas»: la pintas de agua (la cambias a '0'). Así ya no la vuelves a ver como tierra y no la cuentas dos veces. Y desde esa casilla te vas caminando a sus vecinas de tierra e inundas todas también, hasta que toda la isla quede bajo el agua. Cuando terminas, sigues leyendo la cuadrícula buscando la siguiente tierra sin pisar.

```

1  1  0  0  0
1  1  0  0  0
0  0  1  0  0
0  0  0  1  1

```

En esta cuadrícula hay **3** islas: el bloque 2×2 de arriba a la izquierda, la celda central, y el par de abajo a la derecha.

Cómo lo resuelve el código, paso a paso. Recorremos todas las casillas con dos ciclos (uno para las filas, otro para las columnas). Cuando encontramos un '1':

1. Sumamos 1 al contador de islas.
2. Llamamos a una pequeña ayudante llamada `hundir`, que se encarga de inundar toda la isla.

La ayudante `hundir` hace algo muy natural: se para en una casilla de tierra, la convierte en agua y luego se llama a sí misma sobre sus cuatro vecinas (arriba, abajo, izquierda, derecha). Que una función se llame a sí misma se llama *recursión*; piénsalo como una ficha de dominó que tira a las de al lado, y esas a las suyas, hasta que ya no queda tierra que tocar. Antes de inundar, `hundir` revisa dos cosas para no equivocarse: que no se salga de la cuadrícula y que la casilla sea de verdad tierra. Este recorrido que avanza «hacia el fondo» por los vecinos se conoce como *Depth-First Search* (DFS, búsqueda en profundidad). El código completo (haz clic en el título para abrirlo en tu editor):

```

1 //
2 // =====
3 //
4 // 200. Number of Islands (Medium)
5 //
6 // -----
7 //
8 // Dada una cuadrícula de '1' (tierra) y '0' (agua), cuenta cuántas islas hay.
9 // Una isla se forma conectando tierras adyacentes en horizontal o vertical
10 // (no en diagonal). Los bordes de la cuadrícula son todos agua.

```

```

7 //
8 // Ejemplo:
9 //   grid =
10 //       1 1 0 0 0           Aquí hay 3 islas:
11 //       1 1 0 0 0           - bloque 2x2 arriba-izquierda
12 //       0 0 1 0 0           - celda central
13 //       0 0 0 1 1           - bloque de 2 abajo-derecha
14 //
15 // Diagrama de la exploración DFS desde una celda de tierra:
16 //
17 //       (r-1,c)
18 //       |
19 //   (r,c-1)-(r,c)-(r,c+1)   Visitamos las 4 vecinas; cada tierra
20 //       |                   visitada la marcamos como '0' (hundida)
21 //       (r+1,c)             para no contarla otra vez.
22 //
23 // Idea: recorremos la cuadrícula. Cuando encontramos un '1' no visitado,
24 // sumamos una isla y "hundimos" (marcamos como agua) toda su tierra conectada
25 // con DFS. Así cada isla se cuenta exactamente una vez.
26 //
27 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita una vez),
28 // espacio  $O(m*n)$  peor caso por la pila de recursión.
29 //
=====

30 #include <vector>
31 #include <cassert>
32 #include <iostream>
33 using namespace std;
34
35 class Solution {
36 public:
37     int numIslands(vector<vector<char>>& grid) {
38         if (grid.empty() || grid[0].empty()) return 0;
39         int m = grid.size(), n = grid[0].size();
40         int islas = 0;
41         for (int r = 0; r < m; ++r) {
42             for (int c = 0; c < n; ++c) {
43                 if (grid[r][c] == '1') { // tierra sin visitar => nueva isla
44                     ++islas;
45                     hundir(grid, r, c, m, n);
46                 }
47             }
48         }
49         return islas;
50     }
51
52 private:
53     // Marca como agua toda la tierra conectada a (r,c).
54     void hundir(vector<vector<char>>& grid, int r, int c, int m, int n) {
55         if (r < 0 || r >= m || c < 0 || c >= n) return; // fuera de límites
56         if (grid[r][c] != '1') return; // agua o ya
57             visitada
58         grid[r][c] = '0'; // hundir esta celda

```

```

58     hundir(grid, r - 1, c, m, n);    // arriba
59     hundir(grid, r + 1, c, m, n);    // abajo
60     hundir(grid, r, c - 1, m, n);    // izquierda
61     hundir(grid, r, c + 1, m, n);    // derecha
62 }
63 };
64
65 int main() {
66     Solution s;
67
68     vector<vector<char>> g1 = {
69         {'1','1','0','0','0'},
70         {'1','1','0','0','0'},
71         {'0','0','1','0','0'},
72         {'0','0','0','1','1'},
73     };
74     assert(s.numIslands(g1) == 3);
75
76     vector<vector<char>> g2 = {
77         {'1','1','1','1','0'},
78         {'1','1','0','1','0'},
79         {'1','1','0','0','0'},
80         {'0','0','0','0','0'},
81     };
82     assert(s.numIslands(g2) == 1);
83
84     vector<vector<char>> g3 = {{'0','0'},{'0','0'}};
85     assert(s.numIslands(g3) == 0);
86
87     vector<vector<char>> vacio = {};
88     assert(s.numIslands(vacio) == 0);
89
90     cout << "200 Number of Islands: todas las pruebas pasaron.\n";
91     return 0;
92 }

```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

¿Qué significa esa complejidad? Aquí m es el número de filas y n el de columnas, así que $m \cdot n$ es el total de casillas. Decir que el tiempo es $O(m \cdot n)$ solo quiere decir que, en el peor caso, tocamos cada casilla una vez (y ni una casilla se cuenta dos veces gracias al truco de inundarla). Es lo más rápido posible: no puedes contar islas sin mirar el mapa completo al menos una vez.

Consejos para la entrevista (Amazon). Este es un problema clásico de Amazon. Un buen detalle que puedes mencionar: como vamos pintando de agua la tierra ya visitada, no necesitamos una lista aparte para recordar «por dónde ya pasé». Si el entrevistador te pide *no* modificar el mapa original, entonces sí necesitas esa lista de visitados (o resolverlo con una cola, técnica llamada BFS). No olvides los casos raros: un mapa vacío, un mapa todo de agua (0 islas) o un mapa todo de tierra (1 sola isla).

994. Rotting Oranges

(M)

Código fuente: [src/01-grafos/994-rotting-oranges.cpp](#)▷ Ver animación · [local](#) | [web](#)

Tenemos una cuadrícula donde 0 es una celda vacía, 1 es una naranja fresca y 2 una naranja podrida. Cada minuto, toda naranja fresca adyacente (arriba, abajo, izquierda, derecha) a una podrida se pudre. Queremos el número de minutos hasta que no quede ninguna fresca, o -1 si alguna nunca se pudre.

La idea, con un ejemplo de la vida real. Imagina un cajón de naranjas. Algunas ya están podridas y cada minuto contagian a las naranjas buenas que tienen pegadas (arriba, abajo, izquierda o derecha). No se contagia solo una: *todas* las podridas contagian a la vez, como cuando echas varias gotas de tinta en agua y todas se van esparciendo al mismo tiempo. Cada minuto la mancha crece un poquito hacia afuera. Queremos saber cuántos minutos pasan hasta que ya no queda ninguna naranja buena.

Para simularlo usamos una técnica llamada *Breadth-First Search* (BFS, o búsqueda en anchura), que significa explorar por capas, como los círculos que se forman cuando tiras una piedra al agua: primero el círculo más cercano, luego el siguiente, y así. Como partimos de *muchas* naranjas podridas a la vez, se le llama *BFS multi-fuente* (varios puntos de partida). Cada capa nueva de contagio es un minuto que pasa.

	2	1	1		2	2	2	
minuto 0	1	1	0	→	minuto 4	2	2	0
	0	1	1			0	2	2

Cómo lo resuelve el código, paso a paso. Usamos una *cola*: una fila de espera como la del supermercado, donde el primero que entra es el primero que sale. Ahí guardamos las casillas que ya están podridas.

1. Recorremos toda la cuadrícula. Metemos en la cola todas las naranjas podridas del inicio y, de paso, contamos cuántas naranjas buenas hay.
2. Procesamos la cola por capas. Una capa es todo lo que había en la cola en ese instante: son las naranjas que se pudrieron en el minuto anterior. Para cada una miramos sus cuatro vecinas; si alguna es una naranja buena, la pudrimos, restamos 1 al conteo de buenas y la metemos a la cola para que contagie en el siguiente minuto.
3. Cada vez que terminamos una capa y todavía había naranjas que pudrir, sumamos 1 al reloj de minutos.
4. Al final, si el conteo de naranjas buenas llegó a cero, devolvemos los minutos. Si quedó alguna naranja buena que nunca se pudo contagiar (estaba aislada), devolvemos -1 porque es imposible.

```

1 //
  =====
2 // 994. Rotting Oranges (Medium)
```

```

3 // -----
4 // Cuadrícula con valores: 0 = vacío, 1 = naranja fresca, 2 = naranja podrida.
5 // Cada minuto, toda naranja fresca adyacente (4 direcciones) a una podrida
6 // se pudre. Devuelve los minutos hasta que ninguna naranja quede fresca, o
7 // -1 si es imposible (queda alguna fresca aislada).
8 //
9 // Ejemplo:
10 //     minuto 0           minuto 1           minuto 2
11 //     2 1 1           2 2 1           2 2 2
12 //     1 1 0   =>     2 1 0   =>     2 2 0
13 //     0 1 1           0 1 1           0 2 1   ... hasta minuto 4
14 //
15 // Diagrama del BFS multi-fuente (todas las podridas empiezan en la cola):
16 //
17 //     cola inicial: [todas las (r,c) con valor 2]
18 //     por cada nivel (minuto): sacamos la capa actual y pudrimos vecinas
19 //     frescas, que entran al siguiente nivel. El número de niveles-1 = minutos.
20 //
21 // Idea: BFS por niveles desde TODAS las podridas a la vez (multi-fuente).
22 // Contamos cuántas frescas hay; cada vez que pudrimos una, la restamos.
23 // Si al final quedan frescas => -1. Si no había frescas al inicio => 0.
24 //
25 // Complejidad: tiempo  $O(m*n)$ , espacio  $O(m*n)$  por la cola.
26 // =====
27 #include <vector>
28 #include <queue>
29 #include <cassert>
30 #include <iostream>
31 using namespace std;
32
33 class Solution {
34 public:
35     int orangesRotting(vector<vector<int>>& grid) {
36         int m = grid.size(), n = grid[0].size();
37         queue<pair<int,int>> cola;    // naranjas podridas (fuentes del BFS)
38         int frescas = 0;
39         for (int r = 0; r < m; ++r)
40             for (int c = 0; c < n; ++c) {
41                 if (grid[r][c] == 2) cola.push({r, c});
42                 else if (grid[r][c] == 1) ++frescas;
43             }
44
45         if (frescas == 0) return 0;    // nada que pudrir
46
47         int minutos = 0;
48         int dr[] = {-1, 1, 0, 0};
49         int dc[] = {0, 0, -1, 1};
50         while (!cola.empty() && frescas > 0) {
51             int capa = cola.size();    // naranjas podridas en este minuto
52             for (int i = 0; i < capa; ++i) {

```

```

53         auto [r, c] = cola.front(); cola.pop();
54         for (int d = 0; d < 4; ++d) {
55             int nr = r + dr[d], nc = c + dc[d];
56             if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
57             if (grid[nr][nc] != 1) continue; // vacío o ya podrida
58             grid[nr][nc] = 2; // se pudre
59             --frescas;
60             cola.push({nr, nc});
61         }
62     }
63     ++minutos; // pasó un minuto completo
64 }
65 return frescas == 0 ? minutos : -1;
66 }
67 };
68
69 int main() {
70     Solution s;
71
72     vector<vector<int>> g1 = {{2,1,1},{1,1,0},{0,1,1}};
73     assert(s.orangesRotting(g1) == 4);
74
75     // Imposible: la naranja de abajo-izquierda nunca se pudre.
76     vector<vector<int>> g2 = {{2,1,1},{0,1,1},{1,0,1}};
77     assert(s.orangesRotting(g2) == -1);
78
79     // Sin naranjas frescas => 0 minutos.
80     vector<vector<int>> g3 = {{0,2}};
81     assert(s.orangesRotting(g3) == 0);
82
83     // Todo vacío => 0.
84     vector<vector<int>> g4 = {{0,0},{0,0}};
85     assert(s.orangesRotting(g4) == 0);
86
87     cout << "994 Rotting Oranges: todas las pruebas pasaron.\n";
88     return 0;
89 }

```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

¿Qué significa esa complejidad? Aquí m es el número de filas y n el de columnas, así que $m \cdot n$ es el total de casillas del cajón. Decir que el tiempo es $O(m \cdot n)$ solo significa que, en el peor caso, cada naranja entra y sale de la cola una sola vez: tocamos cada casilla una vez. Es lo más rápido posible, porque hay que revisar todo el cajón al menos una vez.

Consejos para la entrevista (Amazon). El error típico es contar un minuto de más: pasa cuando sumas al reloj aunque la última capa no haya podrido ninguna naranja nueva. El truco es aumentar los minutos solo mientras todavía queden naranjas buenas por pudrir. No olvides los casos raros: un cajón sin ninguna naranja buena (respuesta 0, no hay que esperar nada), una naranja buena solita sin vecinas podridas (respuesta -1 , jamás se contagia) y un cajón todo vacío (respuesta 0).

695. Max Area of Island

(M)

Código fuente: `src/01-grafos/695-max-area-of-island.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de 0 (agua) y 1 (tierra). Una *isla* es un grupo de tierras conectadas en horizontal o vertical, y su *área* es cuántas celdas de tierra la forman. Queremos el área de la isla más grande (0 si no hay tierra).

La idea, con un ejemplo de la vida real. Es el mismo mapa visto desde un avión que en *Number of Islands*, pero ahora no nos importa cuántas islas hay, sino cuál es la más grande. El *área* de una isla es simplemente cuántas casillas de tierra la forman.

Vamos leyendo la cuadrícula casilla por casilla, como un libro. Cuando pisamos un pedazo de tierra nuevo, recorremos toda su isla contando 1 por cada casilla de tierra que la forma. Igual que antes, en cuanto pisamos una casilla la «inundamos» (la pintamos de agua) para no contarla dos veces. Cuando terminamos de medir una isla, comparamos su tamaño con el récord que llevamos y nos quedamos con el mayor.

```

0  0  1  0  0
0  0  1  1  0
0  1  1  0  0
0  0  0  0  1

```

La isla central conecta **5** celdas; la celda suelta de abajo a la derecha es otra isla de área 1. La respuesta es **5**.

Cómo lo resuelve el código, paso a paso. Recorremos todas las casillas con dos ciclos (uno para filas, otro para columnas). Cuando encontramos tierra, llamamos a una ayudante que mide la isla completa.

1. Si la casilla se sale de la cuadrícula o es agua, la ayudante devuelve 0: no aporta nada.
2. Si es tierra, la inunda (la pinta de agua para no recontarla) y cuenta 1 por ella misma.
3. Luego se llama a sí misma sobre sus cuatro vecinas (arriba, abajo, izquierda, derecha) y suma todo lo que ellas le regresen. Que una función se llame a sí misma se llama *recursión*: es como una ficha de dominó que tira a sus vecinas, y esas a las suyas, hasta que ya no queda tierra que tocar. Este recorrido que avanza «hacia el fondo» por los vecinos se conoce como *DFS*.
4. El total que devuelve es el área de esa isla. Lo comparamos con el récord y guardamos el mayor.

```

1 //
  =====
2 // 695. Max Area of Island (Medium)
3 //
  -----
4 // Cuadrícula de 0 (agua) y 1 (tierra). Una isla es un grupo de tierras
5 // conectadas en horizontal o vertical (no diagonal). El "área" de una isla

```

```

6 // es cuántas celdas de tierra la forman. Devuelve el área de la isla más
7 // grande; si no hay tierra, devuelve 0.
8 //
9 // Ejemplo:
10 // grid =
11 //    0 0 1 0 0      La isla más grande tiene área 5
12 //    0 0 1 1 0      (el bloque conectado del centro).
13 //    0 1 1 0 0      La celda suelta de abajo-derecha es otra isla (área
14 //    1).
15 //    0 0 0 0 1
16 // Diagrama del DFS que mide el área desde una celda de tierra:
17 //
18 //      (r-1,c)
19 //      |
20 // (r,c-1)-(r,c)-(r,c+1)  Cada tierra visitada aporta 1 al área y se
21 //      |                  hunde (marca como agua) para no recontarla.
22 //      (r+1,c)           El área de la isla = 1 + suma de las 4 vecinas.
23 //
24 // Idea: es el patrón de "Number of Islands", pero en vez de contar islas,
25 // el DFS devuelve el TAMAÑO de la isla que explora. Guardamos el máximo.
26 //
27 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita una vez),
28 //                espacio  $O(m*n)$  peor caso por la pila de recursión.
29 //
=====

30 #include <vector>
31 #include <cassert>
32 #include <algorithm>
33 #include <iostream>
34 using namespace std;
35
36 class Solution {
37 public:
38     int maxAreaOfIsland(vector<vector<int>>& grid) {
39         if (grid.empty() || grid[0].empty()) return 0;
40         int m = grid.size(), n = grid[0].size();
41         int maximo = 0;
42         for (int r = 0; r < m; ++r)
43             for (int c = 0; c < n; ++c)
44                 if (grid[r][c] == 1) // tierra sin visitar
45                     maximo = max(maximo, area(grid, r, c, m, n));
46         return maximo;
47     }
48
49 private:
50     // Devuelve el área de la isla conectada a (r,c), hundiéndola a su paso.
51     int area(vector<vector<int>>& grid, int r, int c, int m, int n) {
52         if (r < 0 || r >= m || c < 0 || c >= n) return 0; // fuera de límites
53         if (grid[r][c] != 1) return 0; // agua o visitada
54         grid[r][c] = 0; // hundir esta
55             celda
56         return 1

```

```

56         + area(grid, r - 1, c, m, n)    // arriba
57         + area(grid, r + 1, c, m, n)    // abajo
58         + area(grid, r, c - 1, m, n)    // izquierda
59         + area(grid, r, c + 1, m, n);    // derecha
60     }
61 };
62
63 int main() {
64     Solution s;
65
66     vector<vector<int>> g1 = {
67         {0,0,1,0,0},
68         {0,0,1,1,0},
69         {0,1,1,0,0},
70         {0,0,0,0,1},
71     };
72     assert(s.maxAreaOfIsland(g1) == 5);
73
74     // Toda agua => 0.
75     vector<vector<int>> g2 = {{0,0,0},{0,0,0}};
76     assert(s.maxAreaOfIsland(g2) == 0);
77
78     // Una sola celda de tierra => 1.
79     vector<vector<int>> g3 = {{0,1,0}};
80     assert(s.maxAreaOfIsland(g3) == 1);
81
82     // Toda tierra => área = m*n.
83     vector<vector<int>> g4 = {{1,1},{1,1}};
84     assert(s.maxAreaOfIsland(g4) == 4);
85
86     cout << "695 Max Area of Island: todas las pruebas pasaron.\n";
87     return 0;
88 }

```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

¿Qué significa esa complejidad? Aquí m es el número de filas y n el de columnas, así que $m \cdot n$ es el total de casillas. Que el tiempo sea $O(m \cdot n)$ quiere decir que, en el peor caso, tocamos cada casilla una sola vez (gracias al truco de inundarla, ninguna se cuenta dos veces). No se puede ir más rápido: hay que mirar el mapa completo al menos una vez.

Consejos para la entrevista (Amazon). Si te acaban de preguntar *Number of Islands*, este es la continuación natural: es el mismo DFS, pero en vez de «contar islas» ahora «mide cada isla». Un buen detalle que puedes mencionar: como vamos pintando de agua la tierra ya visitada, no necesitamos una lista aparte de casillas visitadas. Si el entrevistador te pide *no* modificar el mapa original, entonces sí necesitas esa lista (o resolverlo con una cola, técnica llamada *BFS*). El récord del área máxima se lleva en una sola variable que vas actualizando.

542. 01 Matrix

(M)

Código fuente: [src/01-grafos/542-01-matrix.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una matriz de 0 y 1. Para cada celda debemos calcular su distancia (en pasos horizontales o verticales) al 0 más cercano. La distancia entre celdas adyacentes es 1.

La idea, con un ejemplo de la vida real. Para cada casilla queremos saber a cuántos pasos está el 0 más cercano, moviéndonos solo en horizontal o vertical (un paso entre casillas pegadas vale 1).

Lo primero que se te ocurre es: «para cada 1, salgo a buscar el cero más cercano». Pero eso es muy lento, porque repites la búsqueda una y otra vez. El truco es mirarlo al revés: en lugar de que cada 1 salga a buscar, hacemos que *todos* los ceros suelten una onda al mismo tiempo, como cuando tiras muchas piedritas a un charco a la vez. Cada minuto la onda avanza un paso. La *primera* onda que toca a un 1 viene, por fuerza, de su cero más cercano, y el minuto en que lo toca es justo su distancia. Esta técnica de soltar ondas por capas desde muchos puntos de partida se llama *BFS multi-fuente* (*BFS* es explorar por capas; *multi-fuente* es que arrancamos desde varios lugares a la vez).

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{array} \longrightarrow \begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 2 & 1 \end{array}$$

Los ceros valen 0; el 1 del centro tiene un cero pegado (distancia 1); la celda de en medio de la última fila está a 2 pasos.

Cómo lo resuelve el código, paso a paso. Usamos una *cola*: una fila de espera donde el primero que entra es el primero que sale.

1. Recorremos la matriz. A cada casilla con 0 le ponemos distancia 0 y la metemos a la cola (son los puntos de donde salen las ondas). A cada 1 le ponemos distancia «infinito», que significa «todavía no sé, muy lejos».
2. Procesamos la cola. Sacamos una casilla y miramos sus cuatro vecinas. Si a una vecina le podemos dar una distancia mejor (la distancia de la casilla actual más 1), se la ponemos y la metemos a la cola para que siga propagando la onda.
3. Como cada casilla mejora su distancia una sola vez y luego ya no se vuelve a tocar, el recorrido es rápido y ordenado.

```

1 //
  =====
2 // 542. 01 Matrix (Medium)
3 //
  -----
4 // Dada una matriz de 0 y 1, devuelve una matriz del mismo tamaño donde cada
5 // celda contiene la distancia (número de pasos en 4 direcciones) al 0 más
6 // cercano. La distancia entre celdas adyacentes es 1.
```

```

7 //
8 // Ejemplo:
9 //   entrada      salida
10 //   0 0 0        0 0 0
11 //   0 1 0        0 1 0
12 //   1 1 1        1 2 1
13 //
14 // Diagrama del BFS multi-fuente (todos los 0 arrancan en la cola a la vez):
15 //
16 //   cola inicial: [todas las celdas con valor 0]   dist = 0
17 //   nivel 1: vecinas de los 0 que aún no tienen distancia -> dist = 1
18 //   nivel 2: vecinas de esas -> dist = 2 ...
19 //
20 //   La "ola" se expande desde TODOS los ceros simultáneamente, así la primera
21 //   vez que una celda 1 es alcanzada, es por su cero más cercano.
22 //
23 // Idea (truco clave): NO hacer un BFS por cada 1 (sería  $O((m*n)^2)$ ). En vez
24 // de
25 // eso, hacer UN BFS multi-fuente partiendo de todos los ceros. Inicializamos
26 // dist=0 en los ceros e infinito en los unos; la onda del BFS asigna la
27 // distancia mínima a cada uno automáticamente.
28 //
29 // Complejidad: tiempo  $O(m*n)$  (cada celda entra a la cola una vez),
30 // espacio  $O(m*n)$  por la cola y la matriz de distancias.
31 //
32 =====
33
34 #include <vector>
35 #include <queue>
36 #include <cassert>
37 #include <climits>
38 #include <iostream>
39 using namespace std;
40
41 class Solution {
42 public:
43     vector<vector<int>>> updateMatrix(vector<vector<int>>>& mat) {
44         int m = mat.size(), n = mat[0].size();
45         vector<vector<int>>> dist(m, vector<int>(n, INT_MAX));
46         queue<pair<int,int>> cola;
47
48         // Fuentes del BFS: todos los ceros con distancia 0.
49         for (int r = 0; r < m; ++r)
50             for (int c = 0; c < n; ++c)
51                 if (mat[r][c] == 0) {
52                     dist[r][c] = 0;
53                     cola.push({r, c});
54                 }
55
56         int dr[] = {-1, 1, 0, 0};
57         int dc[] = {0, 0, -1, 1};
58         while (!cola.empty()) {
59             auto [r, c] = cola.front(); cola.pop();
60             for (int d = 0; d < 4; ++d) {

```



```

58         int nr = r + dr[d], nc = c + dc[d];
59         if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
60         // Solo mejora si encontramos un camino más corto (aún sin
           fijar).
61         if (dist[nr][nc] > dist[r][c] + 1) {
62             dist[nr][nc] = dist[r][c] + 1;
63             cola.push({nr, nc});
64         }
65     }
66 }
67 return dist;
68 }
69 };
70
71 int main() {
72     Solution s;
73
74     vector<vector<int>> g1 = {{0,0,0},{0,1,0},{1,1,1}};
75     vector<vector<int>> e1 = {{0,0,0},{0,1,0},{1,2,1}};
76     assert(s.updateMatrix(g1) == e1);
77
78     vector<vector<int>> g2 = {{0,0,0},{0,1,0},{0,0,0}};
79     vector<vector<int>> e2 = {{0,0,0},{0,1,0},{0,0,0}};
80     assert(s.updateMatrix(g2) == e2);
81
82     // Fila con un solo cero al inicio.
83     vector<vector<int>> g3 = {{0,1,1,1}};
84     vector<vector<int>> e3 = {{0,1,2,3}};
85     assert(s.updateMatrix(g3) == e3);
86
87     cout << "542 01 Matrix: todas las pruebas pasaron.\n";
88     return 0;
89 }

```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

¿Qué significa esa complejidad? Aquí m es el número de filas y n el de columnas, así que $m \cdot n$ es el total de casillas. Que el tiempo sea $O(m \cdot n)$ significa que cada casilla entra y sale de la cola una sola vez: tocamos cada una una vez. Comparado con la idea lenta de buscar el cero por separado para cada 1, esto es muchísimo más rápido.

Consejos para la entrevista (Amazon). La frase mágica que quieren oír es *BFS multi-fuente*: darte cuenta de que soltar todas las ondas a la vez convierte un problema que parecía lentísimo en uno rápido. Como alternativa puedes mencionar la *programación dinámica* (resolver algo grande a partir de resultados pequeños ya calculados) con dos pasadas: una de arriba-izquierda hacia abajo y otra de abajo-derecha hacia arriba. También es rápida y no necesita cola.

1091. Shortest Path in Binary Matrix

(M)

Código fuente: [src/01-grafos/1091-shortest-path-binary-matrix.cpp](#)▷ Ver animación · [local](#) | [web](#)

En una cuadrícula $n \times n$ de 0 y 1, un *camino claro* va de la esquina $(0, 0)$ a la esquina $(n-1, n-1)$ pisando solo celdas con 0 y moviéndose en las 8 direcciones (incluidas las diagonales). Su longitud es el número de celdas visitadas. Devolvemos la longitud del camino más corto, o -1 si no existe.

La idea, con un ejemplo de la vida real. Queremos ir de una esquina de la cuadrícula a la esquina opuesta pisando solo casillas libres (0), y usando el camino más corto posible. Aquí sí podemos movernos en diagonal, así que cada casilla tiene hasta 8 vecinas: es el movimiento del rey en el ajedrez, que avanza a cualquiera de las casillas que lo rodean.

Para encontrar el camino más corto usamos *BFS*: soltamos una onda desde la casilla de salida, como los círculos que hace una piedra al caer al agua. La onda crece de un paso en un paso y toca antes las casillas cercanas que las lejanas. Por eso, la *primera* vez que la onda toca la casilla de llegada, lo hace por el camino más corto: no hay forma de llegar antes. Ese número de casillas recorridas es la respuesta.

$$\begin{array}{ccc} (r-1, c-1) & (r-1, c) & (r-1, c+1) \\ (r, c-1) & (r, c) & (r, c+1) \\ (r+1, c-1) & (r+1, c) & (r+1, c+1) \end{array}$$

Las 8 celdas que rodean a (r, c) . La primera vez que la onda toca $(n-1, n-1)$, esa es la distancia mínima en celdas.

Cómo lo resuelve el código, paso a paso. Usamos una *cola*: una fila de espera donde el primero que entra es el primero que sale.

1. Si la esquina de salida o la de llegada están bloqueadas (valen 1), devolvemos -1 de inmediato: es imposible.
2. Metemos la esquina de salida a la cola con longitud 1 (ella misma ya cuenta como una casilla del camino).
3. Procesamos la cola por capas. Cada capa es un paso más de la onda. Sacamos una casilla y miramos sus 8 vecinas; si una vecina está libre y no la hemos visitado, le anotamos la longitud del camino (la actual más 1) y la metemos a la cola.
4. Para no volver a pisar una casilla ya visitada, la marcamos cambiándola a 1 en cuanto la usamos. En cuanto sacamos la esquina de llegada de la cola, devolvemos su longitud.
5. Si vaciamos la cola sin haber llegado nunca, devolvemos -1 .

```
1 //
2 // 1091. Shortest Path in Binary Matrix (Medium)
3 //
```

```

4 // Cuadrícula  $n \times n$  de 0 y 1. Un "camino claro" va de la esquina superior
5 // izquierda (0,0) a la inferior derecha (n-1,n-1) pisando solo celdas con 0,
6 // moviéndose en las 8 direcciones (incluye diagonales). La longitud del
7 // camino es el NÚMERO DE CELDAS visitadas. Devuelve la longitud del camino
8 // más corto, o -1 si no existe.
9 //
10 // Ejemplo:
11 //   grid =                               Camino más corto: (0,0)->(1,1)->(2,2)
12 //     0 0 0                               longitud 4 ... espera, n=3 aquí da 4 celdas
13 //     1 1 0                               si vamos 0,0 -> 0,1 -> 1,2 -> 2,2 (por bordes)
14 //     1 1 0
15 //
16 // Diagrama de las 8 vecinas (movimiento tipo rey de ajedrez):
17 //
18 //   (r-1,c-1) (r-1,c) (r-1,c+1)
19 //   (r ,c-1)  (r,c)  (r ,c+1)      BFS: la primera vez que alcanzamos
20 //   (r+1,c-1) (r+1,c) (r+1,c+1)    (n-1,n-1) es por el camino más corto.
21 //
22 // Idea: BFS clásico de camino más corto en grafo no ponderado. Cada celda 0
23 // es un nodo; hay arista entre celdas 0 adyacentes en las 8 direcciones. BFS
24 // desde (0,0) garantiza que la primera vez que tocamos el destino la
25 // distancia
26 // (en celdas) es mínima. Marcamos visitadas mutando la celda a 1.
27 //
28 // Complejidad: tiempo  $O(n^2)$  (cada celda entra a la cola una vez),
29 // espacio  $O(n^2)$  por la cola.
30 //
31 =====
32
33 #include <vector>
34 #include <queue>
35 #include <cassert>
36 #include <iostream>
37 using namespace std;
38
39 class Solution {
40 public:
41     int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
42         int n = grid.size();
43         if (grid[0][0] != 0 || grid[n-1][n-1] != 0) return -1; // extremo
44             bloqueado
45
46         queue<pair<int,int>> cola;
47         cola.push({0, 0});
48         grid[0][0] = 1; // marcar visitada
49         int longitud = 1; // (0,0) ya cuenta como 1 celda
50
51         int dr[] = {-1,-1,-1, 0, 0, 1, 1, 1};
52         int dc[] = {-1, 0, 1,-1, 1,-1, 0, 1};
53         while (!cola.empty()) {
54             int capa = cola.size(); // celdas a distancia actual
55             for (int i = 0; i < capa; ++i) {
56                 auto [r, c] = cola.front(); cola.pop();

```

```

53         if (r == n-1 && c == n-1) return longitud; // llegamos
54         for (int d = 0; d < 8; ++d) {
55             int nr = r + dr[d], nc = c + dc[d];
56             if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
57             if (grid[nr][nc] != 0) continue; // muro o ya visitada
58             grid[nr][nc] = 1; // marcar visitada
59             cola.push({nr, nc});
60         }
61     }
62     ++longitud; // avanzamos una capa (una celda más de camino)
63 }
64 return -1; // nunca alcanzamos el destino
65 }
66 };
67
68 int main() {
69     Solution s;
70
71     vector<vector<int>> g1 = {{0,1},{1,0}};
72     assert(s.shortestPathBinaryMatrix(g1) == 2);
73
74     vector<vector<int>> g2 = {{0,0,0},{1,1,0},{1,1,0}};
75     assert(s.shortestPathBinaryMatrix(g2) == 4);
76
77     // Origen bloqueado => -1.
78     vector<vector<int>> g3 = {{1,0},{0,0}};
79     assert(s.shortestPathBinaryMatrix(g3) == -1);
80
81     // Una sola celda libre => longitud 1.
82     vector<vector<int>> g4 = {{0}};
83     assert(s.shortestPathBinaryMatrix(g4) == 1);
84
85     // Destino inalcanzable => -1.
86     vector<vector<int>> g5 = {{0,1,0},{1,1,0},{0,0,0}};
87     assert(s.shortestPathBinaryMatrix(g5) == -1);
88
89     cout << "1091 Shortest Path in Binary Matrix: todas las pruebas pasaron.\n";
90     return 0;
91 }

```

Complejidad: tiempo $O(n^2)$, espacio $O(n^2)$.

¿Qué significa esa complejidad? La cuadrícula es de $n \times n$, así que tiene n^2 casillas en total. Que el tiempo sea $O(n^2)$ significa que, en el peor caso, cada casilla entra y sale de la cola una sola vez: revisamos cada una una vez. No se puede menos, porque tal vez haya que recorrer todo el cuadrícula para encontrar la salida.

Consejos para la entrevista (Amazon). Explica por qué usamos BFS y no DFS: el DFS encuentra *algún* camino, pero no garantiza que sea el más corto; el BFS sí, porque avanza por capas parejas. Cuida los casos raros: que la salida o la llegada estén bloqueadas (respuesta -1 de

inmediato) y una cuadrícula de una sola casilla con un 0 (respuesta 1). Si además pusieran «costos» distintos por casilla, el siguiente paso sería usar el algoritmo de Dijkstra o el 0-1 BFS.

417. Pacific Atlantic Water Flow

(M)

Código fuente: `src/01-grafos/417-pacific-atlantic.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una matriz `heights` de tamaño $m \times n$ con la altura de cada celda. El océano Pacífico toca el borde superior y el borde izquierdo de la matriz; el océano Atlántico toca el borde inferior y el borde derecho. El agua fluye de una celda a una vecina (arriba, abajo, izquierda, derecha) si la altura de la vecina es **menor o igual** a la de la celda actual (el agua descende o se mantiene, nunca sube). Queremos las coordenadas de las celdas desde las que el agua puede llegar a **ambos** océanos.

La idea, con un ejemplo de la vida real. Imagina que cada casilla es una montañita con cierta altura, y que el agua de lluvia solo puede escurrir hacia una casilla vecina si esa vecina está igual de alta o más baja (el agua baja o se queda, nunca sube). Un océano toca el borde de arriba y el de la izquierda; el otro océano toca el borde de abajo y el de la derecha. Queremos las casillas desde las que el agua puede llegar a *los dos* océanos.

Lo lento sería, para cada casilla, dejar caer una gota y seguirla a ver a qué océano llega. El truco es pensarlo al revés, como si rebobináramos un video: en lugar de salir de cada casilla, nos paramos en la orilla del océano y subimos «río arriba». Desde el borde del mar, avanzamos hacia adentro pero solo hacia vecinas que estén *igual de altas o más altas*, porque si en la vida real el agua bajaba de esa vecina hacia aquí, entonces desde aquí, al revés, podemos subir hasta ella. Así, con un solo recorrido por océano, marcamos todas las casillas cuya agua logra llegar a ese mar. La respuesta son las casillas que quedaron marcadas por *ambos* océanos a la vez.

1	2	2	3	5
3	2	3	4	4
2	4	5	3	1
6	7	1	4	5
5	1	1	2	4

En este ejemplo, el resultado es $(0,4), (1,3), (1,4), (2,2), (3,0), (3,1), (4,0)$: desde cada una de estas celdas el agua puede descender hasta tocar tanto el borde superior/izquierdo como el inferior/derecho.

Diagrama. Recorrido al revés desde cada océano, subiendo de altura hacia el centro de la matriz:

Pacífico (fila 0 y columna 0)	Atlántico (fila $m-1$ y columna $n-1$)
P P P P P	A
P	A
P	(subiendo de altura) A
P	A
P	A A A A A A A A

Cómo lo resuelve el código, paso a paso. Usamos dos matrices de marcas del mismo tamaño que el mapa: uno para el océano Pacífico y otro para el Atlántico. Una marca dice «desde aquí el agua sí puede llegar a este mar».

1. Para el Pacífico, empezamos desde todas las casillas de su orilla (la fila de arriba y la columna de la izquierda) y las marcamos.

2. Desde cada una avanzamos hacia adentro con un *DFS* (búsqueda en profundidad: seguir un camino hasta el fondo y luego regresar). El DFS se llama a sí mismo sobre las vecinas; que una función se llame a sí misma se llama *recursión*, como una ficha de dominó que tira a las de al lado. Solo avanzamos hacia una vecina si su altura es igual o mayor (porque vamos «río arriba»), y solo si no la habíamos marcado ya.
3. Hacemos exactamente lo mismo para el Atlántico, empezando desde su orilla (la fila de abajo y la columna de la derecha).
4. Al final, las casillas que quedaron marcadas en *las dos* matrices son la respuesta: desde ellas el agua llega a ambos océanos.

```

1 //
2 // =====
3 //
4 // 417. Pacific Atlantic Water Flow (Medium)
5 //
6 // -----
7 //
8 // Dada una matriz de alturas `heights` de tamaño  $m \times n$ , el océano Pacífico
9 // toca el borde superior y el borde izquierdo; el océano Atlántico toca el
10 // borde inferior y el borde derecho. El agua fluye de una celda a una vecina
11 // (arriba, abajo, izquierda, derecha) si la altura de la vecina es MENOR O
12 // IGUAL a la altura de la celda actual (el agua desciende o se mantiene).
13 // Debemos devolver las coordenadas de las celdas desde las que el agua puede
14 // llegar a AMBOS océanos.
15 //
16 // Ejemplo (el clásico de LeetCode):
17 // heights =
18 //   1  2  2  3  5
19 //   3  2  3  4  4
20 //   2  4  5  3  1
21 //   6  7  1  4  5
22 //   5  1  1  2  4
23 //
24 // Resultado: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
25 //
26 // Diagrama de la idea (BFS/DFS INVERSO desde cada océano):
27 //
28 // Pacífico toca Atlántico toca
29 // fila 0 y columna 0 fila m-1 y columna n-1
30 // ↓↓
31 // +-----+ +-----+
32 // |P P P P P | | A A |
33 // |P | | A |
34 // |P | (subiendo de | A |
35 // |P | altura hacia | A |
36 // |P | adentro) |A A A A A A A |
37 // +-----+ +-----+
38 //
39 // Idea: hacer BFS/DFS desde cada celda por separado sería  $O((m*n)^2)$ . En su
40 // lugar invertimos el problema: partimos de los bordes de cada océano y
41 // exploramos hacia adentro SUBIENDO de altura (el reverso de "fluir hacia
42 // abajo"). Una celda vecina es alcanzable en el grafo inverso si su altura es

```

```

39 // MAYOR O IGUAL a la de la celda actual (porque en el grafo original el agua
40 // fluiría de esa vecina hacia la celda actual). Así marcamos qué celdas
41 // alcanzan el Pacífico y cuáles alcanzan el Atlántico; la respuesta es la
42 // intersección de ambos conjuntos.
43 //
44 // Complejidad: tiempo  $O(m*n)$  (cada celda se visita a lo más una vez por
45 // océano), espacio  $O(m*n)$  por las matrices de visitados y la
46 // pila de recursión.
47 //
=====

48 #include <vector>
49 #include <algorithm>
50 #include <cassert>
51 #include <iostream>
52 using namespace std;
53
54 class Solution {
55 public:
56     vector<vector<int>>> pacificAtlantic(vector<vector<int>>>& heights) {
57         vector<vector<int>>> resultado;
58         if (heights.empty() || heights[0].empty()) return resultado;
59         int m = heights.size(), n = heights[0].size();
60
61         vector<vector<bool>>> pacifico(m, vector<bool>(n, false));
62         vector<vector<bool>>> atlantico(m, vector<bool>(n, false));
63
64         // Bordes del Pacífico: fila 0 y columna 0.
65         // Bordes del Atlántico: fila m-1 y columna n-1.
66         for (int c = 0; c < n; ++c) {
67             dfs(heights, pacifico, 0, c, m, n);
68             dfs(heights, atlantico, m - 1, c, m, n);
69         }
70         for (int r = 0; r < m; ++r) {
71             dfs(heights, pacifico, r, 0, m, n);
72             dfs(heights, atlantico, r, n - 1, m, n);
73         }
74
75         for (int r = 0; r < m; ++r)
76             for (int c = 0; c < n; ++c)
77                 if (pacifico[r][c] && atlantico[r][c])
78                     resultado.push_back({r, c});
79
80         return resultado;
81     }
82
83 private:
84     // DFS inverso: desde (r,c) avanza hacia una vecina sólo si su altura es
85     // mayor o igual (en el grafo original el agua fluiría de esa vecina hacia
86     // aquí, ya que fluye de mayor/igual altura hacia menor/igual altura).
87     void dfs(vector<vector<int>>>& heights, vector<vector<bool>>>& visitado,
88             int r, int c, int m, int n) {
89         visitado[r][c] = true;
90         int dr[] = {-1, 1, 0, 0};

```



```

91     int dc[] = {0, 0, -1, 1};
92     for (int d = 0; d < 4; ++d) {
93         int nr = r + dr[d], nc = c + dc[d];
94         if (nr < 0 || nr >= m || nc < 0 || nc >= n) continue;
95         if (visitado[nr][nc]) continue;
96         if (heights[nr][nc] < heights[r][c]) continue; // agua no podría
97             subir
98         dfs(heights, visitado, nr, nc, m, n);
99     }
100 };
101
102 // Compara dos listas de coordenadas sin importar el orden.
103 bool mismasCeldas(vector<vector<int>> a, vector<vector<int>> b) {
104     sort(a.begin(), a.end());
105     sort(b.begin(), b.end());
106     return a == b;
107 }
108
109 int main() {
110     Solution s;
111
112     // Caso clásico de LeetCode.
113     vector<vector<int>> h1 = {
114         {1, 2, 2, 3, 5},
115         {3, 2, 3, 4, 4},
116         {2, 4, 5, 3, 1},
117         {6, 7, 1, 4, 5},
118         {5, 1, 1, 2, 4},
119     };
120     vector<vector<int>> esperado1 = {
121         {0, 4}, {1, 3}, {1, 4}, {2, 2}, {3, 0}, {3, 1}, {4, 0}
122     };
123     assert(mismasCeldas(s.pacificAtlantic(h1), esperado1));
124
125     // Caso borde: matriz 1x1. La única celda toca los cuatro bordes a la
126     // vez, así que siempre alcanza ambos océanos sin importar su altura.
127     vector<vector<int>> h2 = {{5}};
128     vector<vector<int>> esperado2 = {{0, 0}};
129     assert(mismasCeldas(s.pacificAtlantic(h2), esperado2));
130
131     // Caso borde: una sola fila. Como m=1, esa fila es a la vez la fila
132     // superior (Pacífico) y la inferior (Atlántico): todas las celdas
133     // alcanzan ambos océanos de inmediato, sin importar la altura.
134     vector<vector<int>> h3 = {{1, 2, 3}};
135     vector<vector<int>> esperado3 = {{0, 0}, {0, 1}, {0, 2}};
136     assert(mismasCeldas(s.pacificAtlantic(h3), esperado3));
137
138     // Caso borde: una sola columna. Como n=1, esa columna es a la vez la
139     // columna izquierda (Pacífico) y la derecha (Atlántico): todas las
140     // celdas alcanzan ambos océanos de inmediato.
141     vector<vector<int>> h4 = {{1}, {2}, {3}};
142     vector<vector<int>> esperado4 = {{0, 0}, {1, 0}, {2, 0}};
143     assert(mismasCeldas(s.pacificAtlantic(h4), esperado4));

```

```
144
145 // Meseta plana: toda la matriz tiene la misma altura, así que el agua
146 // fluye libremente entre vecinas y todas las celdas llegan a ambos
147 // océanos.
148 vector<vector<int>> h5 = {
149     {3, 3, 3},
150     {3, 3, 3},
151     {3, 3, 3},
152 };
153 vector<vector<int>> esperado5 = {
154     {0, 0}, {0, 1}, {0, 2},
155     {1, 0}, {1, 1}, {1, 2},
156     {2, 0}, {2, 1}, {2, 2},
157 };
158 assert(mismasCeldas(s.pacificAtlantic(h5), esperado5));
159
160 cout << "417 Pacific Atlantic: todas las pruebas pasaron.\n";
161 return 0;
162 }
```

Complejidad: tiempo $O(m \cdot n)$, espacio $O(m \cdot n)$.

¿Qué significa esa complejidad? Aquí m es el número de filas y n el de columnas, así que $m \cdot n$ es el total de casillas. Que el tiempo sea $O(m \cdot n)$ significa que cada casilla se visita a lo mucho un par de veces (una por cada océano). Comparado con la idea lenta de seguir una gota desde cada casilla, esto es muchísimo más rápido.

Consejos para la entrevista (Amazon). El corazón del truco es darte cuenta de que conviene recorrer al revés: partir de los océanos en vez de partir de cada casilla. Eso baja el costo de algo lentísimo a algo rápido, y es buena idea decirlo en voz alta al entrevistador. No olvides los casos raros: un mapa de una sola casilla (esa casilla toca los cuatro bordes, así que llega a ambos mares), un mapa de una sola fila o columna (esa misma fila o columna es orilla de los dos océanos) y las zonas planas de igual altura (el agua pasa libremente entre casillas de la misma altura, porque la regla es «igual o más bajo»).

79. Word Search

(M)

Código fuente: [src/01-grafos/79-word-search.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una cuadrícula de letras (**board**) y una palabra (**word**). Queremos saber si la palabra se puede formar recorriendo celdas adyacentes en horizontal o vertical (nunca en diagonal), sin usar la misma celda dos veces dentro de un mismo camino.

La idea, con un ejemplo de la vida real. Es como esos juegos de sopa de letras donde tienes que trazar una palabra saltando de una letra a la de al lado. Empiezas parado en una casilla y vas letra por letra: buscas a cuál vecina moverte para encontrar la siguiente letra de la palabra. Solo puedes moverte a casillas pegadas en horizontal o vertical, y no puedes reutilizar una casilla dentro del mismo trazo.

Si te metes por un camino y te atorras (la siguiente letra no aparece en ninguna vecina), regresas sobre tus pasos y pruebas otro camino. A eso de «probar, equivocarse y regresar para intentar otra opción» se le llama *backtracking* (retroceder). Para no pisar dos veces la misma casilla en el mismo intento, la marcamos temporalmente (por ejemplo poniéndole un '#') mientras la usamos, y le devolvemos su letra original en cuanto retrocedemos, para que otros caminos sí puedan usarla.

A	B	C	E
S	F	C	S
A	D	E	E

En esta cuadrícula, "ABCCED" existe (bajando por la columna central de la C y luego a la E y la D), y "SEE" también existe. En cambio "ABCB" no existe: tras usar la primera B, la única forma de volver a una B obligaría a reutilizar una celda ya visitada en ese mismo camino.

Cómo lo resuelve el código, paso a paso. Como no sabemos dónde empieza la palabra, probamos empezar desde cada casilla de la cuadrícula. Desde cada una lanzamos una ayudante que busca el resto de la palabra con *DFS*. Que la ayudante se llame a sí misma para seguir con la siguiente letra se llama *recursión*.

1. Si la casilla se sale de la cuadrícula o su letra no es la que toca, este camino falla: devolvemos «no».
2. Si la letra coincide y era la última de la palabra, ¡lo logramos!
3. Si coincide pero faltan letras, marcamos la casilla con '#' para no repetirla y probamos sus 4 vecinas buscando la siguiente letra.
4. Si ninguna vecina logra completar la palabra, le devolvemos su letra original a la casilla (esto es el *backtracking*) y regresamos a probar otro camino.

En cuanto algún punto de partida logra formar la palabra completa, devolvemos que sí existe.

```

1 //
  =====
2 // 79. Word Search (Medium)
3 //
  -----

```

```

4 // Dada una cuadrícula de letras (board) y una palabra (word), decide si la
5 // palabra existe formando un camino de celdas adyacentes en horizontal o
6 // vertical (no diagonal), sin reutilizar una misma celda dos veces.
7 //
8 // Ejemplo:
9 //   board =
10 //     A B C E           word = "ABCCED"  -> true (A-B-C-C-E-D, bajando y
11 //     S F C S           recorriendo por la columna central)
12 //     A D E E           word = "SEE"      -> true
13 //                               word = "ABCB"  -> false (la segunda B reutilizaría
14 //                               la celda ya usada por la primera)
15 //
16 // Diagrama de la búsqueda DFS con backtracking desde una celda (r,c) que
17 // coincide con word[i]:
18 //
19 //           (r-1,c)
20 //           |
21 //   (r,c-1)-(r,c)-(r,c+1)   Si board[r][c] == word[i], marcamos la celda
22 //           |               como visitada ('#') y probamos las 4 vecinas
23 //           (r+1,c)         buscando word[i+1]. Si ninguna funciona,
24 //                           restauramos la celda (backtrack) antes de
25 //                           volver, para que otros caminos puedan usarla.
26 //
27 // Idea: para cada celda de la cuadrícula, intentamos que sea el punto de
28 // partida (word[0]) y hacemos DFS con backtracking. Al entrar a una celda la
29 // marcamos temporalmente para no reusarla en el mismo camino; al salir (ya
30 // sea que el camino tuvo éxito o fracasó) la restauramos a su valor original.
31 //
32 // Complejidad: tiempo  $O(m*n*4^L)$  donde  $L$  = longitud de word (en el peor caso,
33 // desde cada una de las  $m*n$  celdas se abren hasta 4 ramas por
34 // cada letra de la palabra),
35 // espacio  $O(L)$  por la profundidad de la pila de recursión.
36 //

```

```

37 #include <vector>
38 #include <string>
39 #include <cassert>
40 #include <iostream>
41 using namespace std;
42
43 class Solution {
44 public:
45     bool exist(vector<vector<char>>& board, string word) {
46         if (board.empty() || board[0].empty() || word.empty()) return false;
47         int m = board.size(), n = board[0].size();
48         for (int r = 0; r < m; ++r) {
49             for (int c = 0; c < n; ++c) {
50                 if (buscar(board, word, r, c, 0, m, n)) return true;
51             }
52         }
53         return false;
54     }
55 }

```

```

56 private:
57     // Intenta continuar el camino de word desde word[i] partiendo de (r,c).
58     bool buscar(vector<vector<char>>& board, const string& word,
59               int r, int c, int i, int m, int n) {
60         if (i == (int)word.size()) return true; // ya se emparejó toda la
61         palabra
62         if (r < 0 || r >= m || c < 0 || c >= n) return false; // fuera de lí
63         mites
64         if (board[r][c] != word[i]) return false; // no coincide o ya
65         visitada ('#')
66
67         char original = board[r][c];
68         board[r][c] = '#'; // marcamos la celda como usada en este camino
69
70         bool encontrada =
71             buscar(board, word, r - 1, c, i + 1, m, n) || // arriba
72             buscar(board, word, r + 1, c, i + 1, m, n) || // abajo
73             buscar(board, word, r, c - 1, i + 1, m, n) || // izquierda
74             buscar(board, word, r, c + 1, i + 1, m, n); // derecha
75
76         board[r][c] = original; // backtrack: restauramos antes de volver
77         return encontrada;
78     }
79 };
80
81 int main() {
82     Solution s;
83
84     // Palabra presente: A-B-C-C-E-D es un camino válido (baja por la columna
85     central de la C y luego va a la E y a la D).
86     vector<vector<char>> b1 = {
87         {'A', 'B', 'C', 'E'},
88         {'S', 'F', 'C', 'S'},
89         {'A', 'D', 'E', 'E'},
90     };
91     assert(s.exist(b1, "ABCCED") == true);
92
93     // Palabra presente, camino corto por la columna derecha.
94     vector<vector<char>> b2 = b1;
95     assert(s.exist(b2, "SEE") == true);
96
97     // Palabra ausente: no existe ningún camino que forme "ABCB" porque,
98     tras usar la primera B, la única forma de volver a una B requeriría
99     reutilizar la celda ya visitada.
100     vector<vector<char>> b3 = b1;
101     assert(s.exist(b3, "ABCB") == false);
102
103     // Palabra de una sola letra: basta con que exista esa letra en la
104     cuadrícula (no requiere adyacencia con nada más).
105     vector<vector<char>> b4 = b1;
106     assert(s.exist(b4, "F") == true);
107     assert(s.exist(b4, "Z") == false);
108
109     // Cuadrícula de una sola celda.

```

```
107     vector<vector<char>> b5 = {{'A'}}};
108     assert(s.exist(b5, "A") == true);
109     assert(s.exist(b5, "AA") == false); // reutilizaría la única celda
110
111     cout << "79 Word Search: todas las pruebas pasaron.\n";
112     return 0;
113 }
```

Complejidad: tiempo $O(m \cdot n \cdot 4^L)$, con L la longitud de `word`, espacio $O(L)$ por la recursión.

¿Qué significa esa complejidad? Aquí $m \cdot n$ es el total de casillas (todos los posibles puntos de partida) y L es cuántas letras tiene la palabra. El 4^L aparece porque en cada letra podemos irnos hacia 4 vecinas, y eso se multiplica letra tras letra, como un árbol que se ramifica. El $O(L)$ de memoria es porque la recursión se mete tan hondo como letras tenga la palabra, guardando el camino que lleva.

Consejos para la entrevista (Amazon). Lo más importante es el *backtracking*: marcar la casilla al entrar y **siempre** devolverle su letra al salir, incluso cuando el camino tuvo éxito. Si el entrevistador te pide *no* modificar la cuadrícula, usa una cuadrícula de marcas aparte en lugar del '#'. No olvides los casos raros: una palabra de una sola letra (basta con que esa letra exista), una cuadrícula de una sola casilla y palabras que obligarían a reutilizar una casilla (deben fallar).

207. Course Schedule

(M)

Código fuente: [src/01-grafos/207-course-schedule.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay `numCourses` cursos numerados de 0 a `numCourses - 1`, y una lista `prerequisites` de pares $[a, b]$ que significan “para tomar el curso a primero hay que tomar el curso b ”. Queremos saber si es posible tomar todos los cursos.

La idea, con un ejemplo de la vida real. Piensa en los cursos como en armar una torre de bloques de juguete. Para poner un bloque arriba, primero tiene que estar puesto el bloque de abajo. Cada par $[a, b]$ quiere decir “antes del bloque a hay que poner el bloque b ”. Si dibujas una flecha de b hacia a (b va antes que a) para cada regla, terminas con un dibujo de puntos y flechas. En matemáticas ese dibujo se llama *grafo dirigido*: los puntos son los cursos y las flechas dicen quién va antes que quién.

Ahora, la pregunta “¿puedo tomar todos los cursos?” es lo mismo que preguntar “¿este dibujo tiene un círculo vicioso?”. Un *círculo vicioso* (en matemáticas, un *ciclo*) es cuando el curso A pide al B, el B pide al C, y el C vuelve a pedir al A: nadie puede ser el primero, así que es imposible. Si no hay ningún círculo vicioso, siempre puedes acomodar los cursos en una fila donde cada uno queda después de todos los que necesita. A esa fila ordenada se le llama *orden topológico*.

Con ciclo (imposible):	Sin ciclo (posible):
$0 \rightarrow 1 \rightarrow 0$ (0 necesita 1, 1 necesita 0)	$0 \rightarrow 1 \rightarrow 3, \quad 0 \rightarrow 2 \rightarrow 3$ orden topológico: 0,1,2,3

Una palabra nueva: “grado de entrada”. El *grado de entrada* de un curso es simplemente cuántos prerequisites le faltan todavía por cumplir, es decir, cuántas flechas le llegan apuntando. Un curso con grado de entrada 0 no le debe nada a nadie, así que lo puedes tomar de inmediato. Piénsalo como una lista de tareas: las que no dependen de ninguna otra son las que puedes empezar hoy mismo.

Cómo lo resuelve el código, paso a paso. La técnica se llama *algoritmo de Kahn*, pero la idea es muy sencilla: ir haciendo primero los cursos que ya no necesitan nada. Usamos una *cola*, que es como la fila del súper: el primero que entra es el primero que sale.

1. Anotamos, para cada curso, la lista de cursos que dependen de él (`adj[b]` guarda los cursos que necesitan a b) y su grado de entrada (cuántos prerequisites le faltan).
2. Metemos en la cola todos los cursos que ya tienen grado de entrada 0, porque esos se pueden tomar sin esperar a nadie.
3. Sacamos un curso de la cola, lo contamos como “ya tomado” y a cada curso que dependía de él le restamos 1 prerequisite. Si a alguno se le acaban los prerequisites pendientes (llega a 0), lo metemos a la cola.
4. Repetimos hasta vaciar la cola.

Al final contamos: si logramos tomar todos los cursos, no había ningún círculo vicioso, así que la respuesta es sí. Si sobró algún curso (nunca bajó su grado de entrada a 0 porque estaba atrapado en un círculo), la respuesta es no.

```

1 //
  =====

2 // 207. Course Schedule (Medium)
3 //
  -----

4 // Hay numCourses cursos, numerados de 0 a numCourses-1. La lista
  prerequisites
5 // contiene pares [a, b] que significan "para tomar el curso a, primero hay
6 // que tomar el curso b". Determina si es posible tomar todos los cursos.
7 //
8 // Esto equivale a preguntar si el grafo dirigido de dependencias (b -> a,
9 // "b habilita a a") tiene un ciclo. Si hay un ciclo, esos cursos se necesitan
10 // entre sí circularmente y nunca se puede empezar. Si no hay ciclo, siempre
11 // existe un orden válido para cursarlos: un orden topológico.
12 //
13 // Ejemplo con ciclo (imposible):
14 //   numCourses = 2, prerequisites = [[1,0],[0,1]]
15 //   0 --> 1
16 //   ^-----|   para tomar 1 necesitas 0, y para tomar 0 necesitas 1:
17 //               ciclo, nunca se puede empezar.
18 //
19 // Ejemplo sin ciclo (posible):
20 //   numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
21 //   0 --> 1 --> 3
22 //   0 --> 2 --> 3   orden topológico válido: 0, 1, 2, 3
23 //
24 // Idea (algoritmo de Kahn, BFS con grados de entrada):
25 // - El "grado de entrada" (in-degree) de un curso es cuántos prerequisites
26 //   le faltan por cumplir antes de poder tomarlo.
27 // - Empezamos metiendo en una cola todos los cursos con grado de entrada 0
28 //   (los que no necesitan nada, se pueden tomar de inmediato).
29 // - Sacamos un curso de la cola, lo "tomamos" (lo contamos como procesado) y
30 //   reducimos en 1 el grado de entrada de todos los cursos que dependían de
31 //   él. Si alguno llega a 0, ya cumplió todos sus prerequisites y entra a
32 //   la cola.
33 // - Si al final procesamos exactamente numCourses cursos, no había ciclo y
34 //   existe un orden topológico completo => true. Si sobran cursos sin
35 //   procesar (se quedaron atorados en un ciclo, su grado de entrada nunca
36 //   baja a 0) => false.
37 //
38 // Complejidad: tiempo  $O(V+E)$  ( $V$  cursos,  $E$  prerequisites),
39 //               espacio  $O(V+E)$  por la lista de adyacencia y los grados.
40 //
  =====

41 #include <vector>
42 #include <queue>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46

```



```

47 class Solution {
48 public:
49     bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
50         vector<vector<int>> adj(numCourses);    // adj[b] = cursos que dependen
51         // de b
52         vector<int> gradoEntrada(numCourses, 0);
53
54         for (auto& pre : prerequisites) {
55             int a = pre[0], b = pre[1];    // para tomar a, primero hay que
56             // tomar b
57             adj[b].push_back(a);
58             ++gradoEntrada[a];
59         }
60
61         queue<int> cola;    // cursos listos para tomarse (sin prerequisites
62         // pendientes)
63         for (int curso = 0; curso < numCourses; ++curso)
64             if (gradoEntrada[curso] == 0) cola.push(curso);
65
66         int procesados = 0;
67         while (!cola.empty()) {
68             int curso = cola.front(); cola.pop();
69             ++procesados;    // "tomamos" este curso
70             for (int siguiente : adj[curso]) {
71                 if (--gradoEntrada[siguiente] == 0) cola.push(siguiente);
72             }
73         }
74
75         return procesados == numCourses;    // si falta alguno, había un ciclo
76     };
77 };
78
79 int main() {
80     Solution s;
81
82     // Sin prerequisites: siempre se puede tomar todo.
83     vector<vector<int>> p1 = {};
84     assert(s.canFinish(2, p1) == true);
85
86     // Cadena lineal: 1 necesita 0, 2 necesita 1, 3 necesita 2. Sin ciclos.
87     vector<vector<int>> p2 = {{1,0},{2,1},{3,2}};
88     assert(s.canFinish(4, p2) == true);
89
90     // Ciclo simple: 0 necesita 1 y 1 necesita 0. Imposible.
91     vector<vector<int>> p3 = {{0,1},{1,0}};
92     assert(s.canFinish(2, p3) == false);
93
94     // Varios cursos con un ciclo escondido entre 1, 2 y 3 (0 sí se puede
95     // tomar).
96     vector<vector<int>> p4 = {{1,0},{2,1},{3,2},{1,3}};
97     assert(s.canFinish(4, p4) == false);
98
99     cout << "207 Course Schedule: todas las pruebas pasaron.\n";
100     return 0;

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

¿Qué significa esa complejidad? Aquí V es el número de cursos (los puntos del dibujo) y E el número de reglas de prerequisite (las flechas). Decir que el tiempo es $O(V + E)$ solo quiere decir que miramos cada curso una vez y cada flecha una vez, y ya. No repetimos trabajo, así que es lo más rápido que se puede: para saber si el plan cabe, hay que leer al menos todas las reglas una vez.

Consejos para la entrevista (Amazon). Este problema es la puerta de entrada al patrón de *orden topológico*, que aparece mucho cuando hay que planear tareas que dependen unas de otras (por ejemplo, en qué orden compilar las partes de un programa o instalar programas que necesitan otros programas). Menciona que se puede resolver de dos maneras: contando prerequisites con una cola (el algoritmo de Kahn, el que usamos aquí, que además te sirve si te piden el orden concreto), o con *DFS* (recorrer hacia el fondo por las flechas) pintando los cursos de colores para cachar el círculo vicioso. No olvides los casos raros: sin prerequisites (siempre se puede), un círculo vicioso que atrapa solo a algunos cursos y no a todos, y cursos que tienen muchos prerequisites o de los que dependen muchos otros.

210. Course Schedule II

(M)

Código fuente: `src/01-grafos/210-course-schedule-ii.cpp`▷ Ver animación · [local](#) | [web](#)

Hay que cursar `numCourses` materias numeradas de 0 a `numCourses-1`. Nos dan una lista de prerequisites `prerequisites[i] = [a, b]`, que significa que para tomar el curso `a` primero hay que haber tomado el curso `b`. Queremos devolver *un* orden válido para cursar todas las materias, o un vector vacío si es imposible (existe un ciclo de dependencias).

La idea, con un ejemplo de la vida real. Es la continuación del problema 207 (Course Schedule). Allá solo preguntaban “¿se puede terminar todo?”; aquí, además, hay que entregar la lista con el orden exacto en que tomar las materias. Imagina que preparas un pastel: no puedes decorarlo antes de hornearlo, ni hornearlo antes de batir la masa. Cada paso tiene otros pasos que deben ir antes. La idea sigue siendo la misma que en el 207: un curso solo se puede tomar cuando ya cumpliste todos sus prerequisites, así que vamos “liberando” cursos conforme se van completando los que necesitan. La técnica se llama *algoritmo de Kahn*, y consiste en ir apuntando en una lista los cursos que ya están listos para tomar.



Con `prerequisites = [[1,0],[2,0],[3,1],[3,2]]`: para tomar 1 y 2 hace falta 0; para tomar 3 hacen falta 1 y 2. Un orden válido es `[0, 1, 2, 3]` (o `[0, 2, 1, 3]`): el orden topológico no es único cuando hay ramas independientes.

Cómo lo resuelve el código, paso a paso. El *grado de entrada* de un curso es cuántos prerequisites le faltan por cumplir. Usamos una *cola*, que es como la fila del cine: el primero que llega es el primero en salir.

1. Para cada curso anotamos quiénes dependen de él (`curso -> [dependientes]`) y cuántos prerequisites le faltan (su grado de entrada).
2. Metemos en la cola todos los cursos con grado de entrada 0, porque esos ya se pueden tomar sin esperar a nadie.
3. Sacamos un curso de la cola, lo escribimos en la lista de resultado y a cada curso que dependía de él le restamos 1 prerequisite. Al que llegue a 0, lo metemos a la cola.
4. Repetimos hasta vaciar la cola.

Si al final la lista de resultado tiene los `numCourses` cursos, ese es un orden válido. Si le faltan cursos, es que había un círculo vicioso (un *ciclo*: cursos que se piden entre sí en redondo) y devolvemos una lista vacía.

```

1 //
  =====
2 // 210. Course Schedule II (Medium)
3 //
  -----

```

```

4 // Hay que cursar "numCourses" materias (0 .. numCourses-1). Nos dan una lista
5 // de prerequisites "prerequisites[i] = [a, b]", que significa: para tomar el
6 // curso a hace falta haber tomado antes el curso b. Devuelve UN orden válido
7 // en el que se pueden cursar todas las materias, o un vector vacío si es
8 // imposible (existe un ciclo de dependencias).
9 //
10 // Es la continuación natural del problema 207 (Course Schedule): ahí solo
11 // preguntaban si era posible terminar todos los cursos (true/false); aquí
12 // además hay que construir el orden topológico concreto.
13 //
14 // Ejemplo:
15 //   numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
16 //   Para tomar 1 y 2 hace falta 0; para tomar 3 hacen falta 1 y 2.
17 //
18 // Diagrama del grafo dirigido (a -> b significa "b depende de a"):
19 //
20 //       0
21 //      / \
22 //     1   2
23 //      \ /
24 //       3
25 //
26 // Un orden válido es [0, 1, 2, 3] (o [0, 2, 1, 3]): el orden topológico no
27 // es único cuando hay ramas independientes.
28 //
29 // Idea (algoritmo de Kahn, BFS con grado de entrada):
30 //   1) Construimos la lista de adyacencia curso -> [cursos que dependen de él]
31 //      y calculamos el grado de entrada (in-degree) de cada curso: cuántos
32 //      prerequisites le faltan.
33 //   2) Metemos en una cola todos los cursos con grado de entrada 0 (se pueden
34 //      tomar de inmediato, sin prerequisites pendientes).
35 //   3) Sacamos un curso de la cola, lo agregamos al resultado, y "liberamos"
36 //      a sus dependientes restándoles 1 al grado de entrada; el que llegue a
37 //      0 entra a la cola.
38 //   4) Si al final el resultado no incluye a los "numCourses" cursos, hay un
39 //      ciclo (algunos cursos nunca bajan a grado de entrada 0) => devolvemos
40 //      un vector vacío.
41 //
42 // Complejidad: tiempo  $O(V + E)$  ( $V$  cursos,  $E$  prerequisites),
43 //               espacio  $O(V + E)$  por la lista de adyacencia y la cola.
44 //
45 =====
46
47 #include <vector>
48 #include <queue>
49 #include <cassert>
50 #include <iostream>
51 using namespace std;
52
53 class Solution {
54 public:
55     vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites)
56     {

```

```

54     vector<vector<int>>> adj(numCourses);    // adj[b] = cursos que dependen
        de b
55     vector<int> grado(numCourses, 0);      // grado de entrada de cada
        curso
56
57     for (auto& pr : prerequisites) {
58         int a = pr[0], b = pr[1];    // b es prerrequisito de a
59         adj[b].push_back(a);
60         ++grado[a];
61     }
62
63     queue<int> cola;    // cursos listos para tomar (sin prerrequisitos
        pendientes)
64     for (int curso = 0; curso < numCourses; ++curso)
65         if (grado[curso] == 0) cola.push(curso);
66
67     vector<int> orden;
68     orden.reserve(numCourses);
69
70     while (!cola.empty()) {
71         int actual = cola.front(); cola.pop();
72         orden.push_back(actual);
73         for (int dependiente : adj[actual]) {
74             if (--grado[dependiente] == 0) cola.push(dependiente);
75         }
76     }
77
78     if ((int)orden.size() != numCourses) return {};    // había un ciclo
79     return orden;
80 }
81 };
82
83 // -----
84 // Verifica que "orden" sea un orden topológico válido para "numCourses"
    cursos
85 // con las dependencias dadas en "prerequisites": debe incluir a todos los
86 // cursos exactamente una vez, y cada prerrequisito debe aparecer antes que el
87 // curso que lo necesita. Como el orden topológico no es único, esta función
88 // nos permite comparar sin depender de un vector fijo.
89 bool esOrdenValido(const vector<int>& orden, int numCourses,
90                   const vector<vector<int>>& prerequisites) {
91     if ((int)orden.size() != numCourses) return false;
92
93     vector<int> posicion(numCourses, -1);
94     vector<bool> visto(numCourses, false);
95     for (int i = 0; i < (int)orden.size(); ++i) {
96         int curso = orden[i];
97         if (curso < 0 || curso >= numCourses || visto[curso]) return false;
98         // repetido o fuera de rango
99         visto[curso] = true;
100        posicion[curso] = i;
101    }
102
103    for (auto& pr : prerequisites) {

```

```

103     int a = pr[0], b = pr[1];    // b debe ir antes que a
104     if (posicion[b] >= posicion[a]) return false;
105 }
106 return true;
107 }
108
109 int main() {
110     Solution s;
111
112     // Sin prerequisites: cualquier permutación de 0..numCourses-1 es válida.
113     vector<vector<int>> p1 = {};
114     auto o1 = s.findOrder(2, p1);
115     assert(esOrdenValido(o1, 2, p1));
116
117     // Cadena lineal: 0 <- 1 <- 2 <- 3 (cada curso depende del anterior).
118     // El orden topológico es único: [0, 1, 2, 3].
119     vector<vector<int>> p2 = {{1, 0}, {2, 1}, {3, 2}};
120     auto o2 = s.findOrder(4, p2);
121     vector<int> esperado2 = {0, 1, 2, 3};
122     assert(o2 == esperado2);
123     assert(esOrdenValido(o2, 4, p2));
124
125     // Grafo con ramas independientes (0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3):
126     // hay más de un orden válido, así que solo verificamos con esOrdenValido.
127     vector<vector<int>> p3 = {{1, 0}, {2, 0}, {3, 1}, {3, 2}};
128     auto o3 = s.findOrder(4, p3);
129     assert(esOrdenValido(o3, 4, p3));
130
131     // Ciclo: 0 depende de 1 y 1 depende de 0 => imposible, vector vacío.
132     vector<vector<int>> p4 = {{0, 1}, {1, 0}};
133     auto o4 = s.findOrder(2, p4);
134     assert(o4.empty());
135
136     // Ciclo más grande escondido entre cursos válidos: 0 -> 1 -> 2 -> 1.
137     vector<vector<int>> p5 = {{1, 0}, {2, 1}, {1, 2}};
138     auto o5 = s.findOrder(3, p5);
139     assert(o5.empty());
140
141     cout << "210 Course Schedule II: todas las pruebas pasaron.\n";
142     return 0;
143 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

¿Qué significa esa complejidad? Aquí V es el número de cursos y E el número de reglas de prerequisite. Que el tiempo sea $O(V + E)$ quiere decir que tocamos cada curso una vez y cada regla una vez, sin repetir trabajo. Es lo más rápido posible: no puedes armar el orden sin leer al menos todas las reglas una vez.

Consejos para la entrevista (Amazon). Como casi siempre hay varias formas válidas de ordenar los cursos (a esa fila ordenada se le llama *orden topológico*), en la entrevista aclara que cualquier orden que respete las dependencias es una respuesta correcta, no hay una sola. Si te piden

comprobar tu propia respuesta, basta con revisar que cada prerequisite aparezca antes que el curso que lo necesita. No olvides los casos raros: sin prerequisites (sirve cualquier orden), una cadena en la que cada curso pide al anterior (entonces el orden es único), y un círculo vicioso (respuesta vacía, igual que el “no se puede” del 207).

269. Alien Dictionary

(H)

Código fuente: [src/01-grafos/269-alien-dictionary.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una lista de palabras que ya están ordenadas según las reglas de un alfabeto alienígena que no conocemos. A partir de esas palabras hay que deducir un orden válido de las letras del alfabeto. Si la lista es inconsistente (no existe ningún orden posible), devolvemos "".

La idea, con un ejemplo de la vida real. Es como jugar a adivinar el orden del abecedario de otro planeta usando su diccionario. Sabemos que las palabras del diccionario ya están ordenadas alfabéticamente, aunque no sabemos en qué orden van sus letras. ¿Cómo se compara dos palabras en un diccionario? Se lee letra por letra hasta encontrar la primera que sea distinta; esa primera diferencia decide cuál palabra va antes.

Entonces, cada par de palabras *vecinas* (una y la que le sigue) nos regala como mucho una pista: la primera letra en la que se diferencian nos dice “esta letra va antes que esa otra”. Las demás letras no dicen nada sobre el orden. Con todas esas pistas dibujamos flechas letra $\rightarrow$ letra (un *grafo dirigido*: puntos que son letras y flechas que dicen quién va antes). Después solo hay que acomodar las letras en una fila que respete todas las flechas; a esa fila se le llama *orden topológico*, y esa es la respuesta.

```
wrt wrf (diff. índice 2: t  $\rightarrow$  f)
wrf er (diff. índice 0: w  $\rightarrow$  e)
er ett (diff. índice 1: r  $\rightarrow$  t)
ett rftt (diff. índice 0: e  $\rightarrow$  r)
```

Grafo resultante: $w \rightarrow e \rightarrow r \rightarrow t \rightarrow f$

En este ejemplo las aristas forman una sola cadena, así que el orden topológico es único: "wertf".

Cómo lo resuelve el código, paso a paso. Primero juntamos las pistas, y luego ordenamos las letras. El *grado de entrada* de una letra es cuántas flechas le llegan, es decir, cuántas letras deben ir antes que ella; una letra con grado de entrada 0 no tiene ninguna letra obligada delante.

1. Comparamos cada palabra con la que le sigue y buscamos la primera letra en la que se diferencian: esa es la única pista (la única flecha) que aporta ese par.
2. Con todas las flechas juntas aplicamos el *algoritmo de Kahn*: metemos en una *cola* (la fila del súper: primero en entrar, primero en salir) las letras con grado de entrada 0.
3. Sacamos una letra, la ponemos en la respuesta y a las letras que iban después de ella les quitamos esa flecha; la que se quede con grado de entrada 0 entra a la cola.
4. Repetimos hasta vaciar la cola.

Si al final acomodamos todas las letras, esa fila es el orden del alfabeto. Si sobró alguna letra (quedó atrapada en un círculo vicioso, un *ciclo*, del tipo “a antes que b, b antes que a”), no existe orden posible y devolvemos "".

Ojo con una trampa fácil de olvidar: si dos palabras vecinas empiezan igual pero la primera es *más larga* que la segunda (por ejemplo ["abc", "ab"]), eso rompe el orden del diccionario, porque una palabra corta siempre debe ir antes que la palabra larga que empieza igual. En ese caso devolvemos "" de inmediato, sin siquiera ponernos a ordenar letras.


```
48 #include <string>
49 #include <queue>
50 #include <unordered_map>
51 #include <unordered_set>
52 #include <cassert>
53 #include <iostream>
54 using namespace std;
55
56 class Solution {
57 public:
58     string alienOrder(vector<string>& words) {
59         unordered_map<char, unordered_set<char>> adyacencia; // letra ->
60             letras que van después
61         unordered_map<char, int> gradoEntrada; // cuántas
62             letras deben ir antes
63
64         // Registramos todas las letras que aparecen (con grado de entrada 0
65         // por defecto), para no perder letras aisladas sin restricciones.
66         for (const string& palabra : words)
67             for (char c : palabra) {
68                 adyacencia[c];
69                 gradoEntrada[c] = 0;
70             }
71
72         // Comparamos cada par de palabras adyacentes para deducir aristas.
73         for (size_t i = 0; i + 1 < words.size(); ++i) {
74             const string& w1 = words[i];
75             const string& w2 = words[i + 1];
76             size_t minLen = min(w1.size(), w2.size());
77             bool difieren = false;
78             for (size_t j = 0; j < minLen; ++j) {
79                 if (w1[j] != w2[j]) {
80                     if (!adyacencia[w1[j]].count(w2[j])) {
81                         adyacencia[w1[j]].insert(w2[j]);
82                         ++gradoEntrada[w2[j]];
83                     }
84                     difieren = true;
85                     break;
86                 }
87             }
88
89             // Prefijo inválido: w1 es más larga que w2 pero coinciden en todo
90             // el prefijo común => w2 debería haber ido antes que w1.
91             if (!difieren && w1.size() > w2.size()) return "";
92         }
93
94         // Algoritmo de Kahn: arrancamos con las letras sin dependencias.
95         queue<char> cola;
96         for (auto& [letra, grado] : gradoEntrada)
97             if (grado == 0) cola.push(letra);
98
99         string resultado;
100         while (!cola.empty()) {
101             char actual = cola.front(); cola.pop();
102             resultado += actual;
103         }
104     }
105 }
```

```

100         for (char siguiente : adyacencia[actual]) {
101             if (--gradoEntrada[siguiente] == 0) cola.push(siguiente);
102         }
103     }
104
105     // Si no procesamos todas las letras, había un ciclo => imposible.
106     if (resultado.size() != gradoEntrada.size()) return "";
107     return resultado;
108 }
109 };
110
111 // -----
112 // Verifica que "resultado" sea un orden de letras compatible con "words":
113 // debe
114 // contener exactamente el mismo conjunto de letras (sin repetidas) y respetar
115 // cada arista deducida (letra c1 antes que c2). Como el orden topológico no
116 // es
117 // único cuando hay letras sin relación entre sí, esta función permite
118 // comparar sin depender de un string fijo.
119 bool ordenValido(const string& resultado, const vector<string>& words) {
120     unordered_set<char> letras;
121     for (const string& palabra : words)
122         for (char c : palabra) letras.insert(c);
123
124     if (resultado.size() != letras.size()) return false;
125
126     unordered_map<char, int> posicion;
127     for (int i = 0; i < (int)resultado.size(); ++i) {
128         char c = resultado[i];
129         if (!letras.count(c) || posicion.count(c)) return false; // letra
130                             repetida o ajena
131         posicion[c] = i;
132     }
133
134     for (size_t i = 0; i + 1 < words.size(); ++i) {
135         const string& w1 = words[i];
136         const string& w2 = words[i + 1];
137         size_t minLen = min(w1.size(), w2.size());
138         bool difieren = false;
139         for (size_t j = 0; j < minLen; ++j) {
140             if (w1[j] != w2[j]) {
141                 if (posicion[w1[j]] >= posicion[w2[j]]) return false; //
142                                     arista violada
143                 difieren = true;
144                 break;
145             }
146         }
147         if (!difieren && w1.size() > w2.size()) return false; // prefijo invá
148                             lido
149     }
150     return true;
151 }
152
153 int main() {

```

```

149     Solution s;
150
151     // Ejemplo clásico: las aristas forman una cadena w->e->r->t->f, así que
152     // el
153     // orden topológico es único y podemos comparar directo contra "wertf".
154     vector<string> w1 = {"wrt", "wrf", "er", "ett", "rftt"};
155     assert(s.alienOrder(w1) == "wertf");
156     assert(ordenValido(s.alienOrder(w1), w1));
157
158     // Prefijo inválido: "abc" es más larga que "ab" pero "ab" es su prefijo,
159     // eso viola el orden alfabético => "".
160     vector<string> w2 = {"abc", "ab"};
161     assert(s.alienOrder(w2) == "");
162
163     // Ciclo de 2 letras: hacen falta 3 palabras para que dos comparaciones
164     // consecutivas den las dos aristas opuestas. "z","x" da z->x; "x","z" da
165     // x->z => ciclo z<->x, imposible (caso clásico de LeetCode).
166     vector<string> w3 = {"z", "x", "z"};
167     assert(s.alienOrder(w3) == "");
168
169     // Ambiguo: solo se deduce b->d; la letra c no tiene ninguna relación con
170     // las demás, así que hay varios órdenes válidos (usamos el verificador).
171     vector<string> w4 = {"ab", "adc"};
172     string r4 = s.alienOrder(w4);
173     assert(ordenValido(r4, w4));
174
175     // Una sola palabra: no hay pares que comparar, cualquier letra presente
176     // debe aparecer y no hay aristas que respetar.
177     vector<string> w5 = {"z"};
178     assert(s.alienOrder(w5) == "z");
179
180     // Ciclo más largo escondido entre tres letras: a->b (de "ab","bc"),
181     // b->c (de "bc","ca") y c->a (de "ca","a") cierran el ciclo a->b->c->a.
182     vector<string> w6 = {"ab", "bc", "ca", "a"};
183     assert(s.alienOrder(w6) == "");
184
185     cout << "269 Alien Dictionary: todas las pruebas pasaron.\n";
186     return 0;
187 }

```

Complejidad: tiempo $O(C)$, espacio $O(1)$.

¿Qué significa esa complejidad? Aquí C es la suma de las longitudes de todas las palabras, es decir, cuántas letras hay que leer en total para comparar todas las palabras vecinas. Ese conteo es lo que manda en el tiempo, porque acomodar las letras es rapidísimo: el alfabeto tiene a lo más 26 letras. Es lo más rápido que se puede, ya que hay que leer las palabras al menos una vez para sacar las pistas.

Consejos para la entrevista (Amazon). Es pariente cercano de Course Schedule (207/210): usa el mismo algoritmo de Kahn, pero aquí lo difícil es armar bien el dibujo de flechas a partir de las palabras. Los dos errores más comunes: (1) olvidar la trampa de la palabra corta que va después de la larga que empieza igual, y (2) tomar más de una letra distinta por cada par de palabras

(solo importa la *primera* diferencia; lo demás no dice nada). Recuerda también que casi siempre hay varios órdenes válidos (una letra que nadie obliga a nada puede ir en varios lugares), así que conviene aclarar que basta con devolver *un* orden que funcione, no “el” orden.

127. Word Ladder

(H)

Código fuente: [src/01-grafos/127-word-ladder.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dadas `beginWord`, `endWord` y una lista `wordList`, buscamos la longitud de la secuencia de transformación *más corta* de `beginWord` a `endWord`, donde cada paso cambia exactamente una letra y cada palabra intermedia debe existir en `wordList`. La longitud cuenta las palabras (incluyendo `begin` y `end`); si no existe tal secuencia, o `endWord` no está en `wordList`, devolvemos 0.

La idea, con un ejemplo de la vida real. Imagina un juego de mesa donde cada palabra es una casilla. Dos casillas están conectadas por un puente si sus palabras se diferencian en una sola letra (por ejemplo `hit` y `hot`). Queremos ir de la casilla inicial a la casilla final dando la menor cantidad de saltos posible.

Para encontrar el camino más corto en un tablero así, la mejor forma es explorar “en anillos”, desde la casilla de salida hacia afuera: primero todas las casillas a las que llegas en un salto, luego todas a las que llegas en dos saltos, y así. En cuanto uno de esos anillos toca la casilla final, sabemos que ese es el número mínimo de saltos, porque exploramos de menos a más. Esa forma de explorar por anillos, o por capas, se llama *BFS*: cada anillo del BFS es un paso de transformación.

<code>hit</code>	<code>dot</code>	<code>lot</code>	
			<code>cog</code>
<code>hot</code>	<code>dog</code>	<code>log</code>	

Cómo lo resuelve el código, paso a paso. No dibujamos todos los puentes de antemano, porque con muchas palabras sería carísimo. En vez de eso, descubrimos los vecinos de una palabra “sobre la marcha”.

1. Usamos una *cola* (una fila: primero en entrar, primero en salir) y metemos la palabra inicial.
2. Para sacar los vecinos de una palabra, recorremos cada posición y probamos cambiarla por las 26 letras del abecedario. Cada palabra que resulte y que esté en `wordList` es una casilla vecina. Guardamos las palabras en un `unordered_set` (una especie de bolsa mágica donde revisar si algo está adentro es instantáneo).
3. Vamos anillo por anillo: al empezar cada nivel miramos cuántas palabras hay en la cola en ese momento (`cola.size()`) y procesamos justo esas antes de pasar al siguiente anillo.
4. Marcamos como visitada cada palabra que ya usamos, para no volver a pasar por ella y dar vueltas en círculo.

En cuanto un anillo alcanza `endWord`, contamos cuántos anillos llevamos y esa es la respuesta.

```

1 //
  =====
2 // 127. Word Ladder (Hard)
3 //
  -----
4 // Dadas beginWord, endWord y una lista wordList, encuentra la longitud de la
5 // secuencia de transformación MÁS CORTA de beginWord a endWord, tal que:
```

```

6 // - cada paso cambia exactamente UNA letra,
7 // - cada palabra intermedia resultante debe existir en wordList.
8 // La longitud cuenta las palabras (incluyendo begin y end). Si no existe tal
9 // secuencia (o endWord no está en wordList), devuelve 0.
10 //
11 // Ejemplo:
12 //   beginWord = "hit", endWord = "cog"
13 //   wordList = ["hot","dot","dog","lot","log","cog"]
14 //
15 //   hit -> hot -> dot -> dog -> cog    (longitud 5)
16 //
17 // Diagrama del grafo implícito (arista si dos palabras difieren en 1 letra):
18 //
19 //       hit
20 //       |
21 //       hot
22 //      / \
23 //     dot  lot
24 //     |    |
25 //     dog  log
26 //     \    /
27 //      cog
28 //
29 // Idea: cada palabra es un nodo; hay arista entre dos palabras si difieren en
30 // exactamente una letra. Buscamos el camino más corto de beginWord a endWord
31 // en ese grafo => BFS por niveles (no ponderado). Cada nivel del BFS
32 // corresponde a un paso de transformación.
33 //
34 // Para no construir el grafo explícitamente (sería carísimo), generamos los
35 // vecinos de una palabra "sobre la marcha": para cada posición, probamos las
36 // 26 letras del alfabeto distintas a la original y verificamos si el
37 // resultado está en el conjunto de wordList (O(1) con unordered_set).
38 //
39 // Complejidad: tiempo O(N * L^2 * 26), espacio O(N * L),
40 //               donde N = número de palabras en wordList, L = longitud de
41 //               cada palabra (todas tienen la misma longitud).
42 //
43 =====
44 #include <vector>
45 #include <string>
46 #include <queue>
47 #include <unordered_set>
48 #include <cassert>
49 #include <iostream>
50 using namespace std;
51
52 class Solution {
53 public:
54     int ladderLength(string beginWord, string endWord, vector<string>&
55     wordList) {
56         unordered_set<string> diccionario(wordList.begin(), wordList.end());
57         if (diccionario.find(endWord) == diccionario.end()) return 0;
58     }
59 };

```

```

57     queue<string> cola; // BFS por niveles
58     unordered_set<string> visitadas;
59     cola.push(beginWord);
60     visitadas.insert(beginWord);
61
62     int nivel = 1; // beginWord ya cuenta como la primera palabra
63     while (!cola.empty()) {
64         int capa = cola.size(); // palabras en el nivel actual
65         for (int i = 0; i < capa; ++i) {
66             string palabra = cola.front(); cola.pop();
67             if (palabra == endWord) return nivel;
68
69             // Generamos todos los vecinos: cambiar cada posición a cada
70             // letra de la 'a' a la 'z' y ver si el resultado es válido.
71             for (size_t pos = 0; pos < palabra.size(); ++pos) {
72                 char original = palabra[pos];
73                 for (char c = 'a'; c <= 'z'; ++c) {
74                     if (c == original) continue;
75                     palabra[pos] = c;
76                     if (diccionario.count(palabra) && !visitadas.count(
77                         palabra)) {
78                         visitadas.insert(palabra);
79                         cola.push(palabra);
80                     }
81                     palabra[pos] = original; // restauramos antes de la
82                     // siguiente posición
83                 }
84             }
85             ++nivel; // pasamos al siguiente nivel (un paso más de
86             // transformación)
87         }
88     }
89
90     return 0; // se agotó el BFS sin alcanzar endWord => imposible
91 }
92
93 int main() {
94     Solution s;
95
96     // Caso clásico: hit -> hot -> dot -> dog -> cog (longitud 5).
97     vector<string> w1 = {"hot", "dot", "dog", "lot", "log", "cog"};
98     assert(s.ladderLength("hit", "cog", w1) == 5);
99
100     // endWord no está en wordList => imposible.
101     vector<string> w2 = {"hot", "dot", "dog", "lot", "log"};
102     assert(s.ladderLength("hit", "cog", w2) == 0);
103
104     // Camino más corto entre dos palabras intermedias del mismo diccionario:
105     // hot -> dot -> dog (longitud 3). La ruta hot->lot->log->dog es más larga
106     .
107     vector<string> w3 = {"hot", "dot", "dog", "lot", "log", "cog"};
108     assert(s.ladderLength("hot", "dog", w3) == 3);
109
110     // beginWord y endWord difieren en una sola letra y endWord está en la

```



```
107 // lista => transformación directa, longitud 2.  
108 vector<string> w4 = {"hot"};  
109 assert(s.ladderLength("hit", "hot", w4) == 2);  
110  
111 cout << "127 Word Ladder: todas las pruebas pasaron.\n";  
112 return 0;  
113 }
```

Complejidad: tiempo $O(N \cdot L^2 \cdot 26)$, espacio $O(N \cdot L)$.

¿Qué significa esa complejidad? Aquí N es cuántas palabras hay en `wordList` y L cuántas letras tiene cada palabra. Para cada palabra probamos cambiar cada una de sus L letras por las 26 del abecedario, y armar cada palabra nueva cuesta otras L letras; por eso aparece L^2 y el 26. En resumen: es el trabajo de generar y revisar todos los cambios de una letra, por todas las palabras.

Consejos para la entrevista (Amazon). Lo primero, sin falta, es revisar que `endWord` esté en `wordList`; si no está, la respuesta es 0 y ni siquiera vale la pena empezar a explorar. El error típico es buscar los vecinos comparando la palabra contra cada palabra de la lista letra por letra, lo cual es lentísimo cuando la lista es grande; es mucho mejor generar los cambios de una letra y preguntarle a la bolsa mágica (`unordered_set`) si esa palabra existe. También puedes mencionar el *BFS bidireccional*: explorar al mismo tiempo desde la palabra inicial y desde la final, para que los dos anillos se encuentren en medio y haya menos trabajo.

126. Word Ladder II

(H)

Código fuente: [src/01-grafos/126-word-ladder-ii.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dadas `beginWord`, `endWord` y una lista `wordList`, cada palabra puede transformarse en otra cambiando exactamente una letra, siempre que el resultado exista en `wordList`. A diferencia del problema 127 (Word Ladder), donde sólo pedían la *longitud* del camino más corto, aquí hay que devolver **todas** las secuencias de transformación más cortas de `beginWord` a `endWord`. Si `endWord` es inalcanzable, se devuelve el vector vacío.

La idea, con un ejemplo de la vida real. Este problema es como el 127 (Word Ladder), pero más exigente: allá bastaba con saber *cuántos* saltos tenía el camino más corto entre dos palabras; aquí hay que entregar **todos** los caminos más cortos, no solo uno.

Piénsalo como un rally donde varios corredores salen de la palabra inicial y avanzan por anillos, cambiando una letra a la vez, igual que en el 127. Ese avance por anillos, de menos a más saltos, se llama *BFS* (búsqueda en anchura). La diferencia es que ahora, cuando un corredor llega a una palabra nueva, anotamos de *qué palabra venía* (le llamamos su “papá”). Como una misma palabra puede alcanzarse desde varias palabras distintas en el mismo anillo, cada palabra puede tener varios papás. Al terminar, tenemos una especie de árbol genealógico de las palabras. Para sacar todos los caminos más cortos, solo hay que caminar hacia atrás desde `endWord` siguiendo a los papás hasta llegar a `beginWord`; como solo guardamos papás del anillo anterior, todos los caminos que salgan tienen la longitud mínima.

```
nivel 0: hit
nivel 1: hot
nivel 2: dot  lot
nivel 3: dog  log
nivel 4: cog
```

Con `wordList = ["hot","dot","dog","lot","log","cog"]` hay dos caminos más cortos, ambos de longitud 5: `hit-hot-dot-dog-cog` y `hit-hot-lot-log-cog`, porque tanto `dog` como `log` tienen a `cog` como hija en el mismo nivel.

Cómo lo resuelve el código, paso a paso. Son dos fases: primero avanzar por anillos anotando papás, y luego reconstruir los caminos hacia atrás.

1. **Avanzar por anillos (BFS)** desde `beginWord`. En cada anillo, para cada palabra probamos cambiar cada letra por las 26 del abecedario; si la palabra que sale está en el diccionario, la mandamos al siguiente anillo y anotamos la palabra actual como uno de sus papás. Cuando terminamos de procesar un anillo completo, borramos del diccionario las palabras que ya usamos, para que un anillo más lejano no pueda *regresar* a ellas (si regresara, el camino ya no sería el más corto). Si `endWord` aparece en un anillo, terminamos de revisar ese anillo completo (por si tiene varios papás) y ahí paramos.
2. **Reconstruir los caminos hacia atrás (DFS con backtracking)** desde `endWord`, siguiendo todos los papás anotados hasta llegar a `beginWord`, y guardando cada camino completo. *DFS* significa seguir un camino hasta el fondo antes de probar otro; *backtracking* quiere decir que, al llegar al final, deshacemos el último paso para probar otra rama. Como armamos cada camino de atrás hacia adelante, al final lo volteamos.


```

97             siguienteNivel.insert(variante);
98             padres[variante].push_back(palabra);
99             if (variante == endWord) encontrado = true;
100         }
101     }
102     variante[pos] = original; // restauramos antes de la
        siguiente posición
103 }
104 }
105 nivelActual = siguienteNivel;
106 }
107
108 if (!encontrado) return resultado; // BFS se agotó sin alcanzar
        endWord
109
110 vector<string> camino{endWord};
111 reconstruir(endWord, beginWord, padres, camino, resultado);
112 for (auto& c : resultado) reverse(c.begin(), c.end());
113 return resultado;
114 }
115
116 private:
117     // DFS con backtracking: desde 'palabra' retrocede por todos sus padres
118     // hasta llegar a beginWord, acumulando cada camino completo (en reversa,
119     // de endWord a beginWord) en resultado.
120     void reconstruir(const string& palabra, const string& beginWord,
121                     unordered_map<string, vector<string>>& padres,
122                     vector<string>& camino, vector<vector<string>>&
                        resultado) {
123         if (palabra == beginWord) {
124             resultado.push_back(camino);
125             return;
126         }
127         for (const string& padre : padres[palabra]) {
128             camino.push_back(padre);
129             reconstruir(padre, beginWord, padres, camino, resultado);
130             camino.pop_back();
131         }
132     }
133 };
134
135 int main() {
136     Solution s;
137
138     // Caso clásico: dos caminos más cortos, ambos de longitud 5.
139     {
140         vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog"};
141         auto res = s.findLadders("hit", "cog", wordList);
142         sort(res.begin(), res.end());
143         vector<vector<string>> esperado = {
144             {"hit", "hot", "dot", "dog", "cog"},
145             {"hit", "hot", "lot", "log", "cog"},
146         };
147         sort(esperado.begin(), esperado.end());

```

```

148     assert(res == esperado);
149 }
150
151 // Sin solución: endWord no está en wordList => vector vacío.
152 {
153     vector<string> wordList = {"hot", "dot", "dog", "lot", "log"};
154     auto res = s.findLadders("hit", "cog", wordList);
155     assert(res.empty());
156 }
157
158 // Un único camino más corto (sin bifurcaciones).
159 {
160     vector<string> wordList = {"dot", "dog"};
161     auto res = s.findLadders("hot", "dog", wordList);
162     vector<vector<string>> esperado = {{ "hot", "dot", "dog" }};
163     assert(res == esperado);
164 }
165
166 // beginWord no pertenece a wordList (sólo endWord y las intermedias
167 // deben estar); aquí sólo existe un camino más corto de longitud 3.
168 {
169     vector<string> wordList = {"hot", "dot", "dog", "lot", "log", "cog", "
        hog"};
170     auto res = s.findLadders("hig", "cog", wordList);
171     vector<vector<string>> esperado = {{ "hig", "hog", "cog" }};
172     assert(res == esperado);
173 }
174
175 cout << "126 Word Ladder II: todas las pruebas pasaron.\n";
176 return 0;
177 }

```

Complejidad: tiempo $O(N \cdot L^2 \cdot 26)$ para el BFS más el costo de reconstruir todos los caminos (exponencial en el peor caso, inherente al problema), espacio $O(N \cdot L)$ para el diccionario y el mapa de padres.

¿Qué significa esa complejidad? Aquí N es cuántas palabras hay y L cuántas letras tiene cada una. La parte de avanzar por anillos cuesta lo mismo que en el 127: probar todos los cambios de una letra en todas las palabras. A eso se le suma el costo de armar todos los caminos, que puede crecer muchísimo cuando hay muchas bifurcaciones; pero eso no es culpa de nuestro método, sino de que el problema en sí puede tener muchísimos caminos válidos que hay que entregar.

Consejos para la entrevista (Amazon). Suelen plantearlo como “ya resolviste el 127, ahora dame todos los caminos”. Lo que buscan es que te des cuenta de que anotar un solo papá por palabra pierde caminos igual de cortos, y que hay que permitir varios papás por anillo antes de borrar las palabras ya usadas. Errores típicos: borrar las palabras del diccionario una por una en vez de por anillo completo (eso rompe caminos válidos del mismo anillo), o intentar generar todos los caminos con puro DFS sin la exploración por anillos (eso se vuelve lentísimo). No olvides los casos raros: `endWord` no está en `wordList` (respuesta vacía), `beginWord` no está en `wordList` (es válido, solo debe iniciar la cadena) y un único camino más corto sin ramificaciones.

261. Graph Valid Tree

(M)

Código fuente: [src/01-grafos/261-graph-valid-tree.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan n nodos etiquetados de 0 a $n - 1$ y una lista de aristas no dirigidas. Queremos saber si ese grafo forma un *árbol* válido.

La idea, con un ejemplo de la vida real. Un *árbol*, en matemáticas, es una red de puntos unidos por líneas que cumple dos cosas: todo está conectado en una sola pieza (puedes llegar de cualquier punto a cualquier otro) y no tiene ningún círculo cerrado. Piensa en el árbol genealógico de una familia, o en las ramas de un árbol de verdad: todo cuelga de un mismo tronco y las ramas nunca se vuelven a juntar para formar un aro.

Hay un truco de conteo muy útil: un árbol con n puntos tiene **exactamente** $n - 1$ líneas. Si tiene menos, es imposible que todo quede conectado (faltan líneas para unir todo); si tiene más, forzosamente sobra alguna línea que cierra un aro. Así que primero descartamos por el número de líneas, y si son justo $n - 1$, solo nos falta comprobar que ninguna de ellas cierre un aro. Un *aro* o círculo cerrado se llama en matemáticas un *ciclo*. Si no hay ciclos y hay $n - 1$ líneas, entonces todo queda conectado automáticamente.

```

      árbol válido (n = 5)
          0
        1  2  3
          4
aristas: [0,1] [0,2] [0,3] [1,4]  →  4 = n-1  →  true

```

Una herramienta nueva: “Union-Find”. Para cachar los aros de forma rápida usamos una herramienta llamada *Union-Find* (también conocida como *Disjoint Set Union* o DSU). Sirve para llevar la cuenta de qué puntos están en el mismo grupo, como cuando en el recreo se van formando equipos y quieres saber al instante si dos niños ya son del mismo equipo. Cada grupo tiene un “capitán” (su representante), y para saber si dos puntos son del mismo grupo basta con ver si tienen el mismo capitán.

- Al principio, cada punto es su propio capitán (cada quien es su propio equipo de una persona). Eso se guarda en un arreglo `padre`.
- `find(x)` averigua quién es el capitán de `x`: va siguiendo la cadena de jefes hacia arriba hasta llegar al de más arriba. Con un truco llamado *compresión de caminos*, cada punto por el que pasamos se reengancha directo al capitán, para que la próxima vez la respuesta sea casi inmediata.
- `unite(x, y)` junta los equipos de `x` y `y`. Primero mira quién es el capitán de cada uno. Si resulta que ya tienen el mismo capitán, quiere decir que `x` y `y` ya estaban conectados por otro camino, y esta línea nueva cerraría un aro (un *ciclo*): devolvemos `false`. Si tienen capitanes distintos, unimos los dos equipos colgando el equipo pequeño del grande (eso se llama *unión por rango*), para que las cadenas de jefes se queden cortas y todo siga siendo rápido.

Cómo lo resuelve el código, paso a paso.

1. Primero contamos las líneas: si no son exactamente $n - 1$, ya sabemos que no es un árbol y respondemos `false`.
2. Si sí son $n - 1$, recorreremos las líneas una por una llamando a `unite` para juntar los equipos de sus dos extremos.
3. Si en alguna línea los dos extremos ya eran del mismo equipo, es que se forma un aro: respondemos `false` de inmediato.
4. Si recorremos todas las líneas sin encontrar ningún aro, entonces sí es un árbol válido.

```

1  //
   =====
2  // 261. Graph Valid Tree (Medium)
3  //
   -----
4  // Dados n nodos etiquetados de 0 a n-1 y una lista de aristas no dirigidas,
5  // determina si forman un árbol válido.
6  //
7  // Un grafo no dirigido con n nodos es un árbol si y sólo si cumple DOS
8  // condiciones a la vez:
9  //   1) Está completamente conectado (un solo componente).
10 //   2) No tiene ciclos.
11 //
12 // Un dato clave: un árbol con n nodos tiene EXACTAMENTE n-1 aristas. Si hay
13 // menos de n-1 aristas, es imposible estar conectado (false de inmediato).
14 // Si hay más de n-1, forzosamente hay un ciclo (false de inmediato). Con
15 // exactamente n-1 aristas, basta revisar que no exista ciclo para garantizar
16 // que el grafo es un único componente conectado (n-1 aristas sin ciclos ya
17 // implica conectividad total).
18 //
19 // Ejemplo (árbol válido), n = 5:
20 //
21 //      0
22 //     / | \
23 //    1  2  3
24 //     |
25 //     4
26 // aristas: [[0,1],[0,2],[0,3],[1,4]] -> 4 aristas = n-1 -> true
27 //
28 // Ejemplo (con ciclo), n = 5:
29 //      0 - 1 - 2 - 0 (ciclo)      3 - 4 (componente aparte)
30 // aristas: [[0,1],[1,2],[2,0],[3,4]] -> false
31 //
32 // Técnica: Union-Find (Disjoint Set Union, DSU) con unión por rango y
33 // compresión de caminos. Es un patrón NUEVO en este libro, así que se explica
34 // a detalle:
35 //
36 // - Cada nodo empieza siendo su propio representante (su propio conjunto).
37 // - find(x): sigue los punteros "padre" hasta la raíz del conjunto de x.

```



```

38 //      Con COMPRESIÓN DE CAMINOS, cada nodo visitado en el camino se re-liga
39 //      directamente a la raíz, para que futuras consultas sean casi  $O(1)$ .
40 //      - unite(x, y): busca las raíces de x y y.
41 //      * Si ya son la MISMA raíz, x y y ya estaban conectados por otro
42 //      camino => añadir esta arista formaría un CICLO => false.
43 //      * Si son distintas, se fusionan los conjuntos. Con UNIÓN POR RANGO
44 //      (o por tamaño), la raíz del árbol más pequeño/bajo cuelga de la
45 //      raíz del árbol más grande, para mantener los árboles del DSU
46 //      planos y las operaciones rápidas.
47 //
48 // Recorremos todas las aristas llamando a unite(). Si alguna unite() detecta
49 // que ambos extremos ya comparten raíz, hay un ciclo -> false. Si procesamos
50 // todas las aristas sin ciclos Y el número de aristas es exactamente  $n-1$ ,
51 // el grafo es un árbol válido.
52 //
53 // Complejidad: tiempo casi  $O(n + e * \text{alfa}(n))$  (alfa = inversa de Ackermann,
54 // prácticamente constante gracias a rango + compresión),
55 // espacio  $O(n)$  para los arreglos padre/rango.
56 //
=====

57 #include <vector>
58 #include <cassert>
59 #include <iostream>
60 using namespace std;
61
62 class Solution {
63 public:
64     bool validTree(int n, vector<vector<int>>& edges) {
65         // Un árbol con n nodos tiene exactamente  $n-1$  aristas.
66         if ((int)edges.size() != n - 1) return false;
67
68         padre.resize(n);
69         rango.assign(n, 0);
70         for (int i = 0; i < n; ++i) padre[i] = i;    // cada nodo es su propia
            raíz
71
72         for (auto& e : edges) {
73             int a = e[0], b = e[1];
74             if (!unite(a, b)) return false;    // ya estaban conectados =>
                ciclo
75         }
76         return true;    //  $n-1$  aristas y sin ciclos => un solo árbol conectado
77     }
78
79 private:
80     vector<int> padre;
81     vector<int> rango;
82
83     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
84     int find(int x) {
85         if (padre[x] != x) padre[x] = find(padre[x]);    // compresión de
            caminos
86         return padre[x];

```

```

87     }
88
89     // Une los conjuntos de a y b. Devuelve false si ya estaban en el mismo
90     // conjunto (es decir, unir esta arista formaría un ciclo).
91     bool unite(int a, int b) {
92         int ra = find(a), rb = find(b);
93         if (ra == rb) return false;    // mismo conjunto => ciclo
94
95         // Unión por rango: la raíz de menor rango cuelga de la de mayor rango
96
97         if (rango[ra] < rango[rb]) swap(ra, rb);
98         padre[rb] = ra;
99         if (rango[ra] == rango[rb]) ++rango[ra];
100        return true;
101    };
102
103    int main() {
104        Solution s;
105
106        // Árbol válido: 5 nodos, 4 aristas (n-1), todo conectado, sin ciclos.
107        vector<vector<int>> e1 = {{0,1},{0,2},{0,3},{1,4}};
108        assert(s.validTree(5, e1) == true);
109
110        // Con ciclo: 0-1-2-0 forma un ciclo; además 3-4 queda desconectado del
111        // resto. La detección de ciclo en unite() basta para regresar false.
112        vector<vector<int>> e2 = {{0,1},{1,2},{2,0},{3,4}};
113        assert(s.validTree(5, e2) == false);
114
115        // Desconectado: dos componentes separados {0,1} y {2,3}; sólo 2 aristas
116        // pero se necesitan 3 (n-1 con n=4) para siquiera intentar ser árbol.
117        vector<vector<int>> e3 = {{0,1},{2,3}};
118        assert(s.validTree(4, e3) == false);
119
120        // Un solo nodo sin aristas: componente trivial, es un árbol válido.
121        vector<vector<int>> e4 = {};
122        assert(s.validTree(1, e4) == true);
123
124        cout << "261 Graph Valid Tree: todas las pruebas pasaron.\n";
125        return 0;
126    }

```

Complejidad: tiempo casi $O(n + e \cdot \alpha(n))$, espacio $O(n)$.

¿Qué significa esa complejidad? Aquí n es el número de puntos y e el de líneas. En pocas palabras, tocamos cada punto y cada línea una sola vez. Ese símbolo raro, $\alpha(n)$, es una función que crece tan pero tan despacio que para cualquier cantidad de puntos que veas en la vida real vale prácticamente lo mismo que 3 o 4: por eso decimos que cada operación de Union-Find es “casi instantánea”.

Consejos para la entrevista (Amazon). Menciona primero el atajo de contar líneas ($n - 1$), porque descarta la mayoría de los casos fáciles sin ni siquiera usar Union-Find. Explica por

qué los dos trucos (colgar el equipo pequeño del grande y reenganchar los puntos al capitán) son importantes: sin ellos, buscar al capitán podría volverse lento en el peor caso (una cadena larguísima de jefes), mientras que con los dos trucos cada consulta es casi instantánea. No olvides los casos raros: $n = 1$ sin líneas (es un árbol trivial válido), un grafo con un aro, un grafo partido en varios pedazos sueltos, y tener más o menos líneas de las $n - 1$ que debe tener un árbol.

323. Number of Connected Components

(M)

Código fuente: [src/01-grafos/323-connected-components.cpp](#)

► Ver animación · [local](#) | [web](#)

Hay n nodos, numerados de 0 a $n - 1$, y una lista `edges` de pares $[a, b]$ que representan aristas **no dirigidas**. Queremos contar cuántos componentes conexos tiene el grafo.

La idea, con un ejemplo de la vida real. Un *componente conexo* es un grupo de puntos que están unidos entre sí; contar componentes es contar cuántos grupos separados hay. Imagina un salón donde al principio cada niño está solo, sin amigos: hay n grupitos de una sola persona. Cada arista (cada línea que une dos puntos) es como presentar a dos niños para que se hagan amigos.

- Si los dos niños ya eran del mismo grupo de amigos, presentarlos otra vez no cambia nada; sigue habiendo los mismos grupos.
- Si eran de grupos distintos, al hacerse amigos los dos grupos se juntan en uno solo, así que hay un grupo menos: el contador baja en 1.

Cuando terminamos de procesar todas las líneas, el contador nos dice cuántos grupos separados quedaron.

```
n = 5, edges = [[0, 1], [1, 2], [3, 4]]
0 --- 1 --- 2          3 --- 4
2 componentes: {0, 1, 2} y {3, 4}
```

Cómo lo resuelve el código, paso a paso. La herramienta ideal para esto de “ir juntando grupos” se llama *Union-Find* (o *Disjoint Set Union*, DSU). Cada punto tiene un “jefe”, y siguiendo la cadena de jefes hacia arriba llegas al *capitán* del grupo; dos puntos son del mismo grupo si tienen el mismo capitán. Tiene dos operaciones:

- `find(x)`: averigua el capitán de x siguiendo la cadena de jefes hacia arriba. Usamos un truco, la *compresión de caminos*: cada punto por el que pasamos se reengancha directo al capitán, para que la próxima vez la respuesta sea casi inmediata.
- `unir(a, b)`: si a y b tienen capitanes distintos, juntamos los dos grupos colgando el grupo pequeño del grande (eso se llama *unión por rango*, y sirve para que las cadenas de jefes se queden cortas) y bajamos en 1 el contador de grupos.

Arrancamos con cada punto siendo su propio capitán (n grupos) y recorremos las líneas una sola vez, uniendo cuando hace falta. El contador final es la respuesta.

```
1 //
2 // =====
3 //
4 // 323. Number of Connected Components in an Undirected Graph (Medium)
5 //
6 // -----
7 //
8 // Hay n nodos, numerados de 0 a n-1, y una lista edges de pares [a, b] que
9 // representan aristas NO dirigidas. Cuenta cuántos componentes conexos tiene
10 // el grafo.
```

```

7 //
8 // Ejemplo 1:
9 //   n = 5, edges = [[0,1],[1,2],[3,4]]
10 //
11 //   0 --- 1 --- 2           3 --- 4
12 //
13 //   Hay 2 componentes: {0,1,2} y {3,4}.
14 //
15 // Ejemplo 2:
16 //   n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
17 //
18 //   0 --- 1 --- 2 --- 3 --- 4
19 //
20 //   Todos quedan conectados en cadena: 1 solo componente.
21 //
22 // Idea (Union-Find / DSU, Disjoint Set Union):
23 // - Empezamos suponiendo que cada nodo es su propio componente: n
24 //   componentes en total, cada uno representado por su propia raíz.
25 // - Por cada arista [a, b], buscamos la raíz de a y la raíz de b (find).
26 //   Si ya comparten raíz, la arista no cambia nada (ya estaban en el mismo
27 //   componente). Si tienen raíces distintas, unimos los dos conjuntos
28 //   (union) y decrementamos el contador de componentes en 1: dos
29 //   componentes se acaban de fusionar en uno solo.
30 // - Al terminar de procesar todas las aristas, el contador tiene la
31 //   respuesta final.
32 // - Usamos unión por rango y compresión de caminos para que find/union
33 //   sean casi O(1) amortizado (O(alfa(n))), la función inversa de Ackermann).
34 //
35 // Diagrama de una unión:
36 //
37 //   componente(a): [a]--[x]--[raiz_a]           componente(b): [b]--[raiz_b]
38 //
39 //   find(a) = raiz_a, find(b) = raiz_b, son distintas => unimos:
40 //
41 //   raiz_b pasa a colgar de raiz_a (o viceversa, según el rango) y
42 //   --componentes.
43 //
44 // Nota: es hermano del problema 261 "Graph Valid Tree" (mismo DSU: un grafo
45 // es árbol si y solo si tiene exactamente n-1 aristas y termina con un solo
46 // componente, es decir, ninguna arista intenta unir un nodo consigo mismo
47 // en el mismo componente) y del patrón de "contar islas" (200, 695): en
48 // vez de recorrer una cuadrícula con DFS/BFS, aquí contamos componentes de
49 // un grafo general dado por lista de aristas. También se puede resolver con
50 // DFS/BFS marcando visitados y contando cuántas veces arrancamos una
51 // exploración nueva, igual que en Number of Islands.
52 //
53 // Complejidad: tiempo O(n + e*alfa(n)) (n nodos, e aristas),
54 //               espacio O(n) por los arreglos padre y rango.
55 //
=====
56 #include <vector>
57 #include <cassert>
58 #include <iostream>

```

```

59 using namespace std;
60
61 class Solution {
62 public:
63     int countComponents(int n, vector<vector<int>>& edges) {
64         padre.resize(n);
65         rango.assign(n, 0);
66         for (int i = 0; i < n; ++i) padre[i] = i;    // cada nodo es su propia
67                                                     raíz
68
69         int componentes = n;    // al inicio, cada nodo es un componente
70                                 aislado
71         for (auto& arista : edges) {
72             int a = arista[0], b = arista[1];
73             int raizA = find(a);
74             int raizB = find(b);
75             if (raizA != raizB) {
76                 unir(raizA, raizB);
77                 --componentes;    // dos componentes se fusionaron en uno
78             }
79         }
80         return componentes;
81     }
82 private:
83     vector<int> padre;
84     vector<int> rango;
85
86     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
87     int find(int x) {
88         if (padre[x] != x) padre[x] = find(padre[x]);    // compresión de
89                                                         caminos
90         return padre[x];
91     }
92
93     // Une los conjuntos de dos raíces distintas usando unión por rango.
94     void unir(int raizA, int raizB) {
95         if (rango[raizA] < rango[raizB]) swap(raizA, raizB);
96         padre[raizB] = raizA;    // la raíz de menor rango cuelga
97                                 de la mayor
98         if (rango[raizA] == rango[raizB]) ++rango[raizA];
99     }
100 };
101
102 int main() {
103     Solution s;
104
105     // Dos componentes: {0,1,2} en cadena y {3,4} aparte.
106     vector<vector<int>> e1 = {{0,1},{1,2},{3,4}};
107     assert(s.countComponents(5, e1) == 2);
108
109     // Cadena que conecta todo: un solo componente.
110     vector<vector<int>> e2 = {{0,1},{1,2},{2,3},{3,4}};
111     assert(s.countComponents(5, e2) == 1);

```

```
109 // Sin aristas: cada nodo es su propio componente.
110 vector<vector<int>> e3 = {};
111 assert(s.countComponents(4, e3) == 4);
112
113 // Un solo nodo, sin aristas: un componente.
114 vector<vector<int>> e4 = {};
115 assert(s.countComponents(1, e4) == 1);
116
117 cout << "323 Connected Components: todas las pruebas pasaron.\n";
118 return 0;
119 }
120 }
```

Complejidad: tiempo $O(n + e \cdot \alpha(n))$, espacio $O(n)$.

¿Qué significa esa complejidad? Aquí n es el número de puntos y e el de líneas. En pocas palabras, tocamos cada punto y cada línea una sola vez. Ese símbolo raro, $\alpha(n)$, es una función que crece tan pero tan despacio que para cualquier cantidad de puntos de la vida real vale casi lo mismo que 3 o 4; por eso cada operación de Union-Find se considera “casi instantánea”.

Consejos para la entrevista (Amazon). Este es el ejemplo más clásico de Union-Find y es hermano directo del 261. **Graph Valid Tree:** allá, en vez de contar grupos, preguntan si el grafo termina siendo un solo árbol (justo $n - 1$ líneas y ninguna que una dos puntos ya del mismo grupo, es decir, ningún aro). También es pariente de “contar islas” (200, 695): en vez de recorrer una cuadrícula pintando lo visitado, aquí el grafo te llega como lista de líneas, pero el fondo es el mismo, contar cuántos grupos conectados hay. De hecho, este problema también se puede resolver con *DFS* o *BFS* (recorrer los puntos vecinos) armando una lista de vecinos y contando cuántas veces arrancamos una exploración desde un punto no visitado. Menciona las dos formas en la entrevista: recorrer con DFS/BFS es más fácil de explicar, pero Union-Find gana cuando las líneas van llegando de a poco (por ejemplo, si el entrevistador pregunta “¿y si las líneas se agregan una por una y quiero saber cuántos grupos hay después de cada una?”). No olvides los casos raros: sin líneas (cada punto es su propio grupo), un solo punto, y líneas repetidas o que no cambian el número de grupos.

128. Longest Consecutive Sequence

(M)

Código fuente: [src/01-grafos/128-longest-consecutive.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de enteros sin ordenar. Queremos la longitud de la secuencia de enteros *consecutivos* más larga (por ejemplo 3, 4, 5, 6), sin importar el orden en que aparezcan en el arreglo. El algoritmo debe correr en $O(n)$.

La idea, con un ejemplo de la vida real. Imagina un montón de niños en el patio, cada uno con un número escrito en la playera, pero revueltos y sin formarse. Los niños con números seguidos (por ejemplo 3, 4, 5, 6) quieren tomarse de la mano y formar una fila. Nuestra tarea es encontrar la fila más larga que se pueda armar.

El truco para no marearnos: en lugar de empezar a contar desde cualquier niño, solo empezamos desde el que va *al frente* de su fila, es decir, el que no tiene a nadie con el número justo anterior. Si un niño tiene el 4 y existe alguien con el 3, ese niño no es el frente: dejamos que el que esté más adelante empiece a contar. Así cada fila se cuenta una sola vez. Para preguntar rápido “¿existe un niño con tal número?” guardamos todos los números en un *conjunto* (`unordered_set`): es como una caja mágica en la que preguntar si algo está adentro es instantáneo, sin tener que revisar uno por uno.

```

nums = {100, 4, 200, 1, 3, 2}
100 → ¿existe 99? no → inicio, longitud 1
4   → ¿existe 3?  sí → no es inicio, se ignora
200 → ¿existe 199? no → inicio, longitud 1
1   → ¿existe 0?  no → inicio, cuenta 1,2,3,4 → longitud 4
3, 2 → no son inicio, se ignoran

```

El máximo es 4, por la secuencia 1, 2, 3, 4.

Cómo lo resuelve el código, paso a paso.

1. Metemos todos los números en la caja mágica (`unordered_set`). Los repetidos se funden en uno solo, y eso está bien: un número repetido no alarga la fila.
2. Recorremos los números uno por uno. Para cada número `num` preguntamos: ¿existe `num - 1` en la caja? Si *sí* existe, este número no es el frente de su fila, así que lo saltamos.
3. Si `num - 1` *no* existe, entonces `num` es el frente. Empezamos a contar hacia arriba: ¿está `num + 1`?, ¿y `num + 2`? Seguimos mientras el siguiente exista, contando cuántos niños se unen a la fila.
4. Guardamos la longitud de la fila más larga que hayamos visto. Esa es la respuesta.

El código completo (abre el archivo con clic en el título):

```

1 //
  =====
2 // 128. Longest Consecutive Sequence (Medium)

```



```

3 // -----
4 // Dado un arreglo de enteros sin ordenar, devuelve la longitud de la
5 // secuencia de enteros CONSECUTIVOS más larga (por ejemplo 3,4,5,6). El
6 // arreglo no está ordenado y el algoritmo debe correr en  $O(n)$ .
7 //
8 // Ejemplo:
9 //   nums = [100, 4, 200, 1, 3, 2]
10 //   La secuencia consecutiva más larga es 1, 2, 3, 4 => longitud 4.
11 //
12 // Diagrama de la idea (metemos todo en un unordered_set):
13 //
14 //   set = { 100, 4, 200, 1, 3, 2 }
15 //
16 //   100 -> ¿existe 99? no  => 100 ES inicio de secuencia. Cuenta: 100 (long
17 //   1)
18 //   4 -> ¿existe 3?  sí  => 4 NO es inicio, se ignora aquí (ya se contará
19 //   cuando lleguemos al verdadero inicio: 1)
20 //   200 -> ¿existe 199? no => 200 ES inicio. Cuenta: 200 (long 1)
21 //   1 -> ¿existe 0?  no  => 1 ES inicio de secuencia. Cuenta hacia arriba:
22 //   1,2,3,4 (existen en el set) -> longitud 4
23 //   3 -> ¿existe 2?  sí  => NO es inicio, se ignora
24 //   2 -> ¿existe 1?  sí  => NO es inicio, se ignora
25 //
26 //   Máximo encontrado: 4
27 //
28 // Idea clave (por qué es  $O(n)$  y no  $O(n^2)$ ): si para CADA número intentáramos
29 // contar hacia arriba su secuencia, cada elemento de una secuencia larga
30 // dispararía un conteo completo desde ahí, y sumaríamos trabajo repetido
31 // (1 dispara conteo de longitud 4, pero 2, 3 y 4 también dispararían sus
32 // propios conteos de longitud 3, 2, 1 =>  $O(n^2)$  en el peor caso, como una
33 // secuencia 1..n completa).
34 //
35 // El truco es filtrar: SOLO contamos hacia arriba cuando el número es un
36 // "inicio de secuencia", es decir, cuando (num - 1) NO está en el set. Así,
37 // cada secuencia consecutiva se cuenta EXACTAMENTE UNA VEZ, empezando desde
38 // su mínimo. Como cada número pertenece a una única secuencia, y esa
39 // secuencia sólo se recorre una vez completa (desde su inicio), la suma total
40 // de pasos de conteo a lo largo de todo el algoritmo es a lo sumo n. Cada
41 // número se visita cuando mucho dos veces: una para meterlo al set / revisar
42 // si es inicio, y otra (a lo sumo) cuando se cuenta como parte de la
43 // secuencia de su verdadero inicio. Por eso el algoritmo completo es  $O(n)$ .
44 //
45 // Complejidad: tiempo  $O(n)$  (cada número se procesa  $O(1)$  veces en promedio
46 // gracias al unordered_set y al filtro de "inicio de secuencia")
47 //
48 // ,
49 //   espacio  $O(n)$  por el unordered_set.
50 // =====
51
52 #include <vector>
53 #include <unordered_set>
54 #include <cassert>

```

```

51 #include <iostream>
52 using namespace std;
53
54 class Solution {
55 public:
56     int longestConsecutive(vector<int>& nums) {
57         unordered_set<int> presentes(nums.begin(), nums.end());
58         int mejor = 0;
59
60         for (int num : presentes) {
61             // Sólo arrancamos el conteo si num es el INICIO de su secuencia,
62             // es decir, si num - 1 no pertenece al conjunto. Esto evita
63             // recomtar la misma secuencia varias veces.
64             if (presentes.count(num - 1)) continue;
65
66             int actual = num;
67             int longitud = 1;
68             while (presentes.count(actual + 1)) { // cuenta hacia arriba
69                 ++actual;
70                 ++longitud;
71             }
72             mejor = max(mejor, longitud);
73         }
74         return mejor;
75     }
76 };
77
78 int main() {
79     Solution s;
80
81     vector<int> n1 = {100, 4, 200, 1, 3, 2};
82     assert(s.longestConsecutive(n1) == 4); // secuencia 1,2,3,4
83
84     vector<int> n2 = {0, 3, 7, 2, 5, 8, 4, 6, 0, 1};
85     assert(s.longestConsecutive(n2) == 9); // secuencia 0..8 (el 0 duplicado
86         // no afecta)
87
88     vector<int> vacio = {};
89     assert(s.longestConsecutive(vacio) == 0); // sin elementos, no hay
90         // secuencia
91
92     vector<int> n3 = {1, 2, 2, 3};
93     assert(s.longestConsecutive(n3) == 3); // duplicados no alargan la
94         // secuencia: 1,2,3
95
96     cout << "128 Longest Consecutive: todas las pruebas pasaron.\n";
97     return 0;
98 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? $O(n)$ quiere decir que el trabajo total crece igual que la cantidad de números: si hay el doble de números, tardamos más o menos el doble, nada peor. Quizá

te preocupe que la fila se cuente muchas veces, pero no pasa: como solo el niño del frente empieza a contar, cada fila se recorre *una sola vez* completa. Sumando todo, cada número se toca a lo mucho dos veces (una al revisar si es frente y otra al contarle dentro de su fila). Si en cambio dejáramos que *cualquier* niño empezara a contar, una fila larga se recontaría desde cada uno de sus miembros y el trabajo se dispararía a $O(n^2)$, mucho más lento.

Consejos para la entrevista (Amazon). El error más común es ordenar el arreglo primero; eso funciona, pero tarda $O(n \log n)$ y no cumple el $O(n)$ que suelen pedirte de forma explícita. Explica que el filtro de “solo el frente cuenta” es justo lo que evita el trabajo repetido: sin él, el algoritmo se vuelve $O(n^2)$ cuando hay una sola fila muy larga. Menciona los casos raros: arreglo vacío (la respuesta es 0), números repetidos (no alargan la fila, por eso usamos un conjunto que los funde, no un *multiset* ni un conteo de frecuencias) y números negativos (funcionan igual, al algoritmo no le importa el signo).

547. Number of Provinces

(M)

Código fuente: [src/01-grafos/547-number-of-provinces.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay n ciudades y una matriz de adyacencia `isConnected` de tamaño $n \times n$, donde `isConnected[i][j] == 1` si la ciudad i y la ciudad j están conectadas **directamente** (la matriz es simétrica y siempre `isConnected[i][i] == 1`). Una *provincia* es un grupo de ciudades conectadas directa o indirectamente, sin conexión con ninguna ciudad fuera del grupo. Queremos contar cuántas provincias hay.

La idea, con un ejemplo de la vida real. Piensa en las ciudades como niños y en una provincia como un grupo de amigos. Dos niños son amigos directos si la casilla `isConnected[i][j]` vale 1. Pero la amistad se contagia: si Ana es amiga de Beto y Beto es amigo de Carla, los tres pertenecen al mismo grupo aunque Ana y Carla no se conozcan directamente. Contar provincias es contar cuántos grupos de amigos separados hay.

La matriz solo es otra manera de darnos el mismo mapa de amistades: en lugar de una lista de parejas de amigos, nos entregan una matriz que, para cada par de ciudades (i, j) , dice con un 1 o un 0 si son amigas directas.

1	1	0
1	1	0
0	0	1

En esta matriz hay **2** provincias: $\{0, 1\}$ (conectadas directamente) y $\{2\}$ (aislada, solo conectada consigo misma en la diagonal).

Cómo lo resuelve el código, paso a paso (Union-Find). *Union-Find* (también llamado DSU) es una técnica para agrupar cosas rápidamente. Imagina que cada grupo de amigos elige un *capitán*, y todos en el grupo saben quién es su capitán. Dos ciudades están en la misma provincia exactamente cuando comparten capitán.

1. Al principio nadie se conoce, así que cada ciudad es su propio capitán: hay n grupos (n provincias).
2. Recorremos la matriz. Como es simétrica (si i es amiga de j , entonces j es amiga de i), basta mirar la mitad de arriba ($i < j$) para no repetir trabajo.
3. Cada vez que encontramos un 1 (ciudades i y j conectadas), *unimos* sus grupos: buscamos el capitán de cada una y, si eran distintos, los juntamos bajo un solo capitán y restamos 1 al conteo de provincias (dos grupos se volvieron uno).
4. Al terminar, el contador tiene el número de grupos que quedaron.

Dos detalles hacen que buscar al capitán sea casi instantáneo: la *compresión de caminos* (cada vez que buscamos, dejamos a la ciudad apuntando directo a su capitán, para la próxima vez) y la *unión por rango* (al juntar, colgamos el grupo pequeño del grande, para que las cadenas no se hagan largas).

```

1  //
   =====

2  // 547. Number of Provinces (Medium)
3  //
   -----

4  // Hay n ciudades. Nos dan una matriz de adyacencia isConnected de tamaño
5  // n x n, donde isConnected[i][j] == 1 si la ciudad i y la ciudad j están
6  // conectadas DIRECTAMENTE, e isConnected[i][j] == 0 si no lo están (siempre
7  // isConnected[i][i] == 1, la matriz es simétrica). Una "provincia" es un
8  // grupo de ciudades conectadas directa o indirectamente, y que no está
9  // conectado con ninguna ciudad fuera del grupo. Queremos contar cuántas
10 // provincias hay.
11 //
12 // Ejemplo:
13 //     isConnected =
14 //         1 1 0      Ciudad 0 y 1 están conectadas directamente.
15 //         1 1 0      Ciudad 2 no está conectada con nadie más.
16 //         0 0 1
17 //
18 //     Hay 2 provincias: {0, 1} y {2}.
19 //
20 // Idea: esta matriz de adyacencia NO es más que otra forma de darnos el
21 // mismo grafo no dirigido de 323 "Number of Connected Components". En vez de
22 // recibir una lista de aristas [a, b], recibimos, para cada par de ciudades
23 // (i, j), si existe o no la arista. Así que el problema es idéntico: contar
24 // componentes conexos. Usamos Union-Find (DSU): cada ciudad empieza siendo
25 // su propia provincia; cada vez que isConnected[i][j] == 1 con i != j,
26 // unimos las ciudades i y j (si no estaban ya en la misma provincia,
27 // decrementamos el contador de provincias).
28 //
29 // Diagrama de la matriz interpretada como grafo:
30 //
31 //     isConnected[i][j] == 1 <=> arista (i, j) en el grafo
32 //
33 //         0 --- 1          2 (aislada)
34 //
35 //     (fila/columna 0 y 1 se apuntan entre sí; fila/columna 2 solo se apunta
36 //     a sí misma en la diagonal)
37 //
38 // Complejidad: tiempo  $O(n^2 * \alpha(n))$  (revisamos las  $n^2$  celdas de la
39 // matriz, cada find/unir es casi  $O(1)$  amortizado),
40 // espacio  $O(n)$  por los arreglos padre y rango.
41 //
   =====

42 #include <vector>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 class Solution {

```

```

48 public:
49     int findCircleNum(vector<vector<int>>& isConnected) {
50         int n = isConnected.size();
51         padre.resize(n);
52         rango.assign(n, 0);
53         for (int i = 0; i < n; ++i) padre[i] = i;    // cada ciudad es su
54             propia raíz
55
56         int provincias = n;    // al inicio, cada ciudad es una provincia
57             aislada
58         for (int i = 0; i < n; ++i) {
59             for (int j = i + 1; j < n; ++j) {    // basta la mitad superior (
60                 matriz simétrica)
61                 if (isConnected[i][j] == 1) {
62                     int raizI = find(i);
63                     int raizJ = find(j);
64                     if (raizI != raizJ) {
65                         unir(raizI, raizJ);
66                         --provincias;    // dos provincias se fusionaron en una
67                     }
68                 }
69             }
70         }
71         return provincias;
72     }
73
74 private:
75     vector<int> padre;
76     vector<int> rango;
77
78     // Encuentra la raíz del conjunto de x, comprimiendo el camino recorrido.
79     int find(int x) {
80         if (padre[x] != x) padre[x] = find(padre[x]);    // compresión de
81             caminos
82         return padre[x];
83     }
84
85     // Une los conjuntos de dos raíces distintas usando unión por rango.
86     void unir(int raizA, int raizB) {
87         if (rango[raizA] < rango[raizB]) swap(raizA, raizB);
88         padre[raizB] = raizA;    // la raíz de menor rango cuelga
89             de la mayor
90         if (rango[raizA] == rango[raizB]) ++rango[raizA];
91     }
92 };
93
94 int main() {
95     Solution s;
96
97     // Ciudades 0 y 1 conectadas directamente; ciudad 2 aislada => 2
98     provincias.
99     vector<vector<int>> m1 = {
100         {1, 1, 0},
101         {1, 1, 0},

```

```

96         {0, 0, 1},
97     };
98     assert(s.findCircleNum(m1) == 2);
99
100     // Ninguna ciudad conectada con otra (solo la diagonal) => 3 provincias.
101     vector<vector<int>>> m2 = {
102         {1, 0, 0},
103         {0, 1, 0},
104         {0, 0, 1},
105     };
106     assert(s.findCircleNum(m2) == 3);
107
108     // Todas las ciudades conectadas entre sí => 1 sola provincia.
109     vector<vector<int>>> m3 = {
110         {1, 1, 1},
111         {1, 1, 1},
112         {1, 1, 1},
113     };
114     assert(s.findCircleNum(m3) == 1);
115
116     // Una sola ciudad => 1 provincia.
117     vector<vector<int>>> m4 = {{1}};
118     assert(s.findCircleNum(m4) == 1);
119
120     cout << "547 Number of Provinces: todas las pruebas pasaron.\n";
121     return 0;
122 }

```

Complejidad: tiempo $O(n^2 \cdot \alpha(n))$, espacio $O(n)$.

¿Qué significa esa complejidad? Recorremos las n^2 casillas de la matriz (en la práctica solo la mitad de arriba), y por eso el tiempo depende de n^2 . Buscar o unir capitanes cuesta casi nada, prácticamente un paso, gracias a los dos trucos de arriba; ese “casi nada” se escribe $\alpha(n)$ (la inversa de Ackermann), una función que crece tan increíblemente lento que para cualquier cantidad de ciudades del mundo real vale 4 o menos. En resumen: no puedes bajar de n^2 porque solo *leer* la matriz completa ya cuesta eso.

Consejos para la entrevista (Amazon). Este problema es hermano de **323. Number of Connected Components**: lo único que cambia es cómo te entregan las conexiones (una matriz en vez de una lista de parejas); por dentro es el mismo Union-Find. También se puede resolver con DFS/BFS: por cada ciudad que no hayas visitado, sumas una provincia y “te contagias” visitando a todos sus amigos, y a los amigos de sus amigos, marcándolos como vistos (igual que se “hunde” la tierra en **200. Number of Islands**). Vale la pena mencionar ambos: DFS/BFS es más fácil de explicar cuando la entrada ya es una matriz, pero Union-Find gana si el entrevistador pregunta “¿y si las conexiones llegan una por una en vez de todas de golpe?”. Ojo con el costo: si n es grande, $O(n^2)$ es simplemente lo que cuesta leer la matriz, así que ninguna técnica baja de ahí; con pocas conexiones convendría recibir una lista de parejas en lugar de la matriz completo. Casos raros: una sola ciudad (1 provincia), ninguna ciudad conectada con otra (cada una es su propia provincia) y todas conectadas entre sí (1 provincia).

743. Network Delay Time

(M)

Código fuente: `src/01-grafos/743-network-delay-time.cpp`▷ Ver animación · [local](#) | [web](#)

Hay n nodos numerados de 1 a n , conectados por aristas *dirigidas y con peso*: `times[i] = [u, v, w]` significa que una señal enviada desde u tarda w unidades de tiempo en llegar a v . Enviamos una señal desde el nodo k . Queremos el tiempo mínimo que tarda en que la señal llegue a *todos* los nodos, o -1 si algún nodo nunca la recibe.

La idea, con un ejemplo de la vida real. Imagina que quieres esparcir un chisme por un grupo de amigos. Le cuentas el chisme al amigo k , y cada amigo, en cuanto se entera, se lo pasa a otros; pero cada llamada tarda un rato distinto (unas personas contestan rápido, otras tardan). Queremos saber en qué momento se entera *el último* amigo, porque hasta entonces todo el grupo ya lo sabe.

En problemas anteriores todas las conexiones “costaban” lo mismo (un paso), así que bastaba avanzar por capas con BFS. Aquí cada conexión tarda distinto, y avanzar por capas ya no sirve: el amigo que descubres primero no siempre es al que el chisme le llega antes en tiempo real. Para eso usamos el algoritmo de **Dijkstra**: en vez de una fila normal, usamos una *min-heap* (o cola de prioridad), que es como una fila mágica que siempre deja pasar primero al amigo que se va a enterar más temprano.

```
2 --1--> 1
2 --1--> 3 --1--> 4
```

Con `times = [[2,1,1],[2,3,1],[3,4,1]]`, $n = 4$, $k = 2$: desde 2 la señal llega a 1 en tiempo 1, a 3 en tiempo 1 y a 4 (pasando por 3) en tiempo 2. El nodo más lento en enterarse es 4, así que la respuesta es 2.

Cómo lo resuelve el código, paso a paso. Guardamos, para cada amigo, en cuánto tiempo se entera del chisme en un arreglo `dist`. Al principio nadie sabe nada, así que todos empiezan en “infinito” (un número gigante que significa “todavía no se entera”), salvo el amigo k , que arranca en 0 porque él ya lo sabe.

1. Metemos a k en el min-heap con tiempo 0.
2. Sacamos de la fila al amigo con el menor tiempo pendiente. Si el tiempo con el que sale ya está viejo (porque mientras tanto encontramos una forma más rápida de que se enterara), lo ignoramos.
3. Si no, miramos a cada vecino al que ese amigo le puede pasar el chisme. Calculamos a qué hora se enteraría el vecino pasando por este amigo. Si es más temprano que lo que teníamos apuntado, actualizamos su `dist` y lo metemos a la fila. A esto se le llama *relajar* una conexión: encontrar un camino que llega antes y quedarnos con él.
4. Repetimos hasta vaciar la fila.

¿Por qué funciona? Como ningún tiempo es negativo, la primera vez que un amigo sale de la fila su hora ya es la definitiva: cualquier otro camino tendría que pasar por alguien que se entera igual o más tarde, y sumarle más tiempo solo lo empeora. (Si hubiera tiempos negativos esta garantía se rompería, y haría falta otro método llamado Bellman-Ford.)

Al final, si algún amigo se quedó en “infinito” es porque el chisme nunca le llegó: devolvemos -1. Si todos tienen una hora real, la respuesta es la *más grande* de todas: el grupo entero se entera justo cuando se entera el más lento.

```

1 //
  =====
2 // 743. Network Delay Time (Medium)
3 //
  -----
4 // Hay n nodos numerados de 1 a n. Nos dan times[i] = [u, v, w]: una señal que
5 // sale de u tarda w unidades de tiempo en llegar a v (arista DIRIGIDA y
6 // PONDERADA). Enviamos una señal desde el nodo k. Devuelve el tiempo mínimo
7 // que tarda en que la señal llegue a TODOS los n nodos, o -1 si algún nodo
8 // nunca la recibe.
9 //
10 // Ejemplo: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
11 //
12 //      2 --1--> 1
13 //      |
14 //      1
15 //      v
16 //      3 --1--> 4
17 //
18 // Desde 2: a 1 le llega en 1, a 3 en 1, a 4 (via 3) en 1+1 = 2.
19 // El nodo que tarda más es 4, con tiempo 2 => respuesta 2.
20 //
21 // Diagrama de Dijkstra (cola de prioridad, min-heap por distancia):
22 //
23 // dist[k] = 0, resto = infinito
24 // heap: (0, k)
25 // mientras el heap no esté vacío:
26 //   sacar (d, u) con menor d
27 //   si d > dist[u] ya conocido, es una entrada obsoleta: ignorar
28 //   para cada arista (u -> v, peso w):
29 //     si dist[u] + w < dist[v]: // relajación de la arista
30 //       dist[v] = dist[u] + w
31 //       heap.push((dist[v], v))
32 //
33 // Idea: es un BFS "por distancia" en vez de por número de aristas. Como los
34 // pesos no son todos iguales, no basta con procesar por capas (como en BFS
35 // normal): hay que procesar siempre el nodo con la MENOR distancia acumulada
36 // todavía no confirmada, y para eso sirve el min-heap. Esto es Dijkstra:
37 // mientras las aristas sean todas de peso >= 0, la primera vez que sacamos
38 // un nodo del heap su distancia ya es la mínima posible (no puede mejorar
39 // después, porque cualquier otro camino que pase por un nodo con distancia
40 // mayor solo puede sumar más peso, nunca restar). Si hubiera pesos negativos
41 // esta garantía se rompe y haría falta Bellman-Ford.
42 // El heap puede tener entradas duplicadas u obsoletas para un mismo nodo
43 // (llegamos a él por varios caminos); las descartamos comparando contra la
44 // distancia ya confirmada en dist[].
45 //
46 // Al final, si algún nodo se quedó en infinito, es inalcanzable => -1.

```

```

47 // Si no, la respuesta es la mayor de las distancias mínimas (el último nodo
48 // en enterarse es el que determina cuánto tarda la red completa).
49 //
50 // Complejidad: tiempo  $O(E \log V)$ , espacio  $O(V + E)$ .
51 //
=====
52 #include <vector>
53 #include <queue>
54 #include <climits>
55 #include <cassert>
56 #include <iostream>
57 using namespace std;
58
59 class Solution {
60 public:
61     int networkDelayTime(vector<vector<int>>& times, int n, int k) {
62         // Lista de adyacencia: adj[u] = {(v, peso), ...}
63         vector<vector<pair<int,int>>> adj(n + 1);
64         for (auto& t : times) {
65             int u = t[0], v = t[1], w = t[2];
66             adj[u].push_back({v, w});
67         }
68
69         vector<int> dist(n + 1, INT_MAX);
70         dist[k] = 0;
71
72         // Min-heap por distancia: pair<distancia, nodo>.
73         priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> heap;
74         heap.push({0, k});
75
76         while (!heap.empty()) {
77             auto [d, u] = heap.top();
78             heap.pop();
79             if (d > dist[u]) continue; // entrada obsoleta, ya mejoramos u
80
81             for (auto& [v, w] : adj[u]) {
82                 if (dist[u] + w < dist[v]) { // relajación de la arista
83                     dist[v] = dist[u] + w;
84                     heap.push({dist[v], v});
85                 }
86             }
87         }
88
89         int maxDist = 0;
90         for (int nodo = 1; nodo <= n; ++nodo) {
91             if (dist[nodo] == INT_MAX) return -1; // nodo inalcanzable
92             maxDist = max(maxDist, dist[nodo]);
93         }
94         return maxDist;
95     }
96 };
97
98 int main() {

```

```

99     Solution s;
100
101     // Caso normal: la señal llega a todos, el nodo 4 es el más lento (t=2).
102     vector<vector<int>> t1 = {{2,1,1},{2,3,1},{3,4,1}};
103     assert(s.networkDelayTime(t1, 4, 2) == 2);
104
105     // Nodo 2 es inalcanzable desde 1 (solo hay arista 1 -> 2, no llega a 2
106     // el resto de nodos aunque exista arco 1->2... aquí forzamos un nodo sin
107     // ningún arco de entrada).
108     vector<vector<int>> t2 = {{1,2,1}};
109     assert(s.networkDelayTime(t2, 3, 1) == -1);    // nodo 3 nunca recibe la se
        ñal
110
111     // Un solo nodo: la señal ya está en k, tarda 0.
112     vector<vector<int>> t3 = {};
113     assert(s.networkDelayTime(t3, 1, 1) == 0);
114
115     // Camino donde el orden de procesamiento del heap importa: hay una ruta
116     // directa cara y una ruta indirecta barata.
117     vector<vector<int>> t4 = {{1,2,5},{1,3,1},{3,2,1}};
118     // Desde 1: directo a 2 cuesta 5, pero via 3 cuesta 1+1=2 (mejor).
119     assert(s.networkDelayTime(t4, 3, 1) == 2);
120
121     cout << "743 Network Delay Time: todas las pruebas pasaron.\n";
122     return 0;
123 }

```

Complejidad: tiempo $O(E \log V)$, espacio $O(V + E)$.

¿Qué significa esa complejidad? V es el número de amigos (nodos) y E el número de conexiones (aristas). $O(E \log V)$ significa que revisamos cada conexión y, cada vez que metemos algo en el min-heap, mantenerla ordenada cuesta un poquito ($\log V$, que crece muy despacio). En pocas palabras: es casi tan rápido como mirar todas las conexiones una vez, con un pequeño costo extra por tener la fila siempre ordenada.

Consejos para la entrevista (Amazon). Este es el ejemplo estrella de Dijkstra con cola de prioridad. Suelen usarlo para ver si entiendes *por qué* el algoritmo necesita que los tiempos no sean negativos (conviene mencionarlo aunque no te lo pregunten). Otro punto que evalúan: cómo manejas las entradas “viejas” del min-heap. Como en la cola de prioridad de C++ no hay una forma barata de bajarle el valor a algo que ya metiste, simplemente metemos duplicados y, al sacarlos, descartamos los que ya no coinciden con `dist`. Casos raros: un solo amigo (k se entera en tiempo 0), un amigo al que nadie le puede llamar (nunca se entera, -1), y la trampa de creer que el camino con menos saltos es siempre el más rápido cuando en realidad hay que comparar el tiempo acumulado.

787. Cheapest Flights Within K Stops

(M)

Código fuente: `src/01-grafos/787-cheapest-flights-k-stops.cpp`▷ Ver animación · [local](#) | [web](#)

Hay n ciudades numeradas de 0 a $n-1$ y una lista `flights` de vuelos `[from, to, price]` (aristas dirigidas y ponderadas, sin pesos negativos). Dados `src`, `dst` y `k`, buscamos el precio más barato de `src` a `dst` usando a lo más k escalas (paradas intermedias), o -1 si no existe tal ruta. Usar a lo más k escalas equivale a usar a lo más $k+1$ vuelos (aristas) en el camino.

La idea, con un ejemplo de la vida real. Quieres volar de una ciudad a otra lo más barato posible, pero con un límite: como mucho k *escalas* (paradas intermedias). Una ruta con muchas escalas puede salir más barata, pero si se pasa del límite no te sirve, aunque cueste menos.

Podríamos pensar en Dijkstra (el del problema anterior), pero Dijkstra solo se fija en el precio total y no lleva la cuenta de *cuántos vuelos* usaste. Y aquí eso importa muchísimo: dos rutas que cuestan igual no valen lo mismo si una usa más escalas que la otra. Necesitamos un método que avance *un vuelo a la vez* y nos deje detenernos justo después de $k+1$ vuelos.

```
n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]
0 --100--> 1 --100--> 2 --200--> 3
1 -----600-----> 3
```

Con `src=0`, `dst=3`, `k=1`: la ruta $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ cuesta $100 + 100 + 200 = 400$ pero usa 2 escalas (nodos 1 y 2), más de las permitidas. La única ruta con a lo más 1 escala es $0 \rightarrow 1 \rightarrow 3$, que cuesta $100 + 600 = 700$. Respuesta: 700.

Cómo lo resuelve: Bellman-Ford por rondas (patrón nuevo). *Bellman-Ford* es otro algoritmo de caminos. Su idea es sencilla: hace *rondas*, y en cada ronda revisa **todos** los vuelos e intenta abaratar el precio de llegar a cada ciudad. Lo bonito es esto: después de r rondas, el precio apuntado para cada ciudad es lo más barato que se puede llegar *usando a lo más r vuelos*. Como k escalas son $k+1$ vuelos, hacemos **exactamente $k+1$ rondas**: ni una más (usaríamos más escalas de las permitidas) ni una menos (nos quedaríamos cortos).

El truco de la copia (la parte clave). En cada ronda trabajamos sobre una *fotografía congelada* de los precios de la ronda anterior. Es decir: construimos un arreglo nuevo, `nuevaDist`, mirando siempre los precios “de ayer” (`dist`), y solo al terminar toda la ronda reemplazamos los viejos por los nuevos.

¿Por qué? Si actualizáramos los precios “en el momento” (como hace Dijkstra), una sola ronda podría encadenar varios vuelos seguidos: abaratar $0 \rightarrow 1$ y, en esa misma ronda, usar ya el nuevo precio de 1 para abaratar $1 \rightarrow 2$. Eso metería dos vuelos (dos escalas) de contrabando en una sola ronda, y daría una ruta más barata pero *tramposa*, porque usa más escalas de las permitidas. Al copiar la fotografía, cada ronda solo puede sumar *exactamente un* vuelo por camino, sin importar en qué orden revisemos la lista de vuelos.

```
1 //
  =====
2 // 787. Cheapest Flights Within K Stops (Medium)
```



```

53 // que iteremos las aristas.
54 //
55 // Complejidad: tiempo  $O(k \cdot E)$  ( $k+1$  rondas, cada una recorre las  $E$  aristas),
56 //                espacio  $O(n)$  para los arreglos de distancia.
57 //
=====

58 #include <vector>
59 #include <cassert>
60 #include <iostream>
61 using namespace std;
62
63 class Solution {
64 public:
65     int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int
        dst, int k) {
66         const int INF = 1e9;
67         vector<int> dist(n, INF);
68         dist[src] = 0;
69
70         //  $k+1$  rondas: cubrimos caminos de hasta  $k$  escalas ( $k+1$  aristas).
71         for (int ronda = 0; ronda <= k; ++ronda) {
72             vector<int> nuevaDist = dist;    // copia: fotografía de la ronda
              anterior
73             for (auto& f : flights) {
74                 int u = f[0], v = f[1], precio = f[2];
75                 if (dist[u] == INF) continue;    // u todavía inalcanzable en
              esta ronda
76                 if (dist[u] + precio < nuevaDist[v]) {
77                     nuevaDist[v] = dist[u] + precio;
78                 }
79             }
80             dist = nuevaDist;
81         }
82
83         return dist[dst] == INF ? -1 : dist[dst];
84     }
85 };
86
87 int main() {
88     Solution s;
89
90     // Caso clásico: con  $k=1$  el camino directo  $0 \rightarrow 1 \rightarrow 3$  ( $600+100=700$ ) gana
91     // porque  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$  usa 2 escalas y no está permitido.
92     vector<vector<int>> f1 =
93         {{0,1,100},{1,2,100},{2,0,100},{1,3,600},{2,3,200}};
94     assert(s.findCheapestPrice(4, f1, 0, 3, 1) == 700);
95
96     // Con  $k=0$  (sin escalas) y sin vuelo directo  $0 \rightarrow 3$ , es imposible.
97     assert(s.findCheapestPrice(4, f1, 0, 3, 0) == -1);
98
99     // Con más escalas permitidas ( $k=4$ , sobran rondas) sí se puede aprovechar
100    // la ruta de 2 escalas  $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ , más barata ( $100+100+200=400$ ).
    assert(s.findCheapestPrice(4, f1, 0, 3, 4) == 400);

```

```
101 // Ejemplo clásico de LeetCode: hay una ruta directa cara y una ruta con
102 // una escala más barata.
103 vector<vector<int>> f2 = {{0,1,100},{1,2,100},{0,2,500}};
104 assert(s.findCheapestPrice(3, f2, 0, 2, 1) == 200); // 0->1->2 = 200
105 assert(s.findCheapestPrice(3, f2, 0, 2, 0) == 500); // solo directo: 500
106
107 // Destino totalmente inalcanzable, sin importar cuántas escalas demos.
108 vector<vector<int>> f3 = {{0,1,100}};
109 assert(s.findCheapestPrice(3, f3, 0, 2, 1) == -1);
110
111 // src == dst: el costo es 0 sin necesidad de volar.
112 assert(s.findCheapestPrice(3, f3, 0, 0, 1) == 0);
113
114 cout << "787 Cheapest Flights: todas las pruebas pasaron.\n";
115 return 0;
116 }
117 }
```

Complejidad: tiempo $O(k \cdot E)$, espacio $O(n)$.

¿Qué significa esa complejidad? $O(k \cdot E)$ significa que hacemos $k + 1$ rondas y en cada ronda miramos las E conexiones (los vuelos) una vez; es decir, el trabajo es simplemente “número de rondas por número de vuelos”. El espacio $O(n)$ es porque solo guardamos un precio por cada una de las n ciudades (más su copia).

Consejos para la entrevista (Amazon). Este problema es la puerta de entrada al patrón de **Bellman-Ford con rondas limitadas**, muy útil cuando el límite no es sobre el costo sino sobre el *número de aristas* del camino (aquí, las escalas). El error más común es olvidar la copia y actualizar sobre el mismo arreglo, lo que deja colar rutas con más escalas de las permitidas; Amazon suele preguntar justo por qué esa copia es necesaria. Otro tropiezo es confundir “ k escalas” con “ k vuelos”: hay que hacer $k+1$ rondas, no k . Casos raros: `src == dst` (cuesta 0, ni siquiera vuelas), destino imposible de alcanzar con cualquier número de escalas (-1) y una k tan grande que el límite ya no estorba (el algoritmo simplemente termina de calcular el más barato de todos, sin que las rondas de más dañen el resultado).

133. Clone Graph

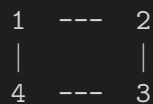
(M)

Código fuente: [src/01-grafos/133-clone-graph.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan una referencia a un nodo de un grafo no dirigido y *conexo*. Cada nodo tiene un valor entero (`val`) y una lista de vecinos (`neighbors`). Queremos devolver una **copia profunda** de todo el grafo: cada nodo de la copia debe ser un objeto nuevo, con el mismo valor y las mismas conexiones que el original.

La idea, con un ejemplo de la vida real. Imagina que tienes un grupo de amigos dibujado con líneas que unen a quienes se conocen, y quieres hacer una *copia nueva* del grupo entero: personas nuevas, de carne y hueso, pero con los mismos nombres y las mismas amistades. A eso se le llama **copia profunda**: no vale con señalar a los amigos originales, hay que crear a cada uno de nuevo.

Recorremos el grupo con un *DFS* (ir metiéndonos por un amigo, luego por un amigo de ese amigo, y así hasta el fondo antes de regresar). El único problema es que las amistades forman *ciclos*: A conoce a B, y B conoce a A. Si no tenemos cuidado, clonaríamos a A, luego a B, luego a A otra vez, luego a B otra vez...para siempre. La solución es llevar una *libreta* que anota, para cada persona original, cuál es su copia ya hecha. Antes de recorrer los amigos de alguien, apuntamos su copia en la libreta; así, si volvemos a toparnos con él, simplemente usamos la copia que ya está anotada en vez de crearla de nuevo.



Lista de adyacencia de este ejemplo: 1 : [2, 4], 2 : [1, 3], 3 : [2, 4], 4 : [1, 3]. Al clonar desde el nodo 1, el DFS visita 2, que a su vez intenta visitar 1 otra vez: como 1 ya está en el mapa, se reutiliza su copia en lugar de recrearla.

Cómo lo resuelve el código, paso a paso. Usamos un mapa `unordered_map<Node*, Node*>` como esa libreta: dado un nodo original, te dice cuál es su copia. La función recursiva, al llegar a un nodo, hace:

1. Si el nodo ya está en la libreta, devolvemos directamente su copia (ya lo clonamos antes). Esto es lo que rompe los ciclos y evita la recursión infinita.
2. Si no está, creamos su copia (mismo valor) y la anotamos en la libreta *de inmediato*, antes de tocar a sus vecinos.
3. Recorremos cada vecino del original, lo clonamos con la misma función y vamos agregando esas copias a la lista de vecinos de nuestra copia.
4. Devolvemos la copia ya con todos sus vecinos conectados.

```

1 //
  =====
2 // 133. Clone Graph (Medium)
3 //
  =====

```



```

4  // Nos dan una referencia a un nodo de un grafo no dirigido y CONEXO. Cada
5  // nodo tiene un valor entero y una lista de vecinos. Debemos devolver una
6  // COPIA PROFUNDA (deep copy) de todo el grafo: cada nodo de la copia debe ser
7  // un objeto nuevo (distinta dirección de memoria), con el mismo valor y las
8  // mismas conexiones que su original.
9  //
10 // Ejemplo (grafo cuadrado de 4 nodos):
11 //
12 //   1 -- 2           Lista de adyacencia:
13 //   |     |           1: [2, 4]
14 //   4 -- 3           2: [1, 3]
15 //                     3: [2, 4]
16 //                     4: [1, 3]
17 //
18 // Diagrama del DFS con mapa original -> copia:
19 //
20 //   visitar(1) -> ¿existe copia de 1? no: la creamos, la guardamos en el mapa
21 //               y recorremos sus vecinos [2, 4]
22 //               visitar(2) -> creamos copia de 2, recorremos [1, 3]
23 //               visitar(1) -> YA existe copia de 1 en el mapa:
24 //                           la devolvemos sin recrearla (evita
25 //                           recursión infinita en el ciclo)
26 //               visitar(3) -> creamos copia de 3, recorremos [2, 4]
27 //               visitar(2) -> ya existe, se devuelve del mapa
28 //               visitar(4) -> creamos copia de 4, recorremos [1,
29 //               3]
30 //                           (1 y 3 ya existen, se devuelven del mapa)
31 //
32 // Idea: DFS clásico, pero en vez de un simple "visitado" usamos un
33 // unordered_map<Node*, Node*> que traduce cada nodo original a su copia. La
34 // primera vez que visitamos un nodo original creamos su copia y la
35 // registramos en el mapa ANTES de recorrer sus vecinos; así, si el grafo
36 // tiene un ciclo (como el cuadrado de arriba), al volver a toparnos con un
37 // nodo ya visitado simplemente reutilizamos la copia guardada en vez de
38 // clonar de nuevo o entrar en un bucle infinito.
39 //
40 // Complejidad: tiempo  $O(V+E)$  (visitamos cada nodo y cada arista una vez),
41 // espacio  $O(V)$  por el mapa y la pila de recursión.
42 //
43 =====
44
45 #include <vector>
46 #include <unordered_map>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 // Definición del nodo del grafo (la da el enunciado de LeetCode).
52 struct Node {
53     int val;
54     vector<Node*> neighbors;
55     Node() : val(0), neighbors(vector<Node*>()) {}
56     Node(int _val) : val(_val), neighbors(vector<Node*>()) {}
57     Node(int _val, vector<Node*> _neighbors) : val(_val), neighbors(_neighbors

```

```

    ) {}
55 };
56
57 class Solution {
58 public:
59     Node* cloneGraph(Node* nodo) {
60         if (nodo == nullptr) return nullptr;    // grafo vacío => copia vacía
61         unordered_map<Node*, Node*> copias;      // original -> copia
62         return clonar(nodo, copias);
63     }
64
65 private:
66     Node* clonar(Node* original, unordered_map<Node*, Node*>& copias) {
67         auto it = copias.find(original);
68         if (it != copias.end()) return it->second;    // ya clonado: reusar
69
70         Node* copia = new Node(original->val);
71         copias[original] = copia;    // registrar ANTES de recorrer vecinos
72                                     // (rompe los ciclos del grafo)
73         for (Node* vecino : original->neighbors) {
74             copia->neighbors.push_back(clonar(vecino, copias));
75         }
76         return copia;
77     }
78 };
79
80 // -----
81 // Utilidades de prueba: liberar la memoria de un grafo clonado y comparar
82 // que dos grafos (original y copia) tengan la misma estructura pero nodos
83 // con direcciones de memoria distintas.
84 // -----
85
86 // Verifica recursivamente que "copia" sea una réplica fiel de "original":
87 // mismos valores, misma cantidad de vecinos en el mismo orden, pero cada
88 // nodo de la copia debe ser un puntero DISTINTO al original correspondiente.
89 void verificarClonSonIguales(Node* original, Node* copia,
90                             unordered_map<Node*, Node*>& visitados) {
91     assert(original != nullptr && copia != nullptr);
92     assert(original != copia);    // direcciones distintas
93     assert(original->val == copia->val);    // mismo valor
94     assert(original->neighbors.size() == copia->neighbors.size());
95
96     visitados[original] = copia;
97     for (size_t i = 0; i < original->neighbors.size(); ++i) {
98         Node* vecinoOriginal = original->neighbors[i];
99         Node* vecinoCopia = copia->neighbors[i];
100         if (visitados.count(vecinoOriginal)) {
101             // Ya visitado: sólo confirmamos que apunte a la MISMA copia
102             // (así comprobamos que no se duplicó el clon de un nodo).
103             assert(visitados[vecinoOriginal] == vecinoCopia);
104         } else {
105             verificarClonSonIguales(vecinoOriginal, vecinoCopia, visitados);
106         }
107     }

```

```

108 }
109
110 // Libera todos los nodos alcanzables desde "raiz" (evita fugas de memoria
111 // entre las distintas pruebas del main).
112 void liberarGrafo(Node* raiz) {
113     if (raiz == nullptr) return;
114     unordered_map<Node*, bool> visitados;
115     vector<Node*> pila = {raiz};
116     visitados[raiz] = true;
117     vector<Node*> todos;
118     while (!pila.empty()) {
119         Node* actual = pila.back();
120         pila.pop_back();
121         todos.push_back(actual);
122         for (Node* vecino : actual->neighbors) {
123             if (!visitados.count(vecino)) {
124                 visitados[vecino] = true;
125                 pila.push_back(vecino);
126             }
127         }
128     }
129     for (Node* n : todos) delete n;
130 }
131
132 int main() {
133     Solution s;
134
135     // --- Caso 1: grafo cuadrado de 4 nodos (1-2-3-4-1), con ciclo ---
136     Node* n1 = new Node(1);
137     Node* n2 = new Node(2);
138     Node* n3 = new Node(3);
139     Node* n4 = new Node(4);
140     n1->neighbors = {n2, n4};
141     n2->neighbors = {n1, n3};
142     n3->neighbors = {n2, n4};
143     n4->neighbors = {n1, n3};
144
145     Node* copia1 = s.cloneGraph(n1);
146     assert(copia1 != nullptr);
147     assert(copia1 != n1);    // el nodo raíz de la copia es un objeto distinto
148     {
149         unordered_map<Node*, Node*> visitados;
150         verificarClonSonIguales(n1, copia1, visitados);
151         assert(visitados.size() == 4);    // se clonaron exactamente 4 nodos
152     }
153     liberarGrafo(copia1);
154     liberarGrafo(n1);
155
156     // --- Caso 2: un solo nodo, sin vecinos ---
157     Node* solo = new Node(1);
158     Node* copiaSolo = s.cloneGraph(solo);
159     assert(copiaSolo != nullptr);
160     assert(copiaSolo != solo);
161     assert(copiaSolo->val == 1);

```

```
162     assert(copiaSolo->neighbors.empty());
163     liberarGrafo(copiaSolo);
164     liberarGrafo(solo);
165
166     // --- Caso 3: grafo nulo (vacío) ---
167     assert(s.cloneGraph(nullptr) == nullptr);
168
169     cout << "133 Clone Graph: todas las pruebas pasaron.\n";
170     return 0;
171 }
```

Complejidad: tiempo $O(V + E)$, espacio $O(V)$.

¿Qué significa esa complejidad? V es el número de nodos (personas) y E el de aristas (amistades). $O(V + E)$ significa que tocamos cada persona una vez y recorremos cada amistad una vez: no se puede copiar el grupo sin mirarlo entero al menos una vez. El espacio $O(V)$ es por la libreta (una entrada por persona) más la pila de la recursión.

Consejos para la entrevista (Amazon). El error típico es anotar la copia en la libreta *después* de recorrer a los vecinos: eso provoca recursión infinita en cuanto hay un ciclo (y casi siempre lo hay, porque el grafo es no dirigido). Menciona también la versión con BFS y una cola (la misma idea de la libreta, pero sin recursión, útil si te preocupa que la pila se llene en grafos enormes). Casos raros: nodo nulo (grafo vacío) devuelve `nullptr`, un solo nodo sin vecinos, y comprueba que las copias sean objetos de verdad nuevos (direcciones de memoria distintas a las del original), no un simple “apodo” que apunta al mismo lugar.

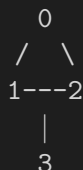
1192. Critical Connections in a Network

(H)

Código fuente: [src/01-grafos/1192-critical-connections.cpp](#)▷ Ver animación · [local](#) | [web](#)

Hay n servidores numerados de 0 a $n - 1$, conectados por una lista de **connections** (aristas **no dirigidas**). El grafo es *conexo*: siempre se puede llegar de cualquier servidor a cualquier otro. Una conexión es **crítica** (un *punte*) si, al quitarla, el grafo se parte en dos o más componentes; es decir, es la única forma en que un grupo de servidores se comunica con el resto. Debemos devolver **todas** las conexiones críticas.

La idea, con un ejemplo de la vida real. Piensa en cada conexión como un cable físico entre dos servidores. Si un cable forma parte de un *ciclo* (o sea, hay otro camino alternativo para ir de un extremo al otro dando la vuelta), quitarlo no deja a nadie incomunicado: la información se va por el otro lado. Pero si un cable es la *única* manera de llegar a cierta parte de la red —como una rama que cuelga y solo se sostiene de ahí— quitarlo sí la desconecta. Ese cable tan importante es lo que llamamos un **punte**, y son justo los que queremos encontrar.



Ejemplo con $n = 4$, **connections** = $[[0, 1], [1, 2], [2, 0], [1, 3]]$: el triángulo 0-1-2 tiene tres rutas distintas entre cualquier par de sus nodos, así que ninguna de sus tres aristas es puente. Pero 1-3 es la *única* forma de llegar al servidor 3: quitarla lo aísla. La única conexión crítica es $\{1, 3\}$.

Cómo lo resuelve el código, paso a paso: puentes de Tarjan. Este es un patrón nuevo en el libro, así que vamos con calma. Hacemos un *DFS* (recorrer la red metiéndonos hasta el fondo por un camino antes de regresar) empezando desde cualquier servidor, y a cada nodo le anotamos dos números:

- **disc[u]**: el *orden de llegada*, es decir, en qué turno el DFS visitó a u por primera vez. El primero que visitamos tiene **disc** = 0, el segundo **disc** = 1, y así. Una vez puesto, este número ya no cambia. Piénsalo como el número que te dan al llegar a una fila.
- **low[u]**: el *turno más antiguo* (el **disc** más pequeño) al que se puede “trepar de regreso” saliendo desde u o desde cualquier nodo que cuelgue debajo de él, usando los cables del recorrido y, como mucho, **un** cable “de atajo” que salte hacia un nodo visitado antes (a esos atajos se les llama *aristas de retroceso*). Este número sí se va ajustando (solo puede bajar) conforme el DFS regresa de sus hijos.

La idea es esta: si desde la parte que cuelga debajo de un hijo v existe un atajo que trepa hasta u o más arriba, entonces esa parte tiene otra “cuerda” que la sostiene y el cable (u, v) no es imprescindible: no es puente. En cambio, si no hay ningún atajo así, ese cable es la única entrada y salida, y sí es puente. En fórmula:

$$(u, v) \text{ es puente} \iff \text{low}[v] > \text{disc}[u]$$

que se lee: “si desde todo lo que cuelga de v es imposible trepar hasta u o algún ancestro suyo sin volver a usar el cable (u, v) , entonces ese cable es un puente”.

En el ejemplo de arriba: al visitar 2 (hijo de 1) encontramos un atajo hacia 0, que ya estaba visitado y no es su padre, así que $\text{low}[2]$ baja hasta $\text{disc}[0] = 0$; ese 0 se contagia hacia arriba y deja $\text{low}[1] = 0$ y $\text{low}[0] = 0$. Ni $(0, 1)$ ni $(1, 2)$ son puente, porque desde ellos sí se puede trepar hasta el turno 0. En cambio, 3 no tiene ningún atajo (su único vecino es su padre, 1), así que $\text{low}[3]$ se queda en su propio turno $\text{disc} = 3$; como $3 > \text{disc}[1] = 1$, el cable $(1, 3)$ sí es un puente.

```

1 //
2 // =====
3 //
4 // 1192. Critical Connections in a Network (Hard)
5 //
6 // -----
7 // Hay n servidores numerados de 0 a n-1 conectados por "connections" (aristas
8 // NO dirigidas). El grafo es conexo (se puede llegar de cualquier servidor a
9 // cualquier otro). Una conexión es CRÍTICA (un "puente") si, al quitarla, el
10 // grafo se parte en dos o más componentes: es decir, esa arista es la ÚNICA
11 // forma de comunicar a los servidores de un lado con los del otro. Debemos
12 // devolver TODAS las conexiones críticas.
13 //
14 // Ejemplo, n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]:
15 //
16 //      0
17 //      / \
18 // 1---2      3 cuelga de 1 con una sola arista.
19 //      |
20 //      3
21 //
22 // El triángulo 0-1-2 tiene tres caminos distintos entre cualquier par de sus
23 // nodos (quitar una arista del triángulo no lo desconecta), así que NINGUNA
24 // arista del triángulo es puente. Pero 1-3 es la ÚNICA forma de llegar a 3:
25 // si la quitamos, el servidor 3 queda aislado. Por eso [[1,3]] es la única
26 // conexión crítica.
27 //
28 // Técnica: PUENTES DE TARJAN, una variante del DFS clásico con dos arreglos
29 // nuevos en este libro:
30 //
31 // disc[u] = el "número de orden" en que el DFS descubrió (visitó por
32 // primera vez) al nodo u. Es simplemente un contador global que
33 // aumenta cada vez que visitamos un nodo nuevo: el primer nodo
34 // visitado tiene disc = 0, el segundo disc = 1, etc. Una vez
35 // asignado, disc[u] YA NO CAMBIA.
36 //
37 // low[u] = el disc MÁX PEQUEÑO al que u puede "regresar" usando, como
38 // máximo, UNA arista de retroceso (back-edge) hacia un ancestro
39 // en el árbol de DFS. Es decir: partiendo de u, moviéndonos por
40 // aristas del árbol hacia descendientes y (a lo más) una arista
41 // que salte hacia arriba a un ancestro ya visitado, ¿qué tan
42 // "arriba" en el orden de descubrimiento podemos llegar? Este
43 // valor SÍ se va actualizando (sólo baja, nunca sube) conforme
44 // el DFS regresa de sus hijos.
45 //

```

```

43 // Diagrama del árbol de DFS para el ejemplo (raíz = 0):
44 //
45 //         0   disc=0
46 //         |
47 //         1   disc=1
48 //         /|
49 //        2 3   disc=2   disc=3
50 //         |
51 //        (back-edge a 0)
52 //
53 // Al procesar 2, encontramos una arista de retroceso hacia 0 (ya visitado,
54 // y no es el padre de 2). Eso significa que 2 puede "regresar" hasta
55 // disc=0 sin usar la arista 1-2 ni la 0-1: por eso low[2] = 0. Ese 0 se
56 // propaga hacia arriba: low[1] = min(low[1], low[2]) = 0, y luego
57 // low[0] = min(low[0], low[1]) = 0. Así, ni la arista 0-1 ni la 1-2 son
58 // puente, porque el subárbol de abajo tiene una "cuerda" (back-edge) que
59 // lo mantiene conectado al resto incluso sin esa arista del árbol.
60 //
61 // En cambio, 3 no tiene ninguna arista de retroceso (su único vecino es su
62 // padre, 1), así que low[3] se queda en su propio disc = 3. Ese 3 NO puede
63 // "escapar" hacia arriba de disc[1] = 1.
64 //
65 // La condición clave: para una arista de árbol (u, v) donde v es hijo de u,
66 //
67 //         (u,v) es puente  <=>  low[v] > disc[u]
68 //
69 // ¿Por qué funciona? low[v] > disc[u] significa que, desde el subárbol de v
70 // (usando aristas del árbol hacia abajo y a lo más una arista de retroceso),
71 // es IMPOSIBLE alcanzar a u o a cualquier ancestro de u. La única forma en
72 // que v y su subárbol se comunican con el resto del grafo es a través de la
73 // arista (u, v). Quitárla los desconecta: es un puente. Si en cambio
74 // low[v] <= disc[u], existe una "cuerda" que rodea la arista (u, v) y la
75 // vuelve prescindible.
76 //
77 // Aplicando esto al ejemplo: low[3] = 3 > disc[1] = 1 => (1,3) es puente.
78 // low[2] = 0, disc[1] = 1 => 0 > 1 es falso => (1,2) NO es puente.
79 // low[1] = 0, disc[0] = 0 => 0 > 0 es falso => (0,1) NO es puente.
80 //
81 // Al final del DFS habremos revisado cada arista del árbol exactamente una
82 // vez con esta prueba, y habremos recolectado todos los puentes.
83 //
84 // Complejidad: tiempo  $O(V + E)$  (un DFS normal, cada nodo y arista se visitan
85 // una vez), espacio  $O(V + E)$  para la lista de adyacencia más
86 //  $O(V)$  de los arreglos disc/low y la pila de recursión.
87 //
=====
88 #include <vector>
89 #include <cassert>
90 #include <algorithm>
91 #include <iostream>
92 using namespace std;
93
94 class Solution {

```

```

95 public:
96     vector<vector<int>> criticalConnections(int n, vector<vector<int>>&
        connections) {
97         vector<vector<int>> adj(n);
98         for (auto& c : connections) {
99             adj[c[0]].push_back(c[1]);
100             adj[c[1]].push_back(c[0]);
101         }
102
103         disc.assign(n, -1);
104         low.assign(n, -1);
105         timer = 0;
106         vector<vector<int>> puentes;
107
108         // El grafo es conexo (garantía del enunciado), así que un solo DFS
109         // desde el nodo 0 recorre todos los servidores.
110         dfs(0, -1, adj, puentes);
111
112         return puentes;
113     }
114
115 private:
116     vector<int> disc;    // disc[u]: momento de descubrimiento de u
117     vector<int> low;    // low[u]: disc mínimo alcanzable desde el subárbol de
        u
118     int timer;
119
120     void dfs(int u, int padre, vector<vector<int>>& adj, vector<vector<int>>&
        puentes) {
121         disc[u] = low[u] = timer++;
122
123         for (int v : adj[u]) {
124             if (v == padre) continue;    // no regresar por la misma arista al
                padre
125
126             if (disc[v] == -1) {
127                 // v aún no visitado: es hijo de u en el árbol de DFS.
128                 dfs(v, u, adj, puentes);
129                 low[u] = min(low[u], low[v]);    // propagar hacia arriba
130                 if (low[v] > disc[u]) {
131                     // Ninguna cuerda desde el subárbol de v llega hasta u
132                     // (ni más arriba): la arista (u, v) es un puente.
133                     puentes.push_back({min(u, v), max(u, v)});
134                 }
135             } else {
136                 // v ya visitado y no es el padre: es una arista de retroceso
137                 // (back-edge) hacia un ancestro. Actualiza low[u] con el
138                 // disc de ese ancestro (no con su low, por eso se usa disc[v
                ]).
139                 low[u] = min(low[u], disc[v]);
140             }
141         }
142     }
143 };

```



```

144
145 int main() {
146     Solution s;
147
148     // Normaliza (ya vienen como [min,max] desde el algoritmo) y ordena el
149     // vector de puentes para comparar sin depender del orden de recorrido.
150     auto ordenar = [](vector<vector<int>> v) {
151         sort(v.begin(), v.end());
152         return v;
153     };
154
155     // Caso 1 (el del enunciado): triángulo 0-1-2 con una arista colgante 1-3.
156     // El triángulo no tiene puentes; la única conexión crítica es 1-3.
157     vector<vector<int>> c1 = {{0,1},{1,2},{2,0},{1,3}};
158     vector<vector<int>> esperado1 = {{1,3}};
159     assert(ordenar(s.criticalConnections(4, c1)) == esperado1);
160
161     // Caso 2: camino lineal 0-1-2-3. Sin ciclos, así que TODAS las aristas
162     // son puentes (quitar cualquiera desconecta el grafo en dos pedazos).
163     vector<vector<int>> c2 = {{0,1},{1,2},{2,3}};
164     vector<vector<int>> esperado2 = {{0,1},{1,2},{2,3}};
165     assert(ordenar(s.criticalConnections(4, c2)) == esperado2);
166
167     // Caso 3: dos triángulos (0-1-2 y 3-4-5) unidos por una sola arista
168     // puente (2-3). Ningún lado del triángulo es puente, pero la conexión
169     // que los une sí lo es.
170     vector<vector<int>> c3 = {{0,1},{1,2},{2,0},{2,3},{3,4},{4,5},{5,3}};
171     vector<vector<int>> esperado3 = {{2,3}};
172     assert(ordenar(s.criticalConnections(6, c3)) == esperado3);
173
174     // Caso 4: anillo (ciclo) de 4 nodos. Cada arista tiene una ruta
175     // alternativa por el resto del anillo, así que no hay puentes.
176     vector<vector<int>> c4 = {{0,1},{1,2},{2,3},{3,0}};
177     assert(ordenar(s.criticalConnections(4, c4)).empty());
178
179     cout << "1192 Critical Connections: todas las pruebas pasaron.\n";
180     return 0;
181 }

```

Complejidad: tiempo $O(V + E)$, espacio $O(V + E)$.

¿Qué significa esa complejidad? V es el número de servidores y E el de cables. $O(V + E)$ significa que un solo recorrido DFS toca cada servidor una vez y pasa por cada cable una vez; encontrar *todos* los puentes cuesta lo mismo que recorrer la red entera una sola vez, que es lo más rápido posible.

Consejos para la entrevista (Amazon). Este problema mide si conoces algoritmos de grafos más allá del BFS/DFS básico, así que conviene explicar *para qué* sirve `low[]` y no solo recitar la fórmula. Puntos clave: (1) al usar un atajo (arista de retroceso), actualiza `low[u]` con el `disc` del nodo al que llegas, *no* con su `low`; esa distinción evita “inventar” caminos que no existen. (2) Cuidado con el cable que va hacia el padre: aquí evitamos regresar al padre comparando quién es, pero si la red permitiera dos cables idénticos entre los mismos dos servidores habría que ignorar

solo *uno* de ellos (por ejemplo, usando el número del cable en lugar del nodo padre); el enunciado de LeetCode promete que no hay cables repetidos. (3) El orden en que devuelves los puentes no importa, pero escribir cada cable como (min, max) ayuda a comparar respuestas. Casos raros: $n = 1$ sin conexiones (no hay puentes posibles) y un árbol puro (sin ningún ciclo), donde *todos* los cables son puentes.

Capítulo 2

Árboles

236. Lowest Common Ancestor of a Binary Tree

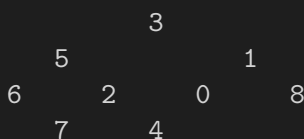
(M)

Código fuente: `src/02-arboles/236-lowest-common-ancestor.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan un árbol binario y dos nodos p y q que están garantizados presentes en el árbol. Queremos encontrar su *ancestro común más bajo* (LCA, *Lowest Common Ancestor*): el nodo más profundo que tiene tanto a p como a q entre sus descendientes (un nodo se considera descendiente de sí mismo).

La idea, con un ejemplo de la vida real. Antes que nada, unas palabras nuevas. Un *árbol binario* es como un árbol genealógico dibujado al revés: arriba está el jefe de la familia (lo llamamos *raíz*) y de él cuelgan personas. Cada persona es un *nodo*, y cada nodo puede tener como mucho dos hijos: uno a la izquierda y otro a la derecha. Un *ancestro* de alguien es cualquiera que esté por encima de él en el árbol (su papá, su abuelo, su bisabuelo...). El *ancestro común más bajo* de dos personas es el pariente compartido más cercano a ellas, el primero donde se juntan sus dos líneas familiares.

Imagina que buscas al pariente común de dos primos en ese árbol genealógico. Te paras en el jefe de la familia y preguntas: «¿alguno de los dos que busco está en mi lado izquierdo?» y «¿alguno está en mi lado derecho?». Si uno está a la izquierda y el otro a la derecha, entonces tú, justo donde estás parado, eres el punto donde se juntan sus caminos: eres la respuesta. Si los dos están del mismo lado, la respuesta está más abajo por ese lado, así que dejas que esa rama siga buscando y te avise.



En este árbol, $LCA(5, 1) = 3$ porque cada uno cuelga de una rama distinta de la raíz. En cambio $LCA(5, 4) = 5$ porque 5 es ancestro directo de 4 (4 vive dentro del propio subárbol de 5). Y $LCA(6, 4) = 5$ porque tanto 6 como 4 cuelgan del subárbol de 5, aunque en ramas distintas dentro de él.

Cómo lo resuelve el código, paso a paso. Usamos una función que se llama a sí misma para revisar cada nodo. Que una función se llame a sí misma se llama *recursión*: es como un mensajero que, para responder una pregunta, manda a dos mensajeros más pequeños (uno a la rama izquierda y otro a la derecha) y espera lo que ellos le contesten. Empezamos por abajo del todo y vamos subiendo con las respuestas; a ese orden (primero los hijos, luego el padre) se le llama *post-order*. Cada llamada devuelve «el nodo interesante que encontró»: p , q , la respuesta ya hallada, o *nada* (`nullptr`) si por ahí no había ninguno.

1. Si el nodo actual está vacío, o es p , o es q , lo devolvemos tal cual y dejamos de bajar por ahí: ya encontramos uno de los buscados (o se acabó la rama).
2. Si no, preguntamos a la rama izquierda y a la derecha.
3. Si *ambas* ramas devuelven algo, quiere decir que p y q quedaron uno a cada lado: el nodo actual es el ancestro común más bajo, así que se devuelve a sí mismo.
4. Si solo una rama devuelve algo, esa es la respuesta y la pasamos hacia arriba sin cambiarla.


```

47
48 struct TreeNode {
49     int val;
50     TreeNode *left;
51     TreeNode *right;
52     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
53 };
54
55 class Solution {
56 public:
57     TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
58         if (root == nullptr || root == p || root == q) return root;
59
60         TreeNode* izq = lowestCommonAncestor(root->left, p, q);
61         TreeNode* der = lowestCommonAncestor(root->right, p, q);
62
63         if (izq != nullptr && der != nullptr) return root; // p y q en ramas
64         // distintas
65         return (izq != nullptr) ? izq : der; // propaga el que
66         // exista
67     }
68 };
69
70 int main() {
71     Solution s;
72
73     // Árbol de prueba:
74     //           3
75     //        /  \
76     //       5    1
77     //      / \  / \
78     //     6  2 0  8
79     //        / \
80     //       7  4
81
82     TreeNode* n3 = new TreeNode(3);
83     TreeNode* n5 = new TreeNode(5);
84     TreeNode* n1 = new TreeNode(1);
85     TreeNode* n6 = new TreeNode(6);
86     TreeNode* n2 = new TreeNode(2);
87     TreeNode* n0 = new TreeNode(0);
88     TreeNode* n8 = new TreeNode(8);
89     TreeNode* n7 = new TreeNode(7);
90     TreeNode* n4 = new TreeNode(4);
91
92     n3->left = n5;    n3->right = n1;
93     n5->left = n6;    n5->right = n2;
94     n1->left = n0;    n1->right = n8;
95     n2->left = n7;    n2->right = n4;
96
97     // Caso 1: p y q en subárboles distintos de la raíz => LCA es la raíz.
98     TreeNode* r1 = s.lowestCommonAncestor(n3, n5, n1);
99     assert(r1 != nullptr && r1->val == 3);
100
101     // Caso 2: uno es ancestro directo del otro => LCA es el ancestro (5).

```

```

99     TreeNode* r2 = s.lowestCommonAncestor(n3, n5, n4);
100     assert(r2 != nullptr && r2->val == 5);
101
102     // Caso 3: ambos cuelgan del subárbol de 2 (hijo de 5) => LCA es 2.
103     TreeNode* r3 = s.lowestCommonAncestor(n3, n7, n4);
104     assert(r3 != nullptr && r3->val == 2);
105
106     // Caso 4: árbol pequeño de dos nodos, p es la raíz y q su único hijo.
107     TreeNode* a = new TreeNode(1);
108     TreeNode* b = new TreeNode(2);
109     a->left = b;
110     TreeNode* r4 = s.lowestCommonAncestor(a, a, b);
111     assert(r4 != nullptr && r4->val == 1);
112
113     // Caso 5: dos hojas hermanas => LCA es su padre directo (1).
114     TreeNode* r5 = s.lowestCommonAncestor(n3, n0, n8);
115     assert(r5 != nullptr && r5->val == 1);
116
117     delete n6; delete n7; delete n4; delete n2; delete n5;
118     delete n0; delete n8; delete n1; delete n3;
119     delete b; delete a;
120
121     cout << "236 Lowest Common Ancestor: todas las pruebas pasaron.\n";
122     return 0;
123 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

¿Qué significa esa complejidad? Aquí n es la cantidad de nodos y h es la *altura* del árbol (cuántos pisos tiene desde la raíz hasta el nodo más hondo). El tiempo $O(n)$ quiere decir que, en el peor caso, miramos cada nodo una sola vez. La memoria $O(h)$ es porque la recursión va apilando llamadas mientras baja: nunca hay más llamadas amontonadas que la profundidad del árbol, como una torre de platos que solo llega tan alto como el árbol es hondo.

Consejos para la entrevista (Amazon). Es una pregunta muy común. Un detalle bonito que puedes mencionar: no comparamos los números de los nodos, sino su «dirección en memoria» (el puntero), así que funciona aunque haya valores repetidos. Otra cosa: si el árbol estuviera ordenado (un *árbol de búsqueda binario*, donde los valores menores van a la izquierda y los mayores a la derecha), habría un truco aún más rápido que baja directo hacia el lado correcto sin revisar las dos ramas. No olvides los casos raros: que p sea ancestro de q (o al revés), o que los dos sean hermanos que cuelgan del mismo padre.

1644. Lowest Common Ancestor of a Binary Tree II

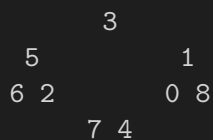
(M)

Código fuente: `src/02-arboles/1644-lca-ii.cpp`▷ Ver animación · [local](#) | [web](#)

Es la variante del problema 236 donde **p y/o q pueden NO existir** en el árbol. Si alguno de los dos falta, debemos devolver `nullptr` en vez de un ancestro cualquiera.

La idea, con un ejemplo de la vida real. Recuerda que el *ancestro común más bajo* de dos nodos es el pariente compartido más cercano a ambos en el árbol (piensa en un árbol genealógico). En el problema 236 nos prometían que las dos personas buscadas sí estaban en la familia, así que en cuanto encontrábamos a una podíamos confiar y detenernos. Aquí ya no hay esa promesa: puede que una de las dos, o las dos, ni siquiera existan en el árbol.

Imagina que buscas a dos alumnos en la lista de una escuela para saber quién es su tutor común. Si te detienes en cuanto encuentras al primero y resulta que el segundo no está inscrito, darías un tutor equivocado. Para no equivocarte, revisas la lista *entera* y vas apuntando en un contador cuántos de los dos encontraste de verdad. Solo si el contador llega a 2 (los dos existen) te crees la respuesta; si es 0 o 1, falta alguno y contestas «nada» (`nullptr`).



Con este árbol: $LCA(5, 1) = 3$, $LCA(5, 4) = 5$, pero $LCA(5, 99) = \text{nullptr}$ porque 99 no está en el árbol, y $LCA(99, 88) = \text{nullptr}$ porque ninguno de los dos existe.

Cómo lo resuelve el código, paso a paso. Usamos la misma función que se llama a sí misma (*recursión*) del problema 236, que sube la respuesta desde abajo, pero con dos cambios importantes:

1. Nunca cortamos por lo sano: aunque encontremos a **p** o a **q**, siempre revisamos primero las dos ramas de cada nodo antes de decidir nada. Así visitamos el árbol completo.
2. Llevamos un contador compartido. Cada vez que el nodo actual resulta ser **p** o **q**, le sumamos 1 a ese contador.

La función encuentra un candidato a respuesta igual que en 236 (cuando los dos buscados quedan uno en cada rama). Pero al final solo damos por buena esa respuesta si el contador terminó valiendo exactamente 2, es decir, si de verdad encontramos a los dos:

```

1 //
  =====
2 // 1644. Lowest Common Ancestor of a Binary Tree II (Medium)
3 //
  -----
4 // Dado un árbol binario y dos nodos p y q, encuentra su ancestro común más
5 // bajo (LCA). A diferencia del problema 236, aquí p y/o q PUEDEN NO EXISTIR
6 // en el árbol. Si alguno de los dos falta, debemos devolver nullptr.
7 //
8 // Ejemplo (mismo árbol que en 236):

```



```

9  //
10 //      3
11 //     / \
12 //    5   1
13 //   / \ / \
14 //  6  2 0  8
15 //     / \
16 //    7  4
17 //  LCA(5, 1) = 3      (ambos existen, en subárboles distintos)
18 //  LCA(5, 4) = 5      (ambos existen, 5 es ancestro directo de 4)
19 //  LCA(5, 99) = null  (99 no está en el árbol)
20 //  LCA(99, 88) = null (ninguno de los dos está en el árbol)
21 //
22 // Diferencia clave con 236: en 236 podíamos cortar la búsqueda en cuanto
23 // encontrábamos p o q ("if root == p or root == q, devuélvelo YA"), porque
24 // el enunciado GARANTIZABA que ambos existían en el árbol. Aquí esa garantía
25 // no existe: si cortáramos temprano y resultara que el otro nodo no está en
26 // el árbol, devolveríamos un LCA falso. Por eso NO podemos confiar en la
27 // primera coincidencia; hay que recorrer el árbol COMPLETO y, además, llevar
28 // la cuenta de cuántos de los dos nodos {p, q} encontramos de verdad.
29 //
30 // Diagrama de la técnica (DFS post-order con contador por referencia):
31 //
32 //      nodo
33 //     /  \
34 //  dfs(izq)  dfs(der)    <- ambas llamadas SIEMPRE se hacen completas
35 //           /  \        (no hay retorno anticipado)
36 //        ?nodo es p o q?
37 //        sí -> contador++, este nodo es candidato a LCA
38 //        no -> si izq y der no son nulos, este nodo es el LCA;
39 //              si sólo uno no es nulo, se propaga hacia arriba
40 //
41 // Al final, sólo confiamos en el candidato a LCA si el contador llegó a 2
42 // (es decir, se confirmó que TANTO p COMO q existen en el árbol). Si el
43 // contador es 0 o 1, alguno de los dos nodos no existe: devolvemos nullptr.
44 //
45 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo exactamente una vez, sin
46 // atajos, porque necesitamos confirmar la existencia de ambos),
47 // espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
48 //
=====

49 #include <cassert>
50 #include <iostream>
51 using namespace std;
52
53 struct TreeNode {
54     int val;
55     TreeNode *left;
56     TreeNode *right;
57     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
58 };
59
60 class Solution {

```

```

61 public:
62     TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
63         int contador = 0;
64         TreeNode* candidato = dfs(root, p, q, contador);
65         return (contador == 2) ? candidato : nullptr; // sólo válido si ambos
        // existen
66     }
67
68 private:
69     // Recorre TODO el subárbol (sin retorno anticipado) y va sumando en
70     // "contador" cada vez que encuentra a p o a q. Devuelve el candidato a
71     // LCA para este subárbol, igual que en 236.
72     TreeNode* dfs(TreeNode* nodo, TreeNode* p, TreeNode* q, int& contador) {
73         if (nodo == nullptr) return nullptr;
74
75         TreeNode* izq = dfs(nodo->left, p, q, contador);
76         TreeNode* der = dfs(nodo->right, p, q, contador);
77
78         if (nodo == p || nodo == q) {
79             ++contador; // confirmamos que este nodo objetivo existe
80             return nodo;
81         }
82
83         if (izq != nullptr && der != nullptr) return nodo; // p y q en ramas
        // distintas
84         return (izq != nullptr) ? izq : der; // propaga el que
        // exista
85     }
86 };
87
88 int main() {
89     Solution s;
90
91     // Árbol de prueba (el mismo que en 236):
92     //
93     //           3
94     //          / \
95     //         5   1
96     //        / \ / \
97     //       6  2 0  8
98     //      / \
99     //     7  4
100
101     TreeNode* n3 = new TreeNode(3);
102     TreeNode* n5 = new TreeNode(5);
103     TreeNode* n1 = new TreeNode(1);
104     TreeNode* n6 = new TreeNode(6);
105     TreeNode* n2 = new TreeNode(2);
106     TreeNode* n0 = new TreeNode(0);
107     TreeNode* n8 = new TreeNode(8);
108     TreeNode* n7 = new TreeNode(7);
109     TreeNode* n4 = new TreeNode(4);
110
111     n3->left = n5;    n3->right = n1;
112     n5->left = n6;    n5->right = n2;
113     n1->left = n0;    n1->right = n8;

```

```

112     n2->left = n7;    n2->right = n4;
113
114     // Caso 1: p y q existen y están en subárboles distintos de la raíz
115     // => LCA es la raíz (idéntico al caso feliz de 236).
116     TreeNode* r1 = s.lowestCommonAncestor(n3, n5, n1);
117     assert(r1 != nullptr && r1->val == 3);
118
119     // Caso 2: p y q existen, uno es ancestro directo del otro => LCA es 5.
120     TreeNode* r2 = s.lowestCommonAncestor(n3, n5, n4);
121     assert(r2 != nullptr && r2->val == 5);
122
123     // Caso 3: q NO existe en el árbol (nodo suelto, no enlazado a nada).
124     // Aunque p (n5) sí existe, como q falta debemos devolver nullptr.
125     TreeNode* suelto99 = new TreeNode(99);
126     TreeNode* r3 = s.lowestCommonAncestor(n3, n5, suelto99);
127     assert(r3 == nullptr);
128
129     // Caso 4: NINGUNO de los dos existe en el árbol => nullptr.
130     TreeNode* suelto88 = new TreeNode(88);
131     TreeNode* r4 = s.lowestCommonAncestor(n3, suelto99, suelto88);
132     assert(r4 == nullptr);
133
134     // Caso 5: dos hojas hermanas que sí existen => LCA es su padre directo
135     // (1).
136     TreeNode* r5 = s.lowestCommonAncestor(n3, n0, n8);
137     assert(r5 != nullptr && r5->val == 1);
138
139     delete n6; delete n7; delete n4; delete n2; delete n5;
140     delete n0; delete n8; delete n1; delete n3;
141     delete suelto99; delete suelto88;
142
143     cout << "1644 LCA II: todas las pruebas pasaron.\n";
144     return 0;
145 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

¿Qué significa esa complejidad? n es el número de nodos y h la *altura* del árbol (cuántos pisos tiene). $O(n)$ significa que miramos cada nodo una vez, porque ahora sí recorreremos el árbol completo. La memoria $O(h)$ es lo que ocupa la torre de llamadas de la recursión mientras baja, que nunca es más alta que la profundidad del árbol.

Consejos para la entrevista (Amazon). Di en voz alta la diferencia con el 236: aquí nadie te promete que p y q existan, así que detenerte apenas encuentras uno (el atajo `if root == p or root == q: return root`) está mal, porque puede darte una respuesta falsa cuando el otro no está. El truco del contador compartido es la forma limpia de confirmar que los dos existen sin recorrer el árbol dos veces. También vale hacerlo en dos vueltas (una para comprobar que ambos están, otra para calcular la respuesta como en 236), pero eso hace doble trabajo. Casos raros: que falte p o q , que falten los dos, que uno sea ancestro del otro, y un árbol de un solo nodo.

103. Binary Tree Zigzag Level Order Traversal

(M)

Código fuente: [src/02-arboles/103-zigzag-level-order.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dada la raíz de un árbol binario, queremos su recorrido por niveles, pero alternando la dirección: el nivel 0 se lee de izquierda a derecha, el nivel 1 de derecha a izquierda, el nivel 2 otra vez de izquierda a derecha, y así sucesivamente.

La idea, con un ejemplo de la vida real. Primero, unas palabras. Un *nivel* del árbol es todo lo que está a la misma altura: la raíz es el nivel 0, sus hijos el nivel 1, los nietos el nivel 2, y así. Recorrer «por niveles» es como leer un edificio piso por piso: primero toda la planta baja, luego todo el primer piso, etc. A esa forma de recorrer se le llama *Breadth-First Search* (BFS, búsqueda en anchura), y para hacerla usamos una *cola*: una fila de espera donde el primero que entra es el primero que atendemos, igual que en la fila del supermercado.

Lo único distinto aquí es que leemos cada piso en zigzag, como una serpiente: el primer piso de izquierda a derecha, el siguiente de derecha a izquierda, el siguiente otra vez de izquierda a derecha...Truco para no perder tiempo: en lugar de leer normal y luego voltear la lista, colocamos cada valor directamente en el lugar que le toca dentro del piso. Si el piso tiene *capa* nodos, el que sería el número *i* contando desde la izquierda lo guardamos en la posición *capa-1-i* cuando toca ir al revés.

3 9 20 15 7

Árbol [3,9,20,null,null,15,7]: el nivel 0 es [3] (izquierda a derecha), el nivel 1 se recorre de derecha a izquierda dando [20, 9], y el nivel 2 vuelve a izquierda a derecha con [15, 7]. Resultado: [[3], [20, 9], [15, 7]].

Cómo lo resuelve el código, paso a paso. Recorremos el árbol por niveles con una cola:

1. Metemos la raíz en la cola.
2. Mientras la cola no esté vacía, miramos cuántos nodos hay ahora mismo en ella: ese número es cuántos nodos tiene el piso actual.
3. Sacamos justo esos nodos uno por uno, guardamos su valor en la lista del piso y metemos a la cola a sus hijos (que formarán el piso siguiente).
4. Llevamos un interruptor de encendido/apagado (una *bandera*) que cambia en cada piso. Cuando está encendido, guardamos los valores en la posición espejo para que ese piso quede al revés.

Casos raros: si el árbol está vacío devolvemos una lista vacía de inmediato; si solo hay un nodo, devolvemos ese único valor sin voltear nada, porque hay un solo piso.

```

1 //
  =====
2 // 103. Binary Tree Zigzag Level Order Traversal (Medium)
3 //
  -----
4 // Dado la raíz de un árbol binario, devuelve el recorrido por niveles de sus

```

```

5 // valores en zigzag: el nivel 0 va de izquierda a derecha, el nivel 1 va de
6 // derecha a izquierda, el nivel 2 de nuevo izquierda a derecha, y así
7 // alternando.
8 //
9 // Ejemplo:
10 //      3
11 //     /\
12 //    9 20
13 //     /\
14 //    15 7
15 //
16 // nivel 0 (izq->der): [3]
17 // nivel 1 (der->izq): [20, 9]      <- se invierte el orden normal [9, 20]
18 // nivel 2 (izq->der): [15, 7]
19 //
20 // Resultado: [[3], [20, 9], [15, 7]]
21 //
22 // Diagrama del BFS por niveles (igual que 994, pero con struct.size() como
23 // tamaño de la capa):
24 //
25 // cola: [3]
26 // nivel 0 -> saco 3, encolo 9 y 20      -> capa = [3]
27 // nivel 1 -> saco 9, 20, encolo 15 y 7  -> capa = [9, 20] (se invierte)
28 // nivel 2 -> saco 15, 7, no hay hijos   -> capa = [15, 7]
29 //
30 // Idea: BFS estándar por niveles (igual que en 994-rotting-oranges). La única
31 // diferencia es que, en los niveles impares, invertimos el vector del nivel
32 // antes de agregarlo al resultado. Usamos un contador de nivel (o el tamaño
33 // del resultado) para saber si toca invertir.
34 //
35 // Complejidad: tiempo  $O(n)$  (cada nodo se visita una vez),
36 // espacio  $O(n)$  por la cola y el resultado.
37 //
=====

38 #include <vector>
39 #include <queue>
40 #include <cassert>
41 #include <iostream>
42 #include <algorithm>
43 using namespace std;
44
45 struct TreeNode {
46     int val;
47     TreeNode* left;
48     TreeNode* right;
49     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
50 };
51
52 class Solution {
53 public:
54     vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
55         vector<vector<int>> resultado;
56         if (root == nullptr) return resultado;    // árbol vacío => []

```

```

57     queue<TreeNode*> cola;
58     cola.push(root);
59     bool derechaAIzquierda = false;    // nivel 0 va izquierda a derecha
60
61     while (!cola.empty()) {
62         int capa = cola.size();    // nodos en este nivel
63         vector<int> nivel(capa);
64         for (int i = 0; i < capa; ++i) {
65             TreeNode* nodo = cola.front(); cola.pop();
66
67             // Posición dentro del nivel: normal o invertida según direcci
68             ón.
69             int idx = derechaAIzquierda ? (capa - 1 - i) : i;
70             nivel[idx] = nodo->val;
71
72             if (nodo->left != nullptr) cola.push(nodo->left);
73             if (nodo->right != nullptr) cola.push(nodo->right);
74         }
75         resultado.push_back(nivel);
76         derechaAIzquierda = !derechaAIzquierda;    // alterna para el
77             siguiente nivel
78     }
79     return resultado;
80 };
81
82 // Construye el árbol de ejemplo:
83 //      3
84 //    / \
85 //   9  20
86 //  / \
87 // 15  7
88 TreeNode* construirArbolEjemplo() {
89     TreeNode* raiz = new TreeNode(3);
90     raiz->left = new TreeNode(9);
91     raiz->right = new TreeNode(20);
92     raiz->right->left = new TreeNode(15);
93     raiz->right->right = new TreeNode(7);
94     return raiz;
95 }
96
97 int main() {
98     Solution s;
99
100     // Caso clásico: [3,9,20,null,null,15,7] => [[3],[20,9],[15,7]]
101     TreeNode* arbol1 = construirArbolEjemplo();
102     vector<vector<int>> esperado1 = {{3}, {20, 9}, {15, 7}};
103     assert(s.zigzagLevelOrder(arbol1) == esperado1);
104
105     // Árbol vacío => [].
106     assert(s.zigzagLevelOrder(nullptr).empty());
107
108     // Un solo nodo => [[x]].

```

```
109     TreeNode* unNodo = new TreeNode(1);
110     vector<vector<int>> esperadoUnNodo = {{1}};
111     assert(s.zigzagLevelOrder(unNodo) == esperadoUnNodo);
112
113     cout << "103 Zigzag Level Order: todas las pruebas pasaron.\n";
114     return 0;
115 }
```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es el número total de nodos. $O(n)$ en tiempo quiere decir que tocamos cada nodo una sola vez. $O(n)$ en memoria es porque, en el peor caso, la cola y las listas del resultado llegan a guardar todos los nodos a la vez. Es lo más rápido posible: no puedes escribir todos los valores sin visitarlos al menos una vez.

Consejos para la entrevista (Amazon). El error típico es leer el piso normal y luego voltear la lista con una llamada aparte (`reverse`); funciona, pero colocar cada valor directo en su lugar ahorra esa vuelta extra y demuestra que entiendes cómo está armado el piso. También puedes mencionar la variante con dos pilas (una para cada dirección) como alternativa clásica. Casos raros: árbol vacío, un solo nodo, y árboles muy torcidos donde algún piso tiene un único nodo.

199. Binary Tree Right Side View

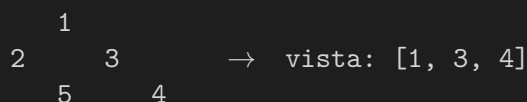
(M)

Código fuente: [src/02-arboles/199-right-side-view.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dado un árbol binario, imagina que te paras del lado *derecho* del árbol y miras hacia él. Queremos los valores de los nodos que alcanzas a ver, de arriba hacia abajo (uno por cada nivel: el nodo más a la derecha de ese nivel).

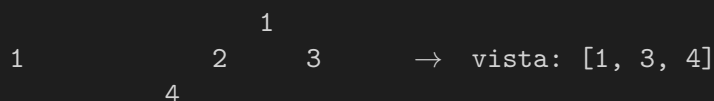
La idea, con un ejemplo de la vida real. Un *nivel* del árbol es todo lo que está a la misma altura: la raíz es el nivel 0, sus hijos el nivel 1, y así hacia abajo. Recorrer «por niveles» es leer el árbol piso por piso, de arriba abajo; a esa forma de recorrer se le llama *BFS*.

Imagina que te asomas al árbol desde la derecha, como quien mira una fila de personas desde un costado: de cada fila solo alcanzas a ver a la persona que está más a la orilla, la de más a la derecha. Entonces, si en cada piso leemos los nodos de izquierda a derecha, el que se ve desde fuera es simplemente el *último* que leemos en ese piso. No importa si ese nodo cuelga de la rama izquierda o derecha de su padre; lo único que importa es que sea el último de su piso, porque el último de izquierda a derecha es el que queda más a la derecha.



Nótese el nivel 2: el nodo 5 (que cuelga de 2) queda oculto detrás del nodo 4 (que cuelga de 3), simplemente porque 4 está más a la derecha dentro de ese nivel.

Caso importante. Un nivel puede tener su nodo visible colgando de una rama *izquierda*, si la rama derecha de ese nivel ya se quedó sin hijos (es más corta):



Aquí 4 cuelga de la izquierda de 2, pero en el nivel 2 no existe ningún nodo del lado derecho: 4 es el único nodo de ese nivel y, por lo tanto, el último leído de izquierda a derecha. Sí es visible desde la derecha aunque provenga de una rama izquierda.

Cómo lo resuelve el código, paso a paso. Recorremos el árbol por pisos usando una *cola* (una fila de espera donde el primero que entra es el primero que sale):

1. Metemos la raíz en la cola.
2. Mientras la cola no esté vacía, vemos cuántos nodos hay ahora: ese es el tamaño del piso actual.
3. Sacamos esos nodos uno por uno de izquierda a derecha y vamos metiendo a sus hijos para el piso siguiente.
4. El último nodo que sacamos de ese piso es el visible desde la derecha: guardamos su valor.

También se puede resolver con *recursión* (una función que se llama a sí misma) visitando primero la rama derecha y luego la izquierda, y anotando el primer nodo que encontramos en cada altura: como bajamos primero por la derecha, ese primero es justo el más a la derecha, y cualquier otro de esa misma altura queda tapado detrás de él.

```

1 //
  // =====
2 // 199. Binary Tree Right Side View (Medium)
3 //
  // -----
4 // Dado un árbol binario, imagina que te paras del lado DERECHO del árbol y
5 // miras hacia él. Devuelve, de arriba hacia abajo, los valores de los nodos
6 // que alcanzas a ver (uno por cada nivel).
7 //
8 // Ejemplo clásico:
9 //
10 //      1
11 //     / \
12 //    2   3
13 //     \   \
14 //      5   4
15 //
16 // Vista derecha: [1, 3, 4]
17 // Nivel 0: solo 1      -> visible: 1
18 // Nivel 1: 2 y 3, gana 3 -> visible: 3 (el más a la derecha)
19 // Nivel 2: solo 5      -> visible: 4 (!ojo! el 5 queda oculto
20 //                                detrás del 4, que está en
21 //                                el mismo nivel pero cuelga
22 //                                de la rama derecha de 3)
23 //
24 // Caso con nivel donde el nodo visible viene de la rama IZQUIERDA porque la
25 // rama derecha de ese nivel es más corta (termina antes):
26 //
27 //      1
28 //     / \
29 //    2   3
30 //   /
31 //  4
32 //
33 // Nivel 0: [1]          -> visible: 1
34 // Nivel 1: [2, 3]       -> visible: 3
35 // Nivel 2: [4]          -> visible: 4 (4 cuelga de la IZQUIERDA de 2; en
36 //                                este nivel no hay ningún nodo del
37 //                                lado derecho, así que el último nodo
38 //                                leído de izquierda a derecha SÍ es
39 //                                visible desde la derecha)
40 //
41 // Vista derecha: [1, 3, 4]
42 //
43 // Idea: BFS por niveles (como en 994-rotting-oranges). En cada nivel
44 // recorremos todos sus nodos de izquierda a derecha y nos quedamos con el
45 // ÚLTIMO: ese es, por definición, el más a la derecha del nivel y por lo
46 // tanto el que se ve desde afuera. No importa de qué rama cuelgue (izquierda
47 // o derecha del padre); lo único que importa es su posición dentro del
48 // nivel.

```

```

46 //
47 // Alternativa equivalente: DFS visitando primero la rama derecha y luego la
48 // izquierda, registrando el primer nodo que se visita en cada profundidad
49 // (si ya hay un valor registrado para esa profundidad, el nodo actual queda
50 // oculto detrás de uno visitado antes, que por el orden de visita está más a
51 // la derecha).
52 //
53 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
54 //                espacio  $O(n)$  en el peor caso (la cola puede llegar a
55 //                contener hasta el último nivel completo, que en un árbol
56 //                balanceado es  $\sim n/2$  nodos; el resultado también usa  $O(n)$ ).
57 //
=====

58 #include <vector>
59 #include <queue>
60 #include <cassert>
61 #include <iostream>
62 using namespace std;
63
64 struct TreeNode {
65     int val;
66     TreeNode *left;
67     TreeNode *right;
68     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
69 };
70
71 class Solution {
72 public:
73     vector<int> rightSideView(TreeNode* root) {
74         vector<int> vista;
75         if (root == nullptr) return vista;    // árbol vacío => nada que ver
76
77         queue<TreeNode*> cola;
78         cola.push(root);
79
80         while (!cola.empty()) {
81             int nivel = cola.size();    // cuántos nodos hay en este nivel
82             for (int i = 0; i < nivel; ++i) {
83                 TreeNode* actual = cola.front(); cola.pop();
84
85                 if (i == nivel - 1) vista.push_back(actual->val);    // último
                                del nivel
86
87                 if (actual->left != nullptr) cola.push(actual->left);
88                 if (actual->right != nullptr) cola.push(actual->right);
89             }
90         }
91         return vista;
92     }
93 };
94
95 int main() {
96     Solution s;

```

```

97 // Caso 1 (clásico): [1,2,3,null,5,null,4] => [1,3,4]
98 //
99 //      1
100 //     / \
101 //    2   3
102 //   \   \
103 //   5   4
104 TreeNode* n1 = new TreeNode(1);
105 TreeNode* n2 = new TreeNode(2);
106 TreeNode* n3 = new TreeNode(3);
107 TreeNode* n5 = new TreeNode(5);
108 TreeNode* n4 = new TreeNode(4);
109 n1->left = n2;  n1->right = n3;
110 n2->right = n5;
111 n3->right = n4;
112
113 vector<int> r1 = s.rightSideView(n1);
114 assert((r1 == vector<int>{1, 3, 4}));
115
116 delete n5; delete n4; delete n2; delete n3; delete n1;
117
118 // Caso 2: el nodo visible del último nivel cuelga de la rama IZQUIERDA
119 // porque la rama derecha de ese nivel ya terminó (es más corta).
120 //
121 //      1
122 //     / \
123 //    2   3
124 //   /
125 //  4
126 TreeNode* m1 = new TreeNode(1);
127 TreeNode* m2 = new TreeNode(2);
128 TreeNode* m3 = new TreeNode(3);
129 TreeNode* m4 = new TreeNode(4);
130 m1->left = m2;  m1->right = m3;
131 m2->left = m4;
132
133 vector<int> r2 = s.rightSideView(m1);
134 assert((r2 == vector<int>{1, 3, 4}));
135
136 delete m4; delete m2; delete m3; delete m1;
137
138 // Caso 3: árbol vacío => vista vacía.
139 vector<int> r3 = s.rightSideView(nullptr);
140 assert(r3.empty());
141
142 // Caso 4: un solo nodo => se ve únicamente él.
143 TreeNode* solo = new TreeNode(42);
144 vector<int> r4 = s.rightSideView(solo);
145 assert((r4 == vector<int>{42}));
146 delete solo;
147
148 cout << "199 Right Side View: todas las pruebas pasaron.\n";
149 return 0;
150 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es el número de nodos. $O(n)$ en tiempo quiere decir que visitamos cada nodo una vez. $O(n)$ en memoria es porque la cola puede llegar a guardar todo un piso, que en el peor caso tiene casi la mitad de los nodos del árbol.

Consejos para la entrevista (Amazon). El error típico es creer que el nodo visible de cada piso siempre cuelga de una rama derecha; como muestra el segundo dibujo, puede colgar de una izquierda si la rama derecha se acabó antes. Aclarar esto demuestra que entendiste el problema de verdad y no memorizaste una solución superficial. Casos raros: árbol vacío (vista vacía), un solo nodo (la vista es ese único valor), y árboles muy torcidos donde una rama es mucho más profunda que la otra.

987. Vertical Order Traversal of a Binary Tree

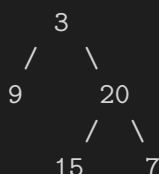
(H)

Código fuente: [src/02-arboles/987-vertical-order.cpp](#)▷ Ver animación · [local](#) | [web](#)

A cada nodo del árbol le asignamos una *columna* y una *fila*: la raíz vive en columna 0, fila 0; el hijo izquierdo se mueve una columna a la izquierda y una fila hacia abajo; el hijo derecho, una columna a la derecha y una fila hacia abajo. Debemos devolver los valores agrupados de columna menor a mayor; dentro de cada columna, ordenados por fila ascendente; y si dos nodos caen exactamente en la misma columna y la misma fila, se desempatan por **valor ascendente**. Este último detalle es justo lo que distingue a este problema del más sencillo #314 y lo que lo hace Hard: no basta con recorrer el árbol, hay que ordenar con un criterio compuesto de tres niveles.

La idea, con un ejemplo de la vida real. Imagina que el árbol está dibujado sobre una hoja cuadriculada. A cada *nodo* (cada personaje del árbol) le damos dos números, como las coordenadas de un mapa: la *columna* dice qué tan a la izquierda o a la derecha está, y la *fila* dice qué tan abajo está. Así cada nodo se convierte en una tarjeta con tres datos: (columna, fila, valor).

Cuando ya tenemos todas las tarjetas, el problema se vuelve tan sencillo como *ordenar un montón de fichas*: primero por columna, y si dos caen en la misma columna, por fila, y si además coinciden en fila, por su valor (el número más chico va antes). Lo bonito es que en el lenguaje de programación una terna de tres números ya se compara así sola, en ese mismo orden. Por eso no hace falta nada raro: recorreremos el árbol apuntando la tarjeta de cada nodo, y al final las ordenamos todas de un solo golpe.



columna	fila	valor
-1	1	9
0	0	3
0	2	15
1	1	20
2	2	7

Al agrupar por columna (ya ordenado) obtenemos `[[9],[3,15],[20],[7]]`: exactamente el resultado esperado.

Cómo lo resuelve el código, paso a paso. La idea son dos fases bien separadas: primero recolectar, luego ordenar.

1. Recorreremos el árbol (da igual por pisos o hacia el fondo) y por cada nodo guardamos su tarjeta (`columna`, `fila`, `valor`) en una lista.
2. Al bajar, el hijo izquierdo hereda `columna-1` y `fila+1` (un paso a la izquierda y uno hacia abajo); el hijo derecho, `columna+1` y `fila+1`.

3. Cuando terminamos, ordenamos toda la lista de tarjetas de una sola vez. Como una terna de tres números se compara primero por el primer número, luego por el segundo y luego por el tercero, ese único ordenamiento ya deja todo como se pide, incluido el desempate por valor.
4. Por último recorreremos la lista ya ordenada y juntamos en un mismo grupo los valores que comparten columna.

Un ejemplo donde el desempate por valor de verdad importa: en el árbol $[1, 2, 3, 4, 5, 6, 7]$, el nodo 5 (hijo derecho de 2) y el nodo 6 (hijo izquierdo de 3) caen los dos en la columna 0, fila 2. Sin la regla de desempate, no sabríamos cuál va primero; con ella, el 5 siempre va antes que el 6.

```

1 //
  =====
2 // 987. Vertical Order Traversal of a Binary Tree (Hard)
3 //
  -----
4 // A cada nodo le asignamos una columna y una fila: la raíz está en columna 0,
5 // fila 0; el hijo izquierdo va una columna a la izquierda (columna-1) y una
6 // fila más abajo; el hijo derecho, una columna a la derecha (columna+1) y una
7 // fila más abajo. Debemos devolver los valores agrupados de columna menor a
8 // mayor; dentro de cada columna, ordenados por fila ascendente; y si dos
9 // nodos caen en la MISMA columna y MISMA fila, se desempata por valor
10 // ascendente (esto es justo lo que distingue a este problema del más
11 // sencillo 314, y lo que lo hace Hard).
12 //
13 // Ejemplo:
14 //           3                      columna -1: [9]
15 //        /   \                   columna  0: [3, 15]
16 //       9     20                  columna  1: [20]
17 //          /   \                  columna  2: [7]
18 //         15    7
19 //
20 //   (3 en c0,f0) (9 en c-1,f1) (20 en c1,f1) (15 en c0,f2) (7 en c2,f2)
21 //
22 // Diagrama del recorrido DFS acumulando (columna, fila, valor):
23 //
24 //      dfs(nodo, col, fila)
25 //      |-- guarda (col, fila, nodo->val)
26 //      |-- dfs(nodo->left, col-1, fila+1)
27 //      |-- dfs(nodo->right, col+1, fila+1)
28 //
29 // Idea: recorreremos el árbol completo (DFS o BFS, da igual) guardando una
30 // terna (columna, fila, valor) por cada nodo. Al final ordenamos todas las
31 // ternas: primero por columna, luego por fila, luego por valor (el orden
32 // natural de una tupla ya hace exactamente esto). Después agrupamos los
33 // valores consecutivos que comparten columna.
34 //
35 // Complejidad: tiempo  $O(n \log n)$  por el ordenamiento de las  $n$  ternas,
36 // espacio  $O(n)$  para guardarlas.
37 //
  =====

```

```

38 #include <vector>
39 #include <tuple>
40 #include <algorithm>
41 #include <functional>
42 #include <climits>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 struct TreeNode {
48     int val;
49     TreeNode *left;
50     TreeNode *right;
51     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
52 };
53
54 class Solution {
55 public:
56     vector<vector<int>> verticalTraversal(TreeNode* root) {
57         vector<tuple<int,int,int>> nodos;    // (columna, fila, valor)
58
59         function<void(TreeNode*, int, int)> dfs = [&](TreeNode* nodo, int col,
60             int fila) {
61             if (!nodo) return;
62             nodos.emplace_back(col, fila, nodo->val);
63             dfs(nodo->left, col - 1, fila + 1);
64             dfs(nodo->right, col + 1, fila + 1);
65         };
66         dfs(root, 0, 0);
67
68         // El orden natural de tuple<int,int,int> ya compara componente a
69         // componente: columna, luego fila, luego valor. Justo lo que pide
70         // el problema (incluido el desempate por valor).
71         sort(nodos.begin(), nodos.end());
72
73         vector<vector<int>> resultado;
74         int colActual = INT_MIN;
75         bool primera = true;
76         for (auto& [col, fila, val] : nodos) {
77             if (primera || col != colActual) {
78                 resultado.push_back({});
79                 colActual = col;
80                 primera = false;
81             }
82             resultado.back().push_back(val);
83         }
84         return resultado;
85     };
86
87     int main() {
88         Solution s;
89
90         // Caso clásico: [3,9,20,null,null,15,7] => [[9],[3,15],[20],[7]]

```

```

91     TreeNode* raiz1 = new TreeNode(3);
92     raiz1->left = new TreeNode(9);
93     raiz1->right = new TreeNode(20);
94     raiz1->right->left = new TreeNode(15);
95     raiz1->right->right = new TreeNode(7);
96     vector<vector<int>> esperado1 = {{9}, {3, 15}, {20}, {7}};
97     assert(s.verticalTraversal(raiz1) == esperado1);
98
99     // Árbol completo [1,2,3,4,5,6,7]: los nodos 5 y 6 comparten columna 0 y
100    // fila 2 (5 = hijo derecho de 2, que está en columna -1; 6 = hijo
101    // izquierdo de 3, que está en columna 1; ambos terminan en columna 0).
102    // Se desempatan por valor ascendente: 5 antes que 6.
103    // Esperado: [[4],[2],[1,5,6],[3],[7]]
104    TreeNode* raiz2 = new TreeNode(1);
105    raiz2->left = new TreeNode(2);
106    raiz2->right = new TreeNode(3);
107    raiz2->left->left = new TreeNode(4);
108    raiz2->left->right = new TreeNode(5);
109    raiz2->right->left = new TreeNode(6);
110    raiz2->right->right = new TreeNode(7);
111    vector<vector<int>> esperado2 = {{4}, {2}, {1, 5, 6}, {3}, {7}};
112    assert(s.verticalTraversal(raiz2) == esperado2);
113
114    // Un solo nodo => [[x]]
115    TreeNode* raiz3 = new TreeNode(42);
116    vector<vector<int>> esperado3 = {{42}};
117    assert(s.verticalTraversal(raiz3) == esperado3);
118
119    cout << "987 Vertical Order: todas las pruebas pasaron.\n";
120    return 0;
121 }

```

Complejidad: tiempo $O(n \log n)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es el número de nodos. El tiempo $O(n \log n)$ viene del ordenamiento final: ordenar n fichas cuesta un poco más que solo mirarlas una vez, y ese «poco más» es el factor $\log n$ (piénsalo como el número de veces que puedes partir el montón a la mitad). La memoria $O(n)$ es porque guardamos una tarjeta por cada nodo.

Consejos para la entrevista (Amazon). El error típico es creer que un recorrido normal (por pisos o en orden) ya deja todo listo: no es así, porque nodos de columnas distintas pueden mezclarse en el mismo piso del recorrido. La forma segura es separar las dos fases: primero *recolectar* la posición de cada nodo sin preocuparse por el orden, y después *ordenar* con la regla completa. Vale la pena decir que este es la versión «difícil» del LeetCode 314: toda la diferencia está en el desempate por valor cuando dos nodos coinciden en columna y fila, algo fácil de olvidar si lo resuelves de memoria.

124. Binary Tree Maximum Path Sum

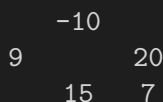
(H)

Código fuente: [src/02-arboles/124-max-path-sum.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un árbol binario cuyos valores pueden ser negativos. Un *camino* es cualquier secuencia de nodos donde cada par consecutivo está conectado por una arista, sin repetir ningún nodo. El camino **no** necesita pasar por la raíz ni terminar en una hoja: puede empezar y terminar en cualquier nodo. Queremos la suma máxima de valores que puede lograr algún camino.

La idea, con un ejemplo de la vida real. Un *camino* es como recorrer un sendero entre nodos conectados sin repetir ninguno, y buscamos el sendero cuyos números sumen lo más grande posible (ojo: hay números negativos, que restan). Piensa en cada nodo como una parada de autobús con un valor.

Párate en cada nodo y hazte dos preguntas distintas. La primera: «si quiero seguir mi camino hacia arriba, hacia mi padre, ¿cuánto es lo más que puedo aportar bajando por una sola de mis dos ramas?». Solo una rama, porque si un camino sube hacia el padre no puede a la vez irse por los dos lados: se partiría en dos. La segunda pregunta: «si mi camino *dobla* justo aquí como una V, usando mis dos ramas a la vez y sin subir más, ¿cuánto sumaría?». La primera es lo que le cuentas a tu padre para que siga armando su camino; la segunda solo sirve para ir guardando, en secreto, la mejor suma que hemos visto en todo el árbol.



En este árbol el camino óptimo es $15 \rightarrow 20 \rightarrow 7 = 42$: conviene ignorar por completo la raíz -10 y la rama de 9, porque sumarlas solo empeoraría el resultado.

Cómo lo resuelve el código, paso a paso. Usamos una función que se llama a sí misma (*recursión*) y que trabaja de abajo hacia arriba: primero resuelve a los hijos y luego al padre (a ese orden se le llama *post-order*). La función `gananciaMaxima(nodo)` devuelve, para ese nodo y todo lo que cuelga de él, la mejor ganancia que puede aportar a un camino que sigue subiendo, usando una sola rama:

```

gananciaIzq = max(0, gananciaMaxima(nodo->left))
gananciaDer = max(0, gananciaMaxima(nodo->right))
devuelve = nodo->val + max(gananciaIzq, gananciaDer)

```

El `max(0, ...)` sirve para tirar a la basura una rama si su ganancia es negativa: más vale no meter esa rama al camino, porque sumar algo negativo solo empeora el total. Es como decir «si esta rama me hace perder puntos, mejor la ignoro y cuento cero».

Aquí está el truco más importante, no lo confundas: lo que la función *devuelve* al padre usa una sola rama (porque un camino que sube solo puede continuar por un lado), pero lo que usamos para actualizar la *mejor respuesta de todo el árbol* suma las dos ramas a la vez, imaginando el camino que dobla en V justo en ese nodo y ya no sube más:

```

candidato = nodo->val + gananciaIzq + gananciaDer
mejorGlobal = max(mejorGlobal, candidato)

```

Vamos de abajo hacia arriba (primero los hijos, luego el padre) para que cada nodo ya sepa la ganancia de sus dos ramas antes de calcular su propio candidato. El código completo (abre el archivo con clic en el título):

```

1  //
   =====
2  // 124. Binary Tree Maximum Path Sum (Hard)
3  //
   -----
4  // Dado un árbol binario (los valores pueden ser negativos), un "camino" es
5  // cualquier secuencia de nodos donde cada par consecutivo está conectado por
6  // una arista. Un nodo no puede aparecer dos veces en el mismo camino. El
7  // camino NO necesita pasar por la raíz ni terminar en una hoja: puede
8  // empezar y terminar en cualquier nodo. Queremos la suma máxima de valores
9  // de nodo que se puede lograr con algún camino.
10 //
11 // Ejemplo 1:
12 //      1
13 //    / \
14 //   2  3
15 // Camino óptimo: 2 -> 1 -> 3, suma = 6.
16 //
17 // Ejemplo 2:
18 //     -10
19 //    /  \
20 //   9   20
21 //    /  \
22 //   15   7
23 // Camino óptimo: 15 -> 20 -> 7 (ignora la raíz -10 y la rama de 9),
24 // suma = 42.
25 //
26 // Ejemplo 3 (un solo nodo negativo):
27 //     -3
28 // El único camino posible es el nodo solo: suma = -3. La respuesta SÍ
29 // puede ser negativa si todos los valores lo son.
30 //
31 // Idea clave: en cada nodo hay dos cantidades distintas que no hay que
32 // confundir:
33 //
34 // 1) "Ganancia hacia abajo por UNA sola rama" (lo que devuelve la
35 // recursión hacia su padre): el mejor camino que empieza en este nodo
36 // y baja por a lo más UNO de sus dos hijos, porque un camino que sigue
37 // subiendo hacia el padre solo puede continuar por un lado.
38 //
39 //     gananciaIzq = max(0, ganancia(nodo->left))
40 //     gananciaDer = max(0, ganancia(nodo->right))
41 //     devuelve   = nodo->val + max(gananciaIzq, gananciaDer)
42 //
43 // El max(0, ...) descarta una rama si resulta negativa: es mejor no
44 // extender el camino por ahí (sumar un número negativo solo empeora).
45 //
46 // 2) "Mejor camino que DOBLA en este nodo" (usa las dos ramas a la vez,

```

```

47 //      solo sirve para actualizar la respuesta global, nunca se devuelve
48 //      hacia el padre porque un camino no puede bifurcarse dos veces):
49 //
50 //      candidato = nodo->val + gananciaIzq + gananciaDer
51 //      mejorGlobal = max(mejorGlobal, candidato)
52 //
53 // Diagrama en un nodo cualquiera:
54 //
55 //      nodo (val)
56 //      /      \
57 //  gananciaIzq  gananciaDer      (cada una ya con max(0, ...) aplicado
58 // )
59 // - Lo que se DEVUELVE al padre: val + max(gananciaIzq, gananciaDer)
60 //   (una sola rama, para que el padre pueda seguir extendiendo el camino).
61 // - Lo que actualiza la RESPUESTA GLOBAL: val + gananciaIzq + gananciaDer
62 //   (las dos ramas juntas, el camino "dobla" aquí y ya no puede crecer más
63 //   hacia arriba).
64 //
65 // Recorremos el árbol en post-order (primero hijos, luego nodo actual) para
66 // que cada nodo ya conozca la ganancia de sus dos subárboles antes de
67 // calcular su propio candidato.
68 //
69 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
70 //               espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
71 //
=====

72 #include <climits>
73 #include <cassert>
74 #include <iostream>
75 using namespace std;
76
77 struct TreeNode {
78     int val;
79     TreeNode *left;
80     TreeNode *right;
81     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
82 };
83
84 class Solution {
85 public:
86     int maxPathSum(TreeNode* root) {
87         mejorGlobal = INT_MIN;
88         gananciaMaxima(root);
89         return mejorGlobal;
90     }
91
92 private:
93     int mejorGlobal;
94
95     // Devuelve la ganancia máxima que aporta este subárbol a un camino que
96     // sigue subiendo hacia el padre (una sola rama). De paso, actualiza
97     // mejorGlobal con el mejor camino que "dobla" en este nodo (dos ramas).

```

```

98     int gananciaMaxima(TreeNode* nodo) {
99         if (nodo == nullptr) return 0;
100
101         // Ganancia de cada rama; si es negativa, mejor no tomarla (0).
102         int gananciaIzq = max(0, gananciaMaxima(nodo->left));
103         int gananciaDer = max(0, gananciaMaxima(nodo->right));
104
105         // Camino que dobla en este nodo: usa las dos ramas a la vez.
106         // Solo sirve para la respuesta global, nunca se propaga al padre.
107         int candidato = nodo->val + gananciaIzq + gananciaDer;
108         mejorGlobal = max(mejorGlobal, candidato);
109
110         // Lo que sí se propaga al padre: este nodo más, a lo más, UNA rama.
111         return nodo->val + max(gananciaIzq, gananciaDer);
112     }
113 };
114
115 int main() {
116     Solution s;
117
118     // Caso 1: árbol pequeño sin valores negativos.
119     //      1
120     //    / \
121     //   2  3
122     // Camino óptimo: 2 -> 1 -> 3 = 6.
123     TreeNode* n1 = new TreeNode(1);
124     TreeNode* n2 = new TreeNode(2);
125     TreeNode* n3 = new TreeNode(3);
126     n1->left = n2; n1->right = n3;
127     assert(s.maxPathSum(n1) == 6);
128     delete n2; delete n3; delete n1;
129
130     // Caso 2: la raíz es negativa y conviene descartarla junto con la rama
131     // de 9; el camino óptimo vive por completo en el subárbol derecho.
132     //      -10
133     //     /  \
134     //    9   20
135     //   /  \
136     //  15  7
137     // Camino óptimo: 15 -> 20 -> 7 = 42.
138     TreeNode* r = new TreeNode(-10);
139     TreeNode* rIzq = new TreeNode(9);
140     TreeNode* rDer = new TreeNode(20);
141     TreeNode* rDerIzq = new TreeNode(15);
142     TreeNode* rDerDer = new TreeNode(7);
143     r->left = rIzq; r->right = rDer;
144     rDer->left = rDerIzq; rDer->right = rDerDer;
145     assert(s.maxPathSum(r) == 42);
146     delete rIzq; delete rDerIzq; delete rDerDer; delete rDer; delete r;
147
148     // Caso 3: un solo nodo negativo. No hay otra opción de camino más que
149     // el nodo solo, así que la respuesta debe poder ser negativa.
150     TreeNode* neg = new TreeNode(-3);
151     assert(s.maxPathSum(neg) == -3);

```

```

152     delete neg;
153
154     // Caso 4: árbol con una rama completamente negativa que conviene
155     // descartar (max(0, ...) la corta) en vez de arrastrarla al camino.
156     //           5
157     //        / \
158     //       -8  9
159     //      / \
160     //     -2 -3
161     // El mejor camino ignora por completo el subárbol de -8: la respuesta
162     // es 5 -> 9 = 14, no 5 + (-8) + 9 ni nada que incluya -2 o -3.
163     TreeNode* raiz2 = new TreeNode(5);
164     TreeNode* izq = new TreeNode(-8);
165     TreeNode* der = new TreeNode(9);
166     TreeNode* izqIzq = new TreeNode(-2);
167     TreeNode* izqDer = new TreeNode(-3);
168     raiz2->left = izq; raiz2->right = der;
169     izq->left = izqIzq; izq->right = izqDer;
170     assert(s.maxPathSum(raiz2) == 14);
171     delete izqIzq; delete izqDer; delete izq; delete der; delete raiz2;
172
173     // Caso 5: todos los valores negativos; la mejor respuesta es el nodo
174     // individual menos negativo (un camino de un solo nodo).
175     //           -3
176     //        / \
177     //       -2 -1
178     TreeNode* n = new TreeNode(-3);
179     TreeNode* nIzq = new TreeNode(-2);
180     TreeNode* nDer = new TreeNode(-1);
181     n->left = nIzq; n->right = nDer;
182     assert(s.maxPathSum(n) == -1);
183     delete nIzq; delete nDer; delete n;
184
185     cout << "124 Max Path Sum: todas las pruebas pasaron.\n";
186     return 0;
187 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

¿Qué significa esa complejidad? n es el número de nodos y h la altura del árbol (cuántos pisos tiene). $O(n)$ en tiempo quiere decir que visitamos cada nodo una sola vez. La memoria $O(h)$ es lo que ocupa la torre de llamadas de la recursión mientras baja, que nunca es más alta que la profundidad del árbol.

Consejos para la entrevista (Amazon). El error más común es devolver al padre $val + gananciaIzq + gananciaDer$ (las dos ramas) en vez de $val + \max(gananciaIzq, gananciaDer)$ (una sola): eso rompería la regla de que un camino que sube no se parte en dos. Otro punto clave: la respuesta puede ser negativa (por ejemplo, un árbol con un único nodo de valor negativo), así que la mejor suma debe empezar en el número más pequeño posible (`INT_MIN`) y nunca forzarla a 0. Casos raros: un solo nodo, todos los valores negativos, y una rama negativa que conviene ignorar con $\max(0, \dots)$ en lugar de arrastrarla al camino.

543. Diameter of Binary Tree

(Fácil)

Código fuente: [src/02-arboles/543-diameter-of-binary-tree.cpp](#)▷ Ver animación · [local](#) | [web](#)**Complejidad:** tiempo $O(N)$, espacio $O(H)$.

En este problema se nos pide encontrar la longitud del camino más largo entre dos nodos de un árbol binario (esa longitud se mide contando cuántos “palitos” o aristas hay que caminar de un nodo a otro).

La idea, con un ejemplo de la vida real. Un *árbol binario* es como el organigrama de una familia dibujado de cabeza: hasta arriba está el jefe (la *raíz*) y de cada persona pueden colgar como mucho dos personas por debajo, una a la izquierda y otra a la derecha. A cada persona la llamamos *nodo*, y a las que ya no tienen a nadie colgando les decimos *hojas*. Imagina que este árbol es el mapa de un parque de diversiones y que los palitos que unen a los nodos son caminitos. El problema nos pregunta: ¿cuál es el paseo más largo que se puede hacer caminando de una atracción a otra?

Mucha gente cree que ese paseo tan largo siempre pasa por el jefe de hasta arriba, pero no es cierto: podría estar escondido en una rama muy profunda y ni siquiera tocar la raíz. El truco para verlo bien es este: todo paseo entre dos nodos sube desde uno hasta cierto “punto de encuentro” (un antepasado común) y desde ahí baja hacia el otro. Parados en ese punto de encuentro, el paseo más largo mide *lo que baja por la rama izquierda* más *lo que baja por la rama derecha*.

Cómo lo resuelve el código, paso a paso. Recorremos el árbol de abajo hacia arriba (a esto se le llama recorrido en *post-order*: primero visitamos a los hijos y solo al final al padre). Que una función se llame a sí misma para bajar por las ramas se llama *recursión*; piénsalo como preguntarle a tus subordinados y que ellos le pregunten a los suyos, hasta llegar a los de más abajo, y que las respuestas vayan subiendo.

1. Para cada nodo le preguntamos a su rama izquierda y a su rama derecha: “¿qué tan hondo llegas?” (esa profundidad es la *altura* de la rama).
2. Sumamos las dos alturas. Ese número es el paseo más largo que “dobla” justo en este nodo. Lo comparamos con el mejor paseo que llevamos guardado en una variable y, si es más grande, lo actualizamos.
3. A su propio padre, el nodo le devuelve $1 + \max(\text{alturaIzq}, \text{alturaDer})$, es decir, qué tan hondo se puede bajar desde él por una sola dirección (la más larga de sus dos ramas).

```
1 //  
    =====  
2 // 543. Diameter of Binary Tree (Easy)  
3 //  
    -----  
4 // Dado la raíz de un árbol binario, devuelve la longitud del diámetro del  
5 // árbol.  
6 //  
7 // El diámetro de un árbol binario es la longitud del camino más largo entre  
8 // DOS NODOS CUALESQUIERA del árbol. Este camino puede o no pasar por la raíz.
```

```

9  //
10 // La longitud del camino entre dos nodos se representa por el número de
11 // aristas (edges) entre ellos, no el número de nodos.
12 //
13 // Ejemplo 1:
14 //      1
15 //     /\
16 //    2  3
17 //   /\
18 //  4  5
19 //   Diámetro óptimo: 4 -> 2 -> 1 -> 3 o 5 -> 2 -> 1 -> 3 (3 aristas).
20 //
21 // Ejemplo 2:
22 //      1
23 //     /
24 //    2
25 //   Diámetro óptimo: 1 -> 2 (1 arista).
26 //
27 // Idea clave:
28 // El camino más largo en el árbol puede o no pasar por la raíz.
29 // Sin embargo, si analizamos cualquier nodo como si fuera el punto más
30 // alto (o el "doblez") de un camino, la longitud del camino que pasa por él
31 // es exactamente la altura de su subárbol izquierdo más la altura de su
32 // subárbol derecho.
33 //
34 // Por lo tanto, en cada nodo, calculamos la "altura" (o profundidad) de sus
35 // hijos.
36 // - Lo que devolvemos hacia arriba (al padre) es 1 + max(alturaIzq,
37 //   alturaDer).
38 // - Lo que actualiza nuestra respuesta global (el máximo diámetro visto
39 //   hasta
40 //   ahora) es alturaIzq + alturaDer.
41 //
42 // Al recorrer el árbol en post-order, garantizamos que al procesar un nodo,
43 // ya tenemos las alturas calculadas de sus subárboles.
44 //
45 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
46 //                espacio  $O(h)$  por la pila de recursión ( $h$  = altura del árbol).
47 //
48 // =====
49
50 #include <algorithm>
51 #include <cassert>
52 #include <iostream>
53 using namespace std;
54
55 struct TreeNode {
56     int val;
57     TreeNode *left;
58     TreeNode *right;
59     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
60 };
61
62 class Solution {

```

```

59 public:
60     int diameterOfBinaryTree(TreeNode* root) {
61         maxDiametro = 0;
62         altura(root);
63         return maxDiametro;
64     }
65
66 private:
67     int maxDiametro;
68
69     int altura(TreeNode* nodo) {
70         if (nodo == nullptr) return 0;
71
72         int alturaIzq = altura(nodo->left);
73         int alturaDer = altura(nodo->right);
74
75         // Camino que "dobla" en este nodo: suma de las aristas a izquierda y
76         // derecha
77         maxDiametro = max(maxDiametro, alturaIzq + alturaDer);
78
79         // Se propaga la altura máxima al padre + 1 (por la arista que conecta
80         // a este nodo)
81         return 1 + max(alturaIzq, alturaDer);
82     }
83 };
84
85 int main() {
86     Solution s;
87
88     // Caso 1: árbol básico.
89     //      1
90     //     /\
91     //    2 3
92     //   /\
93     //  4 5
94     // Diámetro: 3 (4->2->1->3 o 5->2->1->3).
95     TreeNode* r1 = new TreeNode(1);
96     r1->left = new TreeNode(2);
97     r1->right = new TreeNode(3);
98     r1->left->left = new TreeNode(4);
99     r1->left->right = new TreeNode(5);
100     assert(s.diameterOfBinaryTree(r1) == 3);
101
102     delete r1->left->left;
103     delete r1->left->right;
104     delete r1->left;
105     delete r1->right;
106     delete r1;
107
108     // Caso 2: árbol muy pequeño.
109     //      1
110     //     /
111     //    2
112     // Diámetro: 1 (1->2).

```



```

111     TreeNode* r2 = new TreeNode(1);
112     r2->left = new TreeNode(2);
113     assert(s.diameterOfBinaryTree(r2) == 1);
114
115     delete r2->left;
116     delete r2;
117
118     // Caso 3: árbol que dobla en un hijo y no pasa por la raíz.
119     //           1
120     //          /
121     //         2
122     //        / \
123     //       3  4
124     //      /  \
125     //     5   6
126     // El diámetro más largo es entre 5 y 6: 5->3->2->4->6 (4 aristas).
127     // Pasa por 2, no por 1.
128     TreeNode* r3 = new TreeNode(1);
129     r3->left = new TreeNode(2);
130     r3->left->left = new TreeNode(3);
131     r3->left->right = new TreeNode(4);
132     r3->left->left->left = new TreeNode(5);
133     r3->left->right->right = new TreeNode(6);
134     assert(s.diameterOfBinaryTree(r3) == 4);
135
136     delete r3->left->left->left;
137     delete r3->left->right->right;
138     delete r3->left->left;
139     delete r3->left->right;
140     delete r3->left;
141     delete r3;
142
143     cout << "543 Diameter of Binary Tree: todas las pruebas pasaron.\n";
144     return 0;
145 }

```

¿Qué significa esa complejidad? N es la cantidad de nodos del árbol y H su altura (qué tan alto es). Decir que el tiempo es $O(N)$ significa que solo pasamos una vez por cada nodo, ni uno se visita dos veces. La memoria $O(H)$ es el espacio que ocupa la recursión: es la “pila” de preguntas pendientes, y como mucho tenemos tantas preguntas abiertas como niveles tiene el árbol.

Consejos para la entrevista (Amazon). El punto que más impresiona es explicar por qué el camino más largo no siempre pasa por la raíz, y por qué necesitamos una variable aparte para el máximo (porque lo que devolvemos al padre es solo una rama, no la suma de las dos). No olvides los casos raros: un árbol vacío (diámetro 0) o un árbol de un solo nodo (también 0, porque no hay ningún palito que caminar).

863. All Nodes Distance K in Binary Tree

(Media)

Código fuente: [src/02-arboles/863-all-nodes-distance-k-in-binary-tree.cpp](#)▷ Ver animación · [local](#) | [web](#)**Complejidad:** tiempo $O(N)$, espacio $O(N)$.

Se nos da un árbol binario, un nodo objetivo y un número K , y hay que devolver todos los nodos que están exactamente a K pasos de distancia del objetivo.

La idea, con un ejemplo de la vida real. En un *árbol binario* cada *nodo* (cada “persona”) solo sabe quiénes cuelgan de él (sus hijos izquierdo y derecho), pero no sabe quién está arriba (su *padre*). Es como un juego de las escondidas donde solo puedes ver hacia abajo. El problema es que los nodos que buscan pueden estar hacia abajo del objetivo, pero también hacia arriba y luego bajando por otra rama, y para llegar a esos hay que poder subir.

La solución es como tratar el árbol igual que un mapa de amistades: si en vez de “padre e hijos” pensamos que cada nodo simplemente tiene *vecinos*, entonces cada nodo tiene hasta tres vecinos: su hijo izquierdo, su hijo derecho y su padre. Ese mapa de vecinos (donde las conexiones se pueden recorrer en los dos sentidos) se llama *grafo*, y ahí sí podemos caminar libremente en todas direcciones.

Cómo lo resuelve el código, paso a paso. Lo hacemos en dos etapas.

1. **Averiguar quién es el padre de cada quien.** Recorremos el árbol una vez bajando por las ramas y guardamos en un diccionario (un *mapa hash*, que es como una agenda que empareja cada nodo con un dato) quién es el padre de cada nodo. Así cada nodo ya conoce a sus tres vecinos.
2. **Expandirnos como una gota de tinta en agua.** Partimos del objetivo y avanzamos por niveles, tocando primero a los vecinos que están a 1 paso, luego a los que están a 2 pasos, y así. Esta forma de explorar “en círculos que crecen” se llama *BFS*. Usamos una *cola* (una fila, como la del supermercado: el primero que entra es el primero que sale) y un *conjunto* de visitados para no regresar por donde ya pasamos. Cuando hemos avanzado exactamente K niveles, todos los nodos que quedaron en la fila son justo los que están a distancia K .

```

1 //
  =====
2 // 863. All Nodes Distance K in Binary Tree (Medium)
3 //
  -----
4 // Dado la raíz de un árbol binario, un nodo objetivo (target) y un entero K,
5 // devuelve los valores de todos los nodos que están a una distancia K del
6 // nodo objetivo.
7 //
8 // Ejemplo:
9 //           3
10 //          / \
11 //         5   1
12 //        / \ / \
13 //       6  2 0  8

```

```

14 //      / \
15 //     7  4
16 // target = 5, K = 2
17 // Salida: [7, 4, 1]
18 // Explicación: Los nodos a distancia 2 de 5 son 7 y 4 (hacia abajo) y 1
19 // (hacia arriba, pasando por la raíz 3).
20 //
21 // Idea clave:
22 // En un árbol binario normal, solo podemos movernos hacia abajo (hijos).
23 // Sin embargo, para encontrar todos los nodos a distancia K, necesitamos
24 // expandirnos en todas direcciones: hacia los hijos y hacia los ancestros.
25 //
26 // La solución tiene dos pasos:
27 // 1. Recorrer el árbol (DFS o BFS) para construir un mapa que asocie cada
28 //    nodo con su padre: unordered_map<TreeNode*, TreeNode*> padres. Esto
29 //    convierte el árbol en un grafo no dirigido.
30 // 2. Hacer una búsqueda en anchura (BFS) comenzando desde el nodo 'target'.
31 //    Llevamos un conjunto (set) de nodos visitados para no retroceder. En
32 //    cada paso de la BFS, nos expandimos a: hijo izquierdo, hijo derecho y
33 //    padre. Cuando el nivel de la BFS sea exactamente K, los nodos en la
34 //    cola son nuestra respuesta.
35 //
36 // Complejidad: tiempo  $O(N)$  para construir el mapa y hacer la BFS, espacio
37 //                $O(N)$  para el mapa, el conjunto de visitados y la cola.
38 //

```

```

39 #include <vector>
40 #include <unordered_map>
41 #include <unordered_set>
42 #include <queue>
43 #include <algorithm>
44 #include <cassert>
45 #include <iostream>
46 using namespace std;
47
48 struct TreeNode {
49     int val;
50     TreeNode *left;
51     TreeNode *right;
52     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
53 };
54
55 class Solution {
56 public:
57     vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
58         // Paso 1: Mapear padres
59         unordered_map<TreeNode*, TreeNode*> padre;
60         mapearPadres(root, nullptr, padre);
61
62         // Paso 2: BFS desde el target
63         queue<TreeNode*> q;
64         unordered_set<TreeNode*> visitados;
65

```

```

66     q.push(target);
67     visitados.insert(target);
68     int distanciaActual = 0;
69
70     while (!q.empty()) {
71         if (distanciaActual == k) {
72             vector<int> resultado;
73             while (!q.empty()) {
74                 resultado.push_back(q.front()->val);
75                 q.pop();
76             }
77             return resultado;
78         }
79
80         int tamanoNivel = q.size();
81         for (int i = 0; i < tamanoNivel; ++i) {
82             TreeNode* actual = q.front();
83             q.pop();
84
85             // Intentar hijo izquierdo
86             if (actual->left && visitados.find(actual->left) == visitados.
                end()) {
87                 visitados.insert(actual->left);
88                 q.push(actual->left);
89             }
90             // Intentar hijo derecho
91             if (actual->right && visitados.find(actual->right) ==
                visitados.end()) {
92                 visitados.insert(actual->right);
93                 q.push(actual->right);
94             }
95             // Intentar padre
96             if (padre[actual] && visitados.find(padre[actual]) ==
                visitados.end()) {
97                 visitados.insert(padre[actual]);
98                 q.push(padre[actual]);
99             }
100         }
101         distanciaActual++;
102     }
103
104     return {};
105 }
106
107 private:
108     void mapearPadres(TreeNode* nodo, TreeNode* p, unordered_map<TreeNode*,
        TreeNode*>& padre) {
109         if (!nodo) return;
110         padre[nodo] = p;
111         mapearPadres(nodo->left, nodo, padre);
112         mapearPadres(nodo->right, nodo, padre);
113     }
114 };
115

```

```

116 int main() {
117     Solution s;
118
119     // Caso 1
120     //           3
121     //        /  \
122     //       5    1
123     //      / \  / \
124     //     6  2 0  8
125     //      / \
126     //     7  4
127     TreeNode* r1 = new TreeNode(3);
128     TreeNode* n5 = new TreeNode(5);
129     TreeNode* n1 = new TreeNode(1);
130     r1->left = n5; r1->right = n1;
131     n5->left = new TreeNode(6);
132     n5->right = new TreeNode(2);
133     n5->right->left = new TreeNode(7);
134     n5->right->right = new TreeNode(4);
135     n1->left = new TreeNode(0);
136     n1->right = new TreeNode(8);
137
138     vector<int> res1 = s.distanceK(r1, n5, 2);
139     sort(res1.begin(), res1.end());
140     assert((res1 == vector<int>{1, 4, 7}));
141
142     delete n5->right->left;
143     delete n5->right->right;
144     delete n5->right;
145     delete n5->left;
146     delete n1->left;
147     delete n1->right;
148     delete n1;
149     delete n5;
150     delete r1;
151
152     // Caso 2: target es raíz
153     TreeNode* r2 = new TreeNode(1);
154     r2->left = new TreeNode(2);
155     r2->right = new TreeNode(3);
156
157     vector<int> res2 = s.distanceK(r2, r2, 1);
158     sort(res2.begin(), res2.end());
159     assert((res2 == vector<int>{2, 3}));
160
161     delete r2->left;
162     delete r2->right;
163     delete r2;
164
165     cout << "863 All Nodes Distance K: todas las pruebas pasaron.\n";
166     return 0;
167 }

```

¿Qué significa esa complejidad? N es la cantidad de nodos. Tanto el tiempo como la memoria son $O(N)$: en el peor caso visitamos cada nodo una sola vez, y el mapa de padres más la cola guardan a lo mucho un dato por nodo. No se puede hacer más rápido, porque para estar seguros hay que poder mirar todo el árbol.

Consejos para la entrevista (Amazon). La idea clave que el entrevistador quiere oír es “convertir el árbol en un grafo para poder subir a los padres”. Marcar los nodos como visitados es imprescindible: si se te olvida, la búsqueda rebota entre un nodo y su padre para siempre. Menciona también los casos raros: si K es 0 la respuesta es solo el propio objetivo, y si K es más grande que el árbol, la respuesta queda vacía.

98. Validate Binary Search Tree

(M)

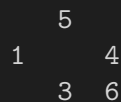
Código fuente: [src/02-arboles/98-validate-binary-search-tree.cpp](#)

► Ver animación · [local](#) | [web](#)

Dada la raíz de un árbol binario, determina si es un árbol de búsqueda binaria (BST) válido. Para que un BST sea válido, el subárbol izquierdo de cada nodo debe contener solo nodos con claves **menores** que la clave del nodo, el subárbol derecho debe contener solo nodos con claves **mayores**, y ambos subárboles deben ser también BST válidos.

La idea, con un ejemplo de la vida real. Un *BST* (árbol de búsqueda binaria) es un árbol ordenado con una regla estricta: para **cada** nodo, todo lo que cuelga a su izquierda debe ser menor que él y todo lo que cuelga a su derecha debe ser mayor. Piénsalo como acomodar libros por número: si estás en el estante “5”, absolutamente todos los libros del ala izquierda tienen que ser menores que 5, no solo el que está justo a un lado.

Ahí está la trampa más común: mucha gente revisa únicamente que el hijo de la izquierda sea menor y el de la derecha mayor que el nodo actual, y eso *no* basta. Un nodo escondido muy abajo, a la derecha de la raíz, podría tener un valor menor que la raíz y “colarse” aunque localmente parezca correcto. La regla vale para todo el subárbol, no solo para los hijos inmediatos.



En este ejemplo, $3 < 4$ y $6 > 4$, por lo que el nodo 4 y sus hijos parecen correctos localmente. Sin embargo, el nodo 3 está en el subárbol derecho de la raíz 5, pero no es mayor que 5. ¡El árbol es inválido!

Cómo lo resuelve el código, paso a paso. En vez de comparar solo padre con hijo, cada nodo lleva una “ventana” de valores permitidos, un *rango* (mínimo, máximo), y el nodo tiene que caer dentro de esa ventana. Que la función se llame a sí misma para bajar por el árbol se llama *recursión*; es como ir dando instrucciones que se van pasando hacia abajo. La ventana se va apretando conforme bajamos:

- Al principio la raíz puede valer lo que sea: su ventana es de menos infinito a más infinito, o sea (`LONG_MIN`, `LONG_MAX`).
- Cuando bajamos hacia la izquierda, todo debe ser menor que el nodo actual, así que el **techo** de la ventana baja hasta el valor de ese nodo.
- Cuando bajamos hacia la derecha, todo debe ser mayor, así que el **piso** de la ventana sube hasta el valor de ese nodo.

Si en algún momento un nodo se sale de su ventana, el árbol no es un BST válido.

Un detalle técnico: usamos el tipo `long` de C++ para el piso y el techo porque un nodo puede valer justo `INT_MIN` o `INT_MAX` (los enteros más pequeño y más grande que caben en un `int`); si usáramos `int` para la ventana, las comparaciones se romperían en esos casos extremos. El código:

1 `//`

`=====`

```

2 // 98. Validate Binary Search Tree (Medium)
3 //
4 -----
4 // Dada la raíz de un árbol binario, determina si es un árbol de búsqueda
   binaria
5 // (BST) válido.
6 //
7 // Un BST válido se define como sigue:
8 // - El subárbol izquierdo de un nodo contiene solo nodos con claves MENORES
   que
9 //   la clave del nodo.
10 // - El subárbol derecho de un nodo contiene solo nodos con claves MAYORES que
11 //   la clave del nodo.
12 // - Ambos subárboles (izquierdo y derecho) deben ser también BST válidos.
13 //
14 // Ejemplo 1:
15 //     2
16 //    / \
17 //   1  3
18 // Es un BST válido. Todo a la izquierda de 2 es < 2, todo a la derecha es >
   2.
19 //
20 // Ejemplo 2:
21 //     5
22 //    / \
23 //   1  4
24 //    / \
25 //   3  6
26 // NO es válido. Aunque en el nodo 4, 3 < 4 y 6 > 4, el nodo 3 está en el
27 // subárbol derecho de 5 y 3 NO es mayor que 5.
28 //
29 // Idea clave: un error común es comparar solo el nodo actual con sus dos
30 // hijos inmediatos. Esto falla porque un nodo muy abajo en el subárbol
   derecho
31 // podría violar la condición de ser mayor que la raíz original.
32 //
33 // Solución correcta: recursión bajando un RANGO PERMITIDO (min, max).
34 // - Al ir a la izquierda, el valor máximo permitido se actualiza al valor
   actual.
35 // - Al ir a la derecha, el valor mínimo permitido se actualiza al valor
   actual.
36 // (Usamos el tipo long long para evitar problemas de overflow con INT_MAX/
   INT_MIN).
37 //
38 // Complejidad: tiempo  $O(n)$  (visitamos cada nodo una vez),
39 //                espacio  $O(h)$  por la pila de recursión ( $h$  = altura).
40 //
41 =====
41 #include <climits>
42 #include <cassert>
43 #include <iostream>
44 using namespace std;

```



```

45
46 struct TreeNode {
47     int val;
48     TreeNode *left;
49     TreeNode *right;
50     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
51 };
52
53 class Solution {
54 public:
55     bool isValidBST(TreeNode* root) {
56         return validar(root, LLONG_MIN, LLONG_MAX);
57     }
58
59 private:
60     bool validar(TreeNode* nodo, long long minimo, long long maximo) {
61         if (nodo == nullptr) return true;
62
63         // Si el valor actual está fuera del rango permitido, no es un BST.
64         if (nodo->val <= minimo || nodo->val >= maximo) {
65             return false;
66         }
67
68         // El subárbol izquierdo debe tener valores menores que nodo->val.
69         // El subárbol derecho debe tener valores mayores que nodo->val.
70         return validar(nodo->left, minimo, nodo->val) &&
71             validar(nodo->right, nodo->val, maximo);
72     }
73 };
74
75 int main() {
76     Solution s;
77
78     // Caso 1: Árbol válido simple.
79     //      2
80     //    / \
81     //   1  3
82     TreeNode* n1 = new TreeNode(2);
83     n1->left = new TreeNode(1);
84     n1->right = new TreeNode(3);
85     assert(s.isValidBST(n1) == true);
86     delete n1->left; delete n1->right; delete n1;
87
88     // Caso 2: Árbol inválido por nodo profundo.
89     //      5
90     //    / \
91     //   1  4
92     //    / \
93     //   3  6
94     TreeNode* n2 = new TreeNode(5);
95     n2->left = new TreeNode(1);
96     n2->right = new TreeNode(4);
97     n2->right->left = new TreeNode(3);
98     n2->right->right = new TreeNode(6);

```

```

99     assert(s.isValidBST(n2) == false);
100     delete n2->right->left; delete n2->right->right;
101     delete n2->left; delete n2->right; delete n2;
102
103     // Caso 3: Solo un nodo (siempre válido).
104     TreeNode* n3 = new TreeNode(10);
105     assert(s.isValidBST(n3) == true);
106     delete n3;
107
108     // Caso 4: Valores en el límite de los enteros de 32 bits (INT_MAX).
109     TreeNode* n4 = new TreeNode(INT_MAX);
110     assert(s.isValidBST(n4) == true);
111     delete n4;
112
113     cout << "98 Validate BST: todas las pruebas pasaron.\n";
114     return 0;
115 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

¿Qué significa esa complejidad? Aquí n es la cantidad de nodos y h la altura del árbol. El tiempo $O(n)$ quiere decir que tocamos cada nodo una sola vez. La memoria $O(h)$ es el espacio de la recursión: la “pila” de llamadas abiertas es tan alta como el árbol. Si el árbol está bien equilibrado, h es pequeño ($O(\log n)$); si es un árbol chueco que parece una cadena, h llega a ser n .

Consejos para la entrevista (Amazon). Ten mucho cuidado con los casos límite de los enteros de 32 bits (`INT_MAX` y `INT_MIN`): conviene usar `long` para la ventana, o usar punteros nulos para representar “sin límite”. Explica también que el espacio $O(h)$ se vuelve $O(n)$ en el peor caso (un árbol degenerado que parece una lista ligada) y $O(\log n)$ en el mejor caso (un árbol balanceado).

105. Construct Binary Tree from Preorder and Inorder Traversal (M)

Código fuente: [src/02-arboles/105-construct-binary-tree-from-preorder-and-inorder-traversal.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Dados dos arreglos de enteros `preorder` e `inorder`, donde el primero es el recorrido en pre-order de un árbol binario y el segundo es su recorrido en in-order, reconstruye y devuelve el árbol binario original. (Se garantiza que no hay valores duplicados en el árbol).

La idea, con un ejemplo de la vida real. Un “recorrido” es el orden en que anotamos los nodos cuando paseamos por el árbol. Aquí nos dan dos listas de ese paseo, y con las dos juntas tenemos que volver a dibujar el árbol, como armar un mueble con dos hojas de instrucciones distintas. Fíjate en la forma de cada lista:

- **Pre-order:** anota primero la raíz y luego lo demás: [RAÍZ, [Subárbol Izquierdo], [Subárbol Derecho]]. Su primer elemento siempre es el jefe del árbol.
- **In-order:** anota primero todo lo de la izquierda, luego la raíz y al final lo de la derecha: [[Subárbol Izquierdo], RAÍZ, [Subárbol Derecho]].

Junta las dos pistas y sale magia: `preorder` te regala quién es la raíz (su primer elemento). Buscas ese valor en `inorder` y, como ahí la raíz está justo en medio, todo lo que quede a su izquierda es el subárbol izquierdo y todo lo de su derecha es el subárbol derecho. Ya separaste el árbol en dos mitades más chiquitas.

Por ejemplo, si la raíz es 3 y en `inorder` aparece en la posición i , entonces hay exactamente i nodos en el subárbol izquierdo. Eso también te dice que los siguientes i valores de `preorder` forman esa mitad izquierda, y el resto la derecha.

Cómo lo resuelve el código, paso a paso. Repetimos ese mismo truco una y otra vez sobre pedazos cada vez más pequeños (eso es *recursión*, y esta estrategia de partir el problema en mitades se llama *dividir y vencer*):

1. Tomamos el siguiente valor de `preorder`: ese es la raíz del pedazo que estamos armando.
2. Buscamos dónde está esa raíz en `inorder` para saber cuántos nodos van a la izquierda y cuántos a la derecha.
3. Construimos primero el subárbol izquierdo y luego el derecho, cada uno con el mismo procedimiento.

Para que buscar la raíz dentro de `inorder` sea instantáneo (y no lento, lo que volvería todo $O(n^2)$), al principio armamos un *mapa hash* (una agenda que empareja cada valor con su posición) y así encontramos cualquier índice al instante.

```
1 //
  =====
2 // 105. Construct Binary Tree from Preorder and Inorder Traversal (Medium)
3 //
  -----
```

```

4 // Dados dos arreglos de enteros preorder e inorder donde preorder es el
5 // recorrido en preorden de un árbol binario e inorder es el recorrido en
6 // inorden del mismo árbol, construye y devuelve el árbol binario.
7 //
8 // Ejemplo 1:
9 //      3
10 //     /\
11 //    9 20
12 //   /\
13 //  15 7
14 // Entrada: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
15 // Salida: [3,9,20,null,null,15,7] (representación en niveles)
16 //
17 // Idea clave:
18 // 1. El primer elemento de 'preorder' siempre es la RAÍZ del árbol actual.
19 // 2. Buscamos el valor de esa raíz en el arreglo 'inorder'.
20 // 3. Todo lo que está a la izquierda de la raíz en 'inorder' pertenece al
21 //    subárbol izquierdo. Todo lo que está a la derecha pertenece al subárbol
22 //    derecho.
23 // 4. Con el tamaño del subárbol izquierdo (digamos, k elementos), sabemos que
24 //    los siguientes k elementos en 'preorder' (después de la raíz) son
25 //    exactamente el preorden del subárbol izquierdo.
26 //
27 // Para que la búsqueda en 'inorder' sea rápida, usamos un mapa hash (O(1)) en
28 // lugar de hacer escaneo lineal (O(n)).
29 //
30 // Complejidad: tiempo O(n) (hash map + recursión lineal), espacio O(n)
31 //                (mapa hash y pila de recursión).
32 //
33 =====
34
35 #include <vector>
36 #include <unordered_map>
37 #include <cassert>
38 #include <iostream>
39 using namespace std;
40
41 struct TreeNode {
42     int val;
43     TreeNode *left;
44     TreeNode *right;
45     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
46 };
47
48 class Solution {
49 public:
50     TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
51         // Mapa para encontrar el índice de cualquier valor en inorder en O(1)
52         unordered_map<int, int> inorderIndex;
53         for (int i = 0; i < inorder.size(); i++) {
54             inorderIndex[inorder[i]] = i;
55         }
56
57         // Puntero global para avanzar en el arreglo preorder

```

```

56         int preIndex = 0;
57         return construir(preorder, inorderIndex, preIndex, 0, inorder.size() -
58             1);
59     }
60 private:
61     TreeNode* construir(const vector<int>& preorder,
62                         unordered_map<int, int>& inorderIndex,
63                         int& preIndex, int inStart, int inEnd) {
64
65         // Caso base: no hay elementos para este subárbol
66         if (inStart > inEnd) {
67             return nullptr;
68         }
69
70         // El elemento actual en preorden es la raíz de este subárbol
71         int rootVal = preorder[preIndex++];
72         TreeNode* root = new TreeNode(rootVal);
73
74         // Encontramos dónde está esta raíz en el arreglo inorden
75         int mid = inorderIndex[rootVal];
76
77         // Construimos recursivamente los subárboles izquierdo y derecho.
78         // Importante: primero construimos el izquierdo porque los siguientes
79         // elementos en preorder pertenecen al subárbol izquierdo.
80         root->left = construir(preorder, inorderIndex, preIndex, inStart, mid
81             - 1);
82
83         root->right = construir(preorder, inorderIndex, preIndex, mid + 1,
84             inEnd);
85
86         return root;
87     };
88
89     // Función auxiliar para liberar la memoria del árbol
90     void liberarArbol(TreeNode* root) {
91         if (!root) return;
92         liberarArbol(root->left);
93         liberarArbol(root->right);
94         delete root;
95     }
96
97     int main() {
98         Solution s;
99
100         // Caso 1:
101         //      3
102         //     / \
103         //    9  20
104         //     / \
105         //    15  7
106         vector<int> pre1 = {3, 9, 20, 15, 7};
107         vector<int> in1 = {9, 3, 15, 20, 7};
108         TreeNode* root1 = s.buildTree(pre1, in1);

```

```
107     assert(root1->val == 3);
108     assert(root1->left->val == 9);
109     assert(root1->right->val == 20);
110     assert(root1->left->left == nullptr);
111     assert(root1->left->right == nullptr);
112     assert(root1->right->left->val == 15);
113     assert(root1->right->right->val == 7);
114     liberarArbol(root1);
115
116
117     // Caso 2: Un solo nodo
118     vector<int> pre2 = {-1};
119     vector<int> in2 = {-1};
120     TreeNode* root2 = s.buildTree(pre2, in2);
121     assert(root2->val == -1);
122     assert(root2->left == nullptr && root2->right == nullptr);
123     liberarArbol(root2);
124
125     cout << "105 Construct Binary Tree: todas las pruebas pasaron.\n";
126     return 0;
127 }
```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es la cantidad de nodos. El tiempo $O(n)$ significa que creamos cada nodo una sola vez, y gracias al mapa hash ya no perdemos tiempo buscando. La memoria $O(n)$ viene de dos cosas: el propio mapa hash guarda un dato por nodo, y la recursión abre una pila de llamadas.

Consejos para la entrevista (Amazon). Es un ejemplo clásico de “dividir y vencer”. El detalle técnico que más cuenta es pasar el índice de **preorder** por **referencia** (`int& preIndex`), para que el avance de la raíz se mantenga sincronizado entre todas las llamadas recursivas. Si lo pasas por valor (una copia), los subárboles de la derecha tomarían la raíz equivocada y el árbol saldría mal.

337. House Robber III

(M)

Código fuente: [src/02-arboles/337-house-robber-iii.cpp](#)▷ Ver animación · [local](#) | [web](#)

El ladrón ha descubierto un nuevo vecindario para robar. Las casas en este vecindario forman un árbol binario. La regla de oro (y única forma de alertar a la policía) es que **no puedes robar dos casas que estén conectadas directamente**. Es decir, si robas un nodo, no puedes robar a sus hijos inmediatos.

Debes determinar la cantidad máxima de dinero que el ladrón puede llevarse sin alertar a la policía.

La idea, con un ejemplo de la vida real. Las casas están acomodadas formando un *árbol binario*: cada casa (cada *nodo*) tiene otras casas colgando debajo (sus hijos), y no puedes robar dos casas que estén conectadas directamente. Parado frente a cualquier casa solo tienes dos opciones:

- **Opción 1: robas esta casa.** Como la alarma se dispararía, tienes *prohibido* robar las casas hijas que cuelgan justo debajo. Tu botín sería el dinero de esta casa más lo mejor que puedas sacar de los nietos hacia abajo, pero *sin* tocar a los hijos directos.
- **Opción 2: no robas esta casa.** Al dejarla en paz, quedas *libre* para hacer lo que quieras con las casas hijas: robarlas o no. De cada hijo eliges lo que te dé más dinero.

Si calculas estas dos opciones con una recursión ingenua, acabas recalculando los mismos pedazos del árbol una y otra vez, y el trabajo explota a $O(2^n)$ (una cantidad monstruosa).

Cómo lo resuelve el código, paso a paso. El truco para no repetir trabajo es que cada casa devuelva **las dos respuestas a la vez**, en lugar de una sola. En C++ usamos un `std::pair<int, int>` (una parejita de dos números) con la forma `{max_SIN_robar, max_ROBANDO}`.

1. Recorremos el árbol de abajo hacia arriba (recorrido en *post-order*: primero los hijos, al final el padre), lo que garantiza que cuando decidimos en una casa ya tenemos las dos respuestas de sus hijos.
2. Para calcular `max_ROBANDO` de una casa: su dinero más el `max_SIN_robar` de cada hijo (porque si robas, los hijos quedan prohibidos).
3. Para calcular `max_SIN_robar`: de cada hijo tomamos lo mejor entre sus dos opciones y lo sumamos.
4. La respuesta final es la mejor de las dos opciones de la raíz.

Así visitamos cada casa una sola vez y ni siquiera necesitamos una libreta extra para recordar cálculos (*memoización*).

```

1 //
  =====
2 // 337. House Robber III (Medium)
3 //
  -----
4 // El ladrón ha descubierto un nuevo vecindario para robar. Hay una sola

```

```

5 // entrada a este lugar, llamada "raíz". Además de la raíz, cada casa tiene
6 // exactamente una casa matriz (un padre). Después de un recorrido, el ladrón
7 // se da cuenta de que todas las casas en este lugar forman un árbol binario.
8 //
9 // Automáticamente se contactará a la policía si se roban dos casas unidas
10 // directamente en la misma noche (es decir, no puedes robar un nodo y a
11 // su padre).
12 //
13 // Determina la cantidad máxima de dinero que el ladrón puede robar esta noche
14 // sin alertar a la policía.
15 //
16 // Ejemplo 1:
17 //      3
18 //     /\
19 //    2  3
20 //     /\
21 //    3  1
22 // Robo óptimo = 3 (raíz) + 3 (hijo de 2) + 1 (hijo de 3) = 7.
23 //
24 // Ejemplo 2:
25 //      3
26 //     /\
27 //    4  5
28 //   /\  \
29 //  1  3  1
30 // Robo óptimo = 4 (hijo izq) + 5 (hijo der) = 9.
31 //
32 // Idea clave:
33 // Para cada nodo, tenemos dos opciones:
34 // 1. Robar el nodo actual. Si lo hacemos, NO podemos robar sus hijos,
35 //    así que sumamos el valor del nodo más lo que sacaríamos de los "nietos"
36 //    (es decir, robando los subárboles de los hijos, pero eligiendo no
37 //    robar los hijos inmediatos).
38 // 2. NO robar el nodo actual. Si hacemos esto, somos libres de decidir si
39 //    robar o no robar cada uno de los hijos (tomando el máximo entre ambas
40 //    opciones para cada hijo).
41 //
42 // Para evitar recalcular estados (que daría  $O(2^n)$ ), usamos DFS en post-order
43 // devolviendo para cada nodo un par: [max_sin_robar_raiz, max_robando_raiz].
44 //
45 // Complejidad: tiempo  $O(n)$ , espacio  $O(h)$  para la pila de recursión.
46 //
=====
47 #include <algorithm>
48 #include <cassert>
49 #include <iostream>
50 using namespace std;
51
52 struct TreeNode {
53     int val;
54     TreeNode *left;
55     TreeNode *right;

```



```

56     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
57 };
58
59 class Solution {
60 public:
61     int rob(TreeNode* root) {
62         pair<int, int> resultado = robarDFS(root);
63         return max(resultado.first, resultado.second);
64     }
65
66 private:
67     // Devuelve un par {maximo_SIN_robear_el_nodo, maximo_ROBANDO_el_nodo}
68     pair<int, int> robarDFS(TreeNode* nodo) {
69         if (!nodo) {
70             return {0, 0};
71         }
72
73         pair<int, int> izq = robarDFS(nodo->left);
74         pair<int, int> der = robarDFS(nodo->right);
75
76         // Si decidimos NO robar el nodo actual, somos libres de elegir
77         // la mejor opción (robar o no robar) para cada uno de sus hijos.
78         int sinRobar = max(izq.first, izq.second) + max(der.first, der.second)
79             ;
80
81         // Si decidimos ROBAR el nodo actual, obligatoriamente NO podemos
82         robar
83         // a sus hijos directos.
84         int robando = nodo->val + izq.first + der.first;
85
86         return {sinRobar, robando};
87     }
88 };
89
90 int main() {
91     Solution s;
92
93     // Caso 1:
94     //      3
95     //     / \
96     //    2  3
97     //     \  \
98     //      3  1
99     TreeNode* r1 = new TreeNode(3);
100    r1->left = new TreeNode(2);
101    r1->right = new TreeNode(3);
102    r1->left->right = new TreeNode(3);
103    r1->right->right = new TreeNode(1);
104    assert(s.rob(r1) == 7);
105
106    // Caso 2:
107    //      3
108    //     / \
109    //    4  5

```

```

108 //      / \      \
109 //     1  3      1
110 TreeNode* r2 = new TreeNode(3);
111 r2->left = new TreeNode(4);
112 r2->right = new TreeNode(5);
113 r2->left->left = new TreeNode(1);
114 r2->left->right = new TreeNode(3);
115 r2->right->right = new TreeNode(1);
116 assert(s.rob(r2) == 9);
117
118 cout << "337 House Robber III: todas las pruebas pasaron.\n";
119
120 // Cleanup manual (solo para no ensuciar la salida, en CP a veces se omite
121 // pero aquí somos limpios).
122 delete r1->left->right; delete r1->right->right;
123 delete r1->left; delete r1->right; delete r1;
124
125 delete r2->left->left; delete r2->left->right; delete r2->right->right;
126 delete r2->left; delete r2->right; delete r2;
127
128 return 0;
129 }

```

Complejidad: tiempo $O(n)$, espacio $O(h)$.

¿Qué significa esa complejidad? n es la cantidad de casas y h la altura del árbol. El tiempo $O(n)$ quiere decir que pasamos por cada casa una sola vez. La memoria $O(h)$ es el espacio de la recursión: la pila de llamadas abiertas es tan alta como el árbol.

Consejos para la entrevista (Amazon). Muchos se lanzan de inmediato a una recursión con memoización usando un `unordered_map<TreeNode*, int>`. Funciona, pero la solución más elegante es devolver los dos estados en una parejita. Contar la evolución de la idea (de la ingenua $O(2^n)$, a la memoizada, y por fin a esta de estado doble que corre en $O(n)$ y sin la carga de un mapa hash) deja muy buena impresión.

297. Serialize and Deserialize Binary Tree

(D)

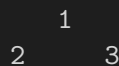
Código fuente: [src/02-arboles/297-serialize-and-deserialize-binary-tree.cpp](#)

▷ Ver animación · [local](#) | [web](#)

El proceso de *serializar* consiste en transformar una estructura de datos (como un árbol) en un formato lineal (como una cadena de texto) que pueda ser enviado por la red o guardado en disco. El proceso inverso, *deserializar*, toma esa cadena y reconstruye la estructura original en memoria de forma exacta. Diseña el algoritmo para lograr esto con un árbol binario.

La idea, con un ejemplo de la vida real. *Serializar* es convertir el árbol en una sola tira de texto, como desarmar un mueble y anotar todas sus piezas en una lista para poder mandarlo por correo. *Deserializar* es lo contrario: leer esa lista y volver a armar el mueble idéntico. La clave es que la lista tenga suficiente información para que no haya dudas al rearmar.

Hay varias formas de recorrer el árbol y anotarlo, pero la más cómoda de programar es el *Depth-First Search* (DFS) en *pre-order* (primero la raíz, luego la rama izquierda, luego la derecha). El detalle mágico es que cada vez que nos topemos con un hueco (un *puntero nulo*, o sea “aquí no hay ningún nodo”) anotamos un símbolo especial, digamos "N". Con esos huecos marcados, la tira de texto describe el árbol sin ninguna ambigüedad.



Usando pre-order y comas como separadores, el árbol de arriba genera la cadena: "1,2,N,N,3,N,N,".

Cómo lo resuelve el código, paso a paso. Las dos mitades usan *recursión* (una función que se llama a sí misma para bajar por las ramas):

1. **Al desarmar (serializar):** vamos en pre-order escribiendo el valor del nodo seguido de una coma; si el nodo es un hueco, escribimos "N". Después hacemos lo mismo con su rama izquierda y luego con la derecha.
2. **Al armar (deserializar):** leemos la tira *en el mismo orden* en que la escribimos. El primer valor es siempre la raíz. Si leemos "N", ahí no va nada (hueco). Tras crear un nodo, la recursión sigue leyendo: lo siguiente que aparezca es su rama izquierda, y cuando esa termine, su rama derecha.

Como los valores están separados por comas, en C++ un `stringstream` pasado por referencia deja ir “comiendo” un pedazo a la vez de forma muy sencilla, y todas las llamadas recursivas avanzan sobre la misma tira sin perderse.

```

1 //
  =====
2 // 297. Serialize and Deserialize Binary Tree (Hard)
3 //
  -----
4 // La serialización es el proceso de convertir una estructura de datos o un
5 // objeto en una secuencia de bits para que pueda almacenarse en un archivo o
6 // búfer de memoria, o transmitirse a través de una conexión de red para
7 // reconstruirse posteriormente en el mismo entorno o en otro entorno.

```

```

8  //
9  // Diseña un algoritmo para serializar y deserializar un árbol binario. No hay
10 // restricciones sobre cómo tu algoritmo de serialización/deserialización debe
11 // funcionar. Solo necesitas asegurarte de que un árbol binario se pueda
12 // serializar en una cadena y esta cadena se pueda deserializar en la
13 // estructura de árbol original.
14 //
15 // Ejemplo 1:
16 //      1
17 //     /\
18 //    2  3
19 //     /\
20 //    4  5
21 // Entrada: root = [1,2,3,null,null,4,5]
22 // Salida: [1,2,3,null,null,4,5]
23 //
24 // Idea clave:
25 // Usaremos un recorrido en Preorden (Preorder: Raíz, Izquierda, Derecha).
26 // - Serializar: Recorremos el árbol y agregamos el valor del nodo seguido de
27 // una coma (e.g., "1,"). Si encontramos un puntero nulo, agregamos un
28 // marcador especial como "N,".
29 // - Deserializar: Leemos los tokens separados por coma uno por uno. Como
30 // guardamos el árbol en preorden, el primer token es garantizadamente la
31 // raíz. Luego construimos recursivamente el subárbol izquierdo y luego el
32 // derecho usando el estado compartido (por ejemplo, pasando un stringstream
33 // por referencia).
34 //
35 // Complejidad: tiempo  $O(n)$ , espacio  $O(n)$  tanto para serializar como para
36 // deserializar (por la cadena generada y la pila de recursión).
37 //
=====

38 #include <string>
39 #include <sstream>
40 #include <cassert>
41 #include <iostream>
42 using namespace std;
43
44 struct TreeNode {
45     int val;
46     TreeNode *left;
47     TreeNode *right;
48     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
49 };
50
51 class Codec {
52 public:
53     // Codifica un árbol a una cadena de texto.
54     string serialize(TreeNode* root) {
55         if (!root) return "N,";
56
57         string res = to_string(root->val) + ",";
58         res += serialize(root->left);
59         res += serialize(root->right);

```

```
60
61     return res;
62 }
63
64 // Decodifica tu cadena de texto a un árbol.
65 TreeNode* deserialize(string data) {
66     stringstream ss(data);
67     return decodificar(ss);
68 }
69
70 private:
71     TreeNode* decodificar(stringstream& ss) {
72         string token;
73         // Leemos el siguiente token hasta la coma
74         if (!getline(ss, token, ',')) {
75             return nullptr;
76         }
77
78         // Si el token es "N", es un puntero nulo
79         if (token == "N") {
80             return nullptr;
81         }
82
83         // De lo contrario, es un número. Lo convertimos y creamos el nodo.
84         TreeNode* nodo = new TreeNode(stoi(token));
85
86         // Como serializamos en preorden, lo siguiente pertenece a la
87             izquierda
88         // y luego a la derecha.
89         nodo->left = decodificar(ss);
90         nodo->right = decodificar(ss);
91
92         return nodo;
93     };
94
95 // Función auxiliar para verificar si dos árboles son idénticos
96 bool sonIden(TreeNode* r1, TreeNode* r2) {
97     if (!r1 && !r2) return true;
98     if (!r1 || !r2) return false;
99     return (r1->val == r2->val) &&
100         sonIden(r1->left, r2->left) &&
101         sonIden(r1->right, r2->right);
102 }
103
104 // Función auxiliar para liberar memoria
105 void liberarArbol(TreeNode* root) {
106     if (!root) return;
107     liberarArbol(root->left);
108     liberarArbol(root->right);
109     delete root;
110 }
111
112 int main() {
```

```

113     Codec codec;
114
115     // Caso 1: Árbol normal
116     //      1
117     //    /  \
118     //   2   3
119     //    /  \
120     //   4   5
121     TreeNode* root1 = new TreeNode(1);
122     root1->left = new TreeNode(2);
123     root1->right = new TreeNode(3);
124     root1->right->left = new TreeNode(4);
125     root1->right->right = new TreeNode(5);
126
127     string ser1 = codec.serialize(root1);
128     TreeNode* ans1 = codec.deserialize(ser1);
129     assert(sonIden(root1, ans1));
130     liberarArbol(root1);
131     liberarArbol(ans1);
132
133     // Caso 2: Árbol vacío
134     TreeNode* root2 = nullptr;
135     string ser2 = codec.serialize(root2);
136     TreeNode* ans2 = codec.deserialize(ser2);
137     assert(ans2 == nullptr);
138
139     cout << "297 Serialize/Deserialize BST: todas las pruebas pasaron.\n";
140     return 0;
141 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es la cantidad de nodos. Tanto el tiempo como la memoria son $O(n)$: al desarmar y al armar tocamos cada nodo una sola vez, y la tira de texto (más la pila de la recursión) guarda un pedacito por cada nodo. No se puede pedir menos, porque hay que escribir y leer todo el árbol.

Consejos para la entrevista (Amazon). Es un problema “Hard” y los entrevistadores quieren ver que dominas las herramientas del lenguaje: usar `std::stringstream` con `getline` en C++ se ve muy maduro. Suma puntos si mencionas que el recorrido BFS (por niveles) también funciona, pero suele pedir más código (colas y manejar los huecos nivel por nivel), mientras que el DFS en pre-order gana por lo cortito de su recursión. Evita usar variables globales para llevar la posición en la tira; en su lugar pasa el estado (el stream) por referencia.

Capítulo 3

Intervalos

56. Merge Intervals

(M)

Código fuente: [src/03-intervalos/56-merge-intervals.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de intervalos `intervals[i] = [inicio, fin]`. Queremos *fusionar* todos los intervalos que se solapan entre sí (o que apenas se tocan en un punto) y devolver el arreglo resultante de intervalos que ya no se solapan, cubriendo exactamente el mismo rango total que el arreglo original.

La idea, con un ejemplo de la vida real. Piensa en tu calendario de reuniones de la semana, donde cada reunión ocupa un bloque de tiempo. Si dos reuniones se traslapan (o una empieza justo cuando termina la otra), en la práctica ocupan tu agenda como si fueran una sola reunión larga. Fusionar intervalos es exactamente eso: convertir un montón de bloques que se pisan entre sí en la lista más corta posible de bloques “limpios” que representan lo mismo.

```

1--3
2-----6      8--10      15---18

```

Con `intervals = [[1,3],[2,6],[8,10],[15,18]]`: `[1,3]` y `[2,6]` se solapan (porque $3 \geq 2$), así que se funden en `[1,6]`. Los otros dos no tocan a nadie más. El resultado es `[[1,6],[8,10],[15,18]]`.

Cómo lo resuelve el código, paso a paso.

1. Ordenamos los intervalos por su valor de `inicio`. Esto es el paso clave: una vez ordenados, cualquier intervalo que se pueda solapar con uno anterior tiene que estar cerca de él en la lista, no disperso por todo el arreglo.
2. Recorremos los intervalos ya ordenados manteniendo un “**resultado**” que empieza vacío.
3. Para cada intervalo, lo comparamos contra el *último* grupo que llevamos armado en **resultado**: si el fin de ese último grupo es menor que el inicio del intervalo actual, no hay solape, así que el intervalo actual abre un grupo nuevo.
4. Si sí hay solape (o se tocan justo en un punto), no agregamos un grupo nuevo: simplemente estiramos el fin del último grupo hasta el mayor de los dos fines.
5. Al terminar el recorrido, **resultado** contiene los intervalos fusionados, en orden.

```

1 //
  =====
2 // 56. Merge Intervals (Medium)
3 //
  -----
4 // Nos dan un arreglo de intervalos intervals[i] = [inicio, fin]. Queremos
5 // fusionar todos los intervalos que se solapan (o que se tocan justo en un
6 // punto) y devolver el arreglo resultante de intervalos que ya no se
7 // solapan entre sí, cubriendo exactamente el mismo rango total.
8 //
9 // Ejemplo: intervals = [[1,3],[2,6],[8,10],[15,18]]

```



```

10 //
11 // 1--3
12 // 2-----6      8--10      15---18
13 //
14 // [1,3] y [2,6] se solapan (3 >= 2) -> se funden en [1,6].
15 // [8,10] y [15,18] no tocan a nadie -> quedan igual.
16 // Resultado: [[1,6],[8,10],[15,18]]
17 //
18 // Diagrama del algoritmo:
19 //
20 // ordenar intervals por su inicio
21 // resultado = []
22 // para cada intervalo iv en intervals (ya ordenados):
23 //     si resultado está vacío o el fin del último en resultado < iv.inicio:
24 //         // no se solapan: iv abre un grupo nuevo
25 //         resultado.push(iv)
26 //     si no:
27 //         // se solapan (o se tocan): extender el fin del último grupo
28 //         resultado.last().fin = max(resultado.last().fin, iv.fin)
29 //
30 // Por qué funciona: una vez ordenados por inicio, dos intervalos que no se
31 // solapan con el grupo "abierto" actual tampoco pueden solaparse con ningún
32 // grupo anterior (porque sus inicios son aún mayores). Así que basta con
33 // comparar cada intervalo nuevo contra el ÚLTIMO grupo armado hasta ahora:
34 // si el intervalo nuevo empieza antes (o justo cuando) termina ese grupo,
35 // pertenece al mismo grupo y solo hay que estirar su fin; si no, es un
36 // grupo nuevo. El orden por inicio es lo que hace posible resolverlo en una
37 // sola pasada.
38 //
39 // Complejidad: tiempo  $O(n \log n)$  (dominado por el ordenamiento), espacio
40 //  $O(n)$  para el resultado (sin contar lo que pida el ordenamiento in-place).
41 //
=====

42 #include <vector>
43 #include <algorithm>
44 #include <cassert>
45 #include <iostream>
46 using namespace std;
47
48 class Solution {
49 public:
50     vector<vector<int>> merge(vector<vector<int>>& intervals) {
51         vector<vector<int>> result;
52         if (intervals.empty()) return result;
53
54         sort(intervals.begin(), intervals.end(),
55             [](const vector<int>& a, const vector<int>& b) {
56                 return a[0] < b[0];
57             });
58
59         for (auto& iv : intervals) {
60             if (result.empty() || result.back()[1] < iv[0]) {
61                 // No se solapa con el último grupo: abrimos uno nuevo.

```

```

62         result.push_back(iv);
63     } else {
64         // Se solapa (o se tocan): estiramos el fin del último grupo.
65         result.back()[1] = max(result.back()[1], iv[1]);
66     }
67 }
68 return result;
69 }
70 };
71
72 int main() {
73     Solution s;
74
75     // Caso normal: dos solapes que se funden, dos intervalos sueltos.
76     vector<vector<int>> in1 = {{1,3},{2,6},{8,10},{15,18}};
77     assert(s.merge(in1) == (vector<vector<int>>{{1,6},{8,10},{15,18}}));
78
79     // Intervalos que solo se TOCAN (fin == inicio del siguiente): también
80     // deben fusionarse, según el enunciado.
81     vector<vector<int>> in2 = {{1,4},{4,5}};
82     assert(s.merge(in2) == (vector<vector<int>>{{1,5}}));
83
84     // Caso límite: arreglo vacío.
85     vector<vector<int>> in3 = {};
86     assert(s.merge(in3) == (vector<vector<int>>{}));
87
88     // Caso límite: un solo intervalo, no hay nada que fusionar.
89     vector<vector<int>> in4 = {{1,4}};
90     assert(s.merge(in4) == (vector<vector<int>>{{1,4}}));
91
92     // Caso donde el orden de procesamiento importa: el intervalo que
93     // solapa llega ANTES en el arreglo original, así que si no ordenamos
94     // por inicio primero, el algoritmo de una sola pasada falla.
95     vector<vector<int>> in5 = {{2,3},{1,5}};
96     assert(s.merge(in5) == (vector<vector<int>>{{1,5}}));
97
98     cout << "56. Merge Intervals: todas las pruebas pasaron.\n";
99     return 0;
100 }

```

Complejidad: tiempo $O(n \log n)$, espacio $O(n)$.

¿Qué significa esa complejidad? El costo dominante es ordenar los n intervalos, que toma $O(n \log n)$; el recorrido que fusiona ya ordenados es una sola pasada, $O(n)$. En espacio, en el peor caso (ningún intervalo se solapa con otro) el resultado tiene los mismos n intervalos que la entrada.

Consejos para la entrevista (Amazon). El error más común es olvidar ordenar primero: sin ordenar, comparar solo contra el “último” grupo no sirve de nada, porque el solape puede estar en cualquier parte del arreglo. Otro punto que preguntan seguido: ¿qué pasa si dos intervalos solo se *tocan* (el fin de uno es igual al inicio del otro), se consideran solapados? Según el enunciado de LeetCode, sí, hay que fusionarlos (usa $<$ estricto al comparar, no $<=$, para decidir cuándo NO se

253. Meeting Rooms II

(M)

Código fuente: `src/03-intervalos/253-meeting-rooms-ii.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de intervalos de reunión `intervals[i] = [inicio, fin]`. Queremos el número *mínimo* de salas necesarias para que todas las reuniones se puedan realizar, sabiendo que dos reuniones que se traslapan en el tiempo no pueden compartir sala.

La idea, con un ejemplo de la vida real. Imagina que eres el encargado de reservar salas de juntas en una oficina, y te llegan todas las solicitudes de reunión del día de golpe. En vez de intentar asignar salas reunión por reunión, es más fácil pensar en dos listas separadas: “a qué horas empieza alguna reunión” y “a qué horas termina alguna reunión”. Si recorres el día cronológicamente y llevas la cuenta de cuántas salas están ocupadas en cada instante, el número máximo que alcanzas en algún momento es la respuesta: necesitas al menos esa cantidad de salas para no quedarte sin espacio.

```

0 -----30
  5----10
    15----20

```

Con `intervals = [[0,30],[5,10],[15,20]]`: `[5,10]` y `[15,20]` no se traslapan entre sí, así que podrían compartir una segunda sala; pero ambas se traslapan con `[0,30]`, que necesita su propia sala durante todo ese tiempo. En total hacen falta 2 salas.

Cómo lo resuelve el código, paso a paso.

1. Separamos los `inicios` de todas las reuniones en un arreglo, y los `fines` en otro. A partir de aquí ya no importa a qué reunión pertenecía cada hora, solo el momento en que ocurre.
2. Ordenamos ambos arreglos por separado, de menor a mayor.
3. Recorremos el tiempo con dos punteros, uno para `inicios` y otro para `fines` (barrido / *sweep line*). En cada paso comparamos el siguiente inicio pendiente contra el siguiente fin pendiente.
4. Si el próximo inicio ocurre *antes* de que la reunión más próxima a terminar acabe, necesitamos una sala adicional: sumamos uno al contador de salas en uso y avanzamos el puntero de `inicios`.
5. Si no (el próximo fin ocurre antes o al mismo tiempo), una sala se libera: restamos uno al contador de salas en uso y avanzamos el puntero de `fines`.
6. Llevamos registro del valor *máximo* que alcanzó el contador de salas en uso durante todo el barrido: esa es la respuesta.

```

1 //
  =====
2 // 253. Meeting Rooms II (Medium)
3 //
  -----
4 // Nos dan un arreglo de intervalos de reunión intervals[i] = [inicio, fin].

```

```

5 // Queremos el número MÍNIMO de salas necesarias para que todas las
6 // reuniones se puedan llevar a cabo, sabiendo que dos reuniones que se
7 // traslapan en el tiempo no pueden compartir sala.
8 //
9 // Ejemplo: intervals = [[0,30],[5,10],[15,20]]
10 //
11 // 0 -----30 (sala A)
12 // 5---10 (sala B, cabe dentro de [0,30])
13 // 15---20 (puede reusar la sala B, ya
14 // libre)
15 // [5,10] y [15,20] no se traslapan entre sí, así que pueden compartir una
16 // segunda sala; pero ambas se traslapan con [0,30], que necesita su
17 // propia sala. Total: 2 salas.
18 //
19 // Diagrama del algoritmo (barrido / sweep line con dos punteros):
20 //
21 // starts = [inicio de cada reunión], ordenado
22 // ends = [fin de cada reunión], ordenado
23 // salasEnUso = 0, maxSalas = 0
24 // si = 0 (puntero en starts), ei = 0 (puntero en ends)
25 // mientras si < n:
26 //     si starts[si] < ends[ei]:
27 //         // una reunión empieza antes de que la más próxima a terminar acabe
28 //         salasEnUso += 1; si += 1
29 //         maxSalas = max(maxSalas, salasEnUso)
30 //     si no:
31 //         // una reunión termina antes (o justo cuando) empieza la siguiente:
32 //         // esa sala queda libre para reusarse
33 //         salasEnUso -= 1; ei += 1
34 //
35 // Por qué funciona: separamos los inicios y los fines y los ordenamos por
36 // SEPARADO (ya no importa a qué reunión pertenecía cada hora). Recorremos
37 // el tiempo en orden cronológico: cada vez que una reunión empieza antes de
38 // que termine la que está más próxima a acabar, necesitamos una sala
39 // adicional; cada vez que una reunión termina, esa sala se libera y puede
40 // reusarse. El máximo de salas ocupadas SIMULTÁNEAMENTE durante todo el
41 // barrido es la respuesta.
42 //
43 // Complejidad: tiempo  $O(n \log n)$  (ordenar starts y ends), espacio  $O(n)$  para
44 // los dos arreglos auxiliares.
45 //
=====

46 #include <vector>
47 #include <algorithm>
48 #include <cassert>
49 #include <iostream>
50 using namespace std;
51
52 class Solution {
53 public:
54     int minMeetingRooms(vector<vector<int>>& intervals) {
55         int n = intervals.size();

```

```

56     if (n == 0) return 0;
57
58     vector<int> starts(n), ends(n);
59     for (int i = 0; i < n; ++i) {
60         starts[i] = intervals[i][0];
61         ends[i] = intervals[i][1];
62     }
63     sort(starts.begin(), starts.end());
64     sort(ends.begin(), ends.end());
65
66     int salasEnUso = 0, maxSalas = 0;
67     int si = 0, ei = 0;
68     while (si < n) {
69         if (starts[si] < ends[ei]) {
70             // Empieza una reunión antes de que libere la más próxima.
71             ++salasEnUso;
72             ++si;
73             maxSalas = max(maxSalas, salasEnUso);
74         } else {
75             // Se libera una sala (fin <= inicio actual).
76             --salasEnUso;
77             ++ei;
78         }
79     }
80     return maxSalas;
81 }
82 };
83
84 int main() {
85     Solution s;
86
87     // Caso normal: [5,10] y [15,20] pueden compartir sala, [0,30] no.
88     vector<vector<int>> in1 = {{0,30},{5,10},{15,20}};
89     assert(s.minMeetingRooms(in1) == 2);
90
91     // Reuniones que no se traslapan en absoluto: basta una sala.
92     vector<vector<int>> in2 = {{7,10},{2,4}};
93     assert(s.minMeetingRooms(in2) == 1);
94
95     // Caso límite: una sola reunión.
96     vector<vector<int>> in3 = {{1,5}};
97     assert(s.minMeetingRooms(in3) == 1);
98
99     // Caso límite: arreglo vacío, no hace falta ninguna sala.
100    vector<vector<int>> in4 = {};
101    assert(s.minMeetingRooms(in4) == 0);
102
103    // Caso donde el orden de procesamiento importa: hay un empate exacto
104    // entre un inicio y un fin ([9,10] termina justo cuando otra reunión
105    // empieza en 9). Si se procesara el inicio antes que el fin en el
106    // empate, se contaría una sala de más.
107    vector<vector<int>> in5 = {{9,10},{4,9},{4,17}};
108    assert(s.minMeetingRooms(in5) == 2);
109

```

```
110     cout << "253. Meeting Rooms II: todas las pruebas pasaron.\n";  
111     return 0;  
112 }
```

Complejidad: tiempo $O(n \log n)$, espacio $O(n)$.

¿Qué significa esa complejidad? Ordenar los dos arreglos de n horas cada uno domina el costo, $O(n \log n)$; el barrido con dos punteros después es lineal. En espacio guardamos dos arreglos de tamaño n (los inicios y los fines por separado).

Consejos para la entrevista (Amazon). Un desempate que suele pasar desapercibido: cuando un inicio y un fin ocurren *exactamente al mismo tiempo*, ¿libera la sala antes de que se necesite otra, o hace falta una sala nueva de todos modos? La convención estándar (y la que produce la respuesta correcta) es tratar los fines como si ocurrieran primero: una reunión que termina a las 10 libera la sala justo a tiempo para otra que empieza a las 10. Menciona también la alternativa con un min-heap de horas de fin (equivalente en complejidad, más fácil de explicar en pizarrón), y el caso límite de un arreglo vacío, que no necesita ninguna sala.

2402. Meeting Rooms III

(H)

Código fuente: [src/03-intervalos/2402-meeting-rooms-iii.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Hay n salas numeradas de 0 a $n-1$. Nos dan $\text{meetings}[i] = [\text{inicio}, \text{fin}]$, y se procesan en orden de *inicio*. Cada reunión se asigna a la sala *libre* de menor índice disponible en ese momento. Si ninguna está libre, la reunión se *retrasa* hasta que la sala que se desocupe primero quede disponible, conservando la *misma duración* que tenía originalmente, y se asigna a esa sala (la de menor índice si varias se desocupan al mismo tiempo). Queremos la sala que organizó más reuniones en total (la de menor índice en caso de empate).

La idea, con un ejemplo de la vida real. Es la versión “estricta” de repartir salas: en vez de simplemente contar cuántas salas hacen falta (como en el problema anterior), aquí ya existe un número fijo de salas y hay que decidir, reunión por reunión, cuál usa cada una, incluyendo qué pasa cuando llega una reunión y no hay ninguna sala libre en ese instante: se le dice a los organizadores “espera a que alguna sala se desocupe”, y su reunión se recorre en el tiempo sin acortarse. Es como una recepcionista con n salas de juntas que siempre prefiere la sala 1 sobre la sala 2 cuando ambas están libres, y que sabe exactamente cuál sala va a desocuparse primero cuando todas están ocupadas.

Con $n = 2$, $\text{meetings} = [[0,10], [1,5], [2,7], [3,4]]$: $[0,10]$ toma la sala 0, $[1,5]$ toma la sala 1. Cuando llega $[2,7]$ ninguna sala está libre; la sala 1 se desocupa primero (en el tiempo 5), así que la reunión se retrasa: dura $7 - 2 = 5$, ocupa la sala 1 de 5 a 10. Cuando llega $[3,4]$ tampoco hay sala libre; ambas se desocupan en el tiempo 10 (empate), gana la de menor índice, sala 0; dura $4 - 3 = 1$, ocupa la sala 0 de 10 a 11. Cada sala terminó con 2 reuniones: empate, gana la sala 0.

Cómo lo resuelve el código, paso a paso.

1. Ordenamos las reuniones por *inicio*, porque el enunciado exige procesarlas en ese orden.
2. Mantenemos dos *min-heaps* (colas de prioridad que siempre dan el menor elemento primero): uno de **salas libres**, ordenado por índice de sala; otro de **salas ocupadas**, ordenado por la hora en que cada una se desocupa (y, en empate, por índice).
3. Antes de procesar cada reunión, sacamos del heap de ocupadas todas las salas cuya hora de fin ya pasó (es decir, es menor o igual al inicio de la reunión actual) y las movemos al heap de libres.
4. Si después de eso hay alguna sala libre, la reunión la usa (la de menor índice, que es justo lo que da un min-heap por índice) y esa sala vuelve al heap de ocupadas con su hora de fin real.
5. Si no hay ninguna sala libre, tomamos del heap de ocupadas la que se va a desocupar *más pronto*: la reunión se retrasa hasta esa hora, conservando su duración original, y esa sala vuelve al heap de ocupadas con la nueva hora de fin (hora en que se liberó más la duración).
6. En cualquiera de los dos casos, sumamos uno al contador de esa sala.
7. Al final, devolvemos la sala con mayor contador (la de menor índice en caso de empate).


```

49 // la de menor índice (regla de desempate del enunciado). El min-heap de
50 // salas ocupadas, ordenado por hora de fin, nos dice cuál sala se
51 // desocupará primero cuando NINGUNA esté libre; a esa se le asigna la
52 // reunión retrasada, conservando su duración original.
53 //
54 // Complejidad: tiempo  $O(n \log n)$  por el ordenamiento inicial más  $O(n \log k)$ 
55 // por las operaciones de heap (donde  $k \leq n$  es el número de salas), espacio
56 //  $O(n)$  para los heaps y el conteo.
57 //
=====
58 #include <vector>
59 #include <queue>
60 #include <algorithm>
61 #include <cassert>
62 #include <iostream>
63 using namespace std;
64
65 class Solution {
66 public:
67     int mostBooked(int n, vector<vector<int>>& meetings) {
68         sort(meetings.begin(), meetings.end());
69
70         priority_queue<int, vector<int>, greater<int>> libres;
71         for (int i = 0; i < n; ++i) libres.push(i);
72
73         // (horaDeFin, índiceSala), min-heap: primero la que termina antes,
74         // en empate la de menor índice (orden natural de pair).
75         priority_queue<pair<long long, int>, vector<pair<long long, int>>,
76             greater<>> ocupadas;
77
78         vector<int> conteo(n, 0);
79
80         for (auto& m : meetings) {
81             long long inicio = m[0], fin = m[1];
82
83             while (!ocupadas.empty() && ocupadas.top().first <= inicio) {
84                 libres.push(ocupadas.top().second);
85                 ocupadas.pop();
86             }
87
88             if (!libres.empty()) {
89                 int sala = libres.top();
90                 libres.pop();
91                 ocupadas.push({fin, sala});
92                 conteo[sala]++;
93             } else {
94                 auto [horaLibre, sala] = ocupadas.top();
95                 ocupadas.pop();
96                 long long duracion = fin - inicio;
97                 ocupadas.push({horaLibre + duracion, sala});
98                 conteo[sala]++;
99             }
100         }
101     }
102 }

```

```

100
101     int mejor = 0;
102     for (int i = 1; i < n; ++i) {
103         if (conteo[i] > conteo[mejor]) mejor = i;
104     }
105     return mejor;
106 }
107 };
108
109 int main() {
110     Solution s;
111
112     // Caso normal (ejemplo oficial de LeetCode): empate 2-2, gana sala 0.
113     vector<vector<int>> m1 = {{0,10},{1,5},{2,7},{3,4}};
114     assert(s.mostBooked(2, m1) == 0);
115
116     // Otro ejemplo oficial, con 3 salas: sala 1 organiza más reuniones.
117     vector<vector<int>> m2 = {{1,20},{2,10},{3,5},{4,9},{6,8}};
118     assert(s.mostBooked(3, m2) == 1);
119
120     // Caso límite: una sola sala. Todas las reuniones, se solapen o no,
121     // terminan cayendo en la sala 0 (retrasadas si hace falta).
122     vector<vector<int>> m3 = {{0,5},{5,10}};
123     assert(s.mostBooked(1, m3) == 0);
124
125     // Caso donde el orden de procesamiento importa: las mismas reuniones
126     // del primer caso pero revueltas en el arreglo de entrada. Como el
127     // algoritmo ordena por inicio antes de procesar, el resultado debe ser
128     // idéntico al caso normal (sala 0).
129     vector<vector<int>> m4 = {{2,7},{0,10},{3,4},{1,5}};
130     assert(s.mostBooked(2, m4) == 0);
131
132     cout << "2402. Meeting Rooms III: todas las pruebas pasaron.\n";
133     return 0;
134 }

```

Complejidad: tiempo $O(n \log n + n \log k)$, espacio $O(n + k)$.

¿Qué significa esa complejidad? Ordenar las n reuniones cuesta $O(n \log n)$. Cada reunión hace un número constante de operaciones sobre heaps de tamaño a lo más k (el número de salas), así que procesarlas todas cuesta $O(n \log k)$. En espacio guardamos los dos heaps (tamaño total acotado por k) más el arreglo de contadores, tamaño k , además de una copia ordenada de las reuniones, tamaño n .

Consejos para la entrevista (Amazon). Este problema mezcla dos ideas del bloque de intervalos: el barrido por tiempo (como en Meeting Rooms II) y el uso de dos heaps con roles distintos (como en Find Median from Data Stream). El punto más fácil de arruinar es la regla de retraso: hay que conservar la *duración*, no la hora de fin original, al mover una reunión en el tiempo. Otro detalle fino: cuando varias salas se desocupan exactamente a la misma hora, el desempate por índice debe respetarse tanto al elegir sala libre como al elegir cuál sala ocupada se desocupa primero; por eso el heap de ocupadas ordena por el par (hora de fin, índice) y no solo por la hora. Menciona

el caso límite de una sola sala, donde todas las reuniones, se traslapen o no, terminan cayendo ahí, posiblemente retrasadas en cadena.

1094. Car Pooling

(M)

Código fuente: `src/03-intervalos/1094-car-pooling.cpp`▷ Ver animación · [local](#) | [web](#)

Manejas un coche con cierta `capacity` de pasajeros y solo avanzas hacia adelante, nunca retrocedes. Nos dan `trips[i] = [numPasajeros, desde, hasta]`: en el tramo del camino que va de `desde` a `hasta` suben `numPasajeros` personas, que bajan justo en `hasta`. Queremos saber si es posible llevar a todos los pasajeros de todos los viajes sin exceder la capacidad del coche en ningún punto del camino.

La idea, con un ejemplo de la vida real. Piensa en una app de carpooling universitario: el conductor va publicando en qué kilómetro sube cada grupo de pasajeros y en qué kilómetro se bajan. En vez de simular el viaje pasajero por pasajero, basta con anotar, en cada kilómetro del camino, *cuántos suben y cuántos bajan* ahí mismo. Si vas sumando esos cambios netos conforme avanzas, en todo momento sabes cuántos pasajeros van sentados en el coche, sin importar el orden en que llegaron las solicitudes de viaje.

posición	0	1	2	3	4	5	6	7
viaje A (+2 en 1, -2 en 5)		+2				-2		
viaje B (+3 en 3, -3 en 7)				+3				-3
ocupación acumulada	0	2	2	5	5	3	3	0

Con `trips = [[2,1,5],[3,3,7]]` y `capacity = 4`: en la posición 3 ya van los 2 pasajeros del viaje A (que bajan hasta la posición 5) más los 3 nuevos del viaje B: $2 + 3 = 5 > 4$. No cabe, la respuesta es `false`.

Cómo lo resuelve el código, paso a paso. Esta es la técnica de *arreglo de diferencias* (*difference array*), una variante del barrido de intervalos.

1. Creamos un arreglo `diff` tan largo como el rango de posiciones posible del camino, inicializado en ceros.
2. Para cada viaje (`num`, `desde`, `hasta`), sumamos `num` en `diff[desde]` (aquí suben personas) y restamos `num` en `diff[hasta]` (aquí bajan).
3. Recorremos `diff` de la posición más chica a la más grande, acumulando una variable `ocupacion` que va sumando cada valor de `diff` conforme avanzamos.
4. En cada posición, si `ocupacion` supera `capacity`, devolvemos `false` de inmediato: en algún punto del camino el coche se llenó de más.
5. Si terminamos el recorrido sin exceder nunca la capacidad, devolvemos `true`.

```

1 //
  =====
2 // 1094. Car Pooling (Medium)
3 //
  =====

```

```

4 // Manejas un coche con cierta capacidad de pasajeros y solo viajas hacia
5 // adelante (nunca retrocedes). Nos dan trips[i] = [numPasajeros, desde,
6 // hasta]: en el tramo del camino que va del punto "desde" al punto "hasta"
7 // suben numPasajeros personas (y bajan justo en "hasta"). Devuelve true si
8 // es posible llevar a todos los pasajeros de todos los viajes sin exceder
9 // la capacidad del coche en ningún punto del camino, o false si no.
10 //
11 // Ejemplo: trips = [[2,1,5],[3,3,7]], capacity = 4
12 //
13 //   posición: 0  1  2  3  4  5  6  7
14 //   viaje A:    +2..... -2
15 //   viaje B:                +3..... -3
16 //   ocupación:  2  2  5  5  5  3  3    <- en la posición 3 ya hay 5 > 4
17 //
18 //   En la posición 3 coinciden los 2 pasajeros del viaje A (todavía no han
19 //   bajado, bajan en 5) con los 3 nuevos del viaje B: 2+3=5 > 4. No cabe.
20 //
21 // Diagrama del algoritmo (arreglo de diferencias / sweep line):
22 //
23 //   diff[posición] = cambio neto de pasajeros que ENTRAN o SALEN justo ahí
24 //   para cada viaje (num, desde, hasta):
25 //       diff[desde] += num    // suben "num" personas en este punto
26 //       diff[hasta] -= num   // bajan "num" personas en este punto
27 //   ocupación = 0
28 //   para cada posición del camino, de menor a mayor:
29 //       ocupación += diff[posición]
30 //       si ocupación > capacity: devolver false
31 //       devolver true
32 //
33 // Por qué funciona: en vez de simular pasajero por pasajero, acumulamos
34 // SOLO los cambios netos en cada punto del camino (cuántos suben menos
35 // cuántos bajan). Al barrer el camino de izquierda a derecha y sumar esos
36 // cambios en orden, la variable "ocupación" en cada posición representa
37 // exactamente cuántos pasajeros van en el coche en ese tramo, sin importar
38 // en qué orden vinieran los viajes en el arreglo de entrada: lo único que
39 // importa es su posición en el camino, no su posición en el arreglo.
40 //
41 // Complejidad: tiempo  $O(n + d)$  donde  $n$  es el número de viajes y  $d$  es el
42 // rango de posiciones del camino (barremos el arreglo de diferencias
43 // completo), espacio  $O(d)$  para el arreglo de diferencias.
44 //

```

```

=====

45 #include <vector>
46 #include <cassert>
47 #include <iostream>
48 using namespace std;
49
50 class Solution {
51 public:
52     bool carPooling(vector<vector<int>>& trips, int capacity) {
53         // Restricción típica de LeetCode: 0 <= desde < hasta <= 1000.
54         vector<int> diff(1001, 0);
55

```

```

56     for (auto& t : trips) {
57         int num = t[0], desde = t[1], hasta = t[2];
58         diff[desde] += num;
59         diff[hasta] -= num;
60     }
61
62     int ocupacion = 0;
63     for (int pos = 0; pos < 1001; ++pos) {
64         ocupacion += diff[pos];
65         if (ocupacion > capacity) return false;
66     }
67     return true;
68 }
69 };
70
71 int main() {
72     Solution s;
73
74     // Caso normal: en la posición 3 se juntan 5 pasajeros, excede capacidad
75     4.
76     vector<vector<int>> t1 = {{2,1,5},{3,3,7}};
77     assert(s.carPooling(t1, 4) == false);
78
79     // Mismos viajes, capacidad 5: alcanza justo, cabe.
80     vector<vector<int>> t2 = {{2,1,5},{3,3,7}};
81     assert(s.carPooling(t2, 5) == true);
82
83     // Caso límite: sin viajes, siempre cabe.
84     vector<vector<int>> t3 = {};
85     assert(s.carPooling(t3, 1) == true);
86
87     // Caso límite: un solo viaje que ya excede la capacidad desde el inicio.
88     vector<vector<int>> t4 = {{5,0,3}};
89     assert(s.carPooling(t4, 4) == false);
90
91     // Caso donde el orden de procesamiento (por posición, no por índice del
92     arreglo) importa: el primer viaje baja a sus pasajeros EXACTAMENTE en
93     la posición donde suben los del segundo viaje. No deben contarse como
94     simultáneos: la ocupación nunca debe superar 3.
95     vector<vector<int>> t5 = {{3,0,2},{3,2,5}};
96     assert(s.carPooling(t5, 3) == true);
97
98     cout << "1094. Car Pooling: todas las pruebas pasaron.\n";
99     return 0;
100 }

```

Complejidad: tiempo $O(n + d)$, espacio $O(d)$.

¿Qué significa esa complejidad? n es el número de viajes y d es el rango de posiciones del camino (en LeetCode, acotado a 1000). Construir el arreglo de diferencias toma $O(n)$ (una suma y una resta por viaje), y barrerlo entero toma $O(d)$; como d es una constante fija en este problema, en la práctica el algoritmo es lineal en n . El espacio es $O(d)$ por el arreglo de diferencias, también acotado por una constante.

Consejos para la entrevista (Amazon). El error más común es restar en `hasta - 1` en vez de `hasta`, pensando que hay que “incluir” la última posición ocupada; pero el enunciado dice que los pasajeros bajan *en hasta*, así que ese punto ya no debe contarlos. Si el rango de posiciones no viniera acotado por el enunciado (a diferencia de este problema, donde sí lo está a 1000), la alternativa general es usar un mapa ordenado (`std::map`) en vez de un arreglo fijo, con el mismo principio de sumar en `desde` y restar en `hasta`. Vale la pena mencionar también que este mismo patrón de “arreglo de diferencias” resuelve cualquier problema de “aplica un cambio a un rango, muchas veces, y pregunta el valor final en cada punto” sin tener que recorrer cada rango por separado.

Capítulo 4

Heaps y Top-K

215. Kth Largest Element in an Array

(M)

Código fuente: `src/04-heaps-topk/215-kth-largest-element.cpp`

▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo `nums` sin ordenar y un entero `k`. Hay que devolver el `k`-ésimo elemento más grande, contando desde el más grande (el primer más grande es el máximo, el segundo es el que le sigue, y así sucesivamente). Si hay valores repetidos, cada copia cuenta por separado: no es el `k`-ésimo valor *distinto*.

La idea, con un ejemplo de la vida real. Imagina que estás organizando una competencia y solo te interesa saber quién quedó en el `k`-ésimo lugar del podio, sin importarte el resto de la tabla completa. No hace falta ordenar a todos los competidores de principio a fin: basta con quedarte, en todo momento, con los `k` mejores vistos hasta ahora, y descartar de inmediato a cualquiera que no logre meterse en ese grupo selecto.

Esa es la idea de usar un *min-heap* (montículo mínimo, una estructura que siempre te da acceso rápido a su elemento más pequeño) de tamaño fijo `k`: metemos cada número, y si el montículo crece más de `k` elementos, sacamos el más chico de ese grupo, porque ya sabemos que no puede ser el `k`-ésimo más grande de todo el arreglo.

```
nums = [3,2,1,5,6,4], k = 2
ordenado desc: 6, 5, 4, 3, 2, 1 -> 2do lugar = 5
```

Cómo lo resuelve el código, paso a paso.

1. Creamos un min-heap vacío.
2. Por cada número del arreglo, lo insertamos en el heap.
3. Si el heap tiene más de `k` elementos, sacamos el más pequeño: ya perdió toda oportunidad de ser el `k`-ésimo mayor.
4. Al terminar de recorrer el arreglo, el heap contiene exactamente los `k` valores más grandes vistos, y como es un min-heap, su cima (el menor de esos `k`) es justo la respuesta.

```
1 //
2 // =====
3 //
4 // 215. Kth Largest Element in an Array (Medium)
5 //
6 // Nos dan un arreglo nums (sin ordenar) y un entero k. Hay que devolver el
7 // k-ésimo elemento más grande, contando desde el más grande (el 1er más
8 // grande es el máximo, el 2do es el segundo máximo, etc.). No es el k-ésimo
9 // valor DISTINTO: si hay duplicados, cada copia cuenta por separado.
10 //
11 // Ejemplo: nums = [3,2,1,5,6,4], k = 2
12 // Ordenado de mayor a menor: 6, 5, 4, 3, 2, 1
13 // El 2do más grande es 5 => respuesta 5.
14 //
15 // Diagrama del algoritmo (min-heap de tamaño k):
```

```

14 //
15 //  heap = []  (min-heap, la cima siempre es el MENOR del heap)
16 //  para cada x en nums:
17 //      heap.push(x)
18 //      si heap.size() > k:
19 //          heap.pop()          // se saca el menor: ya no puede ser el k-ésimo
    mayor
20 //  al terminar, la cima del heap es la respuesta
21 //
22 // Idea: en vez de ordenar todo el arreglo (que nos da más información de la
23 // que necesitamos), mantenemos solo los k elementos más grandes vistos hasta
24 // el momento, en un min-heap de tamaño k. Cada vez que el heap se pasa de
25 // tamaño k, el elemento que sobra es forzosamente el MENOR de esos k+1 (el
26 // que menos posibilidades tiene de ser top-k), así que lo sacamos. Al final,
27 // el heap contiene exactamente los k mayores de todo el arreglo, y como es un
28 // min-heap, su cima (el menor de esos k) es justo el k-ésimo más grande.
29 //
30 // Por qué funciona: es un invariante que se mantiene en cada paso. Después de
31 // procesar cualquier prefijo del arreglo, el heap siempre contiene los
32 // min(k, elementos_vistos) valores más grandes de ese prefijo. Al llegar al
33 // final, el prefijo es el arreglo completo, así que el heap tiene los k
34 // mayores del arreglo completo, y su cima es el k-ésimo.
35 //
36 // Complejidad: tiempo  $O(n \log k)$ , espacio  $O(k)$ .
37 //
    =====

38 #include <vector>
39 #include <queue>
40 #include <cassert>
41 #include <iostream>
42 using namespace std;
43
44 class Solution {
45 public:
46     int findKthLargest(vector<int>& nums, int k) {
47         // Min-heap de tamaño (a lo más) k.
48         priority_queue<int, vector<int>, greater<>> heap;
49         for (int x : nums) {
50             heap.push(x);
51             if ((int)heap.size() > k) {
52                 heap.pop();    // sacamos el menor: no puede ser top-k
53             }
54         }
55         return heap.top();
56     }
57 };
58
59 int main() {
60     Solution s;
61
62     // Caso normal.
63     vector<int> n1 = {3,2,1,5,6,4};
64     assert(s.findKthLargest(n1, 2) == 5);

```

```

65
66 // k = 1: el máximo del arreglo.
67 vector<int> n2 = {3,2,3,1,2,4,5,5,6};
68 assert(s.findKthLargest(n2, 1) == 6);
69
70 // Un solo elemento, k = 1.
71 vector<int> n3 = {7};
72 assert(s.findKthLargest(n3, 1) == 7);
73
74 // k = tamaño del arreglo: el mínimo de todos.
75 vector<int> n4 = {3,2,3,1,2,4,5,5,6};
76 assert(s.findKthLargest(n4, 9) == 1);
77
78 // Con duplicados donde el orden en que entran al heap importa: el valor
79 // repetido debe contarse cada vez que aparece, no solo una vez.
80 vector<int> n5 = {2,2,2,1};
81 // Ordenado desc: 2,2,2,1 -> 3er más grande sigue siendo 2 (no 1).
82 assert(s.findKthLargest(n5, 3) == 2);
83
84 cout << "215. Kth Largest Element in an Array: todas las pruebas pasaron.\n";
85 return 0;
86 }

```

Complejidad: tiempo $O(n \log k)$, espacio $O(k)$.

¿Qué significa esa complejidad? Recorremos los n elementos del arreglo una vez, y cada operación sobre el heap (insertar o sacar) cuesta $\log k$ porque el heap nunca guarda más de k elementos a la vez. Como normalmente k es mucho más pequeño que n , esto es bastante más rápido que ordenar todo el arreglo ($O(n \log n)$), sobre todo cuando k es chico.

Consejos para la entrevista (Amazon). Este problema suele usarse para ver si sabes elegir la estructura correcta según lo que realmente se necesita: aquí no hace falta el orden completo, solo el k -ésimo valor, así que ordenar todo es desperdiciar trabajo. Vale la pena mencionar la alternativa de *quickselect* (una variante de *quicksort* que solo particiona el lado que le interesa), que logra $O(n)$ en promedio pero $O(n^2)$ en el peor caso, contra el $O(n \log k)$ garantizado del heap. Ten cuidado con la trampa de usar un max-heap de todo el arreglo: funciona, pero gasta $O(n)$ de espacio y $O(n + k \log n)$ de tiempo, peor que la versión con heap de tamaño fijo k .

347. Top K Frequent Elements

(M)

Código fuente: `src/04-heaps-topk/347-top-k-frequent-elements.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo `nums` y un entero `k`. Hay que devolver los `k` valores que aparecen con mayor frecuencia en `nums`. El orden de la respuesta no importa, y se garantiza que la respuesta es única (no hay empates ambiguos justo en la posición `k`).

La idea, con un ejemplo de la vida real. Piensa en la lista de canciones más reproducidas de un usuario en un servicio de música: no importa *cuándo* se reprodujo cada canción, solo cuántas veces se reprodujo cada una. Para armar el top `k`, primero cuentas cuántas veces se repite cada canción, y luego agrupas las canciones en “casilleros” según su número de reproducciones: un casillero para las que se reprodujeron una vez, otro para las que se reprodujeron dos veces, y así sucesivamente. Como el número máximo de reproducciones posible es el total de reproducciones registradas, hay un número limitado y conocido de casilleros. Para armar el top `k` basta recorrer los casilleros empezando por el de más reproducciones y bajando, tomando canciones hasta completar `k`.

```
nums = [1,1,1,2,2,3], k = 2
frecuencias: 1 -> 3, 2 -> 2, 3 -> 1
buckets[3]={1}, buckets[2]={2}, buckets[1]={3}
recorriendo de mayor a menor: 1, luego 2 -> respuesta {1, 2}
```

Cómo lo resuelve el código, paso a paso.

1. Contamos cuántas veces aparece cada valor con un mapa de frecuencias.
2. Creamos un arreglo de *buckets* (casilleros) de tamaño $n+1$, donde `buckets[f]` guardará todos los valores que aparecen exactamente `f` veces.
3. Recorremos el mapa de frecuencias y colocamos cada valor en el casillero que le corresponde según su frecuencia.
4. Recorremos los casilleros de mayor a menor frecuencia, agregando sus valores a la respuesta, hasta juntar `k` elementos.

Esto se conoce como *bucket sort* (ordenamiento por casilleros): en vez de comparar elementos entre sí para ordenarlos, usamos directamente el valor que queremos ordenar (aquí, la frecuencia) como índice de un arreglo. Como cada valor cae en exactamente un casillero, llenar los casilleros cuesta $O(n)$, y recorrerlos también.

```
1 //
  // =====
2 // 347. Top K Frequent Elements (Medium)
3 //
  // -----
4 // Nos dan un arreglo nums y un entero k. Hay que devolver los k valores que
5 // aparecen con mayor frecuencia en nums. El orden de la respuesta no importa
6 // (LeetCode acepta cualquier orden), y se garantiza que la respuesta es única
7 // (no hay empates ambiguos en la posición k).
```

```

8 //
9 // Ejemplo: nums = [1,1,1,2,2,3], k = 2
10 // Frecuencias: 1 aparece 3 veces, 2 aparece 2 veces, 3 aparece 1 vez.
11 // Los 2 más frecuentes son 1 y 2 => respuesta {1, 2} (en algún orden).
12 //
13 // Diagrama del algoritmo (bucket sort por frecuencia,  $O(n)$ ):
14 //
15 // freq = mapa valor -> cuántas veces aparece
16 // buckets = arreglo de listas, tamaño n+1
17 // (buckets[f] = lista de valores que aparecen exactamente f veces)
18 // para cada (valor, f) en freq:
19 // buckets[f].push_back(valor)
20 // recorrer buckets desde el índice más alto (n) hacia 0:
21 // por cada valor en buckets[f], agregarlo a la respuesta
22 // parar en cuanto la respuesta tenga k elementos
23 //
24 // Idea: la frecuencia máxima posible de cualquier valor es n (el arreglo
25 // completo es ese valor), así que en vez de ordenar los valores por
26 // frecuencia con un heap o un sort genérico ( $O(n \log n)$ ), usamos la
27 // frecuencia misma como índice de un "casillero" (bucket). Cada casillero
28 // f contiene todos los valores que aparecen exactamente f veces. Como solo
29 // existen n+1 casilleros posibles (de 0 a n) y cada valor cae en exactamente
30 // uno, llenarlos cuesta  $O(n)$ . Luego basta recorrer los casilleros de mayor a
31 // menor frecuencia y tomar valores hasta juntar k: como cada valor solo se
32 // visita una vez en total (está en un único casillero), este recorrido
33 // también es  $O(n)$  en el peor caso.
34 //
35 // Alternativa equivalente en espíritu (no usada aquí): un min-heap de tamaño
36 // k por frecuencia, igual que en 215, con costo  $O(n \log k)$ .
37 //
38 // Complejidad: tiempo  $O(n)$ , espacio  $O(n)$ .
39 //

```

```

40 #include <vector>
41 #include <unordered_map>
42 #include <unordered_set>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 class Solution {
48 public:
49     vector<int> topKFrequent(vector<int>& nums, int k) {
50         unordered_map<int, int> freq;
51         for (int x : nums) freq[x]++;
52
53         int n = nums.size();
54         vector<vector<int>> buckets(n + 1); // buckets[f] = valores con
55                                           frecuencia f
56         for (auto& [valor, f] : freq) {
57             buckets[f].push_back(valor);
58         }

```

```

59     vector<int> resultado;
60     for (int f = n; f >= 0 && (int)resultado.size() < k; --f) {
61         for (int valor : buckets[f]) {
62             resultado.push_back(valor);
63             if ((int)resultado.size() == k) break;
64         }
65     }
66     return resultado;
67 }
68 };
69
70 int main() {
71     Solution s;
72
73     // Caso normal. El orden no está garantizado por el algoritmo, así que
74     // comparamos el CONJUNTO de valores, no el orden exacto.
75     vector<int> n1 = {1,1,1,2,2,3};
76     vector<int> r1 = s.topKFrequent(n1, 2);
77     unordered_set<int> set1(r1.begin(), r1.end());
78     assert(set1 == unordered_set<int>({1, 2}));
79     assert(r1.size() == 2);
80
81     // Un solo elemento distinto, k = 1.
82     vector<int> n2 = {1};
83     vector<int> r2 = s.topKFrequent(n2, 1);
84     assert(r2 == vector<int>({1}));
85
86     // k igual a la cantidad de valores distintos: todos deben salir.
87     vector<int> n3 = {4,4,5,5,6};
88     vector<int> r3 = s.topKFrequent(n3, 3);
89     unordered_set<int> set3(r3.begin(), r3.end());
90     assert(set3 == unordered_set<int>({4, 5, 6}));
91
92     // Caso donde el orden de recorrido de buckets importa: valores con
93     // frecuencias muy distintas, y el valor de mayor frecuencia absoluta
94     // debe salir primero al truncar en k = 1.
95     vector<int> n4 = {7,7,7,7,8,9,9};
96     vector<int> r4 = s.topKFrequent(n4, 1);
97     assert(r4 == vector<int>({7})); // 7 aparece 4 veces, es el único con
98     // esa frecuencia
99
100     cout << "347. Top K Frequent Elements: todas las pruebas pasaron.\n";
101     return 0;
102 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? Tanto contar frecuencias como llenar y recorrer los casilleros toma un tiempo proporcional al tamaño del arreglo original, sin ningún logaritmo de por medio: es más rápido que cualquier solución basada en ordenar ($O(n \log n)$) o incluso que un min-heap de tamaño k ($O(n \log k)$), aunque ambas alternativas también son aceptables en una entrevista.

Consejos para la entrevista (Amazon). Si te preguntan la primera solución que se te ocurre, es normal mencionar primero el min-heap de tamaño k (el mismo patrón del problema 215): es más fácil de explicar y sigue siendo eficiente. El *bucket sort* es la mejora “inteligente” que muestra que reconoces cuando el rango de valores posibles (aquí, la frecuencia máxima es n) es lo bastante acotado como para usarlo directamente de índice. Menciona explícitamente por qué el tamaño del arreglo de buckets es $n + 1$ y no algo arbitrario: la frecuencia de cualquier valor nunca puede superar el tamaño del arreglo de entrada.

973. K Closest Points to Origin

(M)

Código fuente: `src/04-heaps-topk/973-k-closest-points.cpp`▷ Ver animación · [local](#) | [web](#)

Nos dan una lista de puntos en el plano, `points[i] = [x, y]`, y un entero `k`. Hay que devolver los `k` puntos más cercanos al origen $(0, 0)$, usando la distancia euclidiana. El orden de la respuesta no importa.

La idea, con un ejemplo de la vida real. Imagina que estás eligiendo las `k` sucursales más cercanas a tu casa entre una lista larga de direcciones. No necesitas ordenar *todas* las sucursales por distancia, solo necesitas quedarte con las `k` más cercanas vistas hasta ahora, y cada vez que aparece una candidata mejor, descartar a la peor de tu grupo actual. Ese es exactamente el mismo patrón del problema 215 (*Kth Largest Element*), pero invertido: en vez de quedarnos con los más grandes usando un min-heap, aquí nos quedamos con los más cercanos usando un *max-heap* (montículo máximo), porque lo que queremos descartar rápido es al candidato más **lejano**.

Un detalle práctico: para comparar distancias no hace falta calcular la raíz cuadrada. Si $a^2 < b^2$ con $a, b \geq 0$, entonces $a < b$; comparar las distancias al cuadrado da el mismo orden que comparar las distancias reales, y evita el costo (y los posibles errores de precisión) de `sqrt`.

```
points = [[1,3],[-2,2]], k = 1
d2([1,3]) = 1+9 = 10
d2([-2,2]) = 4+4 = 8
8 < 10 -> respuesta {[-2,2]}
```

Cómo lo resuelve el código, paso a paso.

1. Creamos un max-heap vacío, donde cada elemento es un par (distancia al cuadrado, índice del punto).
2. Por cada punto, calculamos su distancia al cuadrado al origen y lo insertamos en el heap.
3. Si el heap crece más de `k` elementos, sacamos el de mayor distancia: es el candidato menos atractivo para el top-`k` cercano.
4. Al terminar, el heap contiene exactamente los `k` puntos más cercanos al origen de toda la lista.

```
1 //
2 // =====
3 //
4 // 973. K Closest Points to Origin (Medium)
5 //
6 // -----
7 //
8 // Nos dan una lista de puntos en el plano, points[i] = [x, y], y un entero k.
9 // Hay que devolver los k puntos más cercanos al origen (0, 0), usando la
10 // distancia euclidiana. El orden de la respuesta no importa.
11 //
12 // Ejemplo: points = [[1,3],[-2,2]], k = 1
13 // distancia al origen de [1,3] = sqrt(1^2 + 3^2) = sqrt(10) ~= 3.16
14 // distancia al origen de [-2,2] = sqrt((-2)^2+2^2) = sqrt(8) ~= 2.83
15 // [-2,2] está más cerca => respuesta {[-2,2]}.
```

```

12 //
13 // Diagrama del algoritmo (max-heap de tamaño k por distancia al cuadrado):
14 //
15 //   heap = [] (max-heap, la cima siempre es el MÁS LEJANO del heap)
16 //   para cada punto p en points:
17 //       d2 = distancia al cuadrado de p al origen (x*x + y*y, sin sqrt)
18 //       heap.push((d2, p))
19 //       si heap.size() > k:
20 //           heap.pop() // se saca el más lejano: ya no puede ser top-k
//           cercano
21 //   al terminar, el heap contiene los k puntos más cercanos
22 //
23 // Idea: comparar distancias al cuadrado da el mismo orden que comparar
24 // distancias reales (la raíz cuadrada es una función creciente, así que si
25 //  $a^2 < b^2$  con  $a, b \geq 0$  entonces  $a < b$ ). Evitamos calcular sqrt, que es más
26 // lento y puede introducir errores de precisión con floats, sin cambiar el
27 // resultado. Usamos un max-heap (en vez de min-heap como en 215) porque aquí
28 // queremos descartar rápido a los puntos MÁS LEJANOS: mantenemos los k
29 // puntos más cercanos vistos hasta el momento, y cuando el heap crece más
30 // allá de k, el que sobra es el más lejano de esos k+1 (el candidato menos
31 // atractivo para estar en el top-k cercano), así que lo sacamos.
32 //
33 // Por qué funciona: mismo invariante que 215 mirado al revés. Después de
34 // procesar cualquier prefijo de points, el heap contiene los
35 // min(k, puntos_vistos) puntos más CERCANOS de ese prefijo. Al final el
36 // heap tiene los k más cercanos de la lista completa.
37 //
38 // Complejidad: tiempo  $O(n \log k)$ , espacio  $O(k)$ .
39 //
=====

40 #include <vector>
41 #include <queue>
42 #include <unordered_set>
43 #include <cassert>
44 #include <iostream>
45 using namespace std;
46
47 class Solution {
48 public:
49     vector<vector<int>>> kClosest(vector<vector<int>>> points, int k) {
50         // Max-heap por distancia al cuadrado: pair<distancia^2, índice del
//         punto>.
51         priority_queue<pair<long long, int>> heap;
52         for (int i = 0; i < (int)points.size(); ++i) {
53             long long x = points[i][0], y = points[i][1];
54             long long d2 = x * x + y * y;
55             heap.push({d2, i});
56             if ((int)heap.size() > k) {
57                 heap.pop(); // sacamos el más lejano: no puede ser top-k
//                 cercano
58             }
59         }
60     }

```

```

61         vector<vector<int>> resultado;
62         while (!heap.empty()) {
63             resultado.push_back(points[heap.top().second]);
64             heap.pop();
65         }
66         return resultado;
67     }
68 };
69
70 int main() {
71     Solution s;
72
73     // Caso normal.
74     vector<vector<int>> p1 = {{1,3},{-2,2}};
75     vector<vector<int>> r1 = s.kClosest(p1, 1);
76     assert(r1.size() == 1);
77     assert(r1[0] == vector<int>({-2, 2}));
78
79     // Un solo punto, k = 1.
80     vector<vector<int>> p2 = {{5,5}};
81     vector<vector<int>> r2 = s.kClosest(p2, 1);
82     assert(r2.size() == 1);
83     assert(r2[0] == vector<int>({5, 5}));
84
85     // k igual al tamaño de la lista: deben salir todos los puntos (orden no
86     // garantizado, así que comparamos como conjunto de pares).
87     vector<vector<int>> p3 = {{3,3},{5,-1},{-2,4}};
88     vector<vector<int>> r3 = s.kClosest(p3, 3);
89     assert(r3.size() == 3);
90     auto asPairSet = [](vector<vector<int>>& v) {
91         unordered_set<long long> out;
92         for (auto& pt : v) out.insert(((long long)(pt[0] + 100) << 32) ^ (pt[1] + 100));
93         return out;
94     };
95     assert(asPairSet(r3) == asPairSet(p3));
96
97     // Caso donde el orden de procesamiento importa: tres puntos con
98     // 1, 100, 4 (al cuadrado); k = 2 debe quedarse con los de distancia 1 y
99     // 4,
100     // descartando el de distancia 100 sin importar en qué orden llegaron.
101     vector<vector<int>> p4 = {{10,0},{1,0},{0,2}};
102     // d2: [10,0]=100, [1,0]=1, [0,2]=4
103     vector<vector<int>> r4 = s.kClosest(p4, 2);
104     assert(r4.size() == 2);
105     unordered_set<long long> got4;
106     for (auto& pt : r4) got4.insert(((long long)(pt[0] + 100) << 32) ^ (pt[1]
107         + 100));
108     unordered_set<long long> expect4 = {
109         ((long long)(1 + 100) << 32) ^ (0 + 100),
110         ((long long)(0 + 100) << 32) ^ (2 + 100)
111     };
112     assert(got4 == expect4);

```

```
111
112     cout << "973. K Closest Points to Origin: todas las pruebas pasaron.\n";
113     return 0;
114 }
```

Complejidad: tiempo $O(n \log k)$, espacio $O(k)$.

¿Qué significa esa complejidad? Igual que en el problema 215: recorreremos los n puntos una vez, y cada operación del heap cuesta $\log k$ porque nunca guardamos más de k elementos. Cuando k es mucho menor que n , esto es notablemente más rápido que ordenar todos los puntos por distancia.

Consejos para la entrevista (Amazon). El punto clave que suelen evaluar aquí es si reconoces que *no* hace falta `sqrt`: mencionarlo proactivamente muestra atención al detalle. También es común que pregunten por qué se usa un max-heap y no un min-heap, a diferencia del problema 215: la respuesta es que aquí queremos descartar rápido a los **lejanos**, así que necesitamos tener el más lejano del grupo actual siempre a la mano en la cima. Como en 215, quickselect es una alternativa válida con mejor tiempo promedio pero peor peor-caso.

295. Find Median from Data Stream

(H)

Código fuente: [src/04-heaps-topk/295-find-median-data-stream.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Hay que diseñar una estructura `MedianFinder` que reciba números uno a uno mediante `addNum` y en cualquier momento pueda devolver, con `findMedian`, la mediana de todos los números recibidos hasta ese instante. Si la cantidad de números vistos es impar, la mediana es el de en medio; si es par, es el promedio de los dos de en medio.

La idea, con un ejemplo de la vida real. Imagina una fila de personas ordenadas de menor a mayor estatura, y quieres saber en todo momento quién está justo en medio, aunque la fila crezca constantemente y nunca la reordenes por completo. Una forma práctica es partir la fila en dos mitades desde el principio: los más bajitos a la izquierda, los más altos a la derecha, cuidando que ambos lados tengan casi el mismo número de personas. La mediana siempre está en la frontera entre ambas mitades: la persona más alta del lado izquierdo, la más baja del lado derecho, o el promedio de ambas si las dos mitades tienen el mismo tamaño.

Para mantener esa frontera accesible en todo momento sin reordenar la fila completa, guardamos la mitad izquierda en un *max-heap* (para consultar su máximo, es decir, el más alto del lado bajo, en tiempo constante) y la mitad derecha en un *min-heap* (para consultar su mínimo, el más bajo del lado alto, también en tiempo constante).

```
addNum(1) -> stream: [1]
addNum(2) -> stream: [1,2] -> mediana = 1.5
addNum(3) -> stream: [1,2,3] -> mediana = 2
```

Cómo lo resuelve el código, paso a paso.

1. Mantenemos dos heaps: *izquierda* (max-heap, la mitad menor de los números vistos) y *derecha* (min-heap, la mitad mayor).
2. Al insertar un número nuevo, decidimos a qué heap va comparándolo con la cima de *izquierda*: si es menor o igual (o si *izquierda* está vacía), va a *izquierda*; si no, va a *derecha*.
3. Después de cada inserción, rebalanceamos: si algún heap le lleva más de un elemento de ventaja al otro, movemos su cima al heap contrario. Esto mantiene la diferencia de tamaños en, a lo más, 1.
4. Para consultar la mediana: si ambos heaps tienen el mismo tamaño, es el promedio de sus dos cimas; si no, es la cima del heap más grande.

El invariante clave que se mantiene todo el tiempo es que *todo* elemento de *izquierda* es menor o igual que *todo* elemento de *derecha*. Como cada inserción respeta ese orden y el rebalanceo solo mueve el elemento frontera de un heap al otro, ese invariante nunca se rompe, y por eso la mediana siempre puede leerse directamente de las cimas.

```
1 //
  // =====
2 // 295. Find Median from Data Stream (Hard)
```



```

53 #include <queue>
54 #include <vector>
55 #include <cassert>
56 #include <cmath>
57 #include <iostream>
58 using namespace std;
59
60 class MedianFinder {
61 public:
62     MedianFinder() {}
63
64     void addNum(int num) {
65         if (izquierda.empty() || num <= izquierda.top()) {
66             izquierda.push(num);
67         } else {
68             derecha.push(num);
69         }
70
71         // Rebalancear: ningún heap puede tener más de 1 elemento de más.
72         if (izquierda.size() > derecha.size() + 1) {
73             derecha.push(izquierda.top());
74             izquierda.pop();
75         } else if (derecha.size() > izquierda.size() + 1) {
76             izquierda.push(derecha.top());
77             derecha.pop();
78         }
79     }
80
81     double findMedian() {
82         if (izquierda.size() == derecha.size()) {
83             return (izquierda.top() + derecha.top()) / 2.0;
84         } else if (izquierda.size() > derecha.size()) {
85             return izquierda.top();
86         } else {
87             return derecha.top();
88         }
89     }
90
91 private:
92     priority_queue<int> izquierda; // max-
93     // heap, mitad menor
94     priority_queue<int, vector<int>, greater<>> derecha; // min-
95     // heap, mitad mayor
96 };
97
98 int main() {
99     // Caso normal del enunciado.
100     MedianFinder mf1;
101     mf1.addNum(1);
102     assert(fabs(mf1.findMedian() - 1.0) < 1e-9);
103     mf1.addNum(2);
104     assert(fabs(mf1.findMedian() - 1.5) < 1e-9);
105     mf1.addNum(3);

```

```

104     assert(fabs(mf1.findMedian() - 2.0) < 1e-9);
105
106     // Caso límite: un solo número.
107     MedianFinder mf2;
108     mf2.addNum(42);
109     assert(fabs(mf2.findMedian() - 42.0) < 1e-9);
110
111     // Caso donde el orden de llegada importa: números que llegan desordenados
112     // deben seguir balanceándose correctamente entre los dos heaps.
113     MedianFinder mf3;
114     mf3.addNum(5);
115     mf3.addNum(15);
116     mf3.addNum(1);
117     // stream ordenado: 1, 5, 15 -> mediana = 5
118     assert(fabs(mf3.findMedian() - 5.0) < 1e-9);
119     mf3.addNum(3);
120     // stream ordenado: 1, 3, 5, 15 -> mediana = (3+5)/2 = 4
121     assert(fabs(mf3.findMedian() - 4.0) < 1e-9);
122     mf3.addNum(8);
123     // stream ordenado: 1, 3, 5, 8, 15 -> mediana = 5
124     assert(fabs(mf3.findMedian() - 5.0) < 1e-9);
125
126     // Caso con muchos números repetidos.
127     MedianFinder mf4;
128     mf4.addNum(2);
129     mf4.addNum(2);
130     mf4.addNum(2);
131     // stream: 2, 2, 2 -> mediana = 2
132     assert(fabs(mf4.findMedian() - 2.0) < 1e-9);
133
134     cout << "295. Find Median from Data Stream: todas las pruebas pasaron.\n";
135     return 0;
136 }

```

Complejidad: tiempo $O(\log n)$ por `addNum`, $O(1)$ por `findMedian`, espacio $O(n)$.

¿Qué significa esa complejidad? Cada número que entra puede costar hasta dos operaciones de heap (una inserción y, en el peor caso, un rebalanceo), cada una $O(\log n)$ porque los heaps van creciendo con el stream. En cambio, consultar la mediana no requiere ningún trabajo extra: las cimas de ambos heaps ya están calculadas, así que es simplemente leerlas.

Consejos para la entrevista (Amazon). Este es un problema clásico de diseño de estructura de datos con heaps, y suelen evaluar si reconoces cuándo un problema deja de ser “calcula una respuesta” y pasa a ser “mantén una respuesta actualizable eficientemente” conforme llegan datos nuevos. El error más común es rebalancear mal: olvidar el caso en que *izquierda* está vacía al decidir dónde insertar el primer número, o permitir que la diferencia de tamaños crezca más de 1 (rompiendo la garantía de $O(1)$ en `findMedian`). Vale la pena aclarar en voz alta por qué se usa `<=` y no `<` al decidir el heap destino: con duplicados, cualquiera de las dos convenciones funciona mientras se use de forma consistente.

23. Merge k Sorted Lists

(H)

Código fuente: [src/04-heaps-topk/23-merge-k-sorted-lists.cpp](#)

▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de k listas enlazadas, cada una ya ordenada de menor a mayor. Hay que fusionarlas en una sola lista enlazada, también ordenada, y devolver su cabeza.

La idea, con un ejemplo de la vida real. Fusionar dos listas ordenadas ya es un ejercicio clásico: comparas las dos cabezas, te quedas con la más chica, avanzas esa lista, y repites. Aquí el reto es que hay k listas en vez de dos. Imagina k filas de personas, cada fila ya ordenada de menor a mayor estatura, y quieres formar una sola fila combinada ordenada. En cada paso necesitas saber cuál de las k personas al frente de cada fila es la más bajita; compararlas una por una cada vez que avanzas sería lento si k es grande. En vez de eso, metes a las k personas que están al frente de cada fila en un *min-heap*: sacar a la más bajita de un grupo de k candidatos cuesta mucho menos que compararlas todas cada vez.

```
lists = [[1,4,5],[1,3,4],[2,6]]
fusión: 1,1,2,3,4,4,5,6
```

Cómo lo resuelve el código, paso a paso.

1. Metemos al min-heap la cabeza de cada lista no vacía (el primer nodo de cada una).
2. Creamos un nodo `dummy` (ancla vacía) para ir construyendo la lista de respuesta, y un puntero `cola` que empieza apuntando a ese `dummy`.
3. Mientras el heap no esté vacío: sacamos el nodo con menor valor, lo enganchamos al final de la respuesta (`cola->next`) y avanzamos `cola`.
4. Si el nodo que acabamos de sacar tenía un siguiente (su lista original no se había terminado), metemos ese siguiente al heap: se vuelve el nuevo candidato de esa lista.
5. Devolvemos `dummy->next`: el primer nodo real de la fusión.

En cualquier momento, el heap contiene exactamente un candidato por cada lista que todavía tiene nodos sin usar (su nodo actual, el más pequeño no usado de esa lista en particular, porque las listas ya vienen ordenadas). El menor de todos esos candidatos es, por lo tanto, el menor valor no usado de *todas* las listas juntas, así que siempre es seguro colocarlo a continuación.

```
1 //
2 // =====
3 //
4 // 23. Merge k Sorted Lists (Hard)
5 //
6 // -----
7 //
8 // Nos dan un arreglo de k listas enlazadas, cada una ya ordenada de menor a
9 // mayor. Hay que fusionarlas en una sola lista enlazada, también ordenada, y
10 // devolver su cabeza.
11 //
12 // Ejemplo: lists = [[1,4,5],[1,3,4],[2,6]]
13 // Fusión ordenada de las tres: 1,1,2,3,4,4,5,6
```

```

10 //
11 // Diagrama del algoritmo (min-heap con la cabeza actual de cada lista):
12 //
13 //   heap = [] (min-heap por valor del nodo)
14 //   para cada lista no vacía, meter su primer nodo al heap
15 //   dummy = nodo vacío que sirve de ancla para armar la respuesta
16 //   cola = dummy
17 //   mientras el heap no esté vacío:
18 //       sacar el nodo con menor valor
19 //       cola.next = ese nodo; cola = cola.next
20 //       si ese nodo tiene un siguiente, meter ese siguiente al heap
21 //   devolver dummy.next
22 //
23 // Idea: fusionar dos listas ordenadas es fácil (comparar sus cabezas y
24 // avanzar la más chica); fusionar k a la vez es lo mismo pero comparando k
25 // candidatos en lugar de 2. En vez de comparar las k cabezas una por una en
26 // cada paso (que sería  $O(k)$  por paso), las metemos en un min-heap: sacar el
27 // menor de k candidatos cuesta  $O(\log k)$  en vez de  $O(k)$ . Cada vez que
28 // "consumimos" un nodo de una lista, su siguiente nodo se vuelve el nuevo
29 // candidato de esa lista y entra al heap.
30 //
31 // Por qué funciona: en cualquier momento, el heap contiene exactamente un
32 // candidato por cada lista que todavía tiene nodos sin usar (su nodo actual,
33 // el más pequeño no usado de esa lista, porque las listas ya vienen
34 // ordenadas). El menor de todos esos candidatos es forzosamente el menor
35 // valor no usado de TODAS las listas juntas, así que es seguro colocarlo a
36 // continuación en la respuesta.
37 //
38 // Complejidad: tiempo  $O(N \log k)$ , donde N es el total de nodos en todas las
39 // listas y k es el número de listas. Espacio  $O(k)$  para el heap (más  $O(N)$ 
40 // para la lista de salida, que no cuenta como espacio extra si se reutilizan
41 // los nodos originales, como hace este código).
42 //
=====

43 #include <vector>
44 #include <queue>
45 #include <cassert>
46 #include <iostream>
47 using namespace std;
48
49 struct ListNode {
50     int val;
51     ListNode *next;
52     ListNode(int x) : val(x), next(nullptr) {}
53 };
54
55 class Solution {
56 public:
57     ListNode* mergeKLists(vector<ListNode*>& lists) {
58         // Min-heap por valor: pair<valor, nodo>. Se compara por valor primero
59         // ;
60         // si dos valores empatan, comparar punteros solo desempata (no
61         // importa

```

```

60      // el orden real entre ellos).
61      auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
62      priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> heap(cmp);
63
64      for (ListNode* nodo : lists) {
65          if (nodo != nullptr) heap.push(nodo);
66      }
67
68      ListNode dummy(0);
69      ListNode* cola = &dummy;
70
71      while (!heap.empty()) {
72          ListNode* menor = heap.top();
73          heap.pop();
74          cola->next = menor;
75          cola = cola->next;
76          if (menor->next != nullptr) heap.push(menor->next);
77      }
78      cola->next = nullptr;
79
80      return dummy.next;
81  }
82 };
83
84 int main() {
85     Solution s;
86
87     // Caso normal: lists = [[1,4,5],[1,3,4],[2,6]]
88     ListNode* l1a = new ListNode(1);
89     l1a->next = new ListNode(4);
90     l1a->next->next = new ListNode(5);
91
92     ListNode* l1b = new ListNode(1);
93     l1b->next = new ListNode(3);
94     l1b->next->next = new ListNode(4);
95
96     ListNode* l1c = new ListNode(2);
97     l1c->next = new ListNode(6);
98
99     vector<ListNode*> lists1 = {l1a, l1b, l1c};
100    ListNode* r1 = s.mergeKLists(lists1);
101    vector<int> esperado1 = {1,1,2,3,4,4,5,6};
102    ListNode* cur1 = r1;
103    for (int v : esperado1) {
104        assert(cur1 != nullptr);
105        assert(cur1->val == v);
106        cur1 = cur1->next;
107    }
108    assert(cur1 == nullptr);    // la lista terminó justo donde debía
109
110    // Caso límite: arreglo de listas vacío.
111    vector<ListNode*> lists2 = {};
112    ListNode* r2 = s.mergeKLists(lists2);
113    assert(r2 == nullptr);

```

```

114
115 // Caso límite: todas las listas están vacías (nullptr).
116 vector<ListNode*> lists3 = {nullptr, nullptr};
117 ListNode* r3 = s.mergeKLists(lists3);
118 assert(r3 == nullptr);
119
120 // Una sola lista no vacía entre varias vacías.
121 ListNode* l4a = new ListNode(5);
122 vector<ListNode*> lists4 = {nullptr, l4a, nullptr};
123 ListNode* r4 = s.mergeKLists(lists4);
124 assert(r4 != nullptr);
125 assert(r4->val == 5);
126 assert(r4->next == nullptr);
127
128 // Caso donde el orden de procesamiento del heap importa: listas con
129 // valores entrelazados donde tomar la lista "equivocada" primero rompería
130 // el orden final.
131 ListNode* l5a = new ListNode(10);
132 l5a->next = new ListNode(20);
133 ListNode* l5b = new ListNode(1);
134 l5b->next = new ListNode(15);
135 l5b->next->next = new ListNode(30);
136 vector<ListNode*> lists5 = {l5a, l5b};
137 ListNode* r5 = s.mergeKLists(lists5);
138 vector<int> esperado5 = {1,10,15,20,30};
139 ListNode* cur5 = r5;
140 for (int v : esperado5) {
141     assert(cur5 != nullptr);
142     assert(cur5->val == v);
143     cur5 = cur5->next;
144 }
145 assert(cur5 == nullptr);
146
147 cout << "23. Merge k Sorted Lists: todas las pruebas pasaron.\n";
148 return 0;
149 }

```

Complejidad: tiempo $O(N \log k)$, espacio $O(k)$.

¿Qué significa esa complejidad? N es el número total de nodos en todas las listas juntas, y k es el número de listas. Cada uno de los N nodos entra y sale del heap exactamente una vez, y cada una de esas operaciones cuesta $\log k$ porque el heap nunca tiene más de k elementos a la vez (uno por lista activa). El espacio extra es solo $O(k)$ para el heap, ya que reutilizamos los nodos originales para construir la lista de salida en vez de crear nodos nuevos.

Consejos para la entrevista (Amazon). Este problema combina dos técnicas que suelen evaluar por separado: fusión de listas enlazadas y uso de heaps para elegir entre múltiples candidatos. Vale la pena mencionar la alternativa de fusionar las listas de dos en dos (fusión por pares, similar a merge sort), que también logra $O(N \log k)$ pero sin necesitar un heap explícito. Un detalle de implementación en C++ que suelen preguntar: cómo comparar punteros a nodos dentro de un `priority_queue` usando un comparador personalizado, ya que no se puede comparar directamente `ListNode*` por valor. También aclara qué pasa si alguna lista viene vacía (`nullptr`) o si el arreglo

621. Task Scheduler

(M)

Código fuente: [src/04-heaps-topk/621-task-scheduler.cpp](#)▷ Ver animación · [local](#) | [web](#)

Nos dan un arreglo de tareas (cada una representada por una letra de la A a la Z) y un entero n : el número mínimo de unidades de tiempo que deben pasar entre dos ejecuciones de la *misma* tarea (su “enfriamiento”, o *cooldown*). En cada unidad de tiempo se puede ejecutar una tarea o quedarse inactivo. Hay que devolver el número mínimo de unidades de tiempo necesarias para ejecutar todas las tareas respetando el enfriamiento.

La idea, con un ejemplo de la vida real. Imagina una lavadora que, después de lavar una prenda de cierto color, necesita esperar n ciclos antes de volver a lavar otra prenda del mismo color (para evitar que se manchen entre sí). Si tienes muchas prendas del mismo color, conviene lavar primero las que **más** se repiten, porque esas son las que más tiempo de espera van a necesitar acumulado; dejarlas para el final solo alarga más el proceso total. Por eso en cada turno conviene elegir, entre las prendas disponibles en ese momento, la del color con más prendas pendientes: eso es justo lo que resuelve un *max-heap* de conteos.

Las prendas que se acaban de lavar no están disponibles de inmediato: se guardan aparte, en una cola de “enfriamiento”, hasta que les toque volver a estar disponibles.

```
tasks = [A,A,A,B,B,B], n = 2
horario: A, B, idle, A, B, idle, A, B -> 8 unidades
```

Cómo lo resuelve el código, paso a paso.

1. Contamos cuántas veces aparece cada tarea con un mapa de frecuencias, y metemos esos conteos (solo el número, no la letra) en un max-heap.
2. Usamos una cola de enfriamiento donde guardamos pares (conteo restante, momento en que puede volver a estar disponible).
3. En cada unidad de tiempo: primero revisamos si el frente de la cola de enfriamiento ya puede regresar al heap (si su momento de regreso coincide con el tiempo actual), y si es así, lo regresamos.
4. Luego, si el heap no está vacío, sacamos el conteo más alto, le restamos uno (la ejecutamos), y si todavía le queda conteo pendiente, la mandamos a la cola de enfriamiento con su nuevo momento de regreso ($\text{tiempo} + n + 1$).
5. Si el heap estaba vacío en ese instante pero aún quedan tareas en enfriamiento, esa unidad de tiempo se pierde como inactividad.
6. Repetimos hasta que tanto el heap como la cola de enfriamiento estén vacíos.

```
1 //
   =====
2 // 621. Task Scheduler (Medium)
3 //
```



```
55 #include <queue>
56 #include <unordered_map>
57 #include <cassert>
58 #include <iostream>
59 using namespace std;
60
61 class Solution {
62 public:
63     int leastInterval(vector<char>& tasks, int n) {
64         unordered_map<char, int> freq;
65         for (char t : tasks) freq[t]++;
66
67         priority_queue<int> heap;    // max-heap de conteos
68         for (auto& [letra, f] : freq) heap.push(f);
69
70         // cooldown: pares (conteo_restante, tiempo_en_que_puede_volver)
71         queue<pair<int, int>> cooldown;
72
73         int tiempo = 0;
74         while (!heap.empty() || !cooldown.empty()) {
75             tiempo++;
76
77             if (!cooldown.empty() && cooldown.front().second == tiempo) {
78                 heap.push(cooldown.front().first);
79                 cooldown.pop();
80             }
81
82             if (!heap.empty()) {
83                 int cuenta = heap.top();
84                 heap.pop();
85                 cuenta--;
86                 if (cuenta > 0) {
87                     cooldown.push({cuenta, tiempo + n + 1});
88                 }
89             }
90             // si el heap estaba vacío aquí, esta unidad de tiempo fue idle
91         }
92
93         return tiempo;
94     }
95 };
96
97 int main() {
98     Solution s;
99
100     // Caso normal del enunciado.
101     vector<char> t1 = {'A', 'A', 'A', 'B', 'B', 'B'};
102     assert(s.leastInterval(t1, 2) == 8);
103
104     // Sin enfriamiento (n = 0): se ejecutan una tras otra sin huecos.
105     vector<char> t2 = {'A', 'A', 'A', 'B', 'B', 'B'};
106     assert(s.leastInterval(t2, 0) == 6);
107
108     // Una sola tarea repetida: obliga a meter huecos de enfriamiento.
```



```
109     vector<char> t3 = {'A','A','A','A'};  
110     // Horario: A _ _ A _ _ A _ _ A (n=2, entre cada A deben pasar 2 huecos)  
111     assert(s.leastInterval(t3, 2) == 10);  
112  
113     // Caso límite: una sola tarea, una sola vez.  
114     vector<char> t4 = {'A'};  
115     assert(s.leastInterval(t4, 5) == 1);  
116  
117     // Muchas tareas distintas y n grande: como no hay repeticiones que  
118     // choquen, el enfriamiento nunca obliga a meter huecos idle.  
119     vector<char> t5 = {'A','B','C','D'};  
120     assert(s.leastInterval(t5, 10) == 4);  
121  
122     cout << "621. Task Scheduler: todas las pruebas pasaron.\n";  
123     return 0;  
124 }
```

Complejidad: tiempo $O(T)$, espacio $O(1)$.

¿Qué significa esa complejidad? T es el número total de tareas. Aunque cada unidad de tiempo puede requerir una operación de heap, el heap nunca tiene más de 26 elementos distintos (uno por letra del alfabeto), así que su costo logarítmico es una constante que no depende de T ; por eso el tiempo total es lineal en T . El espacio también es constante, ya que como mucho guardamos 26 conteos distintos sin importar cuántas tareas haya en total.

Consejos para la entrevista (Amazon). Este problema es un ejemplo clásico de estrategia *greedy* (voraz): en cada paso se toma la decisión que parece mejor localmente (ejecutar la tarea con más conteo restante), y hay que poder justificar por qué esa elección local nunca perjudica el resultado global. También existe una fórmula matemática cerrada basada en la frecuencia máxima y cuántas tareas empatan en esa frecuencia máxima, que da la respuesta sin simular nada; mencionarla es un plus, pero la simulación con heap y cola de enfriamiento es más fácil de razonar y de explicar en la pizarra, y generaliza mejor si el entrevistador cambia las reglas del problema a mitad de la conversación.

Capítulo 5

Sliding Window y Pilas

3. Longest Substring Without Repeating Characters

(M)

Código fuente: `src/06-sliding-window-stack/3-longest-substring-without-repeating.cpp`

▷ Ver animación · [local](#) | [web](#)

Dada una cadena `s`, devuelve la longitud de la subcadena *contigua* más larga que no repite ningún carácter.

La idea, con un ejemplo de la vida real. Imagina que caminas por un pasillo lleno de gente y quieres encontrar el tramo más largo del pasillo donde no te vuelvas a topar con la misma persona dos veces. Vas avanzando y llevas la cuenta de en qué lugar del pasillo viste por última vez a cada persona. En cuanto te topas otra vez con alguien que ya conoces *dentro del tramo que llevas contando*, sabes que ese tramo ya no sirve desde el principio: tienes que empezar a contar de nuevo justo después del lugar donde la viste la vez anterior. A esta técnica de mantener un rango `[L, R]` que se expande y se contrae según lo que vamos viendo se le llama **sliding window** (ventana deslizante).

```
a b c a b c b b
L           R (al toparnos con la segunda `a`, movemos L)
```

Con `s = "abcabcbb"`, la subcadena más larga sin repetidos es "abc", de longitud 3.

Cómo lo resuelve el código, paso a paso. Guardamos, en un mapa, el último índice en el que vimos cada carácter. Con dos punteros `L` (izquierdo) y `R` (derecho) definimos la ventana actual.

1. Recorremos la cadena con `R` de izquierda a derecha.
2. Para el carácter en `R`, revisamos si ya lo vimos antes *y* si esa aparición anterior cae dentro de la ventana actual (es decir, su índice es mayor o igual que `L`).
3. Si es así, la ventana ya no es válida desde `L`: movemos `L` justo un lugar después de esa aparición anterior.
4. Actualizamos el último índice visto del carácter actual a `R`.
5. La longitud de la ventana en este momento es `R - L + 1`; nos quedamos con la más grande vista hasta ahora.

El detalle importante está en el paso 2: si vimos el carácter antes pero esa aparición ya quedó *fuera* de la ventana actual (porque `L` ya había avanzado por otra razón), no hay que retroceder `L`: eso rompería la ventana sin necesidad.

```
1 //
   =====
2 // 3. Longest Substring Without Repeating Characters (Medium)
3 //
   -----
4 // Dada una cadena s, devuelve la longitud de la subcadena CONTIGUA más larga
5 // que no repite ningún carácter.
6 //
7 // Ejemplo: s = "abcabcbb"
```

```

8 //
9 //   a b c a b c b b
10 //   ^       ^
11 //   L       R   (al llegar a la segunda 'a', hay que mover L)
12 //
13 //   La subcadena más larga sin repetidos es "abc", de longitud 3.
14 //
15 // Diagrama de sliding window (ventana deslizante, dos punteros L/R que
16 // definen un rango contiguo [L, R]):
17 //
18 //   ultimoIndice = mapa vacio // último índice donde vimos cada carácter
19 //   L = 0
20 //   mejor = 0
21 //   para R desde 0 hasta el final de s:
22 //       c = s[R]
23 //       si c ya está en ultimoIndice Y ese índice es >= L:
24 //           // el carácter repetido cae DENTRO de la ventana actual: hay que
25 //           // cerrarla justo después de esa aparición anterior
26 //           L = ultimoIndice[c] + 1
27 //           ultimoIndice[c] = R
28 //           mejor = max(mejor, R - L + 1)
29 //
30 // Idea: en vez de, al encontrar un repetido, mover L de uno en uno hasta
31 // salir del carácter repetido (que sería  $O(n)$  por cada R, y  $O(n^2)$  en
32 // total), guardamos el último índice donde vimos cada carácter y saltamos L
33 // directo justo después de esa posición. Esto es correcto porque cualquier
34 // subcadena que empiece antes de esa posición y llegue hasta R tendría el
35 // carácter repetido dos veces, así que no sirve. El truco de la condición
36 // "ese índice es >= L" es importante: si vimos el carácter antes pero esa
37 // aparición ya quedó fuera de la ventana actual (porque L avanzó por otra
38 // razón), no hay que retroceder L.
39 //
40 // Complejidad: tiempo  $O(n)$ , espacio  $O(\min(n, \text{tamaño del alfabeto}))$ .
41 //
=====

42 #include <string>
43 #include <unordered_map>
44 #include <algorithm>
45 #include <cassert>
46 #include <iostream>
47 using namespace std;
48
49 class Solution {
50 public:
51     int lengthOfLongestSubstring(string s) {
52         unordered_map<char, int> ultimoIndice;
53         int L = 0;
54         int mejor = 0;
55
56         for (int R = 0; R < (int)s.size(); ++R) {
57             char c = s[R];
58             auto it = ultimoIndice.find(c);
59             if (it != ultimoIndice.end() && it->second >= L) {

```

```

60         // El carácter repetido está dentro de la ventana: cerrar L
61         // justo después de esa aparición anterior.
62         L = it->second + 1;
63     }
64     ultimoIndice[c] = R;
65     mejor = max(mejor, R - L + 1);
66 }
67 return mejor;
68 }
69 };
70
71 int main() {
72     Solution sol;
73
74     // Caso normal: "abc" se repite, la ventana máxima sin repetidos es "abc".
75     assert(sol.lengthOfLongestSubstring("abcabcbb") == 3);
76
77     // Caso límite: cadena vacía.
78     assert(sol.lengthOfLongestSubstring("") == 0);
79
80     // Caso límite: un solo carácter.
81     assert(sol.lengthOfLongestSubstring("a") == 1);
82
83     // Todos los caracteres repetidos: la ventana nunca crece más de 1.
84     assert(sol.lengthOfLongestSubstring("bbbb") == 1);
85
86     // Caso donde el orden de procesamiento importa: el carácter repetido
87     // aparece muy atrás en la cadena, pero la ventana relevante en el
88     // momento de la repetición ya lo había dejado atrás (L avanzó antes por
89     // otra repetición), así que NO debe retroceder L de más.
90     // s = "abba": R=0 'a' (L=0, best=1) R=1 'b' (L=0, best=2)
91     //      R=2 'b' repetido en ventana (idx1 >= L=0) -> L=2, best sigue 2
92     //      R=3 'a' visto en idx0, pero idx0 < L=2 -> NO se mueve L, best=max
93     //          (2, 3-2+1=2)=2
94     assert(sol.lengthOfLongestSubstring("abba") == 2);
95
96     // Caso con espacios y símbolos, subcadena más larga en medio de la cadena
97     // .
98     assert(sol.lengthOfLongestSubstring("pwwkew") == 3);    // "wke"
99
100     cout << "3. Longest Substring Without Repeating Characters: todas las
        pruebas pasaron.\n";
101     return 0;
102 }

```

Complejidad: tiempo $O(n)$, espacio $O(\min(n, |\Sigma|))$.

¿Qué significa esa complejidad? Cada carácter entra a la ventana (cuando R avanza) y sale de ella (cuando L avanza) como máximo una vez, así que el trabajo total es proporcional al tamaño de la cadena: n . El espacio depende de cuántos caracteres distintos puede haber: en el peor caso, tantos como el tamaño del alfabeto $|\Sigma|$ (por ejemplo, 128 para ASCII), o n si la cadena es más corta que eso.

Consejos para la entrevista (Amazon). Este problema es el ejemplo canónico de sliding window con tamaño de ventana variable (a diferencia de una ventana de tamaño fijo). El error más común es mover L de uno en uno hasta salir del carácter repetido, en vez de saltar directamente a la posición correcta usando el mapa de últimos índices; ambas versiones son correctas, pero la segunda es la que se espera en $O(n)$. Vale la pena mencionar en voz alta la trampa del paso 2 (comparar contra L , no solo contra “¿ya lo vi?”), porque es la parte donde más fácil se cuele un bug sutil. Casos límite que conviene decir en voz alta: cadena vacía (respuesta 0) y cadena de un solo carácter (respuesta 1).

76. Minimum Window Substring

(H)

Código fuente: `src/06-sliding-window-stack/76-minimum-window-substring.cpp`▷ Ver animación · [local](#) | [web](#)

Dadas dos cadenas `s` y `t`, devuelve la subcadena *contigua* más corta de `s` que contiene a todos los caracteres de `t`, contando repeticiones (si `t` tiene dos 'a', la ventana debe tener al menos dos 'a'). Si no existe tal subcadena, devuelve la cadena vacía.

La idea, con un ejemplo de la vida real. Imagina que tienes una lista de compras (`t`) y caminas por los pasillos de un supermercado muy largo (`s`) tomando productos según los vas encontrando, sin poder saltarte pasillos ni regresar por productos sueltos: solo puedes quedarte con un tramo continuo de pasillo. Quieres encontrar el tramo más corto que, entre todo lo que recogiste ahí, cubra tu lista completa (aunque recojas cosas de más que no estaban en la lista, no importa). En cuanto tu carrito ya cubre la lista completa, empiezas a soltar productos por el lado donde entraste primero, mientras la lista siga cubierta, para ver si el tramo se puede acortar todavía más.

A D O B E C O D E B A N C
[B A N C]

Con `s = "ADOBECODEBANC"`, `t = "ABC"`, la ventana más corta que cubre A, B y C es "BANC", de longitud 4.

Cómo lo resuelve el código, paso a paso. Guardamos en un mapa `necesita` cuántas veces aparece cada carácter en `t`, y en `requeridos` cuántos caracteres *distintos* hay que cubrir. Llevamos otro mapa `ventana` con lo que hay actualmente entre los punteros L y R, y un contador `cubiertos` de cuántos caracteres distintos ya alcanzaron la cantidad necesaria.

1. Avanzamos R y metemos `s[R]` a la ventana.
2. Si `s[R]` es uno de los caracteres necesarios y la ventana acaba de alcanzar justo la cantidad requerida de él, sumamos uno a `cubiertos`.
3. Mientras `cubiertos` sea igual a `requeridos` (la ventana ya cubre todo `t`): comparamos su tamaño contra la mejor guardada, y si es más chica la guardamos; luego sacamos `s[L]` de la ventana y avanzamos L. Si al sacar `s[L]` la cantidad cae por debajo de lo necesario, restamos uno a `cubiertos` (la ventana deja de estar completa y el ciclo de contracción se detiene).
4. Repetimos hasta que R llega al final de `s`.

La clave es que L y R solo avanzan hacia adelante, nunca retroceden: por eso cada carácter de `s` entra y sale de la ventana a lo más una vez, y el algoritmo entero es lineal.

```

1 //
  =====
2 // 76. Minimum Window Substring (Hard)
3 //
  -----
4 // Dadas dos cadenas s y t, devuelve la subcadena CONTIGUA más corta de s que
5 // contiene a todos los caracteres de t (contando repeticiones: si t tiene

```

```

6 // dos 'a', la ventana debe tener al menos dos 'a'). Si no existe tal
7 // subcadena, devuelve la cadena vacía "".
8 //
9 // Ejemplo: s = "ADOBECODEBANC", t = "ABC"
10 //
11 //   A D O B E C O D E B A N C
12 //               [B A N C]   <- ventana más chica que cubre A, B, C
13 //
14 //   Respuesta: "BANC" (longitud 4). Hay ventanas que cubren A, B, C antes,
15 //   pero son más largas.
16 //
17 // Diagrama de sliding window (ventana deslizante de tamaño variable, con
18 // conteo de caracteres "necesarios" vs. "presentes en la ventana"):
19 //
20 //   necesita[c]   = cuántas veces aparece cada carácter c en t
21 //   requeridos    = cuántos caracteres DISTINTOS hay que cubrir
22 //   ventana[c]    = cuántas veces aparece c en la ventana actual [L, R]
23 //   cubiertos     = cuántos caracteres distintos ya están completamente
24 //                  cubiertos (ventana[c] >= necesita[c])
25 //
26 //   L = 0
27 //   para R desde 0 hasta el final de s:
28 //       meter s[R] a la ventana (ventana[s[R]]++)
29 //       si s[R] es necesario y ventana[s[R]] llegó justo a necesita[s[R]]:
30 //           cubiertos++
31 //       mientras cubiertos == requeridos: // la ventana YA cubre todo t
32 //           si esta ventana es más chica que la mejor guardada, guardarla
33 //           sacar s[L] de la ventana (ventana[s[L]]--)
34 //           si s[L] es necesario y ventana[s[L]] cayó por debajo de necesita[s[L]
35 //       ]]:
36 //           cubiertos--
37 //           L++
38 //
39 // Idea: expandimos R hasta que la ventana cubra todo t (cubiertos ==
40 // requeridos), y en ese momento tratamos de encogerla desde L todo lo que se
41 // pueda sin perder la cobertura, guardando la más chica vista. Es la
42 // combinación clásica de sliding window: R solo avanza hacia adelante,
43 // L también solo avanza hacia adelante, así que cada carácter de s entra y
44 // sale de la ventana a lo más una vez.
45 //
46 // Complejidad: tiempo  $O(|s| + |t|)$ , espacio  $O(|s| + |t|)$  (para los mapas de
47 // conteo, acotado por el tamaño del alfabeto).
48 //
49 =====
50
51 #include <string>
52 #include <unordered_map>
53 #include <climits>
54 #include <cassert>
55 #include <iostream>
56 using namespace std;
57
58 class Solution {
59 public:

```



```
57     string minWindow(string s, string t) {
58         if (s.empty() || t.empty()) return "";
59
60         unordered_map<char, int> necesita;
61         for (char c : t) necesita[c]++;
62         int requeridos = (int)necesita.size();
63
64         unordered_map<char, int> ventana;
65         int cubiertos = 0;
66         int L = 0;
67         int mejorLen = INT_MAX, mejorL = 0;
68
69         for (int R = 0; R < (int)s.size(); ++R) {
70             char c = s[R];
71             ventana[c]++;
72             if (necesita.count(c) && ventana[c] == necesita[c]) {
73                 cubiertos++;
74             }
75
76             while (L <= R && cubiertos == requeridos) {
77                 if (R - L + 1 < mejorLen) {
78                     mejorLen = R - L + 1;
79                     mejorL = L;
80                 }
81                 char lc = s[L];
82                 ventana[lc]--;
83                 if (necesita.count(lc) && ventana[lc] < necesita[lc]) {
84                     cubiertos--;
85                 }
86                 L++;
87             }
88         }
89
90         return mejorLen == INT_MAX ? "" : s.substr(mejorL, mejorLen);
91     }
92 };
93
94 int main() {
95     Solution sol;
96
97     // Caso normal: ejemplo clásico del enunciado de LeetCode.
98     assert(sol.minWindow("ADOBECODEBANC", "ABC") == "BANC");
99
100    // Caso límite: alguna de las cadenas está vacía.
101    assert(sol.minWindow("", "A") == "");
102    assert(sol.minWindow("A", "") == "");
103
104    // Caso límite: t pide más repeticiones de un carácter de las que s tiene.
105    assert(sol.minWindow("a", "aa") == "");
106
107    // Caso límite: un solo carácter, coincide exacto.
108    assert(sol.minWindow("a", "a") == "a");
109
110    // Caso donde el orden de procesamiento (contraer desde la izquierda
```

```
111 // después de cada expansión) importa: la ventana mínima real está al
112 // final de s, no al principio, y hay que descartar la cobertura parcial
113 // temprana.
114 assert(sol.minWindow("ab", "b") == "b");
115
116 cout << "76. Minimum Window Substring: todas las pruebas pasaron.\n";
117 return 0;
118 }
```

Complejidad: tiempo $O(|s| + |t|)$, espacio $O(|s| + |t|)$.

¿Qué significa esa complejidad? Recorremos s una vez para expandir y, en total, otra vez más para contraer (nunca más de $|s|$ pasos combinados), y construir el mapa de t cuesta $|t|$. El espacio lo usan los dos mapas de conteo, acotados por cuántos caracteres distintos puede haber.

Consejos para la entrevista (Amazon). Es uno de los problemas de sliding window más pedidos porque combina ventana de tamaño variable con conteo de multiplicidad (no basta con “¿está el carácter?”, hay que contar cuántas veces se necesita). El error más común es comparar `ventana[c] > 0` en vez de `ventana[c] >= necesita[c]`, lo cual falla en cuanto t tiene caracteres repetidos. Otro punto que suelen evaluar: por qué la condición del `while` de contracción usa igualdad exacta (`cubiertos == requeridos`) y no una comparación de mayor o igual —conviene explicar que `cubiertos` nunca puede superar `requeridos` por construcción—. Casos límite a mencionar: s o t vacías, y t pidiendo más repeticiones de un carácter de las que existen en s (no hay solución, se devuelve cadena vacía).

42. Trapping Rain Water

(H)

Código fuente: `src/06-sliding-window-stack/42-trapping-rain-water.cpp`

▷ Ver animación · [local](#) | [web](#)

Dado un arreglo `height` que representa la altura de columnas de un histograma (cada columna tiene ancho 1), calcula cuánta agua de lluvia queda atrapada entre las columnas después de llover.

La idea, con un ejemplo de la vida real. Imagina una fila de edificios de distintas alturas pegados uno junto a otro, y que llueve mucho. El agua se acumula entre dos edificios altos que hacen de “paredes” para uno más bajo en medio: el nivel del agua sobre cualquier edificio no puede subir más que el más bajo de los dos edificios más altos que lo encierran (por la izquierda y por la derecha), porque el agua se derramaría por el lado más bajo. Entonces, sobre cada edificio, el agua atrapada es la diferencia entre ese nivel límite y la altura del edificio mismo (si esa diferencia es negativa, no se atrapa nada ahí).

```
altura: 0 1 0 2 1 0 1 3 2 1 2 1
agua atrapada total: 6
```

Cómo lo resuelve el código, paso a paso. En vez de calcular, para cada columna, el máximo a su izquierda y el máximo a su derecha por separado (lo cual tomaría espacio extra), usamos **two pointers** (dos punteros) que arrancan en los dos extremos del arreglo y se acercan.

1. Mantenemos `leftMax` (el máximo visto hasta el puntero izquierdo) y `rightMax` (el máximo visto hasta el puntero derecho).
2. En cada paso comparamos `height[L]` contra `height[R]`.
3. Si `height[L]` es menor o igual, sabemos con certeza que el edificio más alto que limita el agua sobre L por el lado derecho es *al menos* `height[R]`, que ya es mayor o igual que `height[L]`. Entonces el límite real es `leftMax` (el máximo visto por la izquierda), sin necesidad de saber el máximo exacto a la derecha. Sumamos `leftMax - height[L]` al total y avanzamos L.
4. Si es al revés, hacemos lo simétrico con R y `rightMax`.
5. Repetimos hasta que los punteros se crucen.

```
1 //
2 // =====
3 //
4 // 42. Trapping Rain Water (Hard)
5 //
6 // -----
7 //
8 // Dado un arreglo height que representa la altura de columnas de un
9 // histograma (cada columna tiene ancho 1), calcula cuánta agua de lluvia
10 // queda atrapada entre las columnas después de llover.
11 //
12 // Ejemplo: height = [0,1,0,2,1,0,1,3,2,1,2,1]
13 //
14 //      3
15 //      2      -  |  |  -
```

```

12 // 1 _ |~|_ |~|~|_ | |~|_
13 // 0 |_||_||_||_||_||_||_
14 // 0 1 2 3 4 5 6 7 8 9 ...
15 //
16 // El agua atrapada arriba de cada columna se dibuja con "~". En total se
17 // atrapan 6 unidades de agua.
18 //
19 // Diagrama de two pointers (dos punteros que avanzan uno hacia el otro
20 // desde los extremos del arreglo):
21 //
22 // L = 0, R = n-1
23 // leftMax = 0, rightMax = 0
24 // total = 0
25 // mientras L < R:
26 //     si height[L] <= height[R]:
27 //         leftMax = max(leftMax, height[L])
28 //         total += leftMax - height[L] // agua atrapada sobre la columna L
29 //         L++
30 //     si no:
31 //         rightMax = max(rightMax, height[R])
32 //         total += rightMax - height[R] // agua atrapada sobre la columna R
33 //         R--
34 //
35 // Idea: el agua que se atrapa exactamente sobre la columna i es
36 // min(máximo a su izquierda, máximo a su derecha) - height[i]. El truco de
37 // two pointers evita calcular esos dos máximos por separado con arreglos
38 // extra: en cada paso movemos el puntero del lado con la altura MENOR entre
39 // height[L] y height[R]. ¿Por qué es seguro? Porque si height[L] <=
40 // height[R], entonces el máximo real a la derecha de L es al menos
41 // height[R] (aunque no sepamos su valor exacto todavía), que ya es >=
42 // height[L]; así que el mínimo de los dos máximos que limita el agua sobre
43 // L es, con certeza, leftMax (el máximo visto hasta L por la izquierda), sin
44 // importar cuál sea el máximo real del lado derecho. Simétrico para el otro
45 // caso.
46 //
47 // Complejidad: tiempo O(n), espacio O(1).
48 //

```

```

49 #include <vector>
50 #include <algorithm>
51 #include <cassert>
52 #include <iostream>
53 using namespace std;
54
55 class Solution {
56 public:
57     int trap(vector<int>& height) {
58         int n = (int)height.size();
59         if (n == 0) return 0;
60
61         int L = 0, R = n - 1;
62         int leftMax = 0, rightMax = 0;
63         int total = 0;

```

```

64
65     while (L < R) {
66         if (height[L] <= height[R]) {
67             leftMax = max(leftMax, height[L]);
68             total += leftMax - height[L];
69             L++;
70         } else {
71             rightMax = max(rightMax, height[R]);
72             total += rightMax - height[R];
73             R--;
74         }
75     }
76     return total;
77 }
78 };
79
80 int main() {
81     Solution sol;
82
83     // Caso normal: ejemplo clásico del enunciado de LeetCode.
84     vector<int> h1 = {0,1,0,2,1,0,1,3,2,1,2,1};
85     assert(sol.trap(h1) == 6);
86
87     // Caso límite: arreglo vacío.
88     vector<int> h2 = {};
89     assert(sol.trap(h2) == 0);
90
91     // Caso límite: un solo elemento (no hay dónde atrapar agua).
92     vector<int> h3 = {5};
93     assert(sol.trap(h3) == 0);
94
95     // Caso límite: dos elementos (tampoco se puede atrapar nada).
96     vector<int> h4 = {5, 4};
97     assert(sol.trap(h4) == 0);
98
99     // Caso donde el orden de procesamiento importa: el puntero derecho
100    // domina al principio (columna más alta a la derecha), así que hay que
101    // mover el puntero izquierdo varias veces seguidas antes de que el
102    // lado derecho tome el control.
103    vector<int> h5 = {4, 2, 0, 3, 2, 5};
104    assert(sol.trap(h5) == 9);
105
106    cout << "42. Trapping Rain Water: todas las pruebas pasaron.\n";
107    return 0;
108 }

```

Complejidad: tiempo $O(n)$, espacio $O(1)$.

¿Qué significa esa complejidad? Cada paso del ciclo avanza uno de los dos punteros, y entre los dos recorren el arreglo una sola vez en total, así que el tiempo es lineal en el tamaño del arreglo. El espacio es constante porque solo guardamos un puñado de variables (`leftMax`, `rightMax`, los dos punteros y el total), sin arreglos auxiliares.

Consejos para la entrevista (Amazon). Este problema suele empezar con una solución de dos arreglos auxiliares (`maxIzquierda[]`, `maxDerecha[]`) que ya es $O(n)$ en tiempo pero también $O(n)$ en espacio; el reto real es notar que se puede resolver con espacio constante usando two pointers, y explicar *por qué* es seguro mover el puntero del lado menor sin conocer el máximo real del otro lado (la clave está en que ese máximo real, aunque desconocido, ya es mayor o igual al lado que estamos procesando). Casos límite que conviene mencionar: arreglo vacío o con menos de tres columnas (no se puede atrapar nada, porque hace falta al menos un valle entre dos paredes).

84. Largest Rectangle in Histogram

(H)

Código fuente: [src/06-sliding-window-stack/84-largest-rectangle-histogram.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dado un arreglo `heights` con las alturas de las barras de un histograma (cada barra de ancho 1, todas paradas sobre la misma línea base), encuentra el área del rectángulo más grande que se puede formar usando una o más barras *consecutivas*, con altura igual a la barra más chica del tramo elegido.

La idea, con un ejemplo de la vida real. Imagina una fila de libros de distinto grosor parados uno junto al otro en un librero, vistos desde arriba como barras de distinta altura. Quieres cubrir un tramo continuo de esos libros con una tabla rectangular apoyada encima; la tabla solo puede llegar tan alto como el libro más bajo del tramo que decidas cubrir (si un libro es más bajo, la tabla se hunde hasta ahí). Quieres encontrar qué tramo, y qué altura de tabla, te da el área más grande posible.

```
alturas: 2 1 5 6 2 3
```

```
mejor rectángulo: barras de altura 5 y 6, área = 5 × 2 = 10
```

Cómo lo resuelve el código, paso a paso. Usamos una **pila monótona** (monotonic stack): una pila de índices cuyas alturas correspondientes van siempre en aumento de abajo hacia arriba.

1. Recorremos las barras de izquierda a derecha, y agregamos al final una barra “fantasma” de altura 0 (un centinela) para forzar a que la pila se vacíe por completo al terminar.
2. Para cada barra nueva, mientras la altura del tope de la pila sea mayor o igual a la altura actual, esa barra del tope ya no puede seguir creciendo hacia la derecha: la sacamos de la pila y calculamos el rectángulo más grande que la tiene a *ella* como altura limitante.
3. El ancho de ese rectángulo llega, por la izquierda, hasta la barra que quedó justo debajo en la pila (la última vez que hubo algo más bajo o igual), y por la derecha, hasta la posición actual (exclusiva).
4. Guardamos el área más grande vista, y al final metemos la barra actual a la pila.

El centinela final es lo que garantiza que ninguna barra se quede sin evaluar: si el histograma termina con barras cada vez más altas, sin el centinela nunca se dispararía el “mientras” que las saca de la pila.

```
1 //
  // =====
2 // 84. Largest Rectangle in Histogram (Hard)
3 //
  // -----
4 // Dado un arreglo heights con las alturas de las barras de un histograma
5 // (cada barra tiene ancho 1, todas paradas sobre la misma línea base),
6 // encuentra el área del rectángulo más grande que se puede formar usando
7 // una o más barras consecutivas, con altura igual a la barra más chica del
8 // tramo elegido.
```

```

9 //
10 // Ejemplo: heights = [2,1,5,6,2,3]
11 //
12 //      6
13 //      5      _ | |
14 //      4      | | |
15 //      3      | | | _ _
16 //      2      _ | | | | _ |
17 //      1 | | _ | | | | _ |
18 //      0 | _ | | _ | | _ | | _ |
19 //      0  1  2  3  4  5
20 //
21 // El rectángulo más grande usa las barras de índice 2 y 3 (alturas 5 y
22 // 6), con altura limitada por la más chica de las dos (5) y ancho 2:
23 // área = 5 * 2 = 10.
24 //
25 // Diagrama de pila monótona (monotonic stack: pila de índices cuyas
26 // alturas van SIEMPRE en aumento de abajo hacia arriba):
27 //
28 // pila = [] // guarda índices, heights[pila] es creciente
29 // para i desde 0 hasta n (con un 0 "centinela" al final, i == n):
30 // h = (i == n) ? 0 : heights[i]
31 // mientras pila no vacía Y heights[tope de pila] >= h:
32 // altura = heights[pila.pop()] // barra que sacamos
33 // // su ancho es hasta dónde llega siendo la más chica: desde el
34 // // elemento que quedó justo debajo en la pila (exclusivo) hasta i
35 // ancho = pila.vacíá ? i : i - pila.tope() - 1
36 // area = max(area, altura * ancho)
37 // pila.push(i)
38 //
39 // Idea: cuando una barra nueva es más baja que la del tope de la pila,
40 // significa que la barra del tope ya no puede crecer más hacia la derecha
41 // (la barra nueva la "corta"): es el momento exacto de calcular el
42 // rectángulo más grande que tiene a esa barra del tope como su altura
43 // limitante. El ancho de ese rectángulo llega hasta la barra anterior en la
44 // pila (porque esa fue la última vez que hubo algo más bajo o igual, límite
45 // por la izquierda) y hasta la posición actual i (límite por la derecha,
46 // exclusivo). El 0 al final actúa como centinela para forzar que se vacíe
47 // toda la pila y se evalúen las barras que quedaron pendientes.
48 //
49 // Complejidad: tiempo O(n) (cada índice entra y sale de la pila una vez),
50 // espacio O(n).
51 //
=====

52 #include <vector>
53 #include <stack>
54 #include <algorithm>
55 #include <cassert>
56 #include <iostream>
57 using namespace std;
58
59 class Solution {
60 public:

```



```

61     int largestRectangleArea(vector<int>& heights) {
62         int n = (int)heights.size();
63         stack<int> pila;    // índices, con heights[pila] creciente
64         int maxArea = 0;
65
66         for (int i = 0; i <= n; ++i) {
67             int h = (i == n) ? 0 : heights[i];    // centinela al final
68             while (!pila.empty() && heights[pila.top()] >= h) {
69                 int altura = heights[pila.top()];
70                 pila.pop();
71                 int ancho = pila.empty() ? i : i - pila.top() - 1;
72                 maxArea = max(maxArea, altura * ancho);
73             }
74             pila.push(i);
75         }
76         return maxArea;
77     }
78 };
79
80 int main() {
81     Solution sol;
82
83     // Caso normal: el rectángulo óptimo usa dos barras consecutivas.
84     vector<int> h1 = {2,1,5,6,2,3};
85     assert(sol.largestRectangleArea(h1) == 10);
86
87     // Caso límite: arreglo vacío.
88     vector<int> h2 = {};
89     assert(sol.largestRectangleArea(h2) == 0);
90
91     // Caso límite: una sola barra.
92     vector<int> h3 = {5};
93     assert(sol.largestRectangleArea(h3) == 5);
94
95     // Todas las barras con la misma altura: el rectángulo óptimo usa todas.
96     vector<int> h4 = {4,4,4,4};
97     assert(sol.largestRectangleArea(h4) == 16);
98
99     // Caso donde el orden de procesamiento (y el centinela final) importa:
100    // la barra más chica está en medio, y el rectángulo óptimo se calcula
101    // solo hasta que el centinela fuerza a vaciar la pila.
102    vector<int> h5 = {2,1,2};
103    assert(sol.largestRectangleArea(h5) == 3);
104
105    cout << "84. Largest Rectangle in Histogram: todas las pruebas pasaron.\n"
106         ;
107    return 0;
108 }

```

Complejidad: tiempo $O(n)$, espacio $O(n)$.

¿Qué significa esa complejidad? Aunque hay un ciclo anidado (el `while` dentro del `for`), cada índice entra a la pila exactamente una vez y sale de la pila a lo más una vez, así que el trabajo total

sigue siendo proporcional a n , no a n al cuadrado. El espacio lo usa la pila, que en el peor caso (barras siempre creciendo) guarda todos los índices al mismo tiempo.

Consejos para la entrevista (Amazon). La solución ingenua (para cada barra, expandirse hacia los lados hasta encontrar algo más bajo) es $O(n \text{ al cuadrado})$ y suele ser el punto de partida esperado antes de llegar a la pila monótona. Vale la pena explicar en voz alta por qué el ancho se calcula con el índice que quedó *debajo* en la pila, no con el índice que se acaba de sacar: ese índice inferior es la barrera real por la izquierda. El centinela de altura 0 al final es un detalle fácil de olvidar y una fuente común de bugs (rectángulos que quedan sin evaluar). Casos límite: arreglo vacío (área 0), una sola barra (el área es esa misma altura), y todas las barras con la misma altura (el rectángulo óptimo las usa todas).

394. Decode String

(M)

Código fuente: [src/06-sliding-window-stack/394-decode-string.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dada una cadena codificada *s* con el patrón *k*[cadena], donde *k* es un entero positivo, decodifícala devolviendo la cadena expandida, repitiendo *k* veces lo que está dentro de los corchetes. El patrón puede anidarse (corchetes dentro de corchetes) y también puede haber texto normal fuera de cualquier corchete.

La idea, con un ejemplo de la vida real. Imagina que estás armando cajas dentro de cajas, con una nota pegada afuera de cada una que dice “repetir el contenido tantas veces”. Cuando abres una caja nueva (un `[`), guardas en un lugar seguro lo que llevabas armado hasta ese momento y cuántas veces hay que repetir lo que va adentro, y empiezas a armar el contenido de esa caja desde cero. Cuando cierras la caja (un `]`), repites lo que armaste adentro la cantidad de veces indicada, y lo pegas de vuelta a lo que tenías guardado antes de abrirla. Esa “pila de cajas pendientes” es literalmente una **pila** (stack): cada caja que abres se apila encima de la anterior, y se va desapilando en orden inverso conforme se cierran.

```
3[ a 2[ c ] ]
" c " repetida 2 veces → "cc"
"a"+"cc" = "acc", repetida 3 veces → "accaccacc"
```

Cómo lo resuelve el código, paso a paso. Mantenemos una pila donde cada elemento es un par: la cadena que llevábamos construida *antes* de entrar a ese nivel de corchetes, y el número de repeticiones pendiente para ese nivel. También llevamos dos variables de trabajo: *actual* (lo que se está construyendo en el nivel presente) y *numero* (los dígitos acumulados antes de encontrar un `[`).

1. Si el carácter es un dígito, lo acumulamos en *numero* (multiplicando por diez lo que ya teníamos, para soportar números de más de un dígito).
2. Si es `[`, empujamos a la pila el par (*actual*, *numero*), y reiniciamos *actual* y *numero* para empezar el nuevo nivel limpio.
3. Si es `]`, sacamos de la pila el par que guardamos al entrar a este nivel, y reconstruimos *actual* como la cadena anterior más la cadena de este nivel repetida la cantidad de veces indicada.
4. Si es cualquier otro carácter (una letra), simplemente la agregamos a *actual*.

Al terminar de recorrer *s*, *actual* contiene la cadena completamente decodificada.

```
1 //
// =====
2 // 394. Decode String (Medium)
3 //
// -----
4 // Dada una cadena codificada s con el patrón k[cadena], donde k es un
5 // entero positivo, decodifícala devolviendo la cadena expandida, repitiendo
6 // k veces lo que está dentro de los corchetes. El patrón puede anidarse
7 // (corchetes dentro de corchetes) y también puede haber texto normal fuera
8 // de cualquier corchete.
```

```

9  //
10 // Ejemplo: s = "3[a2[c]]"
11 //
12 //   3[ a 2[ c ] ]
13 //   |      |
14 //   |      +-- "c" repetida 2 veces -> "cc"
15 //   +-- "a" + "cc" = "acc", repetida 3 veces -> "accaccacc"
16 //
17 // Respuesta: "accaccacc"
18 //
19 // Diagrama de pila explícita (stack de pares: cadena acumulada hasta el
20 // momento, número de repeticiones pendiente):
21 //
22 //   pila = []           // cada elemento: (cadenaAnterior, repeticiones)
23 //   actual = ""         // cadena que se está construyendo en el nivel actual
24 //   numero = 0          // dígitos acumulados antes de un '['
25 //   para cada carácter c en s:
26 //       si c es dígito:
27 //           numero = numero * 10 + valor(c)    // los números pueden tener 2+ dí
28 //           gitos
29 //       si c == '[':
30 //           pila.push((actual, numero))    // guardamos dónde íbamos antes de
31 //           anidar
32 //           actual = ""                    // el nuevo nivel empieza limpio
33 //           numero = 0
34 //       si c == ']':
35 //           (cadenaAnterior, repeticiones) = pila.pop()
36 //           actual = cadenaAnterior + (actual repetida "repeticiones" veces)
37 //       si no (letra normal):
38 //           actual += c
39 //       devolver actual
40 //
41 // Idea: cada vez que entramos a un nuevo nivel de corchetes, "congelamos" lo
42 // que llevábamos construido y el número de repeticiones pendiente en la
43 // pila, y empezamos un lienzo limpio para ese nivel. Al cerrar el corchete,
44 // repetimos lo que se construyó en ese nivel y lo pegamos de vuelta a lo que
45 // teníamos congelado. Esto maneja anidamiento arbitrario de forma natural,
46 // porque cada apertura de corchete crea un nuevo "marco" en la pila,
47 // exactamente como una llamada recursiva, pero de forma iterativa.
48 //
49 // Complejidad: tiempo  $O(n * m)$  donde  $n$  es la longitud de  $s$  y  $m$  es la
50 // longitud de la cadena final decodificada (cada carácter se puede copiar
51 // varias veces al expandir repeticiones); espacio  $O(n)$  para la pila y la
52 // cadena resultante.
53 //
54 // =====
55
56 #include <string>
57 #include <stack>
58 #include <cctype>
59 #include <cassert>
60 #include <iostream>
61 using namespace std;

```



```
113     return 0;  
114 }
```

Complejidad: tiempo $O(n \cdot m)$, espacio $O(n)$.

¿Qué significa esa complejidad? n es la longitud de la cadena codificada y m es la longitud de la cadena final ya expandida (que puede ser mucho más grande que n si hay muchas repeticiones anidadas). El tiempo depende de cuánto texto terminamos copiando al reconstruir cada nivel. El espacio es proporcional a n porque la pila, en el peor caso, tiene tantos niveles como corchetes anidados haya.

Consejos para la entrevista (Amazon). Este problema es un buen ejemplo de simular recursión con una pila explícita: cada `[` equivale a “entrar a una llamada” y cada `]` a “regresar de ella”. Vale la pena mencionar que también se puede resolver con recursión real, pero la versión con pila evita el riesgo de desbordar la pila de llamadas si hay muchísimos niveles de anidamiento. Un detalle fácil de pasar por alto: el número antes de un corchete puede tener más de un dígito ("`10[a]`"), así que hay que acumular dígitos, no asumir que siempre es uno solo. Casos límite a mencionar: cadena vacía, cadena sin ningún corchete (se devuelve tal cual), y anidamiento profundo.

Capítulo 6

Programación Dinámica

322. Coin Change

(M)

Código fuente: [src/07-dp/322-coin-change.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dado un arreglo `coins` con las denominaciones de monedas disponibles (cada denominación se puede usar tantas veces como se quiera) y un entero `amount`, calcula el número mínimo de monedas necesarias para juntar exactamente `amount`. Si no existe ninguna combinación que llegue exacto, devuelve `-1`.

La idea, con un ejemplo de la vida real. Piensa en dar cambio en una caja registradora: quieres usar el menor número posible de monedas para completar una cantidad exacta. En vez de probar a fuerza bruta todas las combinaciones posibles de monedas (que crecen exponencialmente), resolvemos primero los montos pequeños y reutilizamos esas respuestas para construir los montos grandes. Esto es **programación dinámica** (*dynamic programming*, DP): guardamos la respuesta óptima de cada subproblema (cada monto menor) para no recalcularla.

```
coins = [1, 2, 5], amount = 11
11 = 5 + 5 + 1 (3 monedas, es el mínimo)
```

Cómo lo resuelve el código, paso a paso.

1. Creamos una tabla `dp` de tamaño `amount + 1`, donde `dp[m]` representa el mínimo número de monedas para llegar exacto al monto `m`. Inicializamos todo en un valor “infinito” (representa “todavía no sabemos cómo llegar a este monto”), salvo `dp[0] = 0` (con cero monedas llegas al monto cero).
2. Para cada monto `m` de 1 a `amount`, probamos cada moneda `c` del arreglo `coins`.
3. Si `c` cabe en `m` (es decir `c <= m`) y ya sabemos cómo llegar a `m - c` (su valor en `dp` no es infinito), entonces una forma de llegar a `m` es usar esa solución y agregar una moneda `c` encima: `dp[m] = min(dp[m], dp[m - c] + 1)`.
4. Al terminar, si `dp[amount]` sigue siendo infinito, no existe ninguna combinación exacta y devolvemos `-1`; si no, devolvemos `dp[amount]`.

```
1 //
2 // =====
3 //
4 // 322. Coin Change (Media)
5 //
6 // -----
7 //
8 // Dado un arreglo coins con las denominaciones de monedas disponibles (puedes
9 // usar cada denominación tantas veces como quieras) y un entero amount,
10 // calcula el número MÍNIMO de monedas necesarias para juntar exactamente
11 // amount. Si no hay ninguna combinación que llegue exacto, devuelve -1.
12 //
13 // Ejemplo: coins = [1,2,5], amount = 11
14 //
15 // 11 = 5 + 5 + 1 -> 3 monedas (es el mínimo posible)
16 //
```



```

13 // Este es un problema clásico de programación dinámica (dynamic programming,
14 // DP): construimos la respuesta para montos pequeños y la reusamos para
15 // montos más grandes, en vez de probar todas las combinaciones posibles
16 // desde cero (lo cual sería exponencial).
17 //
18 // Diagrama de la tabla dp (dp[m] = mínimo número de monedas para llegar
19 // exacto al monto m; usamos un valor "infinito" para representar "todavía
20 // no se sabe cómo llegar a este monto"):
21 //
22 //   coins = [1,2,5]
23 //   monto m:  0  1  2  3  4  5  6  7  8  9 10 11
24 //   dp[m]:    0  1  1  2  2  1  2  2  3  3  2  3
25 //
26 //   dp[0] = 0 (con cero monedas llegas a monto 0, caso base).
27 //   Para cada monto m de 1 a amount, probamos cada moneda c:
28 //       si c <= m y dp[m-c] no es infinito:
29 //           dp[m] = min(dp[m], dp[m-c] + 1)
30 //   Es decir: si ya sabemos el mínimo de monedas para llegar a (m - c),
31 //   nada más agregamos una moneda de valor c encima de esa solución.
32 //
33 // Por qué funciona: la solución óptima para el monto m necesariamente usa
34 // AL MENOS una moneda; sea c esa última moneda usada, entonces el resto
35 // (m - c) se resolvió con el mínimo número de monedas posible para ese
36 // monto menor (si no, podríamos sustituir esa parte por una mejor y
37 // contradecir que dp[m] era el mínimo). Como probamos todas las monedas c
38 // como "última moneda", cubrimos todos los casos y nos quedamos con el
39 // mejor.
40 //
41 // Complejidad: tiempo O(amount * número de monedas), espacio O(amount).
42 //
43 =====
44 #include <vector>
45 #include <algorithm>
46 #include <climits>
47 #include <cassert>
48 #include <iostream>
49 using namespace std;
50
51 class Solution {
52 public:
53     int coinChange(vector<int>& coins, int amount) {
54         const int INF = INT_MAX / 2; // "infinito" seguro contra overflow al
55             sumar 1
56         vector<int> dp(amount + 1, INF);
57         dp[0] = 0;
58
59         for (int m = 1; m <= amount; m++) {
60             for (int c : coins) {
61                 if (c <= m && dp[m - c] != INF) {
62                     dp[m] = min(dp[m], dp[m - c] + 1);
63                 }
64             }
65         }
66     }
67 };

```

```

64         return dp[amount] == INF ? -1 : dp[amount];
65     }
66 };
67
68
69 int main() {
70     Solution sol;
71
72     // Caso normal: ejemplo clásico del enunciado de LeetCode.
73     vector<int> c1 = {1, 2, 5};
74     assert(sol.coinChange(c1, 11) == 3);
75
76     // Caso límite: monto 0 (no se necesita ninguna moneda).
77     vector<int> c2 = {1, 2, 5};
78     assert(sol.coinChange(c2, 0) == 0);
79
80     // Caso límite: una sola denominación que no encaja exacto.
81     vector<int> c3 = {2};
82     assert(sol.coinChange(c3, 3) == -1);
83
84     // Trampa común: monto imposible de alcanzar con las monedas dadas
85     // (todas las monedas son mayores que amount, o no hay combinación
86     // exacta posible).
87     vector<int> c4 = {3, 7};
88     assert(sol.coinChange(c4, 5) == -1);
89
90     // Caso donde conviene combinar denominaciones distintas en vez de
91     // usar solo la moneda más grande posible (greedy fallaría aquí: con
92     // monedas {1,3,4} y amount=6, la solución óptima es 3+3=2 monedas, no
93     // 4+1+1=3 monedas).
94     vector<int> c5 = {1, 3, 4};
95     assert(sol.coinChange(c5, 6) == 2);
96
97     cout << "322. Coin Change: todas las pruebas pasaron.\n";
98     return 0;
99 }

```

Complejidad: tiempo $O(\text{amount} \times k)$, con k el número de denominaciones, espacio $O(\text{amount})$.

¿Qué significa esa complejidad? Para cada uno de los `amount` montos posibles probamos las k denominaciones disponibles, así que el trabajo total es proporcional al producto de ambos. El espacio es lineal en `amount` porque solo guardamos una tabla de una dimensión (no hace falta recordar qué moneda se usó en cada paso para resolver este problema, solo el conteo mínimo).

Consejos para la entrevista (Amazon). Es tentador proponer una solución **greedy** (usar siempre la moneda más grande que quepa), pero esa estrategia falla con denominaciones como $\{1, 3, 4\}$ y `amount = 6`: greedy elegiría $4 + 1 + 1$ (tres monedas) cuando la respuesta óptima es $3 + 3$ (dos monedas). Vale la pena mencionar ese contraejemplo para justificar por qué se necesita DP. También conviene aclarar el caso límite `amount = 0` (la respuesta es 0, sin usar ninguna moneda) y el caso en que ninguna combinación llega exacto (por ejemplo, monedas que solo suman números pares y un monto impar).

139. Word Break

(M)

Código fuente: [src/07-dp/139-word-break.cpp](#)▷ Ver animación · [local](#) | [web](#)

Dada una cadena `s` y un diccionario `wordDict`, determina si `s` se puede partir en una secuencia de una o más palabras del diccionario (las palabras se pueden reutilizar tantas veces como haga falta).

La idea, con un ejemplo de la vida real. Imagina que recibes un mensaje de texto sin espacios, como "leetcode", y tienes un diccionario de palabras válidas. Quieres saber si es posible insertar espacios en algunos lugares para que el mensaje quede formado completamente por palabras del diccionario, por ejemplo "leet" + "code". La pregunta clave para resolverlo con **programación dinámica** (*dynamic programming*, DP) es: ¿el prefijo del mensaje hasta cierta posición se puede partir bien? Si ya sabemos la respuesta para prefijos más cortos, podemos construir la respuesta para prefijos más largos.

```
s = "leetcode", wordDict = ["leet", "code"]
"leetcode" = "leet" + "code" -> true
```

Cómo lo resuelve el código, paso a paso.

1. Guardamos el diccionario en un conjunto (`unordered_set`) para poder buscar una palabra en tiempo promedio constante.
2. Creamos una tabla booleana `dp` de tamaño `n + 1`, donde `dp[i]` indica si el prefijo de `s` de longitud `i` se puede partir completamente en palabras del diccionario. `dp[0] = true` (la cadena vacía siempre se puede “partir”: caso base).
3. Para cada posición `i` de 1 a `n`, probamos cada posible corte anterior `j` menor que `i`.
4. Si `dp[j]` es verdadero (el prefijo hasta `j` ya se sabe partir bien) y el substring entre `j` e `i` está en el diccionario, entonces `dp[i]` también es verdadero: encontramos una forma válida de partir el prefijo de longitud `i`.
5. La respuesta final es `dp[n]`, es decir, si la cadena completa se puede partir.

```
1 //
   =====
2 // 139. Word Break (Media)
3 //
   -----
4 // Dada una cadena s y un diccionario de palabras wordDict, determina si s se
5 // puede partir en una secuencia de una o más palabras del diccionario,
6 // separadas donde se quiera (las palabras del diccionario se pueden reusar
7 // tantas veces como haga falta).
8 //
9 // Ejemplo: s = "leetcode", wordDict = ["leet", "code"]
10 //
11 // "leetcode" = "leet" + "code" -> true
```

```

12 //
13 // Programación dinámica (dynamic programming, DP) 1D booleana: dp[i] indica
14 // si el prefijo de s de longitud i (los primeros i caracteres) se puede
15 // partir completamente en palabras del diccionario.
16 //
17 // Diagrama de la tabla dp para s = "leetcode":
18 //
19 //   i:      0  1  2  3  4  5  6  7  8
20 //   letra:   l  e  e  t  c  o  d  e
21 //   dp[i]:  T  F  F  F  T  F  F  F  T
22 //
23 //   dp[0] = true (la cadena vacía siempre se puede "partir": caso base).
24 //   Para cada i de 1 a n, probamos cada j < i:
25 //       si dp[j] es true y el substring s[j..i) está en el diccionario:
26 //           dp[i] = true
27 //
28 // Por qué funciona: si el prefijo de longitud i se puede partir en palabras
29 // del diccionario, entonces existe algún punto de corte j donde la última
30 // palabra usada es exactamente s[j..i); todo lo que viene antes de j (el
31 // prefijo de longitud j) también tuvo que partirse correctamente, es decir
32 // dp[j] debe ser true. Como probamos todos los posibles puntos de corte j,
33 // cubrimos todas las formas de llegar a i.
34 //
35 // Complejidad: tiempo  $O(n^2)$  (n valores de i, cada uno prueba hasta n
36 // valores de j, y comparar/crear el substring toma tiempo proporcional a su
37 // longitud), espacio  $O(n)$  para la tabla dp (sin contar el diccionario).
38 //
=====
39 #include <vector>
40 #include <string>
41 #include <unordered_set>
42 #include <cassert>
43 #include <iostream>
44 using namespace std;
45
46 class Solution {
47 public:
48     bool wordBreak(string s, vector<string>& wordDict) {
49         unordered_set<string> dict(wordDict.begin(), wordDict.end());
50         int n = (int)s.size();
51         vector<bool> dp(n + 1, false);
52         dp[0] = true;
53
54         for (int i = 1; i <= n; i++) {
55             for (int j = 0; j < i; j++) {
56                 if (dp[j] && dict.count(s.substr(j, i - j))) {
57                     dp[i] = true;
58                     break;
59                 }
60             }
61         }
62
63         return dp[n];

```

```

64     }
65 };
66
67 int main() {
68     Solution sol;
69
70     // Caso normal: ejemplo clásico del enunciado de LeetCode.
71     vector<string> d1 = {"leet", "code"};
72     assert(sol.wordBreak("leetcode", d1) == true);
73
74     // Caso límite: cadena vacía siempre se puede partir (cero palabras).
75     vector<string> d2 = {"a", "b"};
76     assert(sol.wordBreak("", d2) == true);
77
78     // Caso límite: una sola palabra que coincide exacto con el diccionario.
79     vector<string> d3 = {"hola"};
80     assert(sol.wordBreak("hola", d3) == true);
81
82     // Trampa común: palabras del diccionario que se solapan y hacen que un
83     // corte "obvio" falle, pero exista otro corte válido. "catsandog" NO se
84     // puede partir con este diccionario aunque "cats", "and", "sand" y "dog"
85     // existan por separado, porque ningún corte deja "og" cubierto.
86     vector<string> d4 = {"cats", "dog", "sand", "and", "cat"};
87     assert(sol.wordBreak("catsandog", d4) == false);
88
89     // Caso donde SÍ hay solape pero el corte correcto existe: "cat" + "sand"
90     // + "og" no funciona, pero "cats" + "and" + "dog" sí.
91     vector<string> d5 = {"cats", "dog", "sand", "and", "cat"};
92     assert(sol.wordBreak("catsanddog", d5) == true);
93
94     cout << "139. Word Break: todas las pruebas pasaron.\n";
95     return 0;
96 }

```

Complejidad: tiempo $O(n^2)$, espacio $O(n)$.

¿Qué significa esa complejidad? Para cada una de las n posiciones finales probamos hasta n posibles puntos de corte, y comparar o construir cada substring toma tiempo proporcional a su longitud, así que en el peor caso el trabajo es cuadrático en el tamaño de la cadena. El espacio extra es lineal, para la tabla `dp` (el diccionario en sí no cuenta como espacio auxiliar del algoritmo, ya viene dado).

Consejos para la entrevista (Amazon). La trampa más común es asumir que el primer corte que funciona localmente (por ejemplo, tomar siempre la palabra más larga posible desde el inicio) lleva a la solución completa; con diccionarios que tienen palabras que se solapan, como `{"cats", "cat", "sand", "and", "dog"}` sobre la cadena `"catsandog"`, un corte greedy puede fallar aunque exista una partición válida en otra combinación de palabras. Por eso se necesita recordar, para CADA prefijo, si es alcanzable, en vez de intentar solo un camino. También conviene mencionar la variante recursiva con memoización como forma equivalente de llegar a la misma complejidad.

55. Jump Game

(M)

Código fuente: `src/07-dp/55-jump-game.cpp`▷ Ver animación · [local](#) | [web](#)

Dado un arreglo `nums` donde cada elemento `nums[i]` representa el número máximo de posiciones que se puede saltar hacia adelante desde el índice `i`, determina si es posible llegar desde el índice 0 hasta el último índice del arreglo.

La idea, con un ejemplo de la vida real. Imagina piedras colocadas en fila para cruzar un río, y cada piedra tiene escrito cuántas piedras hacia adelante puedes brincar desde ahí como máximo. En vez de probar todas las secuencias posibles de saltos (que se ramifican exponencialmente), basta con llevar la cuenta de *cuál es la piedra más lejana a la que se puede llegar*, considerando todo lo que ya se sabe alcanzable. Si en algún momento la siguiente piedra que toca revisar está más allá de ese alcance máximo, ya no hay forma de llegar ahí y el cruce es imposible.

```
nums = [2, 3, 1, 1, 4]
desde el índice 0 se alcanza el índice 2
desde el índice 1 se alcanza el índice 4 (el último) -> true
```

Cómo lo resuelve el código, paso a paso. Este problema admite una solución de programación dinámica (*dynamic programming*, DP) tabular, pero la solución esperada en entrevista es un algoritmo **greedy** (voraz), más simple y con espacio constante.

1. Mantenemos una sola variable, `alcanceMaximo`, inicializada en 0.
2. Recorremos el arreglo de izquierda a derecha con un índice `i`.
3. Si `i` es mayor que `alcanceMaximo`, significa que ninguna posición ya confirmada como alcanzable puede llegar hasta aquí: devolvemos `false` de inmediato.
4. Si no, actualizamos `alcanceMaximo = max(alcanceMaximo, i + nums[i])`, es decir, consideramos si saltar desde la posición actual extiende el alcance máximo conocido.
5. Si el ciclo termina sin encontrar una posición inalcanzable, devolvemos `true`.

```
1 //
2 // =====
3 //
4 // 55. Jump Game (Media)
5 //
6 // Dado un arreglo nums donde cada elemento nums[i] representa el número
7 // máximo de posiciones que puedes saltar hacia adelante desde el índice i,
8 // determina si es posible llegar desde el índice 0 hasta el último índice
9 // del arreglo.
10 //
11 // Ejemplo: nums = [2,3,1,1,4]
12 //
13 // índice:  0  1  2  3  4
14 // nums[i]: 2  3  1  1  4
```

```

13 //
14 //     desde 0 saltamos hasta 2 posiciones (llegamos como máximo al índice 2),
15 //     desde 1 saltamos hasta 3 posiciones (llegamos como máximo al índice 4,
16 //     que ya es el último) -> true
17 //
18 // Este problema SÍ se puede resolver con programación dinámica (dynamic
19 // programming, DP) tabular, pero admite una solución greedy (voraz) más
20 // simple y más eficiente en espacio, que es la que se espera en una
21 // entrevista: en vez de recordar si cada posición individual es alcanzable,
22 // solo llevamos la cuenta del alcance máximo visto hasta el momento.
23 //
24 // Diagrama del algoritmo greedy:
25 //
26 //     alcanceMaximo = 0
27 //     para i desde 0 hasta el último índice:
28 //         si i > alcanceMaximo:
29 //             // ni siquiera pudimos llegar a esta posición -> imposible
30 //             return false
31 //         alcanceMaximo = max(alcanceMaximo, i + nums[i])
32 //     return true
33 //
34 // Por qué funciona: alcanceMaximo siempre representa el índice más lejano al
35 // que se puede llegar usando únicamente posiciones ya confirmadas como
36 // alcanzables (índices 0..i). Si en algún momento i supera ese alcance,
37 // significa que ninguna posición anterior alcanzable pudo "brincar" hasta i,
38 // así que i (y todo lo que dependa de partir desde i) es inalcanzable. Si el
39 // ciclo completa sin que eso pase, el alcance máximo termina cubriendo el
40 // último índice.
41 //
42 // Complejidad: tiempo  $O(n)$ , espacio  $O(1)$ .
43 //

```

```

44 #include <vector>
45 #include <algorithm>
46 #include <cassert>
47 #include <iostream>
48 using namespace std;
49
50 class Solution {
51 public:
52     bool canJump(vector<int>& nums) {
53         int n = (int)nums.size();
54         int alcanceMaximo = 0;
55
56         for (int i = 0; i < n; i++) {
57             if (i > alcanceMaximo) return false;
58             alcanceMaximo = max(alcanceMaximo, i + nums[i]);
59         }
60
61         return true;
62     }
63 };
64

```

```

65 int main() {
66     Solution sol;
67
68     // Caso normal: ejemplo clásico del enunciado de LeetCode.
69     vector<int> n1 = {2, 3, 1, 1, 4};
70     assert(sol.canJump(n1) == true);
71
72     // Caso límite: un solo elemento, ya estás en el último índice.
73     vector<int> n2 = {0};
74     assert(sol.canJump(n2) == true);
75
76     // Trampa común: un cero en medio del camino que bloquea el avance,
77     // porque ninguna posición anterior alcanza a saltar por encima de él.
78     vector<int> n3 = {3, 2, 1, 0, 4};
79     assert(sol.canJump(n3) == false);
80
81     // Caso donde un cero NO bloquea porque una posición anterior ya tiene
82     // alcance suficiente para saltar por encima de él.
83     vector<int> n4 = {2, 3, 0, 1, 4};
84     assert(sol.canJump(n4) == true);
85
86     // Caso límite: el primer elemento es 0 pero el arreglo tiene un solo
87     // elemento, así que ya llegamos sin necesidad de saltar.
88     vector<int> n5 = {0};
89     assert(sol.canJump(n5) == true);
90
91     cout << "55. Jump Game: todas las pruebas pasaron.\n";
92     return 0;
93 }

```

Complejidad: tiempo $O(n)$, espacio $O(1)$.

¿Qué significa esa complejidad? Recorremos el arreglo una sola vez, actualizando una sola variable en cada paso, así que el tiempo es lineal en el tamaño del arreglo. El espacio es constante porque no se necesita ninguna tabla ni estructura auxiliar, a diferencia de una solución DP tabular que gastaría espacio lineal.

Consejos para la entrevista (Amazon). Vale la pena mencionar primero la versión DP ($dp[i]$ = “¿se puede llegar al índice i ?”), mirando hacia atrás todas las posiciones que podrían saltar hasta ahí) para demostrar que se entiende el problema desde ese ángulo, y luego señalar la mejora greedy y por qué es válida: `alcanceMaximo` resume toda la información relevante de las posiciones ya visitadas, sin necesidad de recordar el detalle de cada una. Un caso límite importante es un cero en medio del arreglo que *no* bloquea el camino porque una posición anterior ya tenía alcance suficiente para saltar por encima de él; hay que distinguirlo del caso en que el cero sí bloquea porque ninguna posición previa alcanza a rebasarlo.

5. Longest Palindromic Substring

(M)

Código fuente: `src/07-dp/5-longest-palindromic-substring.cpp`

▷ Ver animación · [local](#) | [web](#)

Dada una cadena `s`, encuentra el substring (secuencia de caracteres *contiguos*) más largo que sea un palíndromo, es decir, que se lea igual de izquierda a derecha que de derecha a izquierda.

La idea, con un ejemplo de la vida real. Un palíndromo siempre es simétrico respecto a un punto central, como un espejo. Ese centro puede caer exactamente sobre un carácter (palíndromos de longitud impar, como "aba") o justo entre dos caracteres (palíndromos de longitud par, como "abba"). La técnica de **expansión desde el centro** (una variante de *two pointers*, dos punteros) consiste en probar, uno por uno, cada posible centro de la cadena, y desde ahí ir separando dos punteros hacia afuera mientras los caracteres que van quedando en los extremos sigan coincidiendo, igual que cuando revisas si una palabra es capicúa comparando letras desde el centro hacia las puntas.

```
s = "babad"
centro en la 'a' del medio (índice 1) -> se expande a "bab"
también existe "aba" como respuesta válida, de la misma longitud
```

Cómo lo resuelve el código, paso a paso.

1. Para una cadena de longitud n hay $2n - 1$ centros posibles: n centros de un solo carácter y $n - 1$ centros “entre pares” de caracteres consecutivos.
2. Para cada centro, colocamos dos punteros `L` y `R` sobre ese centro (iguales si el centro es un carácter, o adyacentes si el centro es “entre pares”).
3. Mientras `L` y `R` sigan dentro del rango de la cadena y `s[L] == s[R]`, expandimos: `L` retrocede uno y `R` avanza uno.
4. Al detenerse la expansión, el palíndromo válido más reciente es el substring entre `L + 1` y `R - 1`.
5. Si esa longitud es mayor que la mejor encontrada hasta ahora, actualizamos el inicio y la longitud del mejor palíndromo.
6. Al terminar de probar todos los centros, devolvemos el substring guardado como el mejor.

```
1 //
   =====
2 // 5. Longest Palindromic Substring (Media)
3 //
   -----
4 // Dada una cadena s, encuentra el substring (secuencia de caracteres
5 // CONTIGUOS) más largo que sea un palíndromo (se lee igual de izquierda a
6 // derecha que de derecha a izquierda).
7 //
8 // Ejemplo: s = "babad"
```

```

9 //
10 //  "bab" es palíndromo (longitud 3)
11 //  "aba" también es palíndromo (longitud 3)
12 //  cualquiera de los dos es una respuesta válida
13 //
14 // Técnica: expansión desde el centro (una variante de two pointers, dos
15 // punteros que se separan en vez de acercarse). Un palíndromo es simétrico
16 // respecto a un centro, y ese centro puede ser un solo carácter (para
17 // palíndromos de longitud impar, como "aba") o el punto entre dos caracteres
18 // (para palíndromos de longitud par, como "abba"). Para una cadena de
19 // longitud n hay exactamente 2n-1 centros posibles: n centros de un solo
20 // carácter y n-1 centros "entre pares".
21 //
22 // Diagrama para s = "babad", probando el centro en el índice 1 ('a'):
23 //
24 //  b a b a d
25 //      ^
26 //  L=1, R=1 -> s[L]==s[R] ('a'), se expande
27 //  L=0, R=2 -> s[L]==s[R] ('b'), se expande
28 //  L=-1      -> fuera de rango, se detiene
29 //  palíndromo encontrado: s[0..2] = "bab" (longitud 3)
30 //
31 // Algoritmo:
32 //  mejorInicio = 0, mejorLongitud = 1
33 //  para cada centro posible c de 0 a 2n-2:
34 //    si c es par: L = R = c / 2                (centro de un carácter)
35 //    si c es impar: L = c / 2, R = L + 1        (centro entre dos caracteres)
36 //    mientras L >= 0 y R < n y s[L] == s[R]:
37 //      L--
38 //      R++
39 //    // al salir del ciclo, el palíndromo válido es s[L+1 .. R-1]
40 //    si (R - 1) - (L + 1) + 1 > mejorLongitud:
41 //      actualizar mejorInicio y mejorLongitud
42 //    devolver s.substr(mejorInicio, mejorLongitud)
43 //
44 // Por qué funciona: cada palíndromo tiene un centro único (aunque distintos
45 // palíndromos puedan compartir el mismo centro en distintas expansiones), así
46 // que probar los 2n-1 centros posibles y expandir cada uno hasta el máximo
47 // garantiza encontrar el palíndromo más largo posible alrededor de cada
48 // centro, y por lo tanto el más largo de toda la cadena. No hace falta la
49 // tabla 2D de DP (que también es correcta) porque la expansión desde el
50 // centro logra lo mismo con espacio constante extra.
51 //
52 // Complejidad: tiempo O(n^2) (2n-1 centros, cada expansión puede tomar hasta
53 // O(n) pasos), espacio O(1) extra (sin contar la cadena resultado).
54 //
=====

55 #include <string>
56 #include <cassert>
57 #include <iostream>
58 using namespace std;
59
60 class Solution {

```

```

61 public:
62     string longestPalindrome(string s) {
63         int n = (int)s.size();
64         if (n == 0) return "";
65
66         int mejorInicio = 0, mejorLongitud = 1;
67
68         for (int c = 0; c < 2 * n - 1; c++) {
69             int L = c / 2;
70             int R = (c % 2 == 0) ? c / 2 : c / 2 + 1;
71
72             while (L >= 0 && R < n && s[L] == s[R]) {
73                 L--;
74                 R++;
75             }
76
77             // El palíndromo válido más reciente es s[L+1 .. R-1].
78             int longitud = (R - 1) - (L + 1) + 1;
79             if (longitud > mejorLongitud) {
80                 mejorLongitud = longitud;
81                 mejorInicio = L + 1;
82             }
83         }
84
85         return s.substr(mejorInicio, mejorLongitud);
86     }
87 };
88
89 int main() {
90     Solution sol;
91
92     // Caso normal: ejemplo clásico del enunciado de LeetCode. Hay dos
93     // respuestas válidas ("bab" y "aba"); esta implementación, al probar
94     // los centros de izquierda a derecha y solo reemplazar cuando encuentra
95     // una longitud ESTRICTAMENTE mayor, se queda con la primera que
96     // encuentra, que es "bab".
97     assert(sol.longestPalindrome("babad") == "bab");
98
99     // Caso límite: cadena vacía.
100    assert(sol.longestPalindrome("") == "");
101
102    // Caso límite: un solo carácter (es palíndromo trivial de sí mismo).
103    assert(sol.longestPalindrome("a") == "a");
104
105    // Trampa común: palíndromo de longitud par, que requiere un centro
106    // "entre dos caracteres" y no solo centros de un carácter.
107    assert(sol.longestPalindrome("cbbd") == "bb");
108
109    // Caso sin ambigüedad de empates: toda la cadena es un palíndromo.
110    assert(sol.longestPalindrome("racecar") == "racecar");
111
112    cout << "5. Longest Palindromic Substring: todas las pruebas pasaron.\n";
113    return 0;
114 }

```

Complejidad: tiempo $O(n^2)$, espacio $O(1)$ extra.

¿Qué significa esa complejidad? Hay $2n - 1$ centros posibles, y cada expansión puede llegar a recorrer, en el peor caso, hasta $O(n)$ pasos (por ejemplo, cuando toda la cadena es un solo palíndromo), así que el trabajo total es cuadrático. El espacio extra es constante porque solo se usan un puñado de variables (los dos punteros y las marcas del mejor resultado encontrado), sin ninguna tabla auxiliar, a diferencia de la solución alternativa de DP con tabla 2D, que sí gastaría espacio cuadrático.

Consejos para la entrevista (Amazon). Muchos candidatos proponen primero la tabla 2D de DP ($dp[i][j] = \text{"¿el substring entre } i \text{ y } j \text{ es palíndromo?"}$), que es correcta pero usa espacio $O(n^2)$; vale la pena mencionarla y luego señalar que expansión desde el centro llega al mismo tiempo con espacio constante. Un error común es olvidar los centros “entre pares” y solo probar centros de un carácter, lo que hace fallar casos de palíndromos de longitud par como “cbbd”. También conviene aclarar qué hacer ante empates de longitud (cualquier palíndromo de la longitud máxima es una respuesta válida, no hay una única correcta).

Capítulo 7

Ruta de problemas

Lista objetivo por bloque. Los marcados con ✓ ya tienen capítulo; el resto son las siguientes iteraciones.

Bloque 1 — Grafos. ✓ 200 Number of Islands • ✓ 994 Rotting Oranges • ✓ 695 Max Area of Island • ✓ 542 01 Matrix • ✓ 1091 Shortest Path in Binary Matrix • ✓ 417 Pacific Atlantic • ✓ 79 Word Search • ✓ 207 Course Schedule • ✓ 210 Course Schedule II • ✓ 269 Alien Dictionary • ✓ 127 Word Ladder • ✓ 126 Word Ladder II • ✓ 261 Graph Valid Tree • ✓ 323 Connected Components • ✓ 128 Longest Consecutive • ✓ 547 Number of Provinces • ✓ 743 Network Delay Time • ✓ 787 Cheapest Flights K Stops • ✓ 133 Clone Graph • ✓ 1192 Critical Connections.

Bloque 2 — Árboles. ✓ 236 LCA • ✓ 1644 LCA II • ✓ 103 Zigzag Level Order • ✓ 199 Right Side View • ✓ 987 Vertical Order • ✓ 124 Max Path Sum • ✓ 543 Diameter • ✓ 863 Distance K • ✓ 98 Validate BST • ✓ 105 Build Tree Pre+In • ✓ 337 House Robber III • ✓ 297 Serialize/Deserialize.

Bloque 3 — Intervalos. ✓ 56 Merge Intervals • ✓ 253 Meeting Rooms II • ✓ 2402 Meeting Rooms III • ✓ 1094 Car Pooling.

Bloque 4 — Heaps / Top-K. ✓ 215 Kth Largest • ✓ 347 Top K Frequent • ✓ 973 K Closest Points • ✓ 295 Find Median • ✓ 23 Merge k Sorted Lists • ✓ 621 Task Scheduler.

Bloque 5 — Design. 146 LRU Cache • 348 Design Tic-Tac-Toe • 642 Search Autocomplete.

Bloque 6 — Sliding Window / Stack. ✓ 3 Longest Substring • ✓ 76 Minimum Window • ✓ 42 Trapping Rain Water • ✓ 84 Largest Rectangle • ✓ 394 Decode String.

Bloque 7 — DP. ✓ 322 Coin Change • ✓ 139 Word Break • ✓ 55 Jump Game • ✓ 5 Longest Palindromic Substring.